EVANDRO CARLOS TERUEL

PROGRAMAÇÃO ORIENTADA A OBJETOS COM Java™

sem mistérios

UNINOVE – USO EXCLUSIVO PARA ALUNO

UNINOVE

UNINOVE – USO EXCLUSIVO PARA ALUNO

Universidade Nove de Julho – UNINOVE
Rua Vergueiro, 235/249 – 11º andar
01504-001 – Liberdade – São Paulo, SP
Tel.: (11) 3385-9218 - editora@uninove.br

UNINOVE – USO EXCLUSIVO PARA ALUNO

Evandro Carlos Teruel

PROGRAMAÇÃO ORIENTADA A OBJETOS COM Java™

sem mistérios

São Paulo
2016

UNINOVE

© 2016 UNINOVE

Todos os direitos reservados. A reprodução desta publicação, no todo ou em parte, constitui violação do *copyright* (Lei nº 9.610/98). Nenhuma parte desta publicação pode ser reproduzida por qualquer meio, sem a prévia autorização da UNINOVE.

Os conceitos emitidos neste livro são de inteira responsabilidade do autor.

Conselho Editorial
Eduardo Storópoli
Maria Cristina Barbosa Storópoli
Patricia Miranda Guimarães
André Felipe Henriques Librantz
Marcos Alberto Bussab

Capa: Big Time Serviços Editoriais
Editoração eletrônica: Big Time Serviços Editoriais
Revisão: Antonio Marcos Cavalheiro

Catalogação na Publicação (CIP)
Cristiane dos Santos Monteiro - CRB/8 7474

Teruel, Evandro Carlos.

Programação orientada a objetos com JAVA sem mistérios / Evandro Teruel. – São Paulo : Universidade Nove de Julho – UNINOVE, 2016.
386 p. il.

Inclui bibliografia
ISBN: 978-85-89852-26-5 (impresso)
ISBN: 978-85-89852-27-2 (*e-book*)

I. JAVA – programação. II. Teruel, Evandro Carlos. III. Título.

CDU 004.439JAVA

UNINOVE – USO EXCLUSIVO PARA ALUNO

Fabricantes

Java e *NetBeans*: desenvolvido pela Sun Microsystems, hoje de propriedade da Oracle Corporation.

MySQL: desenvolvido pela MySQL AB, propriedade da Sun Microsystems, hoje parte da Oracle Corporation.

Windows: desenvolvido pela Microsoft Corporation.

Microsoft Informática Ltda.

Av Nações Unidas, 12901

Torre Norte – 27º andar

04578-000 – São Paulo

Fone: (11) 5504-2155

Site: www.microsoft.com.br

Oracle Brasil

Rua Dr. Jose Aureo Bustamante 455

Vila Cordeiro – CEP 04710-090 – São Paulo

Fone: +55 11 5189-1000

Site: http://www.oracle.com/br/index.html

UNINOVE – USO EXCLUSIVO PARA ALUNO

Requisitos de Hardware e Software

Software

- Windows XP Professional SP3/Vista SP1/Windows 7/8 Professional
- *NetBeans* 6.8 ou posterior
- Java Development Kit (JDK) versão 6 ou posterior
- MySQL Community Server 5.1.53 ou posterior
- Ø MySQL Workbench 5.2.30 ou posterior

Hardware

- Processador: 2,6 GHz Intel Premium IV ou equivalente
- Memória: 2 GB
- Espaço em disco: 1 GB de espaço livre em disco

UNINOVE – USO EXCLUSIVO PARA ALUNO

Dedicatória

À minha esposa Angela e aos meus filhos, Pedro Henrique e Ana Luiza, que foram compreensivos nos períodos de ausência em que me dediquei à pesquisa e ao trabalho.

Aos meus sogros Iracema e Jorge.

Aos meus pais, José e Aparecida, que mesmo longe me apoiam.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Agradecimentos

Aos colegas gestores e professores da diretoria dos cursos de Informática da UNINOVE.

"Educa a criança no caminho em que deve andar; e até quando envelhecer não se desviará dele."

Provérbios 22:6

UNINOVE – USO EXCLUSIVO PARA ALUNO

Sumário

Prefácio 15
Sobre o autor 17
Apresentação 19
Introdução 21

1. Preparando o ambiente para programar em Java 23
1.1 Plataformas Java 23
1.1.1 Java Standard Edition (SE) 23
1.1.2 JavaFX 26
1.1.3 Java Enterprise Edition (JEE) 26
1.1.4 Java Micro Edition (JME) 26
1.2 Principais IDEs para programação em Java 26
1.3 Criando um aplicativo Java 27
1.3.1 Criando um projeto 27
1.3.2 A classe principal 29
1.3.3 Compilando e executando 31
1.4 Principais recursos do *NetBeans* IDE 32
1.4.1 Depurando o código 32
1.5 Resumo 34
1.6 Exercícios 35

2. Programação Estruturada em Java 37
2.1 Paradigmas de programação 37
2.1.1 Características da Programação Estruturada 38
2.1.2 Características da Programação Orientada a Objetos 39
2.2 Variáveis em Java 41
2.3 Tipos de dados em Java 42
2.3.1 Tipos de dados primitivos em Java 42
2.3.2 Conversões de tipos de dados primitivos 45
2.3.3 Tipos de dados por referência em Java 47
2.4 Tipos de operações realizadas em programas Java 50
2.4.1 Expressões aritméticas 50
2.4.2 Expressões relacionais 52
2.4.3 Expressões lógicas 53
2.4.4 Expressões mistas 53

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.5 Estruturas de seleção utilizadas em Java 54
2.5.1 Estrutura de seleção if...else 54
2.5.2 Estrutura de seleção if...elseif...else 55
2.5.3 Comparando valores String com if...else 56
2.5.4 Estrutura de seleção switch...case...default 57
2.6 Lações de repetição em Java 58
2.6.1 Laços de repetição for 59
2.6.2 Laços de repetição while 60
2.6.3 Laços de repetição do...while 61
2.6.4 *Array*s em Java 61
2.6.5 *Array*s unidimensionais 62
2.6.6 *Array*s multidimensionais 64
2.6.7 Percorrendo *array*s com laços de repetição 65
2.7 Variáveis e constantes em Java 66
2.7.1 Constantes 67
2.7.2 Boas práticas para nomeação de variáveis e constantes em Java 67
2.8 Resumo 68
2.9 Exercícios 69

3. Orientação a objetos – conceitos básicos 73
3.1 Classes 74
3.1.1 Declaração da classe 74
3.1.2 Declaração dos atributos 74
3.1.3 Construtor 75
3.1.4 Métodos 76
3.1.5 Exemplo de classe com atributos, construtor e métodos 79
3.1.6 Diagrama UML para representar uma classe 82
3.1.7 Boas práticas para nomeação de classes 83
3.2 Pacotes 83
3.2.1 Boas práticas para nomeação de pacotes 85
3.2.2 Como utilizar pacotes das bibliotecas Java 85
3.3 Objetos 86
3.3.1 Instância de objetos 86
3.4 Encapsulamento 96
3.5 Criação de objetos com entradas do usuário 96
3.5.1 Entradas do usuário com a classe Scanner 96
3.5.2 Entradas do usuário com a classe JOptionPane 99
3.5.3 Comandos encadeados 102
3.6 Classes sem atributos 103
3.7 *Array* de objetos 106

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.7.1 Quando usar *array*s de objetos 109
3.8 Resumo 110
3.9 Exercícios 111
3.9.1 Exercícios práticos 111
3.9.2 Exercícios conceituais 113

4. Orientação a objetos – conceitos avançados 115
4.1 Herança 115
4.1.1 Exemplo de aplicação com herança 117
4.1.2 Superclasses e subclasses nas relações de herança 126
4.1.3 Herança em cadeia 127
4.1.4 Conseguindo polimorfismo com herança 128
4.2 Interfaces 133
4.2.1 Quando utilizar interfaces 136
4.2.2 Conseguindo polimorfismo com utilização de interfaces 136
4.2.3 Implementação de mais de uma interface 143
4.3 Classes Abstratas 145
4.3.1 Exemplo de aplicação com classes abstratas 146
4.4 Modificadores de visibilidade 153
4.4.1 Modificador public 153
4.4.2 Modificador private 155
4.4.3 Modificador protected 156
4.4.4 Sem modificador 157
4.5 Garbage collector 157
4.6 Anotações 158
4.7 Resumo 158
4.8 Exercícios 160
4.8.1 Exercícios práticos 160
4.8.2 Exercícios conceituais 161

5. Métodos estáticos e tratamento de exceções 163
5.1 Métodos Estáticos 164
5.1.1 Utilizando métodos estáticos de classes das bibliotecas que fazem parte do Java 164
5.1.2 Utilizando métodos estáticos de classes criadas pelo desenvolvedor 166
5.1.3 Utilizando métodos estáticos na classe principal 169
5.1.4 Criando uma biblioteca de classes com métodos estáticos 170
5.2 Tratamento de Exceções 173
5.2.1 Tratamento de Exceções com try...catch 173

UNINOVE – USO EXCLUSIVO PARA ALUNO

5.2.2 Tratamento de Exceções com throws 179
5.3 Resumo 182
5.4 Exercícios 182

6. Criação de interface gráfica 187
6.1 API swing 187
6.1.1 Contêineres swing 187
6.1.2 Formas de criação de *layout* utilizando a API *swing* 189
6.1.3 Aplicação exemplo criando *layout* com posicionamento estático 189
6.2 Criação de interface com gerenciadores de *layout* 199
6.2.1 FlowLayout 199
6.2.2 BorderLayout 202
6.2.3 GridLayout 205
6.2.4 GridBagLayout 207
6.2.5 Utilizando gerenciadores de layout compostos 216
6.2.6 Criação de interfaces gráficas ricas utilizando a API *swing* 223
6.3 Tratamento de eventos 254
6.3.1 Tratamento de eventos de ação 254
6.3.2 Tratamento de eventos de janela 261
6.3.3 Tratamento de eventos de foco 266
6.3.4 Tratamento de eventos de mouse 271
6.4 Criação de aplicação com várias janelas ou com abas 273
6.4.1 Exemplo de aplicação com várias janelas 274
6.4.2 Exemplo de aplicação com abas na mesma janela 279
6.5 Criação de interfaces do usuário utilizando recursos de arrastar/soltar com o *NetBeans* 283
6.5.1 Exemplo de criação de interface gráfica simples utilizando os recursos de arrastar/soltar do *NetBeans* 283
6.6 Resumo 294
6.7 Exercícios 297
6.7.1 Exercícios práticos 297
6.7.2 Exercícios conceituais 300

7. Acesso a banco de dados 303
7.1 Os padrões ODBC e JDBC 303
7.2 Baixando e instalando o MySQL 304
7.2.1 Criando o banco de dados e a tabela com o MySQL Workbench 306
7.2.2 Criando o banco de dados e a tabela com o *NetBeans* 310
7.3 Operações em banco de dados 315
7.3.1 Classes e interfaces usadas em operações com banco de dados 315

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.3.2 Operação de cadastro 316
7.3.3 Operação de consulta 322
7.3.4 Operação de consulta de todos os registros 324
7.3.5 Operação de exclusão 327
7.3.6 Diferenças entre as interfaces PreparedStatement e Statement 331
7.4 Exemplo de aplicação CRUD 332
7.5 O padrão Data Access Object (DAO) 356
7.6 Criando uma aplicação de cadastro com duas camadas 357
7.6.1 A classe DAO 358
7.6.2 A classe modelo da tabela do banco de dados 361
7.6.3 A classe para geração do formulário de cadastro 363
7.6.4 Outras operações na classe DAO 368
7.6.5 Criação de interface DAO 372
7.7 Resumo 374
7.8 Exercícios 376
7.8.1 Exercícios práticos 377
7.8.2 Exercícios conceituais 377

8. Recomendações para complementar a formação em Java 379
8.1 Collections 379
8.2 Mapeamento Objeto-Relacional 380
8.3 Java EE 381
8.4 Padrões de projeto 381
8.5 Frameworks 382
8.6 Android e iOS 383
8.7 Principais certificações Java 383
8.8 Resumo 384

Referências 385

UNINOVE – USO EXCLUSIVO PARA ALUNO

UNINOVE – USO EXCLUSIVO PARA ALUNO

Prefácio

Quando comecei a trabalhar com programação, em 1989, utilizando a linguagem Basic, me deparei com um "admirável mundo novo", o mundo da criação. Era possível fazer as máquinas tomarem decisões, executar tarefas rotineiras e gerenciar o armazenamento de dados que até aquele momento era feito em papel, digitados por meio das incríveis máquinas de datilografia. O pré-requisito para conquistar uma boa vaga de emprego naquela época era ter um curso de datilografia. Cursos de computação eram raros e caros, um privilégio de poucos. Mesmo as empresas, muitas delas, não tinham condições financeiras para comprar um computador.

Naquela época a computação já era dita pela mídia especializada como o conjunto de tecnologias que ditariam os caminhos do futuro, mas nem os maiores entusiastas poderiam imaginar o cenário que temos hoje, com mídias sociais, comunicação instantânea, dinheiro virtual, *smartphones*, globalização e democratização da informação.

Os *mainframes* daquela época, anos depois, na década de 1990, deram lugar aos servidores distribuídos, porém, devido ao grande volume de dados e processamento, voltaram atualmente para assumir de vez seu lugar de destaque no cenário da computação mundial.

Os telefones fixos, caríssimos, começaram a perder espaço para os celulares no final da década de 1990 e a Internet, discada, começou a ser oferecida em banda larga, tudo ainda muito caro. Nesse período, já começava a ter destaque as primeiras linguagens de programação orientadas a objetos, vistas com certa desconfiança pela comunidade de programadores de linguagens estruturadas.

Mais de uma década depois, salvo linguagens estruturadas como Cobol, que continuam em foco para programação de *mainframes*, as linguagens orientadas a objetos dominam o cenário de desenvolvimento em todos os principais ambientes de uso (*desktop*, *mobile* e *web*).

Assim, aprender bem uma linguagem de programação orientada a objeto bem aceita nas empresas é essencial para uma boa colocação no mercado de trabalho.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Java, talvez a linguagem de programação orientada a objeto mais poderosa na atualizadade, é uma excelente alternativa para quem quer aproveitar as milhares de vagas de emprego abertas para programadores no Brasil. Apesar dos atrativos, aprender Java não é uma tarefa fácil nem rápida.

O objetivo deste livro é justamente apresentar os conceitos da orientação a objetos e a linguagem Java com um enfoque prático e didático, com muitos exemplos e exercícios.

Espero que você aprecie a leitura e que o conteúdo seja útil para o seu desenvolvimento profissional.

Prof. Me. Evandro Carlos Teruel

UNINOVE – USO EXCLUSIVO PARA ALUNO

Sobre o autor

UNINOVE – USO EXCLUSIVO PARA ALUNO

Evandro Carlos Teruel é formado em Tecnologia em Processamento de Dados. É especialista Pós-Graduado em Projeto e Desenvolvimento de Sistemas para Web e também em Segurança da Informação. Cursou Mestrado em Tecnologia na linha de pesquisa Gestão da Tecnologia da Informação. É também certificado ITIL v3 Foundation. Já atuou na área de desenvolvimento de *software* em empresa própria e atualmente é gestor e professor universitário.

É autor de diversos livros na área de Tecnologia da Informação, dentre eles: "Web Total: desenvolva sites com tecnologias de uso livre", "Web Mobile", "Arquitetura de Sistemas para WEB com Java Utilizando Design Patterns e Frameworks", "HTML5: Guia prático" e "ITIL v3 atualizado em 2011: conceitos e simulados para certificação ITIL foundation e teste de conhecimento".

UNINOVE – USO EXCLUSIVO PARA ALUNO

Apresentação

Este livro apresenta de maneira prática e simplificada a programação orientada a objetos utilizando Java. Você aprenderá a diferença entre a programação estruturada e orientada a objetos, as estruturas e comandos fundamentais do Java, os conceitos essenciais da programação orientada a objetos (como encapsulamento, herança, polimorfismo, interfaces etc.), a criação de interfaces gráficas para *desktop*, tratamento de exceções e acesso a banco de dados utilizando a API Java Database Connectivity (JDBC).

Cada capítulo é estruturado com uma introdução, apresentação dos conteúdos teóricos com exemplos, resumo e lista de exercícios.

Conteúdo dos capítulos:

Capítulo 1 – Preparando o ambiente para programar em Java

Este capítulo ensina a baixar, instalar e preparar um ambiente de desenvolvimento Java para Windows. Ensina também a criar, desenvolver uma pequena aplicação, compilá-la e executá-la.

Capítulo 2 – Programação Estruturada com Java

Este capítulo apresenta a diferença entre a programação estruturada e orientada a objetos; os tipos de dados primitivos e por referência; declaração e manipulação de variáveis e constantes; operações aritméticas, relacionais, lógicas e mistas; estruturas de seleção; laços de repetição e *array*s.

Capítulo 3 – Orientação a objetos – conceitos básicos

Este capítulo ensina os conceitos fundamentais da orientação a objetos, como classes, atributos, construtores, métodos, objetos, encapsulamento e modificadores de visibilidade. Apresenta vários exemplos de classes e formas de instanciar objetos, manipular valores em seus atributos e acessar seus métodos. Apresenta também as formas de entrada de dados utilizando as classes Scanner e JOptionPane.

Capítulo 4 – Orientação a objetos – conceitos avançados

Este capítulo apresenta conceitos avançados da orientação a objetos como herança, interfaces, classes abstratas e polimorfismo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Capítulo 5 – Métodos estáticos e tratamento de exceções

Este capítulo apresenta o uso de métodos estáticos, criação de bibliotecas de classes para reúso e as formas de tratamento de exceções.

Capítulo 6 – Criação de interface gráfica

Este capítulo ensina as diversas formas de criar interfaces gráficas do usuário utilizando as APIs *swing* e *awt* e a tratar eventos de ação do usuário sobre a interface.

Capítulo 7 – Acesso a banco de dados

Este capítulo ensina como criar aplicações Java que acessam e executam operações em bancos de dados utilizando a *API Java Database Connectivity* (JDBC) e o padrão *Data Access Object* (DAO). Você aprenderá a criar uma aplicação que executa as operações CRUD (*Create*, *Read*, *Update* e *Delete*) no banco de dados.

Capítulo 8 – Recomendações para complementar a formação em Java

Este capítulo faz um direcionamento de estudos para complementar a formação em Java. Recomenda o estudo de coleções, mapeamento objeto-relacional, *design pattern* e *frameworks*, tecnologias Java EE e desenvolvimento para *tablets* e celulares com *Android*. Recomenda ainda a realização das certificações OCAJP e OCPJP.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Introdução

Em meio à infinidade de tecnologias, linguagens e aplicativos de todos os tipos e para todos os fins e plataformas existentes hoje, é difícil escolher um caminho para trilhar na área da informática que possa garantir um futuro profissional de sucesso. Se você escolheu a área de desenvolvimento de sistemas, independente do caminho que irá seguir, precisará conhecer bem uma linguagem de programação orientada a objetos. As linguagens do pacote *Visual Studio .NET* da Microsoft e a plataforma Java da *Oracle Corporation* são as duas grandes plataformas de desenvolvimento orientado a objetos utilizadas hoje na maioria das empresas de software. Sendo assim, é natural você se dirigir a uma destas plataformas. As linguagens do pacote *Visual Studio* são mais fáceis de aprender e utilizar, porém, em contrapartida, são de uso pago e geram aplicações com interfaces bonitas, mas normalmente pesadas para executar. Já a plataforma Java é de uso gratuito, mas o tempo de aprendizagem é maior, assim como o nível de complexidade. Por ser uma plataforma gratuita e de código aberto, ganhou a simpatia da comunidade adepta dos softwares livres, que colaboraram para elevar a plataforma a um nível onde se pode desenvolver praticamente qualquer tipo de software para qualquer tipo de dispositivo existente. Para acomodar e classificar melhor os recursos, a plataforma Java em si foi subdividida em plataformas menores voltadas para domínios específicos, como Java SE, que concentra os recursos essenciais da plataforma e é utilizada para desenvolvimento de aplicações para *desktop*; Java ME, para dispositivos móveis e computação embarcada; Java EE, para web e outras aplicações em que rodam em rede baseadas em servidores de aplicação; e JavaFX para multimídia e interfaces gráficas robustas. Dentro de cada uma destas subplataformas estão concentrados pacotes de classes, APIs, bibliotecas, *frameworks* e recursos que precisariam de um ou mais livros cada um para descrevê-los.

Para que você tenha uma ideia clara do que se tornou a plataforma Java, se você fizesse 4 anos de faculdade apenas para aprender Java, provavelmente não dominaria todos os seus recursos.

Não é pretensão do autor deste livro, com os conteúdos apresentados, fazer com que você domine Java, mas sim apresentar os principais recursos do Java SE (*Standard Edition*) a partir dos quais você terá condições de evoluir no caminho da busca por conhecimentos mais completos da plataforma.

Você aprenderá nessa obra a criar um ambiente de desenvolvimento adequado para Java; a utilizar as estruturas essenciais da programação estruturada como

UNINOVE – USO EXCLUSIVO PARA ALUNO

definição de tipos de dados, operações, variáveis, constantes, *arrays*, laços de repetição, estruturas de seleção e tratamento de exceções; os conceitos fundamentais da orientação a objetos como classes, objetos, atributos, métodos, encapsulamento, herança, polimorfismo, interfaces, modificadores de visibilidade; a criar interfaces gráficas do usuário baseadas em formulários; e a executar conexão e operações em bancos de dados utilizando a API JDBC.

Digitar os exemplos e fazer os exercícios do final de cada capítulo será essencial para um bom aprendizado.

Bons estudos!

UNINOVE – USO EXCLUSIVO PARA ALUNO

1. Preparando o ambiente para programar em Java

Para programar em Java é necessário instalar no computador a distribuição *Java Standard Edition* (Java SE), que inclui as bibliotecas de classes, os recursos para compilação e execução do código e a máquina virtual. É necessário também instalar um ambiente integrado para desenvolvimento (IDE – *Integrated Development Environment*), que integra os recursos da linguagem com um editor de código-fonte. Apesar de você poder digitar seu código-fonte em um editor simples como o bloco de notas do Windows, é comum o uso de um ambiente de desenvolvimento robusto (como o *NetBeans* ou o *Eclipse*) que integra os recursos da linguagem com os recursos do ambiente de desenvolvimento em uma única interface de programação.

Neste Capítulo você aprenderá a baixar e instalar os recursos necessários para criar um ambiente de desenvolvimento Java e a criar, compilar e executar sua primeira aplicação Java.

UNINOVE – USO EXCLUSIVO PARA ALUNO

1.1 Plataformas Java

Desde que surgiu, na década de 1990, por seu uma plataforma livre e muito poderosa, o Java ganhou recursos para o desenvolvimento de qualquer tipo de aplicação, desde para dispositivos móveis, computação embarcada (para microprocessadores embutidos em aparelhos eletrônicos, brinquedos, automóveis), para web, para *desktop* etc. Na medida em que foi crescendo, a plataforma foi dividida em plataformas menores, para domínios (ou áreas) específicos. Desta forma, você tem uma distribuição básica, com os recursos essenciais para todas as plataformas e para o desenvolvimento de aplicações *desktop*, chamada *Java Standard Edition* (Java SE), e plataformas específicas para desenvolvimento de aplicações que rodam em rede (Java EE), para dispositivos móveis e embarcados (Java ME) e para o desenvolvimento de interfaces gráficas robustas e aplicações multimídia (Java FX).

1.1.1 Java Standard Edition (SE)

O Java SE (*Standard Edition*) é a plataforma essencial do Java, contendo a *Java Runtime Environment* (JRE), um ambiente de tempo de execução composto por bibliotecas (APIs), compilador e pela *Java Virtual Machine* (JVM), recursos necessários para executar as aplicações Java.

1.1.1.1 Java Virtual Machine (JVM)

A máquina virtual Java (JVM) é responsável pela portabilidade das aplicações Java. Ela abstrai a execução do código compilado, chamado *bytecode*, da máquina real, ou seja, o código compilado é interpretado e executado na JVM, independente de características específicas do sistema operacional ou do hardware, permitindo a execução da aplicação em qualquer computador que tenha uma JVM instalada. Você encontra implementações da JVM para os diversos tipos de sistemas operacionais e hardwares existentes.

1.1.1.2 Java Development Kit (JDK)

Para que todos os recursos do Java SE possam ser instalados de forma integrada, deve-se baixar e instalar o *Java Development Kit* (JDK). É isso mesmo, não se instala o Java SE, mas sim o JDK, que integra os recursos do Java SE (compilador, recursos para execução da aplicação, bibliotecas de classe, JVM etc.).

NOTA O Java SE deve estar instalado no computador caso você queira utilizar as demais distribuições Java (EE, ME e FX).

1.1.1.2.1 Instalação da JDK

Para baixar o Java SE, que vem na forma de JDK, entre no *link* abaixo:

http://www.oracle.com/technetwork/java/javase/downloads/index.html

Na página que aparece, clique no botão "*Download*" referente à "*Java Platform (JDK) 8u20*". A Figura 1.1 mostra o botão que deve ser clicado.



Figura 1.1: *Link* para baixar a JDK

NOTA Há uma versão da JDK, nesta tela, que já instala o *NetBeans*. Não é interessante usar esta versão, pois ela instala apenas os recursos essenciais do *NetBeans*, deixando de fora alguns recursos que serão necessários para desenvolvimento com as demais distribuições Java e com linguagens como C++ e PHP.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Na tela que aparece você terá que marcar a opção "*Accept License Agreement*", para aceitar os termos da licença de uso gratuito. Na mesma tela, na parte de baixo, você escolhe a versão da JDK para seu sistema operacional. Para Windows, você pode baixar uma das versões apresentadas a seguir:

Para Windows 32 bits – jdk-8u20-windows-i586.exe – 61.08 MB
Para Windows 64 bits – jdk-8u20-windows-x64.exe – 173.08 MB

Após baixar a versão adequada, instale-a no computador de forma padrão.

1.1.1.2.2 Instalação da IDE NetBeans

Neste ponto você já conseguirá programar com o Bloco de Notas do Windows, mas não vai querer isso, pois os recursos do Bloco de Notas são básicos. Precisará então instalar um ambiente de desenvolvimento robusto.

O *NetBeans* é ideal, já que foi desenvolvido pela *Sun Microsystems*, a mesma empresa que criou o Java, hoje, de propriedade da *Oracle Corporation*. Se desejar, poderá utilizar o *Eclipse*, da IBM, que também é um ótimo ambiente de desenvolvimento.

Neste livro, vamos utilizar o *NetBeans*.

NOTA Para instalar o *NetBeans* ou o *Eclipse*, é necessário que antes, você tenha instalado o Java SE, através do JDK.

Para baixar o *NetBeans*, entre no *link*:

https://*NetBeans*.org/downloads/

Você perceberá que há várias colunas com um botão "*Download*" embaixo. Cada coluna mostra versões do *NetBeans* com um grupo de recursos diferente. É ideal você clicar no botão "*Download*" da última coluna da direita, a coluna que tem como título a palavra "Tudo", que traz a distribuição do *NetBeans* mais completa. Esta versão já instala o Java EE, o Java ME, o Java FX, os servidores de aplicação *Glassfish* e *Tomcat* para aplicações Java EE, e recursos para desenvolvimento em C++, PHP, HTML5 etc.

Baixe o *NetBeans* e instale no computador.

DICA Na instalação, na primeira tela que aparece, clique no botão "Personalizar" para selecionar o servidor web "*Tomcat*", muito bom para o desenvolvimento de sites com Java EE.

UNINOVE – USO EXCLUSIVO PARA ALUNO

1.1.2 JavaFX

O JavaFX é a distribuição Java que permite criar aplicações multimídia com interfaces gráficas bonitas para *desktop*, *browser* e telefones celulares. Quando você instala a versão completa do *NetBeans*, como descrito no tópico anterior, o Java FX já é instalado automaticamente.

1.1.3 Java Enterprise Edition (JEE)

O Java EE é a distribuição Java para o desenvolvimento de aplicações que rodam em rede, como sites ou aplicações que integram outras aplicações, centralizando-as em um servidor de aplicações. Sempre que você for desenvolver uma aplicação que necessita de um servidor de aplicações para rodar, essa aplicação será considerada uma aplicação Java EE.

Esta distribuição Java traz um conjunto de APIs e *frameworks* como *JavaServer Pages* (JSP), *Servlet*, *JavaServar Faces* (JSF), *Struts*, *Enterprise JavaBeans* (EJB) etc. Quando você instala a versão completa do *NetBeans*, como descrito anteriormente, o Java EE já é instalado automaticamente.

1.1.4 Java Micro Edition (JME)

Java ME é a distribuição Java que possibilita o desenvolvimento de software para sistemas embarcados, ou seja, para carros, eletrodomésticos, celulares, controle remotos, automóveis etc. O uso mais comum e popular é para o desenvolvimento de aplicativos e jogos para celulares, mas também pode ser utilizada para uma vasta quantidade de outros tipos de dispositivos. Quando você instala a versão completa do *NetBeans*, como descrito anteriormente, o Java ME já é instalado automaticamente.

1.2 Principais IDEs para programação em Java

Como vimos anteriormente, para desenvolver aplicações Java é aconselhável a instalação de um *Integrated Development Environment* (IDE), um ambiente integrado para desenvolvimento de software. Os IDEs mais conhecidos para Java são *NetBeans*, criado pela *Sun Microsystem,* e o *Eclipse*, criado pela IBM, porém, existem outros IDEs menos pesados, como *BlueJ*, *JCreator* e o *Gel*. A seguir você encontra o *link* onde poderá baixar cada um, se desejar.

NetBeans – https://NetBeans.org/downloads/
Eclipse – http://www.eclipse.org/
BlueJ – http://www.bluej.org/
JCreator – http://www.jcreator.com/
Gel – http://www.informaticon.com.br/downloads/gel.exe

UNINOVE – USO EXCLUSIVO PARA ALUNO

1.3 Criando um aplicativo Java

Após entrar no *NetBeans*, você precisará criar um novo projeto para programar em Java. Um projeto nada mais é do que uma pasta que você cria no disco para gravar sua aplicação e concentrar alguns recursos do Java necessários para executá-la.

1.3.1 Criando um projeto

Para criar um novo projeto (aplicação), siga os passos abaixo:

- Clique no menu "Arquivo" e na opção "Novo Projeto" (ou pressione Ctrl + Shift + N).
- Na divisão "Categorias", selecione "Java". Veja que nesta divisão você poderá selecionar outros tipos de aplicação, como HTML 5, C++, PHP, aplicação web etc.
- Na divisão "Projetos", selecione "Aplicação Java".
- Clique no botão "Próximo".
- No campo "Nome do Projeto", dê um nome para a pasta onde ficará sua aplicação.
- No campo "Localização do Projeto", clique no botão "Procurar" e selecione o local do disco onde você deseja criar a pasta do projeto.
- No campo "Criar Classe Principal", digite um nome para a classe (programa) que você deseja criar. Lembre-se que é uma boa prática iniciar o nome da classe com letra maiúscula. Uma classe principal é uma classe de *start* da aplicação, como, por exemplo, um menu principal de abertura da aplicação.

A Figura 1.2 mostra a tela do assistente do NetBeans para nomeação do projeto.

UNINOVE – USO EXCLUSIVO PARA ALUNO

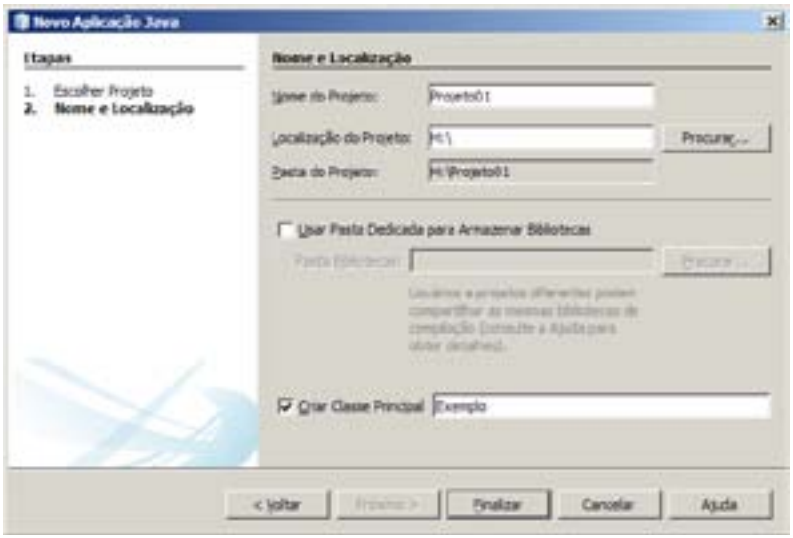


Figura 1.2: Tela do assistente do NetBeans para nomeação do projeto.

– Clique no botão “Finalizar”.

Pronto! Seu projeto está criado.

A Figura 1.3 mostra a área de trabalho do NetBeans.

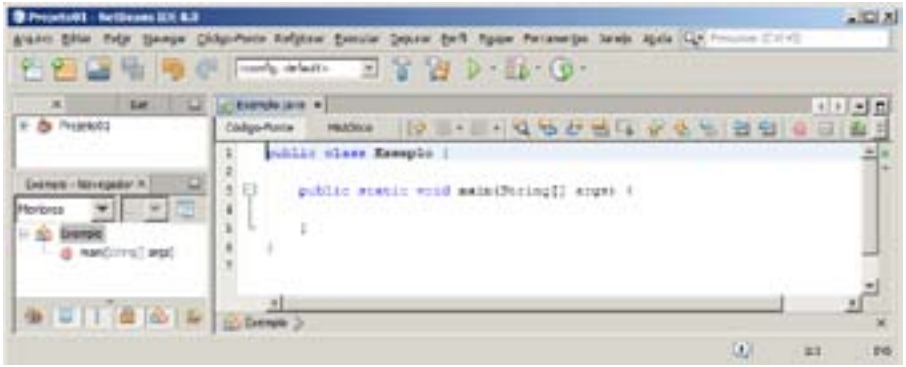


Figura 1.3: Área de trabalho do NetBeans.

UNINOVE – USO EXCLUSIVO PARA ALUNO

1.3.2 A classe principal

Toda aplicação Java SE precisa de uma classe principal, que normalmente é gerada quando se cria um novo projeto. A classe principal é uma classe de "*start*" da aplicação, ou seja, a porta de entrada da aplicação. Você pode ter 20 classes no projeto, mas apenas uma será a classe principal, a partir da qual você acessará as demais classes. O que define uma classe como principal é a instrução *public static void main(String[] args)*. Veja no código-fonte da classe *Exemplo.java* criada no nosso projeto:

```
public class Exemplo {
   public static void main(String[] args) {
      //Digite seu código de inicialização da aplicação aqui
   }
}
```

Toda classe que você cria em um projeto é iniciada pela instrução *public class* seguida pelo nome da classe. Todo o código-fonte da classe deve ficar no interior das chaves.

```
public class Exemplo {
  //Digite a programação da classe aqui
}
```

Como a classe *Exemplo.java* é a classe principal, então no interior das chaves há uma outra estrutura dizendo que trata-se de uma classe principal, a estrutura *public static void main(String[] args)*. Todo o conteúdo de *start* (inicialização) da classe normalmente fica no interior das chaves desta estrutura.

```
public static void main(String[] args) {
  //Digite seu código de inicialização da aplicação aqui
}
```

NOTA Para definir que o conteúdo de uma linha é um comentário, basta utilizar // à esquerda do conteúdo da linha.

Se precisa definir um bloco de linhas como comentário, utilize /* no início do bloco e */ no final.

Os comentários são descartados na execução do código e servem apenas para descrever comandos ou informações adicionais sobre os programas.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Digite o código-fonte a seguir na classe *Exemplo.java*, criada no nosso projeto.

```
public class Exemplo {
   public static void main(String[] args) {
      int n, cont, fat;
      n = 5;
      cont = 2;
      fat = 1;
      while (cont <= n) {
         fat = fat * cont;
         cont++;
      }
      System.out.print(fat);
   }
}
```

O código-fonte apresentado calcula o fatorial do número 5 ou seja, 5 fatoriais, exibindo na tela o resultado que será 120.

No decorrer do livro você vai entender melhor a linha *public static void main(String[] args)*, mas cabe aqui uma breve explicação.

Trata-se de um método chamado *main,* que é invocado automaticamente quando a aplicação é executada. Este método pode ser visto, a grosso modo, como uma espécie de função da programação estruturada, que é chamado automaticamente na inicialização da aplicação.

Quando você executar a aplicação, a JVM (máquina virtual) vai procurar a classe que possui um método chamado *main* para executar. Se nenhuma das classes da aplicação possuir um método chamado *main*, a aplicação não poderá ser executada. Como este método será invocado na máquina virtual de fora da aplicação, então ele precisa ser público (*public*) e estático (*static*) para ser acessado. A instrução *void* indica que quando o método for chamado, não retornará (devolverá) nenhum resultado. O parâmetro *String[] args* indica que se você quiser, na hora de executar a aplicação, poderá passar um ou mais parâmetros para a *array args*. Estes parâmetros, se recebidos, podem ser usados no interior do método *main* para definir características de execução para a aplicação com a finalidade de personalizá-la baseado nos parâmetros.

NOTA A passagem de parâmetros na inicialização de uma aplicação é muito comum em jogos que, quando iniciados com parâmetros, acrescentam poderes adicionais ao personagem do jogo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

1.3.3 COMPILANDO E EXECUTANDO

Se você estivesse usando o Bloco de Notas para criar a aplicação e não o *NetBeans*, teria que abrir o *prompt* de comando do Windows e digitar a instrução a seguir:

```
javac Exemplo.java
```

Esta instrução compila a classe *Exemplo.java*. *Javac* é o compilador Java. Este comando gera um arquivo compilado chamado *Exemplo.class* (conhecido como *bytecode*), que pode ser carregado e interpretado na JVM (máquina virtual Java). Para executar o arquivo compilado carregando-o na JVM, digita-se, no *prompt* de comando, a instrução a seguir:

```
java Exemplo
```

Um pouco complicado, não?

Estes comandos precisam ser digitados na pasta onde se encontra a classe *Exemplo.java* e as configurações de caminho para o compilador e outros recursos necessários precisam estar configuradas nas variáveis de ambiente do Windows.

Calma! Nada de pânico.

Quando você usa o *NetBeans*, tudo é integrado, de forma que você não terá nenhuma dificuldade para compilar e executar sua aplicação.

Ao terminar de digitar a classe principal, basta pressionar a tecla *F6* que a classe será compilada executada na JVM, apresentando o resultado na tela. Isso mesmo! Basta pressionar *F6*.

Se preferir, pode realizar o mesmo trabalho clicando na seta verde da barra de ferramentas que fica na parte superior da tela, abaixo do menu principal. A Figura 1.4 mostra a ferramenta que pode ser clicada para compilar e executar a aplicação no *NetBeans*.



Figura 1.4: Ferramenta para compilar e executar a aplicação no NetBeans.

Se preferir, você poderá também clicar na opção “Executar” do menu superior e, em seguida, na opção “Executar projeto”.

UNINOVE – USO EXCLUSIVO PARA ALUNO

1.4 Principais recursos do *NetBeans* IDE

Você vai notar que o IDE *NetBeans* é uma ferramenta extremamente útil na criação de aplicações Java pelos seguintes motivos:

– Integra a linguagem e seus recursos (bibliotecas, compilador, ferramenta de execução e máquina virtual) com o editor de código-fonte em uma única interface para o usuário.

– Permite compilar e executar o código com um único clique ou pressionamento de tecla.

– Possui depurador em tempo real, que autocompleta comandos que estão sendo digitados, marca comandos escritos de forma incorreta e apresenta sugestões de correção no momento da digitação.

– Importa automaticamente pacotes de classes necessários.

– Permite indentar o código-fonte automaticamente por meio do pressionamento da combinação de teclas *Alt* + *Shift* + *F* ou de um clique no menu "Código-Fonte" e na opção "Formatar".

– Permite depurar (ou debugar) o código-fonte mostrando passo a passo a execução dos comandos e os estados das variáveis na memória, auxiliando no entendimento do programa e na correção de erros.

– Mostra as diversas categorias de instruções como classes, instruções primitivas, conteúdos de variáveis etc. em cores diferentes, facilitando o entendimento do código-fonte.

1.4.1 Depurando o código

A depuração é um processo essencial para que você entenda o código e encontre erros de lógica difíceis de encontrar visualmente. É o processo de acompanhar a execução do código linha a linha, verificando o que ocorre na execução de cada instrução.

Para depurar um código-fonte, você deverá clicar sobre o número da linha onde deseja iniciar o acompanhamento da execução, como mostra a Figura 1.5 a seguir:

UNINOVE – USO EXCLUSIVO PARA ALUNO

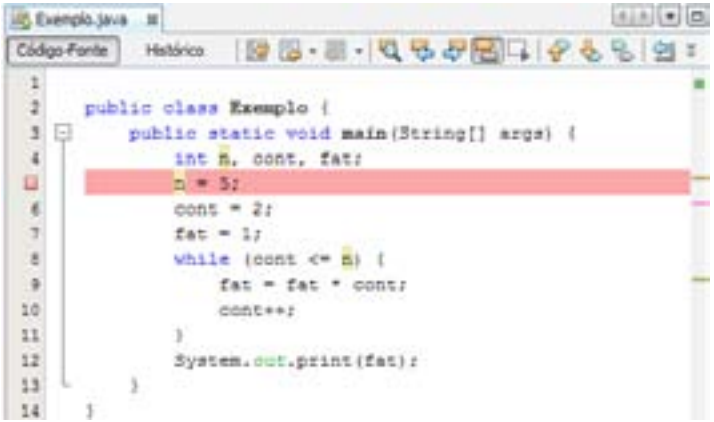


Figura 1.5: Marcação de linha para depuração de código no NetBeans.

Em seguida, deve pressionar a combinação de teclas *Ctrl + F5* ou clicar no menu "Depurar" e "Depurar projeto".

A linha que estava marcada com fundo vermelho ficará marcada com fundo verde, como mostra a Figura 1.6 a seguir:

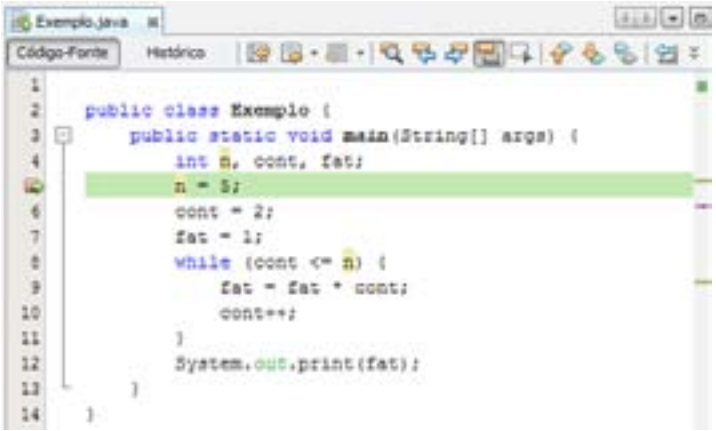


Figura 1.6: Depuração de código no NetBeans.

Agora, basta selecionar abaixo da área onde está o código-fonte a aba "Variáveis" e pressionar a tecla *F8* para executar cada linha do código. Cada vez que você pressionar a tecla *F8,* o depurador executará uma linha e você poderá acompanhar os valores armazenados nas variáveis, como mostra a Figura 1.7 a seguir.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 1.7: Acompanhamento das variáveis na memória durante a depuração no NetBeans.

Caso haja alguma linha de código que chame métodos em outras classes, pressione *F7* para entrar nestes métodos e depurá-los.

Caso queira parar a depuração, basta clicar no menu "Depurar" e em "Finalizar sessão do depurador".

1.5 Resumo

Com Java é possível desenvolver qualquer tipo de aplicação, estruturada ou orientada a objetos. Você pode desenvolver sistemas para *desktop*, sites, jogos para celulares, programas para microcontroladores embutidos em eletrodomésticos, carros etc.

Java, criada pela *Sun Microsystem*, hoje de propriedade da *Oracle Corporation*, cresceu tanto que teve que ser dividida em plataformas (distribuições) especializadas em domínios específicos. Assim, Java *Standard Edition* (SE) é a plataforma fundamental, com a linguagem Java e os recursos necessários para utilizar qualquer outra distribuição. Java *Enterprise Edition* (EE) é a distribuição para desenvolvimento de aplicações que executam em rede, em servidores de aplicações. Com Java EE você pode desenvolver sites ou outras

UNINOVE – USO EXCLUSIVO PARA ALUNO

aplicações corporativas que centralizam processamento em um servidor. *Java Micro Edition* (ME) é a distribuição para desenvolver aplicativos para celulares (incluindo jogos) e programas para serem executados em microcontroladores de eletrodomésticos, carros, controles remotos, sistemas de automação etc. JavaFX é a distribuição para desenvolvimento de aplicações multimídia, normalmente que necessitam de uma interface gráfica mais bonita.

Para utilizar Java EE, Java ME ou JavaFX é necessário ter o Java SE instalado. O Java SE é instalado na forma de *kit*, chamado *Java Development Kit* (JDK), baixado do site da *Oracle*. Após instalar o JDK que contém o Java SE com os recursos fundamentais para a utilização de Java, é necessário instalar um IDE (ambiente de desenvolvimento). Os principais IDEs são *Eclipse* e *NetBeans*, robustos, mas que exigem muitos recursos disponíveis no computador, como memória e espaço em disco. Caso você tenha recursos mais escassos, pode utilizar IDEs mais simples como *JCreator*, *Gel* ou *BlueJ*. O *NetBeans* é o IDE mais indicado para Java, por ter sido criado também pela *Sun Microsystems*, criadora da linguagem. Apesar do *Eclipse*, criado pela IBM, ser o IDE mais antigo e conhecido pela comunidade de desenvolvedores, o *NetBeans* é mais fácil de utilizar e mais integrado com os recursos da plataforma Java.

1.6 Exercícios

1) Quais são as plataformas para desenvolvimento de aplicações com Java?

2) Que plataforma (ou distribuição) Java é necessário utilizar para desenvolver programas embarcados?

3) O que é necessário instalar no computador para utilizar o Java EE, Java ME e JavaFX?

4) Um site feito em Java é considerado uma aplicação Java _____.

5) Quando se compila um programa Java, como é chamado o arquivo compilado, que possui a extensão .class?

6) Onde o arquivo compilado em Java é carregado e executado?

7) Quais são os principais IDEs de desenvolvimento para Java?

UNINOVE – USO EXCLUSIVO PARA ALUNO

8) O que é o processo de depuração ou debugação?

9) Que procedimento precisa ser adotado para indentar o código-fonte automaticamente no *NetBeans*?

10) O que é um projeto criado no *NetBeans*?

11) O que é a classe principal de uma aplicação Java?

12) Como se compila e executa a aplicação Java criada no *NetBeans*?

13) Porque se diz que as aplicações feitas em Java possuem portabilidade?

14) Para que serve o método *main* em uma classe principal?

UNINOVE – USO EXCLUSIVO PARA ALUNO

2. Programação Estruturada em Java

Você deve achar estranho um livro de Programação Orientada a Objeto iniciar com um Capítulo com o título "Programação Estruturada em Java". Pois é, mas não há nada de estranho nisso. A maioria das linguagens de programação orientadas a objeto, incluindo Java, utilizam um conjunto de estruturas derivadas da programação estruturada, como criação de variáveis de tipos primitivos, comparações, laços de repetição, *arrays* (vetores e matrizes) etc.

As aplicações orientadas a objetos normalmente utilizam programação estruturada quando criam variáveis, fazem comparações, executam laços de repetição e manipulam *arrays*. Você pode programar 100% de maneira estruturada utilizando uma linguagem orientada a objeto como Java, porém, estará subutilizando os recursos mais importantes da linguagem.

Este Capítulo apresentará:

– A diferença entre programação estruturada e programação orientada a objetos.
– As instruções fundamentais da linguagem Java para tipificação e criação de variáveis simples, *arrays* e constantes.
– As principais operações aritméticas, relacionais e lógicas.
– Estruturas de seleção e laços de repetição.

No final você encontrará um conjunto de exercícios para praticar e ajudar a fixar conceitos.

O objetivo principal deste Capítulo é apresentar a você as estruturas fundamentais de programação usadas tanto na programação estruturada como na programação orientada a objetos com Java.

2.1 Paradigmas de programação

Um paradigma de programação representa o conjunto de princípios ou estilo utilizado no desenvolvimento de um programa ou aplicação. Existem atualmente dois paradigmas de programação de destaque: Programação Estruturada e Programação Orientada a Objetos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.1.1 Características da Programação Estruturada

Se você já assistiu aulas de disciplinas que tratavam de algoritmos e lógica de programação, utilizou programação estruturada.

A programação estruturada consiste basicamente em dispor um conjunto de instruções sequenciais em um programa de computador para realizar uma tarefa. Sendo assim, um programa desenvolvido de forma estruturada normalmente inicia com a declaração de variáveis, entrada de dados ou atribuição de valores a estas variáveis, eventual realização de operações e comparações com estas variáveis, possíveis laços de repetição, quando necessário, e visualização de resultados. As operações, caso sejam acessadas mais de uma vez, podem ser colocadas em funções ou procedimentos para que possam ser chamadas diversas vezes do corpo da parte principal do programa.

Normalmente, toda a tarefa a que a aplicação se dispõe a realizar é feita dentro de um único programa e, no máximo, em outro programa auxiliar importado no programa principal.

Programação estruturada é uma forma mais antiga de se programar utilizando normalmente Linguagem C, Pascal, Cobol etc. No passado, todas as operações de um *software* podiam ser concentradas em um único programa ou em meia dúzia de programas utilizando normalmente sequências, estruturas de decisão e de repetição, o que não é mais realidade hoje, sendo um dos motivos pelo qual a programação orientada a objeto ganhou espaço.

Para que você possa entender melhor, veja a seguir um exemplo de aplicação estruturada construída com Java, onde são pedidos o nome e duas notas de um aluno. Após a digitação das notas, o programa calcula a média aritmética e mostra uma mensagem indicando se o aluno foi aprovado ou reprovado, considerando média para aprovação maior ou igual a 6.

Não se preocupe neste momento em entender os comandos, pois estes serão apresentados em detalhes no decorrer do livro. Concentre-se nas operações realizadas pelo programa para entender a diferença entre os dois paradigmas.

Considere a classe *Exemplo.java*, cujo código-fonte é apresentado a seguir:

```
import javax.swing.JOptionPane;
public class Exemplo {
 public static void main(String[] args) {
  String nome, mensagem;
  double nota1, nota2, media;
  nome = JOptionPane.showInputDialog("Digite o nome do aluno");
  nota1 = Double.parseDouble(JOptionPane.showInputDialog("Digite a primeira nota"));
  nota2 = Double.parseDouble(JOptionPane.showInputDialog("Digite a segunda nota"));
  media = (nota1 + nota2) / 2;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
 if (media >= 6) {
  mensagem = "aprovado";
 } else {
  mensagem = "reprovado";
 }
 System.out.print("O aluno " + nome + " foi " + mensagem + " com média " + media);
}
}
```

Observe que no programa acontece sequencialmente a declaração das variáveis, a entrada dos dados, o cálculo da média, a comparação e a exibição dos resultados esperados.

No tópico seguinte você verá como esta mesma tarefa será realizada de maneira orientada a objetos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.1.2 Características da Programação Orientada a Objetos

Na programação orientada a objetos você normalmente separa as informações de uma determinada categoria ou classe e as operações sobre estas informações em programas separados do programa principal. Assim, por exemplo, se for necessário aumentar o salário de um funcionário é necessário manipular as informações e operações deste funcionário em um arquivo de programa (classe) criado exclusivamente para esta finalidade. Cada categoria de informação e suas operações são tratadas em programas separados. Por exemplo, se for necessário manipular dados e operações de clientes, cria-se um programa chamado *Cliente.java*; se for necessário manipular informações e operações relacionadas a produtos, cria-se um programa denominado *Produto.java* e assim por diante.

Para que você entenda melhor como se programa orientado a objetos, o mesmo programa apresentado de maneira estruturada no tópico anterior é apresentado a seguir usando o paradigma orientado a objetos.

Serão necessários dois programas, um principal (de *start*), chamado *Exemplo.java*, e outro chamado *Aluno.java*.

O código-fonte do programa *Exemplo.java* é apresentado a seguir:

```
import javax.swing.JOptionPane;
public class Exemplo {
 public static void main(String[] args) {
  String nome, mensagem;
  double nota1, nota2, media;
  nome = JOptionPane.showInputDialog("Digite o nome do aluno");
  nota1 = Double.parseDouble(JOptionPane.showInputDialog("Digite a primeira nota"));
  nota2 = Double.parseDouble(JOptionPane.showInputDialog("Digite a segunda nota"));
```

```
    Aluno aluno = new Aluno (nome, nota1, nota2);
    media = aluno.calcularMedia();
    if (media >= 6) {
       mensagem = "aprovado";
    } else {
       mensagem = "reprovado";
    }
   System.out.print("O aluno " + aluno.getNome() + " foi " + mensagem + " com média " + media);
  }
}
```

O código-fonte do programa *Aluno.java* é apresentado a seguir:

```
public class Aluno {
    private String nome;
    private double nota1;
    private double nota2;
    public Aluno(String nome, double nota1, double nota2) {
       this.nome = nome;
       this.nota1 = nota1;
       this.nota2 = nota2;
    }
    public String getNome() {
       return nome;
    }
    public void setNome(String nome) {
       this.nome = nome;
    }
    public double getNota1() {
       return nota1;
    }
    public void setNota1(double nota1) {
       this.nota1 = nota1;
    }
    public double getNota2() {
       return nota2;
    }
    public void setNota2(double nota2) {
       this.nota2 = nota2;
    }
    public double calcularMedia() {
       return (nota1 + nota2) / 2;
    }
}
```

Analisando o tamanho dos programas você deve estar pensando que é bem mais fácil programar de maneira estruturada. Se pensarmos na simples tarefa que executamos, você tem razão. Agora pense no sistema de uma empresa

UNINOVE – USO EXCLUSIVO PARA ALUNO

onde é necessário manipular dados de funcionários, de clientes, de impostos etc. Neste caso, quando se separa as informações em classes de programas interligados de forma que cada um realize as operações relacionadas à classe na qual pertence, fica mais fácil para modelar, para manter e para reaproveitar operações que podem ser executadas diversas vezes.

NOTA Na programação orientada a objetos cada arquivo de programa é chamado de classe. Assim, por exemplo, se você for construir uma aplicação com 5 arquivos *.java*, terá 5 classes na aplicação.

2.2 Variáveis em Java

Existem linguagens de programação, como PHP e *JavaScript*, que são fracamente tipadas, ou seja, você não precisa especificar o tipo de dado que a variável irá armazenar. Basta atribuir um valor à variável que ela assumirá o tipo referente ao valor armazenado automaticamente. Veja o exemplo abaixo de criação de variáveis em PHP:

```
$idade=40;
$nome="Evandro";
```

Note que a variável *idade* recebeu um valor numérico inteiro, logo, é implicitamente do tipo *int*. Já a variável *nome* recebeu uma cadeia de caracteres, logo, é definida implicitamente como *String*.

Este não é o caso da linguagem Java. Nesta linguagem, considerada fortemente tipada, você precisa obrigatoriamente indicar o tipo da variável antes de atribuir um valor a ela. Por exemplo:

```
int idade=40;
String nome="Evandro";
```

Você pode ainda declarar as variáveis e depois atribuir valores a elas, da seguinte forma:

```
int idade;
String nome;
idade=40;
nome="Evandro";
```

Não há, em termos de execução, diferenças significativas entre declarar as variáveis e já atribuir valores a elas ou declarar as variáveis e atribuir os valores posteriormente.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.3 Tipos de dados em Java

Há duas categorias de tipos de dados em Java, os tipos primitivos e os tipos de referência. Isso significa que você poderá criar variáveis primitivas e variáveis por referência. Veremos a seguir como criar cada um destes tipos de variáveis.

2.3.1 Tipos de dados primitivos em Java

Os tipos de dados primitivos em Java são aqueles tipos essenciais, normalmente existentes na maioria das linguagens de programação, que não necessitam de nenhum programa ou classe especial para permitir sua criação. Estes tipos são mostrados na Tabela 2.1.

	Tipo primitivo	Espaço em *bytes* usado na memória	Exemplo
	char	2	char sexo= 'M';
Inteiros	byte	1	byte idade=55;
	short	2	short x=3456;
	int	4	int y= 678934;
	long	8	long cod=1756453;
Reais	float	4	float pi=3.1415F;
	double	8	double valor=34.56;
	boolean	1	boolean casado=true;

Tabela 2.1: Tipos de dados primitivos em Java.

2.3.1.1 Tipo char

O tipo *char* é destinado a armazenar um único caractere, colocado entre aspas simples. Utiliza dois *bytes* de memória para armazenamento. Isso significa que é possível armazenar 65536 caracteres diferentes e exclusivos em uma variável do tipo *char*, sabe por quê?

Porque dois *bytes* de memória equivalem a 16 *bits*. Cada um destes 16 *bits* pode ser representado pelos algarismos 0 ou 1. Logo, para sabermos quantos caracteres diferentes é possível armazenar em duas posições (ou *bytes*) de memória, basta calcular 2^{16}, onde 2 é a quantidade de valores possíveis para representar cada *bit* e 16 é o número de *bits* equivalentes a dois *bytes* de memória. São exemplos válidos de variáveis do tipo *char*:

```
char classe='B';
char sexo='M';
```

Note que é armazenado um único caractere entres aspas simples.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.3.1.2 TIPO BYTE

Um tipo *byte* é um tipo inteiro muito pequeno, que usa apenas um *byte* de memória para armazenamento. Como mostrado no tópico anterior, que explica o tipo *char*, em um *byte* de memória é possível armazenar apenas 256 valores diferentes e exclusivos, logo, um tipo *byte* pode armazenar apenas 256 valores inteiros exclusivos. Isso porque $2^8 = 256$.

Então a linha abaixo está correta?

```
byte a=240;
```

Não, não está correta. Apesar de você poder armazenar 256 valores inteiros diferentes e exclusivos em uma variável do tipo *byte*, estes valores podem ser negativos ou positivos. Logo, você pode armazenar valores de -128 até 127. Se você contar quantos valores inteiros exclusivos existem entre -128 e 127 (incluindo o zero), perceberá que existem 256 valores que equivalem ao total permitido em uma variável do tipo *byte*. Sendo assim, no exemplo acima, a variável poderia armazenar apenas valores de -128 até 127.

Vamos ver alguns exemplos corretos:

```
byte a=127;
byte b=-80;
byte c=110;
```

Em resumo, use variáveis do tipo *byte* apenas se for armazenar valores inteiros na faixa de -128 até 127. Se precisar armazenar valores maiores, utilize o tipo *short*.

2.3.1.3 TIPO SHORT

O tipo *short* armazena valores inteiros em dois *bytes* (posições) de memória. Em duas posições de memória (que equivalem a dois *bytes* ou 16 *bits*), é possível armazenar até 65536 valores inteiros diferentes e exclusivos. Isso porque $2^{16} = 65536$, onde 2 é o número de valores diferentes para cada *bit* e dois *bytes* de memória utilizados para o armazenamento equivalem a 16 *bits*. Como valores inteiros podem ser negativos ou positivos (duas possibilidades), dividindo-se 65536 por 2 obtêm-se 32768, que é a quantidade de números inteiros negativos e positivos (incluindo o zero), que podem ser armazenados em uma variável do tipo *short*. Sendo assim, variáveis do tipo *short* podem receber valores de -32768 até 32767. A quantidade de números inteiros deste intervalo é 65536. Vamos ver alguns exemplos:

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
short f=-20567;
short g=31789;
short h=32767;
short i=-32768;
```

Se precisar armazenar valores inteiros maiores, deve utilizar o tipo *int*.

2.3.1.4 Tipo int

O tipo *int* é o tipo padrão da linguagem Java para armazenamento de valores inteiros em 4 *bytes* (posições) de memória. Isso nos leva a concluir que é possível armazenar até 4294967296 valores inteiros exclusivos em uma variável do tipo *int*, pois $2^{32} = 4294967296$, onde 2 é a quantidade de valores que representam os *bits* (0 ou 1) e 32 é o número de *bits* referente aos 4 *bytes* de memória usados para armazenar valores do tipo *int*. Como valores inteiros podem ser negativos ou positivos, dividindo-se 4294967296 por 2 obtêm-se 2147483648, que é a quantidade de valores negativos e positivos que podem ser armazenados (incluindo o zero) em uma variável *int*. Desta forma, uma variável do tipo *int* pode armazenar valores de -2147483648 até 2147483647. Se precisar armazenar valores maiores, utilize o tipo *long*.

2.3.1.5 Tipo long

O tipo *long* armazena valores inteiros em 8 *bytes* (posições) de memória. Isso nos leva a concluir que para saber quantos valores inteiros diferentes e exclusivos podem ser armazenados em uma variável do tipo *long* basta calcular $2^{64} = 18446744073709551616$, onde 2 é a quantidade de valores que representam os *bits* (0 ou 1) e 64 é o número de *bits* referente aos 8 *bytes* de memória usados para armazenar valores do tipo *long*. Teoricamente, seguindo a linha de raciocínio utilizada nos tipos inteiros explicados anteriormente, é possível armazenar em uma variável do tipo *long* valores inteiros de -9223372036854775808 a 9223372036854775807.

2.3.1.6 Tipo float

O tipo *float* é utilizado para armazenar valores numéricos reais, ou seja, que normalmente necessitam de casas decimais. Utiliza 4 *bytes* (posições) de memória. Para saber a quantidade de valores reais exclusivos que é possível armazenar em uma variável do tipo *float*, basta calcular 2^{32}, que equivale a 4294967296. Não dá para determinar a faixa de valores permitida, por se tratar de armazenamento de números reais.

Vamos ver alguns exemplos:

```
float pi=3.1415F;
float valor=45.6789F;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Por não ser considerado o tipo padrão para números reais da linguagem Java, já que o tipo padrão é *double*, valores atribuídos a uma variável do tipo *float* devem terminar com a letra F.

2.3.1.7 TIPO DOUBLE

O tipo *double* é o padrão da linguagem Java para armazenamento de valores numéricos reais. Utiliza 8 *bytes* (posições) de memória para armazenamento do valor. Para saber a quantidade de valores reais exclusivos que é possível armazenar em uma variável do tipo *double*, basta calcular 2^{64}, que equivale a 18446744073709551616. Assim como no tipo *float*, não é possível determinar a faixa de valores permitida, por se tratar de armazenamento de números reais.

Vamos ver alguns exemplos:

```
double valorTotal=1765642.45;
double renda=10678.99;
```

2.3.1.8 TIPO BOOLEAN

O tipo *boolean* armazena apenas valores lógicos *true* ou *false* em um *byte* de memória. *True* equivale ao *bit* 1 e *false*, ao *bit* 0. Teoricamente seria necessário apenas um *bit* para armazenar um valor *boolean*, porém, como os computadores trabalham com posições de memória de 8 *bits*, um *byte* será necessário. Vamos ver alguns exemplos:

```
boolean estudante=true;
boolean fumante=false;
```

2.3.2 CONVERSÕES DE TIPOS DE DADOS PRIMITIVOS

Os dados numéricos primitivos em Java podem ser convertidos de um tipo para outro, de acordo com a necessidade. Há dois tipos de conversão: de um tipo de menor precisão para um tipo de maior precisão (*promotion*) e de um tipo de maior precisão para um tipo de menor precisão (*casting*). Estes tipos de conversão são apresentados a seguir.

2.3.2.1 CASTING

Se a conversão for de um tipo de dado de maior precisão para um tipo de dado de menor precisão, o processo de conversão é chamado *casting*.

Trabalharemos neste tópico conversões dos seguintes tipos de dados:

UNINOVE – USO EXCLUSIVO PARA ALUNO

byte
short
int
long
float
double

Casting ocorre sempre que se deseja converter de um tipo mais abaixo, na lista de tipos apresentada, para um tipo mais acima.

Por exemplo, para converter um valor do tipo *int* para *byte*, utiliza-se:

```
int a=10;
byte b=(byte) a;
```

Veja que a variável *int a* recebeu o valor inteiro 10 e este valor foi convertido para *byte* e armazenado na variável *byte b*.

Para fazer *casting*, na conversão, coloca-se o tipo de dado para o qual se quer converter entre parênteses.

Vejamos outros exemplos:

```
double b=10.0;
short c=(short) b;
```

O valor 10 da variável *double b* foi convertido e armazenado na variável *short c*.

```
double d=13.0;
float e=(float) d;
```

O valor 13 da variável *double d* foi convertido e armazenado na variável *float e*.

```
long f=1234;
int g=(int) f;
```

O valor 1234 da variável *long f* foi convertido e armazenado na variável *int g*.

2.3.2.2 Promotion

Quando a conversão ocorre de um tipo de menor precisão para um tipo de maior precisão, ou seja, de um tipo mais acima na lista de tipos apresentada, para um tipo mais abaixo, a conversão será direta, sem precisar de *casting*. Neste caso chamamos a operação de *promotion*.

Vejamos alguns exemplos:

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
int a = 1786;
long c = a;

byte v = 2;
double x=v;

short t = 290;
long u = t;

int k=1675;
float l=k;
```

Veja que as conversões, neste caso, acontecem sem necessidade de indicar entre parênteses o tipo para o qual se está convertendo o valor.

2.3.3 Tipos de dados por referência em Java

Como você aprendeu no tópico anterior, os tipos primitivos são aqueles mais simples, comuns à maioria das linguagens de programação, que não necessitam de nenhum recurso especial para serem declarados. Você deve ter notado que todos os tipos primitivos são escritos em letra minúscula.

Já os tipos de referência, são aqueles que necessitam de uma classe (ou arquivo de programa) para serem declarados e manipulados. Muitas destas classes já vêm instaladas com o Java (como a classe *String.java* e *Date.java*), mas muitas precisarão ser criadas. Quando se usa os tipos de referência, dizemos que o valor é armazenado no objeto da classe de referência. O objeto nada mais é, neste caso, de forma simplificada, uma variável que permite acessar as operações que podem ser aplicadas sobre o valor na classe de referência.

Você pode criar novos tipos de dados por referência, de acordo com suas necessidades, o que torna a linguagem bastante dinâmica no que se refere a manipulação de dados.

Os tipos de dados por referência mais comuns são *Date* e *String*.

NOTA Em linguagens orientadas a objetos, os arquivos de programas são denominados classes. Desta forma, no decorrer dos demais tópicos deste livro, utilizaremos a palavra classe como sinônimo de arquivo de programa.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.3.3.1 Tipo de dado String

O tipo de dado *String* utiliza a classe *String.java*, que é instalada como parte da plataforma Java, para declarar uma variável.

Há duas maneiras de declarar uma variável do tipo *String*:

```
String nome = "Evandro";
```

ou

```
String nome = new String("Evandro");
```

Como visto anteriormente, você ainda poderia separar declaração e atribuição em duas operações distintas, da seguinte forma:

```
String nome;
nome = "Evandro";
```

ou

```
String nome;
nome = new String("Evandro");
```

Em todos os casos, o valor entre aspas é passado para a classe *String.java* e lá cada caracter é manipulado como um tipo primitivo *char*. Basicamente um tipo *String* é uma espécie de *array* de valores do tipo *char*.

Note que a palavra *String*, na declaração do tipo da variável, é escrita em letra maiúscula, já que se refere à classe *String.java*, cujo nome se inicia por letra maiúscula.

2.3.3.2 Tipo de dado Date

Assim como o tipo *String*, *Date* é um tipo de dado de referência, ou seja, utiliza a classe *Date.java* para ser manipulado. Há diversas classes, além da *Date.java*, para manipular datas, mas neste tópico utilizaremos a forma mais simples, apresentada a seguir:

```
Date nascimento = new Date("07/08/1974");
```

Na variável *nascimento* será armazenado o valor "*Mon Jul 08 00:00:00 BRT 1974*". Há classes Java que permitem formatar a forma como a data é armazenada e impressa, mas estas classes não fazem parte do escopo deste tópico.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.3.3.3 OUTROS TIPOS DE DADOS DE REFERÊNCIA

Além dos tipos de referência *String* e *Date*, há diversos outros tipos de referência em Java, para o caso de você não querer utilizar os tipos de dados primitivos para manipulação de valores numéricos. Você pode utilizar as classes como *Byte.java*, *Short.java*, *Integer.java*, *Long.java*, *Float.java* e *Double.java* para manipular estes tipos, como apresentado nos exemplos a seguir.

```
Byte b = new Byte("120");
Short s=new Short("10");
Integer i = new Integer("10");
Long l = new Long("1678");
Double d = new Double("14.5");
Float f = new Float("13.14");
```

Observe que *b* é uma variável que faz referência à classe *Byte.java*, ou seja, é uma variável do tipo *Byte*. Dizemos, neste caso, que *b* é um objeto do tipo *Byte*. Deixaremos os detalhes em relação ao termo objeto para o próximo Capítulo deste livro.

Da mesma forma, *s* é um objeto do tipo *Short*, *i* um objeto do tipo *Integer*, *l* um objeto do tipo *Long,* e assim por diante.

Além das classes de referência para tipos de dados numéricos acima, caso precise armazenar e manipular valores acima dos limites permitidos nos tipos de dados numéricos primitivos, pode utilizar a classe *BigDecimal.java*, como mostra o exemplo a seguir:

```
BigDecimal g = new BigDecimal ("98876716752638949.78");
```

Veja que *g* é um objeto da classe *BigDecimal.java* que armazena um valor superior ao permitido nos tipos de dados primitivos.

2.3.3.4 TIPOS DE DADOS DE REFERÊNCIA PERSONALIZADOS

No tópico "*Programação Orientada a Objetos*", foram apresentadas duas classes, *Exemplo.java* e *Aluno.java*. Na classe *Exemplo.java*, você pôde encontrar a instrução abaixo:

```
Aluno aluno = new Aluno(nome, nota1, nota2);
```

Veja que *aluno* é uma variável que faz referência à classe *Aluno.java*, ou seja, é uma variável do tipo *Aluno*. Neste caso, dizemos que *aluno* é um objeto da classe *Aluno.java*. Esta classe contém recursos para manipular os dados do aluno e executar operações sobre estes dados.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Você pode criar classes para manipular outras categorias de informações e criar objetos (variáveis) de referência a estas classes. Veremos isso nos próximos Capítulos deste livro.

2.4 Tipos de operações realizadas em programas Java

Em toda linguagem de programação orientada a objetos ou não, você pode realizar uma série de operações sobre valores armazenados na memória. Os principais tipos de operações que podem ser realizadas sobre os dados são por meio de expressões aritméticas, relacionais e lógicas.

Este tópico apresenta um resumo destes tipos de expressões fundamentais na lógica de programação.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.4.1 Expressões aritméticas

As expressões aritméticas são cálculos matemáticos realizados normalmente com valores armazenados em variáveis. Por exemplo:

```
int a = 1;
int b = 3;
int c = a + b;
```

Note que a variável *c* armazenará o valor 4. Uma expressão aritmética utiliza operadores aritméticos (sinais) e operandos (valores). A Tabela 2.2 mostra alguns exemplos de expressões aritméticas.

Operador	Significado	Exemplos

*	Multiplicação	double quantidade=20;
		double valor = 109.6;
		double total = quantidade*valor;

/	Divisão	double salario=1987.7;
		double percentual=10.0;
		double aumento=salario*percentual/100;

+	Soma	int a=10;
		int b=20;
		int c=a + b;

-	Subtração	double salario=2456.76;
		double desconto=200.57;
		double salLiquido= salario – desconto;

Tabela 2.2: Operadores aritméticos.

NOTA Operações que envolvem raízes, exponenciação e outras operações matemáticas e de trigonometria, são realizadas utilizando a classe *Math.java*.

É importante lembrar as prioridades matemáticas dos cálculos envolvendo vários operadores aritméticos. Primeiro são realizados multiplicação e divisão e depois, soma e subtração. Por exemplo:

```
double salario=1876.56;
double percentualAumento=1876.56;
double salReajustado=salario + salario * percentualAumento / 100;
```

Note que primeiro é realizado o cálculo de *salario*percentualAumento/100* para depois o valor resultante ser somado ao conteúdo da variável *salario*.

Caso você queira alterar a prioridade natural das operações, forçando a realização de uma operação de menor prioridade antes de uma operação de maior prioridade, deve colocar a operação de menor prioridade entre parênteses. Veja o exemplo a seguir:

```
double nota1=9.5;
double nota2=6.0;
double media=(nota1+nota2)/2;
```

Veja que naturalmente seria realizada a divisão de *nota2* por 2 para depois o valor resultante ser somado ao conteúdo da variável *nota1*. Porém, o parêntese altera esta ordem natural, fazendo com que primeiro o conteúdo da variável *nota1* seja somado ao conteúdo da variável *nota2* para depois o valor resultante ser dividido por 2.

2.4.1.1 Operações de incremento e decremento

Dentro da categoria das expressões aritméticas estão as operações que incrementam e decrementam valores em uma variável. A Tabela 2.3 mostra exemplos destas operações que podem ser realizadas de modo extendido e reduzido. Considere a existência de uma variável *a* declarada da seguinte forma:

```
int a=10;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Modo extendido	Modo reduzido	Valor resultante na variável *a*	Operação
a=a+1;	a++;	11	incremento
a=a+5;	a+=5;	15	incremento
a=a*4;	a*=4;	40	incremento
a=a/2;	a/=2;	5	decremento
a=a-1;	a--;	9	decremento
a=a-6;	a-=6;	4	decremento

Tabela 2.3: Operações de incremento e decremento de variáveis.

Você pode utilizar tanto o modo extendido quanto o modo reduzido que terá o mesmo resultado.

Veja que o conteúdo da variável *a* é utilizado na operação e o resultado é armazenado na própria variável *a*. Quando o valor resultante for maior do que o valor inicial da variável, ocorreu uma operação incremento. Quando o valor resultante for menor, ocorreu um decremento de valor.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.4.2 Expressões relacionais

As expressões relacionais envolvem a comparação de dois valores por meio dos operadores relacionais mostrados na Tabela 2.4. Considere a existência de três variáveis declaradas com os valores seguintes:

```
int a=2;
int b=5;
int c=5;
```

Operador	Significado	Exemplo	Resultado
==	igual	a==b b==c	false true
>	maior	b>a	true
>=	maior ou igual	b>=c a>=c	true false
<	menor	b<c	false
<=	menor ou igual	a<=b	true
!=	diferente	a!=b b!=c	true false

Tabela 2.4: Operadores relacionais.

Note que o resultado de uma operação relacional será sempre um valor *boolean* (*true* ou *false*). Para indicar o que deve ocorrer se a avaliação da expressão for *true* ou *false* utiliza-se o comando *if* que veremos no decorrer neste Capítulo.

2.4.3 Expressões lógicas

Expressões lógicas são aquelas que envolvem dois valores *boolean* e resultam em um terceiro valor *boolean*. Utiliza operadores lógicos como *NÃO*, *OU* e *E*.

A Tabela 2.5 mostra os operadores lógicos em ordem de prioridade, de cima para baixo, e exemplos de expressões. Para entender os exemplos considere a existência de duas variáveis declaradas com os valores seguintes:

```
boolean a=true;
boolean b=false;
```

Operador	Significado	Exemplo	Resultado
!	Não	!a !b	false true
&&	E	a&&b a&&a	false true
||	OU	a||b b||b	true false

Tabela 2.5: Operadores lógicos.

NOTA Para saber mais sobre expressões lógicas, pesquise na Internet sobre o tema "Tabela Verdade". Trata-se de uma tabela que mostra os resultados das principais expressões lógicas.

2.4.4 Expressões mistas

As expressões mistas podem envolver expressões aritméticas, relacionais e/ou lógicas em uma única operação. Quando isto acontece, primeiro são resolvidas as expressões aritméticas, em seguida as operações relacionais e, por último, as expressões lógicas. Veja um exemplo, considerando que as variáveis *a*, *b* e *c* foram inicializadas com os valores abaixo.

```
int a=2;
int b=5;
int c=3;
boolean resultado=(a+b*a)>(b-c) && (b+c)<=(a+c);
```

Na resolução, primeiro são resolvidas as expressões aritméticas, resultando em:

```
12 > 2 && 8 <= 5
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Em seguida, são resolvidas as expressões relacionais, resultando em:

```
true && false
```

Por último, é resolvida a expressão lógica, resultando no valor *false* que é armazenado na variável *boolean resultado*.

2.5 Estruturas de seleção utilizadas em Java

As estruturas de seleção utilizadas em Java comparam valores e, de acordo com o resultado da comparação, realiza determinados procedimentos. Se o resultado for verdadeiro (*true*), executa um tipo de procedimento. Se for falso (*false*), executa outro.

As principais estruturas de seleção utilizadas em Java são *if..else* e *switch... case*. Estas estruturas são apresentadas a seguir.

NOTA Para testar as estruturas apresentadas no restante deste Capítulo, coloque os códigos-fonte apresentados no interior do método *main* de uma classe principal criada em um projeto Java no NetBeans. Para compilar e executar a classe, pressione a tecla F6. Se precisar de mais orientações sobre esse procedimento, reveja o Capítulo 1.

2.5.1 Estrutura de seleção if...else

Utiliza-se, assim como em outras linguagens de programação, a estrutura de seleção com a sintaxe seguinte:

```
if (expressão) {
 procedimento caso o resultado seja true
} else {
 procedimento caso o resultado seja false
}
```

Por exemplo:

```
int a=2;
int b=6;
if (a>=b){
  System.out.println("a contém um valor maior ou igual a b");
} else {
  System.out.println("a não contém um valor maior ou igual a b");
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Neste exemplo, como a variável *a* não contém um valor maior ou igual a *b*, será exibida a mensagem "*a não contém um valor maior ou igual a b*".

Observe que para o caso da expressão ser verdadeira, será executado o comando no interior da primeira chave. Caso seja falsa, será executado o comando no interior da segunda chave.

Para comparar se dois valores numéricos são iguais, utiliza-se a estrutura a seguir:

```
if (a==b) {
 procedimento caso o resultado seja true
} else {
 procedimento caso o resultado seja false
}
```

Para comparar se dois valores numéricos são diferentes, utiliza-se a estrutura a seguir:

```
if (a!=b) {
 procedimento caso o resultado seja true
} else {
 procedimento caso o resultado seja false
}
```

NOTA Quando apenas um comando tiver de ser executado para cada resultado da comparação (*true* ou *false*), as chaves são desnecessárias. Caso seja necessário executar mais de um comando para cada resultado, as chaves serão obrigatórias.

2.5.2 ESTRUTURA DE SELEÇÃO IF...ELSEIF...ELSE

Você pode encadear estruturas *if* caso necessite fazer diversas comparações. Para isso utiliza-se uma estrutura com a seguinte sintaxe:

```
if (expressão 1) {
        procedimento caso o resultado da expressão seja true
} else if (expressão 2) {
        procedimento caso o resultado da primeira expressão tenha sido false e da expressão
        atual tenha sido true.
} else if (expressão 3){
        procedimento caso o resultado da primeira e da segunda expressões tenham sido false
        e da expressão atual tenha sido true.
} else {
       procedimento caso o resultado de todas as expressões anteriores tenha sido false
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja que muitas expressões podem ser avaliadas utilizando esta estrutura.

Por exemplo:

```
if (nota1<nota2 && nota1<nota3) {
 media=(nota2+nota3)/2;
} else if (nota2<nota3){
 media=(nota1+nota3)/2;
} else {
 media=(nota1+nota2)/2;
}
System.out.print(media);
```

Na estrutura apresentada, dadas três notas armazenadas respectivamente nas variáveis *nota1*, *nota2* e *nota3*, será calculada a média aritmética das duas maiores.

2.5.3 Comparando valores String com if...else

Valores *String* podem ser comparados apenas quanto à igualdade e não utilizam o operador relacional == como para comparar valores numéricos. Desta forma, para verificar se uma variável nome contém o valor "Pedro" utiliza-se:

```
if (nome.equals("Pedro") {
 procedimento caso o resultado seja true
} else {
 procedimento caso o resultado seja false
}
```

Caso queira ignorar a forma como o valor está armazenado na variável, ou seja, independente da letra estar maiúscula ou minúscula, utiliza-se:

```
if (nome.equalsIgnoreCase("pedro") {
 procedimento caso o resultado seja true
} else {
 procedimento caso o resultado seja false
}
```

Neste exemplo, se o nome estiver armazenado, por exemplo, como "Pedro", "pedro", "PEDRO" ou "PeDrO", o resultado da expressão será *true*, pois o caixa da letra (alta ou baixa) será ignorado.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.5.4 ESTRUTURA DE SELEÇÃO SWITCH...CASE...DEFAULT

A estrutura de seleção *switch...case...default* em Java, assim como acontece em Linguagem C, aceita apenas tipos como *byte*, *short*, *int*, *long* etc. Esta estrutura realiza apenas verificação de igualdade, ou seja, verifica se uma variável contém um valor específico.

A sintaxe da estrutura de seleção *switch...case...default* é apresentada a seguir:

```
switch(variável) {
  case valor:
        operação a ser realizada se a variável contiver o valor especificado no case
    break;
  case valor:
        operação a ser realizada se a variável contiver o valor especificado no case
    break;
  case valor:
        operação a ser realizada se a variável contiver o valor especificado no case
    break;
  default:
        operação a ser realizada se a variável contiver o valor especificado no case
}
```

Exemplo:

```
import javax.swing.JOptionPane;
public class Principal {
  public static void main(String[] args) {
    int a = Integer.parseInt(JOptionPane.showInputDialog("Digite um valor inteiro
   para a variável a"));
   switch (a) {
     case 0:
       System.out.print("A variável recebeu o valor 0");
        break;
     case 1:
       System.out.print("A variável recebeu o valor 1");
        break;
     case 2:
       System.out.print("A variável recebeu o valor 2");
        break;
     default:
        System.out.print("A variável recebeu qualquer outro valor não testado
        acima");
    }
  }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Note que é solicitado ao usuário um valor inteiro que é armazenado na variável *a*. Em seguida, o conteúdo desta variável é testado pela instrução *switch*. Caso ela tenha recebido o valor 0, a mensagem "*A variável recebeu o valor 0*" será exibida. Caso tenha recebido o valor 1, a mensagem "*A variável recebeu o valor 1*" será exibida. Caso tenha recebido o valor 2, a mensagem "*A variável recebeu o valor 2*" será exibida. Caso tenha recebido qualquer outro valor ainda não testado, a mensagem "*A variável recebeu qualquer outro valor não testado acima*" será exibida.

Veja que após a operação realizada em cada teste *case* há um comando *break*. Este comando encerra os testes *case* quando a condição verdadeira for encontrada. Vale observar a instrução *default*, sempre após o último *case*, para o caso de alguma condição ter sido esquecida nos testes.

A estrutura *switch...case...default* também pode ser utilizada com valores *String*. Veja o exemplo a seguir:

```
import javax.swing.JOptionPane;
public class Principal {
  public static void main(String[] args) {
     String cidade;
     cidade = JOptionPane.showInputDialog("Digite o nome da cidade:");
     switch (cidade) {
       case "São Paulo":
          System.out.println("Cidade da garoa");
          break;
        case "Rio de Janeiro":
           System.out.println("Cidade maravilhosa");
           break;
        default:
           System.out.println("Cidade não reconhecida");
     }
  }
}
```

Note que se a cidade informada pelo usuário for "*São Paulo*", será exibida a mensagem "*Cidade da garoa*". Se for "Rio de Janeiro", será exibida a mensagem "*Cidade maravilhosa*". Se for qualquer outra cidade, será exibida a mensagem "*Cidade não reconhecida*".

2.6 Laçães de repetição em Java

Em Java há três estruturas para criar laços de repetição: *for*, *while* e *do...while*. Estas estruturas serão apresentadas nos tópicos a seguir. Vale relembrar que uma estrutura de repetição é utilizada quando uma mesma operação precisa ser executada várias vezes mudando apenas detalhes desta operação.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.6.1 LAÇOS DE REPETIÇÃO FOR

Os laços de repetição *for* operam sobre variáveis contendo valores numéricos inteiros. Todo laço de repetição possui uma estrutura semelhante contendo inicialização de uma variável que é um contador, o teste desta variável e o incremento ou decremento. Veja a sintaxe da estrutura de repetição *for*:

```
for (valor inicial da variável contador; teste; incremento/decremento){
    operação a ser repetida
}
```

Exemplo de laço com incremento:

```
for (int cont=1; cont<=10; cont++){
 System.out.print(cont);
}
```

Veja que a variável *cont* é inicializada com o valor 1 e testada a cada passagem pelo laço verificando se contém um valor menor ou igual a 10. Em cada passagem pelo laço, esta variável vai sendo incrementada em 1. Quando a variável receber o valor 11, que torna o teste *false*, o laço é finalizado.

Exemplo de laço com decremento:

```
for (int cont=10; cont>=1; cont--){
 System.out.print(cont);
}
```

Veja neste exemplo que a variável *cont* é inicializada com o valor 10 e decrementada em 1 a cada passagem pelo laço, onde o valor é testado e a operação do interior do laço é realizada enquanto a condição for verdadeira. Quando a variável receber o valor 0, o laço será finalizado.

Em alguns casos, a variável *cont* não é inicializada no interior do laço, mas antes dele, sem comprometer ou modificar a execução. Veja um exemplo:

```
int cont=10;
for (; cont>=1; cont--){
 System.out.print(cont);
}
```

Se preferir, pode apenas declarar a variável fora do laço. Veja o exemplo:

```
int cont;
for (cont=10; cont>=1; cont--){
 System.out.print(cont);
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

2.6.2 Laços de repetição while

A maior diferença entre o laço de repetição *for* e o laço *while* é que o laço *while* compara também valores que não são numéricos. Em todo laço haverá inicialização da variável do contador, o teste da variável e o incremento ou decremento. A diferença neste sentido é que estas operações ocorrem em linhas diferentes, e não na mesma linha como é o caso do laço *for*. Veja a sintaxe do laço *while*:

```
declaração e inicialização da variável contador
while(teste da variável contador) {
    operação a ser repetida
    incremento ou decremento da variável contador
}
```

Veja o exemplo abaixo com incremento da variável *cont*:

```
int cont=1;
while (cont<=10){
  System.out.print(cont);
  cont++;
}
```

A instrução de incremento foi escrita de forma reduzida (*cont++*), mas poderia ter sido escrita de forma extendida (*cont = cont +1*) que daria o mesmo resultado. As duas formas podem ser usadas em qualquer tipo de laço de repetição.

Veja agora um exemplo com decremento da variável contador:

```
int cont=10;
while (cont>=1){
  System.out.print(cont);
  cont-=2;
}
```

Veja que neste exemplo, a variável *cont* é decrementada em 2 a cada passagem do laço. A instrução *cont-=2* também poderia ter sido escrita como *cont = cont – 2*.

A estrutura de repetição *while* também permite testes com valores *String*. Veja o exemplo a seguir:

```
String resposta = "sim";
while (resposta.equalsIgnoreCase("sim")) {
  String nome = JOptionPane.showInputDialog("Nome:");
  String telefone = JOptionPane.showInputDialog("Telefone:");
  resposta = JOptionPane.showInputDialog("Deseja continuar[sim/não]?");
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja que o conteúdo do laço será repetido enquanto a resposta da pergunta "*Deseja continuar[sim/não]?*" for "sim". Este exemplo não poderia ser implementado utilizando o laço *for*.

2.6.3 LAÇOS DE REPETIÇÃO DO...WHILE

O laço de repetição *do...while* testa a condição após a operação do interior do laço ser realizada, e não antes, como acontece no laço *while*.

Isso faz com que o laço seja executado pelo menos uma vez, mesmo que a condição retorne *false* no primeiro teste. A sintaxe do comando é apresentada a seguir:

```
declaração e inicialização da variável contador
do {
    operação a ser repetida
    incremento ou decremento da variável contador
} while(teste da variável contador);
```

Veja um exemplo:

```
int cont=1;
do {
  System.out.print(cont);
  cont++;
} while (cont<=10);
```

É necessário ter cuidado ao utilizar esta estrutura de repetição, pois se a condição for falsa já no primeiro teste, mesmo assim o conteúdo do laço será executado. Veja um exemplo deste tipo:

```
int cont=11;
do {
  System.out.print(cont);
  cont++;
} while (cont<=10);
```

Neste caso, a variável *cont* foi inicializada com o valor 11, o que faz com que no primeiro teste o resultado seja *false*. Mesmo assim, a mensagem será exibida uma vez.

2.6.4 *ARRAYS* EM JAVA

Arrays são variáveis que permitem armazenar vários valores, ao contrário de variáveis simples. Há dois tipos de *arrays*, os vetores e as matrizes. Os vetores são chamados de *arrays* unidimensionais, e as matrizes, de *arrays* multidimensionais.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Quando se armazena informações em uma *array* Java, cada posição da *array* é identificada por um índice numérico inteiro, iniciado sempre em 0.

2.6.5 *Arrays* unidimensionais

Nas *arrays* unidimensionais os valores são armazenados em apenas uma dimensão, que normalmente é representada em linha. Cada posição da linha (coluna) recebe um índice numérico inteiro iniciado em 0.

Veja um exemplo de *array* unidimensional, declarada em Java da seguinte forma:

```
String nomes[] = new String[3];
```

nomes

0	1	2

Para armazenar valores nas posições da *array*, utiliza-se as seguintes instruções:

```
nomes[0]="Paulo";
nomes[1]="Pedro";
nomes[2]="Ana";
```

Veja como ficará:

nomes

Paulo	Pedro	Ana
0	1	2

Observe que na representação da *array* unidimensional os nomes são armazenados, em uma representação simbólica, em colunas de uma única linha. Cada coluna é identificada por um valor de índice iniciado por 0.

Note que na declaração da *array* você indicou que ela terá 3 posições, porém, os índices vão de 0 à 2.

Para declarar uma *array* unidimensional de outros tipos utilize sempre a sintaxe:

```
TipoDeDado  nomeDaArray[]=new TipoDeDado[quantidadeDePosições]
```

Por exemplo:

```
String clientes[] = new String[2];
int idades[] = new int[4];
double salarios[]=new double[3];
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Para armazenar valores nestas *arrays* utiliza-se:

```
clientes[0]= "João";
clientes[1]= "Paulo";

idades[0] = 19;
idades[1] = 40;
idades[2] = 29;
idades[3] = 41;

salarios[0]=1678.67;
salarios[1]=1390.00;
salarios[2]=2765.34;
```

Nos exemplos acima declaramos as *arrays* e depois armazenamos os valores em cada posição. Outra maneira de obter o mesmo resultado é já atribuir os valores na declaração da *array*. Ficaria da seguinte forma:

```
String clientes[]={"João","Paulo"};
int idades[]={19,40,29,41};
double salarios[]={1678.67,1390.00,2765.34};
```

O resultado pode ser representado da seguinte maneira:

clientes

João	Paulo
0	1

idades

19	40	29	41
0	1	2	3

salarios

1678.67	1390.00	2765.34
0	1	2

NOTA Na declaração de *array* os colchetes podem vir antes ou depois do nome da *array*. Isto significa, por exemplo, que tanto *int idades[] = new int[4]*; quanto *int[] idades = new int[4]*; são declarações corretas.

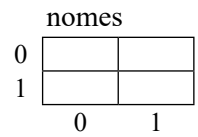
UNINOVE – USO EXCLUSIVO PARA ALUNO

2.6.6 *Arrays* multidimensionais

*Array*s multidimensionais podem ter muitas dimensões, mas normalmente são usadas apenas duas, visualizadas, simbolicamente, como linhas e colunas. As *array*s multidimensionais mais comuns são as de duas dimensões, chamadas bidimensionais, que serão explicadas neste tópico.

Veja o exemplo abaixo de uma *array* bidimensional declarada da seguinte forma:

```
String nomes[][]= new String[2][2];
```



Observe que trata-se de uma *array* de duas dimensões, onde tanto as linhas quanto as colunas possuem índices numéricos iniciados em 0.

Para armazenar valores nesta *array* utiliza-se as instruções:

```
nomes[0][0]= "Paulo";
nomes[0][1]= "Maria";
nomes[1][0]= "Ana";
nomes[1][1]= "Angela";
```

O resultado pode ser representado da seguinte forma:

	nomes	
0	Paulo	Maria
1	Ana	Angela
	0	1

Caso seja necessário visualizar o nome “Angela”, você pode digitar o comando a seguir:

```
System.out.print (nomes[1][1]);
```

Note que há necessidade de indicar a posição da *array* onde o nome se encontra.

Para armazenar em uma posição da *array* uma informação digitada pelo usuário, você pode utilizar a seguinte instrução:

```
nomes[1][1] = JOptionPane.showInputDialog("Digite um nome");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Detalhes sobre os comandos Java acima serão apresentados nos Capítulos seguintes deste livro.

Se desejar declarar a *array* e já atribuir valores às suas posições, você pode fazer da seguinte forma:

```
String [][] nomes = { { "Paulo", "Maria"}, { "Ana", "Angela"} };
```

2.6.7 PERCORRENDO *ARRAYS* COM LAÇOS DE REPETIÇÃO

É comum popular (preencher) *array*s utilizando laços de repetição.

Veja o exemplo utilizando o laço *for*:

```
String[] nomes = new String[3];
for (int indice=0;indice<3;indice++){
  nomes[indice] = JOptionPane.showInputDialog("Digite um nome:");
}
```

Veja o exemplo utilizando o laço *while*:

```
int indice=0;
while (indice<3){
  nomes[indice] = JOptionPane.showInputDialog("Digite um nome:");
  indice++;
}
```

O laço será executado 3 vezes, pois o contador (variável *indice*) vai de 0 à 2, incrementando de 1 em 1. Ao passar pelo laço a primeira vez, o nome digitado será armazenado na posição 0 da *array nomes*. Ao passar pelo laço pela segunda vez o nome digitado será armazenado na posição 1 da *array*. Ao passar pela terceira vez, o nome digitado será armazenado na posição 2 da *array*.

Para exibir os valores armazenados também podem ser utilizados laços de repetição, da seguinte forma:

```
for (int i=0;i<3;i++){
  System.out.println(nomes[i]);
}
```

OU

```
int i=0;
while (i<3){
  System.out.println(nomes[i]);
  i++;
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

A condição de teste do laço poderia ser modificada de *i<3* para *i<nomes.length*. A instrução *nomes.length* captura o tamanho da *array nomes*, que neste caso é 3.

No caso das *array*s bidimensionais são necessários dois laços de repetição encadeados, um para cada dimensão. Veja o exemplo:

```
String[][] nomes = new String[2][2];
for (int indiceLinha = 0; indiceLinha < 2; indiceLinha++) {
  for (int indiceColuna = 0; indiceColuna < 2; indiceColuna++) {
     nomes[indiceLinha][indiceColuna] = JOptionPane.showInputDialog("Digite
     um nome:");
  }
}
```

2.7 Variáveis e constantes em Java

Variáveis são espaços na memória identificados por um nome onde podem ser armazenadas informações sujeitas a mudanças no decorrer da execução do programa. Como os valores que podem ser armazenados podem ser caracteres, números inteiros, números reais, valores booleanos, datas etc, na declaração da variável é necessário, em Java, indicar o tipo de valor que a variável armazenará.

As variáveis podem ser simples, quando armazenam um único valor; *array*s, quando armazenam vários valores; e objetos, que discutiremos mais à frente neste livro.

Vale lembrar que quando se trabalha com variáveis você pode declarar a variável e posteriormente armazenar valores nela (o que é mais indicado) ou já declarar a variável atribuindo valores.

Para declarar uma variável simples e em seguida armazenar um valor nela, basta seguir a sintaxe abaixo:

```
TipoDaVariável nomeDaVariável;
nomeDaVariável=valor;
```

TipoDaVariável pode ser *char*, *byte*, *short*, *int*, *long*, *float*, *double*, *boolean*, *String*, *Date* etc. Veja alguns exemplos:

```
int codigo;
codigo=10;

String cidade;
cidade= "São Paulo";

char sexo;
sexo= 'F';
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Um valor armazenado em uma variável pode mudar no decorrer da execução do programa. Desta forma, se forem executados os comandos abaixo, o valor da variável *idade* irá variar.

```
int idade = 39;
idade = 80;
```

Veja que a variável *idade* recebeu o valor 39 e depois teve o valor modificado para 80.

Outra maneira de trabalhar com variáveis simples é declarando a variável e já atribuindo valor na mesma linha, em uma única instrução. Neste caso a sintaxe será:

```
TipoDaVariável nomeDaVariável=valor;
```

Veja o exemplo:

```
int codigo=1012;
String email= "aluno@uninove.edu.br";
```

2.7.1 Constantes

A diferença entre uma variável e uma constante é que o valor armazenado em uma variável pode mudar no decorrer da execução de um programa e o valor armazenado em uma constante, não.

Em Java, para declarar uma constante utiliza-se a sintaxe seguinte:

```
final TipoDaConstante NomeDaConstante = valor;
```

A palavra reservada *final* identifica que trata-se de uma constante e não de uma variável. Veja um exemplo:

```
final double PI = 3.1415;
```

Veja que *PI* é uma constante, o que significa que o valor armazenado não poderá ser modificado na aplicação.

2.7.2 Boas práticas para nomeação de variáveis e constantes em Java

Existem um conjunto de boas práticas utilizadas em Java para nomear variáveis e constantes. Boas práticas são padrões adotados pela comunidade de desenvolvedores e recomendadas pelo fabricante da linguagem. Não seguir

UNINOVE – USO EXCLUSIVO PARA ALUNO

estas práticas não causa normalmente erros de compilação ou de tempo de execução no programa, porém, é recomendável se você quer demostrar que possui bons conhecimentos da linguagem.

São boas práticas na nomeação de variáveis:

– Se o nome da variável for composto por apenas uma palavra, esta deverá ser escrita em letra minúscula. Exemplo: *int idade=50;*

– Se o nome da variável for composto por mais de uma palavra, a primeira deve ser escrita em letra minúscula e as demais, sem espaços em branco, devem ser escritas com letra inicial maiúscula. Exemplo: *String nomeClienteEspecial="Paulo Freitas";*

– Não utilizar acentos, espaços em branco e caracteres especiais do português nos nomes das variáveis;

– Não iniciar nomes de variáveis por números ou caracteres especiais.

Na nomeação de constantes, todo o nome da constante deve ser escrito em letra maiúscula. Exemplo: *final String CATEGORIA= "Técnico";*

2.8 Resumo

Este Capítulo apresentou um conjunto de conceitos e estruturas utilizados em Java que incluem:

– A diferença entre programação estruturada e orientada a objetos;

– Os tipos de dados para declarar variáveis (primitivos e por referência);

– As categorias de variáveis (simples, *array*s e objetos);

– Operações (aritméticas, relacionais e lógicas);

– Declaração e atribuição de dados às variáveis simples;

– Declaração, atribuição de valores e manipulação de dados em *array*s unidimensionais e bidimensionais;

– Declaração e atribuição de valores à constantes;

– Estruturas de seleção (*if..elseif..else* e *switch...case..default*);

– Estruturas de repetição (*for*, *while* e *do while*).

A programação de computadores pode ser realizada de formas diferentes, de acordo com um conjunto de princípios e recursos disponíveis na linguagem de programação. Estas formas são conhecidas como paradigmas de programação. Os dois principais paradigmas são Programação Estruturada e Programação Orientada a Objetos. Na programação estruturada os comandos são executados sequencialmente, normalmente em um único programa, que executa todas as operações necessárias. Na programação orientada a objetos as operações

UNINOVE – USO EXCLUSIVO PARA ALUNO

referentes à determinadas classes de informações são executadas em arquivos de programa separados do programa principal, denominados classes. Desta forma, as operações, por exemplo, relacionadas a funcionários, são executadas em uma classe *Funcionario.java*, as operações relacionadas a clientes, em uma classe denominada *Cliente.java* e assim por diante.

Tanto nas linguagens orientadas a objetos quanto nas linguagens estruturadas são utilizados recursos básicos de programação que permitem criar variáveis, constantes, estruturas de seleção e repetição e *array*s.

Variáveis são espaços na memória identificados por um nome que armazenam informações que podem mudar no decorrer da execução do programa. Para criar variáveis em Java você precisa declarar seu tipo, pois Java é uma linguagem fortemente tipada. Existem duas categorias de tipos de dados em Java: primitivos e por referência. Os tipos primitivos são os tipos mais simples e essenciais (*char*, *byte*, *short*, *int*, *long*, *float*, *double* e *boolean*). Os tipos por referência necessitam de classes prontas ou da criação de novas classes. Os principais tipos de referência são *Date*, *String*, *Byte*, *Short*, *Integer*, *Long*, *Double*, *Float* e *BigDecimal*. Você pode criar novos tipos de dados em Java, caso necessite.

Caso queira armazenar um valor que não vai mudar durante a execução do programa, utilize uma constante. Um valor armazenado em uma constante não poderá ser alterado.

Para fazer comparações em Java utilizam-se estruturas de seleção. As estruturas de seleção são *if..elseif...else* e *switch...case...default*.

Quando há necessidade de repetir tarefas em um programa utilizam-se os laços de repetição. Em Java existem três tipos de laços de repetição: *for*, *while* e *do...while*.

Além das variáveis simples, que armazenam apenas um valor, em Java é possível criar *array*s unidimensionais (vetores) e multidimensionais (matrizes) para armazenar diversos valores. Cada valor armazenado na *array* é identificado por um índice inteiro. Para armazenar e ler informações em *array*s normalmente são utilizados laços de repetição.

2.9 EXERCÍCIOS

1) Dadas as variáveis abaixo, resolva as expressões mistas A e B indicando o resultado de cada uma:

```
int a=2;
int b=3;
int c=5;
int d = 1;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

A) boolean e = (a+b*(c+d))<=(d*(b+c)/2)||(a+b-c)>(c+d);

B) boolean f = !((b*c/d)==(d*b+c)) && (c*d+a)!=(c+d);

2) O que o trecho a seguir exibirá?

```
int a = 2;
int b = 3;
int c = 5;
int d = 1;

if((b*c/d)==(d*b+c)&&(c*d+a)!=(c+d)||(a+b)<=(c-d)) {
  System.out.print("A");
} else if ((a+b*d)>(b+c/2)||(a-c)>(c+d)) {
  System.out.print("B");
} else if ((a*b/2)<(d*c+8)|| !((c+d*4)<=12)) {
  System.out.print("C");
} else {
  System.out.print("D");
}
```

3) Quais as formas de criar a *array* bidimensional representada abaixo?

nomes

0	Paulo	Maria	Pedro	Marcos
1	Ana	Marcelo	André	Mauro
2	Janaina	José	Renato	Célia
	0	1	2	3

4) Qual a principal diferença entre a programação estruturada e a programação orientada a objeto?

5) É correto afirmar que estruturas fundamentais da programação estruturada, como laços de repetição, estruturas de seleção e manipulação de *array*s não fazem parte da programação orientada a objeto? Justifique sua resposta.

6) Como se escreve em Java os operadores lógicos *OU*, *E* e *NÃO*?

7) Como se escreve em Java o operador relacional *DIFERENTE*?

UNINOVE – USO EXCLUSIVO PARA ALUNO

8) Quando em uma expressão mista há expressão aritmética, lógica e relacional, qual a ordem de resolução?

9) Como se escreve de forma reduzida as operações abaixo:

```
a=a-8;
b=b*5;
c=c+1;
d=d-1;
e=e/8;
```

10) Dada a variável *b* a seguir, converta seu valor para a variável *c* do tipo *byte* e para a variável *d* do tipo *short*.

```
double b=12;
```

11) Dada a variável *g* a seguir, converta seu valor para a variável *h* do tipo *float* e *i* do tipo *long*.

```
int g=6;
```

12) Crie uma *array* unidimensional e armazene nela o salário de 3 funcionários.

13) Crie uma *array* bidimensional para armazenar os dados representados abaixo.

idades

0	60	78
1	23	50
	0	1

UNINOVE – USO EXCLUSIVO PARA ALUNO

14) Dado o algoritmo a seguir, na forma de pseudocódigo, escreva o programa Java correspondente.

```
Algoritmo fatorial
var a, n, cont: inteiro
início
        n←5;
        a← 1
        cont←1
        enquanto (cont<=n) faça
                a ← a* cont
                cont ← cont +1
        fim enquanto
        imprima (a)
fim
```

15) Dado o algoritmo a seguir, na forma de pseudocódigo, escreva o programa Java correspondente.

```
Algoritmo soma
var  a, n, cont : inteiro
início
     n ← 10
    a ← 0
    para cont=1 até  n  passo 1
        a ← a + cont
    fim_para
    imprima(a)
fim
```

16) Dado o algoritmo a seguir, na forma de pseudocódigo, escreva o programa Java correspondente.

```
Algoritmo números
var n1,n2: inteiro
início
 leia (n1,n2)
 se n1>n2
   então imprima ("o número", n1, "é maior que", n2)
   senão se n1=n2
          então imprima ("os números são iguais")
          senão imprima ("o número", n2, "é maior que", n1)
        fim_se
   fim_se
fim
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

3. Orientação a objetos – conceitos básicos

A orientação a objetos foi criada observando-se conceitos aplicados no dia-a-dia das pessoas. Normalmente categorizamos, classificamos, relacionamos e realizamos operações sobre um incontável número de informações. Nosso cérebro está programado para classificar tudo baseado em nosso conjunto de conhecimentos e valores.

Na programação orientada a objetos tudo começa com a categorização ou classificação de informações agrupando-as em classes. As informações de uma determinada classe ou categoria estão sujeitas a um conjunto de operações criadas especificamente para atuar sobre elas. Vamos a um exemplo prático. Em uma empresa podemos classificar as pessoas como clientes, funcionários e, em alguns casos, fornecedores. Cada uma destas classes de pessoas possui um conjunto de informações que as identifica e/ou caracteriza. Estas informações são seus atributos.

Vamos focar, por exemplo, na classe dos funcionários. O funcionário possui matrícula, nome, cargo, salário etc. Sobre estas informações, podem ser realizadas um conjunto de operações. Por exemplo, sobre o salário pode ser realizada uma operação de reajuste, uma operação de desconto de imposto etc. Estas operações, na orientação a objetos, são chamadas de métodos. Deste exemplo podemos obter alguns conceitos essenciais da orientação a objetos, que são:

Classe: Funcionário

Atributos: matrícula, nome, cargo, salário.

Métodos: reajuste, desconto de imposto.

Não só pessoas podem ser categorizadas ou classificadas em uma empresa. Por exemplo, podemos identificar a classe dos produtos que a empresa vende. Estes produtos possuem código, nome, medida, cor, preço etc. Sobre estas informações do produto podem ser realizadas operações. Por exemplo, sobre o preço, podem ser realizadas operações de reajuste, desconto etc. Assim, podemos tirar deste exemplo um conjunto de conceitos da orientação a objetos que são:

UNINOVE – USO EXCLUSIVO PARA ALUNO

Classe: Produto

Atributos: código, nome, medida, cor, preço.

Métodos: reajuste, desconto.

Muito bem, conceitualmente é simples. Mas como transformar isso em programa de computador?

É isso que veremos no próximo tópico deste Capítulo.

3.1 Classes

Tirando a parte conceitual e abstrata, uma classe nada mais é do que um arquivo de programa Java que precisa possuir uma estrutura que atenda aos padrões da linguagem. Elas possuem identificação, a declaração dos atributos ou informações que manipulam, um ou mais construtores e operações que podem ser realizadas sobre estes atributos. Cada uma destas características das classes será apresentada a seguir.

3.1.1 Declaração da classe

Uma classe precisa ser identificada com o mesmo nome do arquivo criado no projeto para a classe. Assim, se o arquivo criado for *Funcionario.java*, a classe deve se chamar *Funcionario*. Neste caso, a declaração da classe ficaria da seguinte forma:

```
public class Funcionario{
 // Conteúdo da classe
}
```

NOTA Veja que a palavra *Funcionario* não foi acentuada, pois não é uma boa prática acentuar nomes de classes, já que a linguagem e seus padrões, apesar de aceitar acentos, foram criados em língua inglesa, que não utiliza acentuação.

3.1.2 Declaração dos atributos

Após declarar a classe você pode declarar os atributos, caso haja necessidade. Um atributo nada mais é do que uma variável especial que vai permitir manipular uma informação daquela classe. Desta forma, a declaração de atributos da classe *Funcionario* poderia ser:

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
public class Funcionario {
   private int matricula;
   private String nome;
   private String cargo;
   private double salario;
}
```

NOTA A palavra *private*, que define que o atributo só poderá ser acessado de dentro da própria classe, será explicada mais a frente, no próximo Capítulo, no tópico sobre modificadores de visibilidade.

3.1.3 Construtor

Construtor é uma estrutura, semelhante a um método ou função, que permite criar (ou instanciar) um objeto (variável) da classe, já incluindo valores a serem armazenados no objeto ou não.

Segundo Horstmann (2009, p. 369), "construtores contêm instruções para inicializar os atributos de um objeto".

Se o construtor permitir o recebimento de parâmetros, então você poderá criar um objeto (variável) da classe e já armazenar as informações nos atributos desta classe em uma mesma operação. Veja um exemplo:

```
public Funcionario(int matricula, String nome, String cargo, double salario) {
      this.matricula = matricula;
      this.nome = nome;
      this.cargo = cargo;
      this.salario = salario;
}
```

Se o construtor não permitir o recebimento de parâmetros, você terá que declarar o objeto da classe e, posteriormente, através dos métodos iniciados pela palavra *set* (*setter*), armazenar os valores nos atributos desta classe. Veja um exemplo de construtor que não recebe parâmetros:

```
public Funcionario() {

}
```

Você deve observar que o construtor, obrigatoriamente, deve ter o mesmo nome da classe. Pode haver diversos construtores em uma classe, desde que eles recebam parâmetros diferentes. O uso de vários construtores em uma classe é chamado de sobrecarga de construtor.

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA Se você não escrever um construtor na classe, implicitamente o compilador vai entender que existe um construtor vazio que não recebe parâmetros. Neste caso, apesar de parecer que a classe não tem construtor, há sim um construtor implícito.

Apesar da semelhança com método, o construtor não é considerado um método, pois não possui tipo de retorno, nem mesmo *void*. Os métodos necessitam da declaração de um tipo de retorno, ao contrário dos construtores.

3.1.4 Métodos

Métodos são semelhantes às funções utilizadas na programação estruturada, porém, ao contrário das funções, estarão sempre associados aos objetos da classe e executarão operações sobre os valores manipulados pela classe. São trechos de código em uma classe que podem ser chamados uma ou mais vezes para realizar procedimentos específicos sobre os atributos da classe.

Os métodos são recursos excelentes para viabilizar o reaproveitamento de código, uma vez que precisam ser digitados uma vez, mas podem ser chamados quantas vezes forem necessárias.

Segundo Horstmann (2009. p. 63), "um método é uma sequência de instruções que acessam os dados internos de um objeto".

Em uma classe Java normalmente existem três categorias de métodos: os métodos *setter* (que incluem valores nos atributos de um objeto); os métodos *getter* (que obtêm valores armazenados nos atributos de um objeto); e métodos de uso geral, que executam operações personalizadas e diversificadas sobre os atributos da classe.

3.1.4.1 Sintaxe dos métodos

Um método tem a seguinte sintaxe:

```
modificadorVisibilidade tipoRetorno nomeMétodo (parâmetros){
  corpo do método
}
```

Onde:

- **ModificadorVisibilidade**: determina se o método terá acesso público, privado, protegido ou padrão. Veremos detalhes sobre esses modificadores no próximo capítulo.
- **TipoRetorno**: determina que tipo de valor o método irá retornar. Um método pode retornar qualquer tipo de valor, seja simples, *array*, objeto etc. Caso o método não retorne valor deve ser escrito *void* neste local.

UNINOVE – USO EXCLUSIVO PARA ALUNO

- **NomeMetodo**: é o nome do método, seguindo as boas práticas de nomeação, as mesmas utilizadas na nomeação de variáveis.
- **Parâmetros**: são valores que o método pode receber para cumprir seu papel. Um método pode receber qualquer tipo de valor como variáveis simples, *array*, objetos etc.
- **Corpo do método**: sempre entre chaves, o corpo do método é o local onde as operações as quais o método se propõe são realizadas. Caso exista um tipo de retorno definido na declaração do método, obrigatoriamente no corpo do método deve existir uma instrução *return*.

Vamos analisar alguns exemplos:

```
public void imprimirNome(String nome){
 System.out.print(nome);
}
```

Veja que este método recebe um nome (do tipo *String*), exibe este nome na tela e não retorna nenhum valor (*void*).

```
public double calcularMedia(double nota1, double nota2){
   return (nota1+nota2)/2;
}
```

Veja que este método recebe duas notas do tipo *double* e retorna o resultado do cálculo da média entre elas, também do tipo *double*.

```
public double calcularMedia(){
   return (8.0+5.0)/2;
}
```

Este método não recebe nada como parâmetro, mas devolve o resultado de um cálculo que é um valor do tipo *double*.

```
public String verificarStatusAluno(double media){
   if (media>=6){
   return "Aprovado";
   } else{
     return "Reprovado";
   }
}
```

Este método recebe a média e retorna “Aprovado”, se for maior ou igual a 6, ou “Reprovado”, se a média for menor que 6.

UNINOVE – USO EXCLUSIVO PARA ALUNO

A linha de declaração de um método com modificador de visibilidade, tipo de retorno, nome do método e parâmetros é chamada de assinatura do método.

No método acima, por exemplo, a assinatura do método é:

```
public String verificarStatusAluno(double media)
```

Quando se está desenvolvendo uma aplicação orientada a objetos, a assinatura do método é a única coisa que é necessário saber para se acessar um método.

3.1.4.2 Métodos getter e setter

Há duas operações padrão que podem ser realizadas sobre os atributos da classe por meio de um objeto desta classe: armazenar valor e obter o valor armazenado.

Métodos destinados a armazenar valores nos atributos são conhecidos como métodos *setter* e aqueles que obtêm os valores armazenados, são conhecidos como métodos *getter*.

3.1.4.2.1 Métodos setter

Para armazenar valores nos atributos da classe, é necessário utilizar métodos normalmente iniciados pela palavra *set*. Estes métodos são conhecidos como métodos *setter*. Por exemplo, para armazenar um nome no atributo nome da classe pode ser utilizado o seguinte método:

```
public void setNome(String nome) {
  this.nome = nome;
}
```

Veja que este método, chamado *setNome*, recebe um *nome* como parâmetro e transfere este nome para o atributo *nome* da classe (*this.nome*). Há duas variáveis *nome* diferentes: a variável local, que recebe o nome como parâmetro; e o atributo *nome*, privado da classe, antecedido pela instrução *this*, que recebe o nome que chegou como parâmetro.

NOTA Normalmente, para cada atributo da classe haverá um método iniciado pela palavra *set* e sucedido pelo nome do atributo com letra inicial maiúscula. Por exemplo, se houver um atributo cargo, haverá um método *setCargo*; se houver um atributo telefone, haverá um método *setTelefone* e assim por diante.

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.1.4.2.2 Métodos getter

Para obter valores armazenados nos atributos de uma classe, utilizam-se métodos *getter*. Um método *getter* é iniciado pela palavra *get* e sucedido pelo nome do método com letra inicial maiúscula. Quando chamado, um método *getter* retorna o valor armazenado no atributo referente a este método. Veja o exemplo:

```
public String getNome() {
      return this.nome;
}
```

NOTA Normalmente, para cada atributo da classe haverá um método iniciado pela palavra *get* e sucedido pelo nome do atributo com letra inicial maiúscula. Por exemplo, se houver um atributo *cargo*, haverá um método *getCargo*; se houver um atributo *telefone*, haverá um método *getTelefone* e assim por diante.

3.1.4.3 Outros métodos

Além dos métodos *setter* e *getter*, você poderá criar tantos métodos quanto precisar para executar operações com os valores manipulados pela classe. Normalmente, são comuns métodos para executar operações matemáticas com valores contidos nos atributos, mas quaisquer outras operações poderão ser realizadas.

3.1.5 Exemplo de classe com atributos, construtor e métodos

Este tópico apresenta uma classe padrão para manipular dados de funcionários, contendo declaração da classe, declaração de atributos; construtores, métodos *setter* e *getter* e um método para aumentar o salário do funcionário.

Para digitar esta classe, entre no *NetBeans* e crie um novo projeto Java, seguindo as orientações a seguir:

– Clique no menu “Arquivo” e na opção “Novo Projeto” (ou pressione *Ctrl + Shift + N*).
– Na divisão “Categorias”, selecione “Java”.
– Na divisão “Projetos”, selecione “Aplicação Java”.
– Clique no botão “Próximo”.

UNINOVE – USO EXCLUSIVO PARA ALUNO

– No campo “Nome do Projeto”, dê um nome para a pasta onde ficará sua aplicação. Pode ser qualquer nome válido como nome de pasta.
– No campo “Localização do Projeto”, clique no botão “Procurar” e selecione o local do disco onde você deseja criar a pasta do projeto.
– Desmarque a opção “Criar Classe Principal”, pois a classe que vamos criar será uma classe de apoio, não a classe de *start* da aplicação. A classe principal será criada mais a frente neste capítulo.
– Clique no botão “Finalizar”.

Agora que o projeto está criado, vamos criar a classe *Funcionario.java*. Para criar esta classe, siga os procedimentos a seguir:

– Clique com o botão direito do mouse no nome da pasta do projeto, localizado na divisão “Projetos” do canto superior esquerdo da tela, clique na opção “Novo” e, em seguida, na opção “Classe Java”.
– No campo “Nome da Classe”, digite *Funcionario* e clique no botão “Finalizar”.

NOTA Não é uma boa prática acentuar nomes de classes e variáveis, por esse motivo, o nome da classe *Funcionario* não deve ser acentuado.

Digite o código-fonte abaixo, da classe *Funcionario.java*.

```
1   public class Funcionario {
2       private int matricula;
3       private String nome;
4       private String cargo;
5       private double salario;
6
7       public Funcionario(int matricula, String nome,String cargo,double salario) {
8           this.matricula = matricula;
9           this.nome = nome;
10          this.cargo = cargo;
11          this.salario = salario;
12      }
13
14      public Funcionario() {
15      }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
16
17        public int getMatricula() {
18            return matricula;
19        }
20
21        public void setMatricula(int matricula) {
22            this.matricula = matricula;
23        }
24
25        public String getNome() {
26            return nome;
27        }
28
29        public void setNome(String nome) {
30            this.nome = nome;
31        }
32
33        public String getCargo() {
34            return cargo;
35        }
36
37        public void setCargo(String cargo) {
38            this.cargo = cargo;
39        }
40
41        public double getSalario() {
42            return salario;
43        }
44
45        public void setSalario(double salario) {
46            this.salario = salario;
47        }
48
49        public void ajustarSalario(double percentual) {
50            this.salario = this.salario + this.salario * percentual / 100;
51        }
52    }
```

A linha 1 declara a classe com nome *Funcionario*. Veja que a chave aberta nesta linha é fechada apenas na linha 52.

Da linha 2 a 5 são declarados os atributos da classe, ou seja, as informações que se deseja manipular. Estes atributos não são nada mais que variáveis privadas da classe, ou seja, variáveis que só podem ser acessadas do interior desta classe.

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas de 7 a 12 compõem o construtor da classe que recebe como parâmetros as informações a serem manipuladas nos atributos. Já as linhas 14 e 15 compõem outro construtor, só que este não recebe parâmetros.

As linhas de 17 a 47 compõem os métodos (operações) para inserir as informações nos atributos da classe e para obter as informações que foram inseridas nestes atributos. Métodos cujos nomes são iniciados pela palavra *set* inserem valores nos atributos. Métodos cujos nomes são iniciados pela palavra *get* obtêm os valores inseridos nos atributos.

As linhas de 49 a 51 compõem um método para aumentar o valor do salário no percentual recebido como parâmetro.

Calma!

Não adianta compilar e executar esta classe que nada vai acontecer. Trata-se de uma classe de apoio, que será acessada de outras classes, sempre que se desejar manipular dados de funcionários. Para testarmos a classe *Funcionario*, vamos criar nos tópicos seguintes uma classe principal (que possui o método *main*) no projeto por meio da qual manipularemos os dados de um funcionário.

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.1.6 Diagrama UML para representar uma classe

Para modelar sistemas desenvolvidos utilizando linguagens orientadas a objetos é comum e popular a utilização da linguagem *Unified Modeling Language* (UML). A UML apresenta a notação para um conjunto de diagramas que podem representar a maioria dos aspectos relevantes para o desenvolvimento do sistema. Um dos diagramas que a UML permite desenvolver é o diagrama de classes que nada mais é do que um desenho da classe usando uma notação específica. Veja a seguir o diagrama de classe da classe *Funcionario.java* desenvolvida nos tópicos anteriores.

Funcionario
- matricula: int - nome: String - cargo: String - salario: double
+ setMatricula (matricula):void + setNome (nome): void + setCargo (cargo): void + setSalario(salario): void + getMatricula(): int + getNome(): String + getCargo(): String + getSalario(): double + ajustarSalario(double percentual): void

O diagrama de classes é dividido em três células contendo, de cima para baixo, o nome da classe, os atributos e os métodos. O sinal de menos (-) indica que os atributos são privados (*private*) e o sinal de mais (+), indica que os métodos são públicos (*public*). O significado das instruções *public* e *private* será visto em detalhes no próximo Capítulo.

Nos diagramas de classe, a representação dos métodos *setter* e *getter*, apesar de poder existir, não são necessárias.

NOTA A ideia do diagrama de classes é desenhar a classe antes do seu desenvolvimento. Normalmente a equipe de análise e/ou projeto desenha as classes e entrega aos programadores para que eles a implementem na linguagem de programação orientada a objeto definida para o projeto.

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.1.7 Boas práticas para nomeação de classes

Existem boas práticas para nomeação de classes utilizadas pela comunidade de desenvolvedores, dentre elas:

- Para nomes com apenas uma palavra, iniciar o nome da classe sempre em letra maiúscula, escrevendo o restante do nome em letra minúscula. Veja exemplos de nomes válidos: *Cliente, Produto, Conta*.
- Para nomes compostos de mais de uma palavra, iniciar cada palavra com letra maiúscula e digitar o restante em letra minúscula. Não deixar espaços ou usar caracteres especiais como *underline* (_) entre as palavras. Veja exemplos de nomes válidos: *ControleClientes, ContaCorrente*.
- Não utilizar acentos ou caracteres especiais no nome de classes.

NOTA Se você não adotar as boas práticas de nomeação, não ocorrerá erro, porém, sua aplicação não será bem vista entre desenvolvedores profissionais. As boas práticas representam recomendações adotadas como padrão atualmente e recomendadas pelos desenvolvedores da linguagem.

3.2 Pacotes

Pacotes são contêineres (ou pastas) que se cria em um projeto para agrupar classes que possuem alguma característica comum ou afinidade.

Horstmann (2009, p. 339), afirma que "um pacote é um conjunto de classes relacionadas".

Por exemplo, em um projeto, pode-se criar um pacote para agrupar as classes que implementam o módulo de controle de funcionários, um pacote para agrupar classes que implementam o módulo de clientes, e assim por diante.

No *NetBeans*, para criar um pacote no projeto utiliza-se o seguinte procedimento:

– Clica-se com o botão direito do mouse sobre o nome do projeto, na divisão "Projetos" do canto superior esquerdo da tela, seleciona-se a opção "Novo" e, em seguida, a opção "Pacote Java".

– Na janela que aparece, no campo "Nome do pacote", digite um nome para o pacote, todo em letra minúscula (que representa uma boa prática), apesar de serem permitidas letras maiúsculas.

– Clique no botão "Finalizar".

Para criar classes no pacote criado, utilize o procedimento a seguir:

– Clique com o botão direito do mouse sobre o nome do pacote e selecione a opção "Novo" e "Classe Java".

– Na janela que aparece, no campo "Nome da Classe", digite o nome desejado para a classe, com letra inicial maiúscula (que é uma boa prática), e clique no botão "Finalizar".

A Figura 3.1 mostra a tela que aparece no processo de criação da classe no pacote:

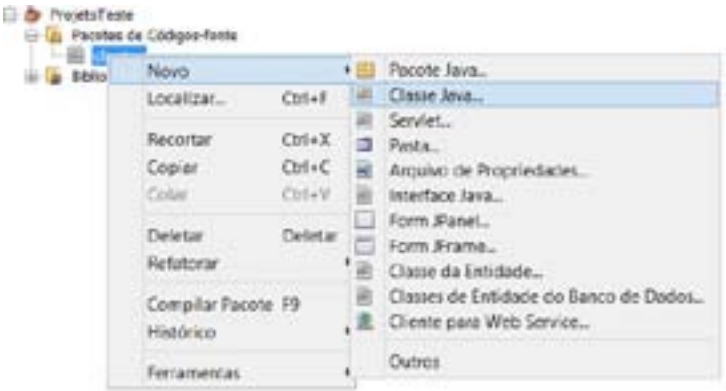


Figura 3.1: Criação de classes em um pacote Java.

A primeira linha de código da classe que está no pacote requer o comando *package* seguido do nome do pacote. Veja o exemplo:

```
package rh;
public classe Funcionário{

}
```

Neste caso, a classe *Funcionario.java* foi criada no pacote *rh*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA Para representar os pacotes de uma aplicação pode ser utilizado o diagrama de pacotes da UML. Se você quiser saber mais sobre o diagrama de pacotes, visite o link a seguir: <http://www.uml-diagrams.org/package--diagrams-overview.html>.

3.2.1 Boas práticas para nomeação de pacotes

Apesar de não ocorrer erro, existem boas práticas utilizadas pela comunidade de desenvolvedores para nomear pacotes, dentre elas estão:

– Utilizar todo o nome do pacote em letras minúsculas. Por exemplo: *controle*.

– Se o nome do pacote tiver mais que uma palavra, utilizar ponto final para separar as palavras. Por exemplo: *cadastro.clientes*

3.2.2 Como utilizar pacotes das bibliotecas Java

Quando você instala o Java no computador, as classes das bibliotecas da linguagem vêm agrupadas APIs que contém pacotes. Por exemplo, o pacote *java.swing* (que será estudado no Capítulo 6) possui as classes *JFrame*, *JOptionPane*, *JLabel*, *JTextField* etc. Já o pacote *java.util* possui as classes *Date*, *Scanner* etc.

Se você desejar utilizar uma destas classes na classe que está escrevendo, terá que importar a classe do pacote correspondente, digitando, antes da declaração da classe, a instrução *import*. Veja o Exemplo:

```
import javax.swing.JOptionPane;
public class Teste{
  public static void main (String[] args){
    JOptionPane.showMessageDialog(null, "Olá!");
   }
}
```

Para importar todas as classes de um pacote de uma só vez, pode-se utilizar o asterisco. Veja o exemplo a seguir:

```
import javax.swing.*;
```

Esta instrução permite utilizar na classe que você está escrevendo qualquer classe que esteja no pacote *javax.swing*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.3 Objetos

A classe *Funcionario.java* que você criou no tópico anterior, é uma classe que permite manipular no projeto criado um novo tipo de dado, um tipo *Funcionario*.

É isso mesmo, agora você pode criar variáveis do tipo *Funcionario* sempre que desejar armazenar informações de funcionários e manipular estas informações. *String*, por exemplo, é um tipo de dado porque há uma classe chamada *String.java*. Da mesma forma, *Funcionario* é um tipo de dado porque existe a classe *Funcionario.java*.

As variáveis que criamos das classes são chamadas objetos. Assim, uma variável do tipo *String* é um objeto, assim como uma variável do tipo *Funcionario* também é um objeto.

Objeto pode ser definido como uma variável de um tipo classe que permite manipular informações nos atributos desta classe através de seus métodos.

Horstmann (2009. p. 63), afirma que "objetos são entidades no programa que você manipula invocando métodos".

Então, pode-se concluir que objetos são variáveis de um tipo de referência a uma classe.

3.3.1 Instância de objetos

Uma instância de um objeto é a alocação de espaço na memória para armazenar e manipular dados neste objeto. Basicamente é a "geração" do objeto na memória. Lembre-se que um objeto é uma variável criada a partir da existência de uma classe para esta finalidade. Para instanciar um objeto de uma classe é necessário chamar o construtor da classe. Por exemplo, para instanciar (ou criar) objetos do tipo da classe *Funcionario*, criada anteriormente, é necessário chamar um dos construtores da classe *Funcionario.java*, a partir do interior de outra classe que deseja manipular dados de funcionários. Neste projeto, vamos criar uma classe principal chamada *Teste.java* a partir da qual vamos manipular os dados de funcionários por meio de objetos da classe *Funcionario.java*.

Para criar a classe principal *Teste.java*, siga as instruções a seguir:

– Clique com o botão direito do mouse no nome da pasta do projeto, localizado na divisão "Projetos" do canto superior esquerdo da tela, clique na opção "Novo" e, em seguida, na opção "Classe Java".

– No campo "Nome da Classe", digite *Teste* e clique no botão "Finalizar".

UNINOVE – USO EXCLUSIVO PARA ALUNO

A classe *Teste.java* será criada com a estrutura básica a seguir:

```
public class Teste {
   public static void main(String[] args) {

   }
}
```

Neste ponto você deve ter no projeto a classe *Funcionario.java* e Teste.java, como mostra a Figura 3.2 a seguir:

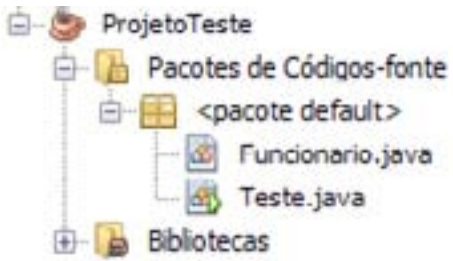


Figura 3.2: ProjetoTeste com suas classes.

3.3.1.1 Instância de objetos utilizando construtor vazio

A classe *Funcionario.java* foi criada com dois construtores: um vazio, que não recebe parâmetros, e um que recebe como parâmetro os dados que se deseja manipular.

No interior do método *main* da classe *Teste.java*, vamos instanciar um objeto chamado *func* da classe *Funcionario.java*, por meio de uma chamada ao construtor que não recebe parâmetros. A instrução deverá ser a seguinte:

```
public class Teste {
   public static void main(String[] args) {
      Funcionario func = new Funcionario();
   }
}
```

Se preferir, pode declarar o objeto *func* primeiro para depois instanciá-lo, da seguinte forma:

```
public class Teste {
   public static void main(String[] args) {
     Funcionario func;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
    func = new Funcionario();
  }
}
```

Veja na Figura 3.3 que a instrução *new Funcionário()* chama o construtor vazio da classe *Funcionario.java*.



Figura 3.3: Chamada ao construtor de uma classe.

A instrução mostrada na Figura 3.3 instancia na memória o objeto chamado *func*, com os atributos vazios, já que a criação foi feita por uma chamada ao construtor da classe que não recebe parâmetros. O objeto *func* pode ser representado, apenas para fins didáticos, como mostra a Figura 3.4.

	func
matricula	0
nome	Null
cargo	Null
salario	0.0
	setMatricula(int matricula): void getMatricula(): int setNome(String nome): void getNome(): String setCargo(String cargo): void getCargo(): String setSalario(double salario): void getSalario(): double ajustarSalario(double percentual):void

Figura 3.4: Criação de um objeto vazio.

Veja que o objeto *func* agrega um conjunto de atributos e as operações que podem ser realizadas sobre estes atributos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Como o objeto foi instanciado vazio por meio do construtor que não recebe parâmetros, para armazenar (setar) valores no objeto deve-se chamar os métodos *setter* da seguinte forma:

```
public class Teste {
   public static void main(String[] args) {
      Funcionario func = new Funcionario();
      func.setMatricula(67543);
      func.setNome("Angela Cristina");
      func.setCargo("Analista");
      func.setSalario(24567.98);
   }
}
```

A chamada ao método *setMatricula* inclui o valor 67543 no atributo *matricula*; a chamada ao método *setNome*, inclui o nome "*Angela Cristina*" no atributo *nome*; a chamada ao método *setCargo*, inclui o cargo "*Analista*" no atributo *cargo;* e a chamada ao método *setSalario*, inclui o salario *24567.98* no atributo *salario*.

Estas operações (*setter*) deixarão o objeto *func* da forma mostrada na Figura 3.5:

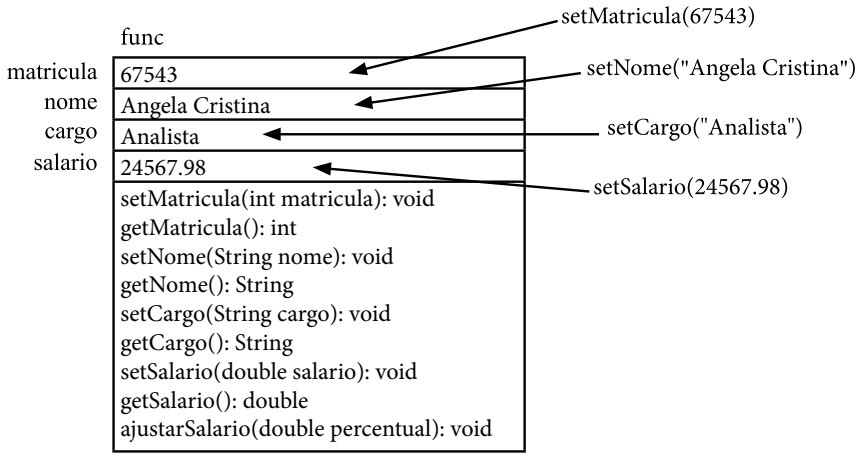


Figura 3.5: Objeto preenchido por meio dos métodos *setter*.

Após instanciar o objeto *func* e armazenar valores em seus atributos, se você deseja mostrar, por exemplo, o conteúdo dos atributos do objeto será necessário obter o conteúdo de cada atributo e exibir na tela. Para obter o conteúdo armazenado nos atributos, utilizam-se os métodos *getter*, das seguintes formas:

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
public class Teste {
   public static void main(String[] args) {
      Funcionario func = new Funcionario();
      func.setMatricula(67543);
      func.setNome("Angela Cristina");
      func.setCargo("Analista");
      func.setSalario(24567.98);
      System.out.println(func.getMatricula());
      System.out.println(func.getNome());
      System.out.println(func.getCargo());
      System.out.println(func.getSalario());
   }
}
```

Note que para obter o valor da matrícula armazenado no atributo *matricula* foi utilizada a instrução *func.getMatricula()*. A instrução *System.out.println* exibe na tela o valor obtido do atributo. Para obter os valores contidos nos demais atributos, foram chamados os métodos *getter* destes atributos. A Figura 3.6 ilustra o uso dos métodos *getter*.

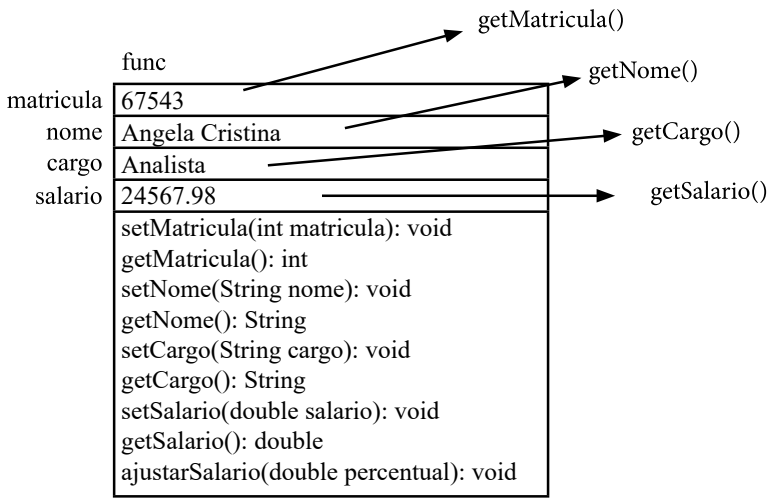


Figura 3.6: Obtendo valores dos atributos.

Uma vez que um objeto é instanciado, para acessar qualquer método contido na classe referente ao objeto, deve-se digitar o nome do objeto, o ponto final e o método que se deseja acessar. Pode-se concluir então, até aqui, que:

"O acesso aos métodos de uma classe ocorre por meio de um objeto instanciado desta classe."

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.3.1.2 INSTÂNCIA DE OBJETOS UTILIZANDO CONSTRUTOR PARAMETRIZADO

Se você quiser instânciar o objeto *func* por meio de uma chamada ao construtor que recebe parâmetros, deve fazer da seguinte forma:

```
public class Teste {
 public static void main(String[] args) {
  Funcionario func = new Funcionario(1367, "Paulo Henrique", "Gerente", 789.56);
 }
}
```

Se preferir, poderá primeiro declarar o objeto para depois instanciá-lo, da seguinte forma:

```
public class Teste {
 public static void main(String[] args) {
   Funcionario func;
   func = new Funcionario(1367, "Paulo Henrique", "Gerente", 789.56);
 }
}
```

Veja na Figura 3.7 que a instrução *new Funcionario* chamou o construtor da classe *Funcionario.java* passando as informações que se deseja armazenar no objeto como parâmetro.



Figura 3.7: Passagem de parâmetros ao construtor de uma classe.

Veja na Figura 3.8 a representação do objeto *func* na memória após a execução da instrução acima:

UNINOVE – USO EXCLUSIVO PARA ALUNO

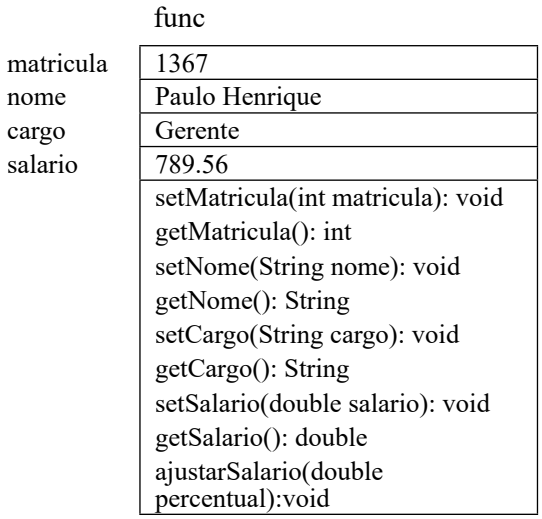


Figura 3.8: Objeto preenchido por meio dos métodos *setter*.

Na instância do objeto, os valores passados como parâmetro já foram armazenados no objeto *func*, sem a necessidade de chamar os métodos *setter*.

Para obter os valores contidos nos atributos do objeto *func*, chamam-se os métodos *getter*, como mostra o exemplo a seguir.

```
public class Teste {
 public static void main(String[] args) {
   Funcionario func = new Funcionario(1367, "Paulo Henrique", "Gerente",
   789.56);
   System.out.println(func.getMatricula());
   System.out.println(func.getNome());
   System.out.println(func.getCargo());
   System.out.println(func.getSalario());
  }
}
```

NOTA Quando você instancia um objeto é feita a chamada a um dos construtores da classe. Neste momento, todas as instruções que estiverem no corpo do construtor, ou seja, dentro das chaves, serão executadas.

3.3.1.3 Instância de vários objetos

Um objeto é uma instância de uma classe, ou variável de uma classe. Assim como qualquer outro tipo de variável, o objeto necessita de uma maneira especial para ser manipulado, como você deve ter percebido. Para armazenar valores, você precisa chamar os métodos *setter* e para obter os valores armazenados, precisa chamar os métodos *getter*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

A nomeação de objetos segue as mesmas regras e boas práticas da nomeação de variáveis comuns. Da mesma forma, é possível ter vários objetos diferentes da mesma classe, assim como é possível ter diversas variáveis diferentes de um mesmo tipo. Veja o exemplo abaixo da classe *Teste.java*.

```
public class Teste {
 public static void main(String[] args) {
        Funcionario func1 = new Funcionario(1367, "Paulo Henrique", "Gerente", 789.56);
        Funcionario func2 = new Funcionario(2567, "Maria Paula", "Vendedora", 1675.55);
        Funcionario func3 = new Funcionario(9634, "Ana Luiza", "Analista", 2765.23);
  }
}
```

Veja que foram instanciados (criados) três objetos: *func1*, *func2*, *func3*. Cada objeto é uma instância da classe *Funcionario.java*. Em cada objeto, os atributos *matricula*, *nome*, *cargo* e *salario* receberam valores diferentes. Se você desejar exibir uma informação contida no atributo de um objeto, deve primeiro obter o valor do atributo, por meio do método *getter* correspondente, para depois exibir o valor obtido. Por exemplo:

```
System.ou.println(func2.getNome());
```

Esta instrução exibirá o nome armazenado no atributo *nome* do objeto *func2*, ou seja, "Maria Paula". Se você quiser exibir o nome "Paulo Henrique", que está no atributo *nome* do objeto *func1*, terá que digitar:

```
System.ou.println(func1.getNome());
```

Se você desejar, para cada funcionário, poderá instanciar um objeto diferente e manipular os dados nele por meio dos métodos *getter* e *setter*.

Mas e se desejar cadastrar 200 funcionários, terá que instanciar 200 objetos?

Não. Nesse caso, você terá que usar um banco de dados e apenas um objeto. Você terá que armazenar os dados do primeiro funcionário no objeto, enviar o objeto para a tabela do banco de dados, armazenar os dados do segundo funcionário no mesmo objeto, enviar o objeto para a tabela do banco de dados, e assim por diante. Isso será visto no Capítulo 7 deste livro. Como um objeto é variável, você poderá substituir o conteúdo dele quantas vezes quiser, bastando chamar os métodos *setter*. Veja o exemplo a seguir:

```
public class Teste {
 public static void main(String[] args) {
      Funcionario func = new Funcionario(1367, "Paulo Henrique", "Gerente", 789.56);
      func.setMatricula(67543);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
        func.setNome("Angela Cristina");
        func.setCargo("Analista");
        func.setSalario(24567.98);
    }
}
```

Note que o objeto *func* foi instanciado com os seguintes valores:

matricula	1367
nome	Paulo Henrique
cargo	Gerente
salario	789.56

Em seguida, estes valores foram alterados, por meio de chamadas aos métodos *setter*, para:

matricula	67543
nome	Angela Cristina
cargo	Analista
salario	24567.98

NOTA Você pode alterar os valores de um objeto quantas vezes desejar, bastando, para isso, chamar os métodos *setter* a partir deste objeto e passar outros valores para seus atributos.

3.3.1.4 Acesso aos métodos

Para acessar os métodos públicos de uma classe, é necessário, antes, instanciar um objeto desta classe. A instância de um objeto ocorre por meio da chamada ao construtor da classe da qual se quer criar o objeto.

Após a instância, é possível chamar os métodos públicos da classe, sejam os métodos *setter*, que incluem valores nos atributos do objeto; sejam os *getter*, que obtêm os valores armazenados nos atributos do objeto; ou outros métodos que atuam sobre os valores dos atributos.

Chamar os métodos *setter* e *getter* você já aprendeu nos tópicos anteriores. Mas e para chamar outros métodos?

A forma é a mesma.

Veja o exemplo do método *ajustarSalario* da classe *Funcionario.java* apresentada anteriormente. Este método recebe um percentual no qual o salário deverá ser aumentado e aplica este percentual sobre o valor contido no atributo *salario*, modificando-o.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja como é feito na classe principal *Teste.Java* o acesso ao método *ajustarSalario*:

```
public class Teste {
   public static void main(String[] args) {
      Funcionario func = new Funcionario();
      func.setMatricula(67543);
      func.setNome("Angela Cristina");
      func.setCargo("Analista");
      func.setSalario(789.56);
      func.ajustarSalario(10);
   }
}
```

Observe que o método *ajustarSalario* é chamado por meio do objeto *func*, passando como parâmetro o valor 10, que é o percentual no qual se deseja aumentar o salário. Na classe *Funcionario.java* o valor passado como parâmetro é recebido pelo método e aplicado sobre o valor do atributo *salario* alterando-o. Assim, o objeto *func*, que tinha o valor 789.56 no atributo salario, passará a ter o valor 868.516.

A Figura 3.9 mostra como ficou a representação do objeto *func*.

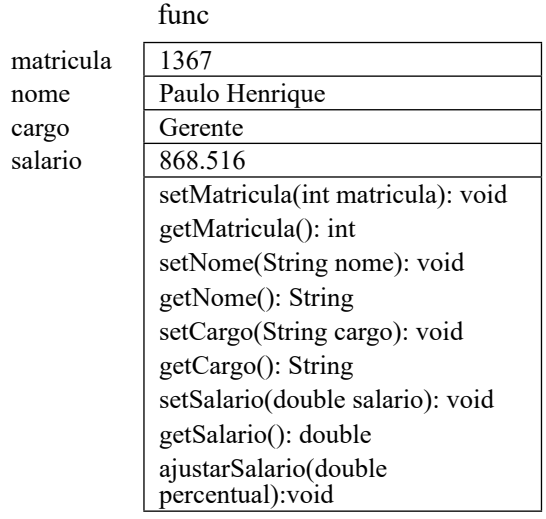


Figura 3.9: Objeto após modificação de valores nos atributos.

Caso você deseje visualizar o novo valor contido no atributo *salario*, basta digitar a instrução *System.out.print(func.getSalario());* após a chamada ao método na classe *Teste.java*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.4 Encapsulamento

Quando você instancia um objeto de uma classe você encapsula os atributos privados da classe no objeto, de forma que possam ser acessados somente por métodos públicos. Dessa forma, podemos ver um objeto como uma cápsula.

Segundo Horstmann (2009, p. 112), "o encapsulamento é o processo de ocultar dados contidos em um objeto e fornecer métodos para acessar estes dados".

Um exemplo de uso do encapsulamento ocorre quando inserimos ou consultamos valores nos atributos de um objeto. Para inserir valores nos atributos privados é necessário chamar os métodos *setter* públicos e para obter os dados presentes nos atributos, é necessário chamar os métodos *getter* públicos. Nesse caso particular, os atributos não podem ser acessados diretamente de fora da classe, pois são privados. Desta forma são necessários métodos públicos para acessá-los.

3.5 Criação de objetos com entradas do usuário

Até agora você criou a classe *Funcionario.java* e a classe principal *Teste.java* em um projeto do *NetBeans*. Na classe *Teste.java*, instanciou um objeto *func* da classe *Funcionario.java* e manipulou dados neste objeto diretamente.

Mas como fazer para armazenar no objeto informações inseridas pelo usuário?

Nesse caso, é necessário utilizar comandos de entrada e armazenar estas entradas no objeto.

Estas entradas podem ser feitas pelo *prompt* de comando, utilizando recursos da classe *Scanner* (contida no pacote *java.util*) ou por uma interface gráfica, utilizando a classe *JOptionPane* (contida no pacote *javax.swing*). Os dois tópicos a seguir apresentam um exemplo de uso destas classes.

NOTA Há outras classes no pacote *javax.swing* que permitem a entrada de dados do usuário utilizando interface gráfica, por meio de formulário. A utilizarão destas classes será explicada no Capítulo 6.

3.5.1 Entradas do usuário com a classe Scanner

A classe *Scanner*, presente no pacote *java.util*, permite a entrada de dados via *prompt* de comando. Para que estas entradas possam ocorrer, é necessário criar um objeto desta classe e, por meio dele, acessar os métodos de entrada. O método *nextInt* é utilizado para entrada de valores inteiros; *nextLine*, para entrada de valores *String*; e *nextDouble*, para entradas de valores *double*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja a seguir o exemplo usando a classe *Scanner*:

```
1   import java.util.Scanner;
2   public class Teste {
3      public static void main(String[] args) {
4         Scanner ler;
5         Funcionario func;
6         ler= new Scanner(System.in);
7         func = new Funcionario();
8         System.out.println("Informe a matrícula:");
9         func.setMatricula(ler.nextInt());
10        ler.nextLine();
11        System.out.println("Informe o nome:");
12        func.setNome(ler.nextLine());
13        System.out.println("Informe o cargo:");
14        func.setCargo(ler.nextLine());
15        System.out.println("Informe o valor do salário:");
16        func.setSalario(ler.nextDouble());
17        System.out.print("\n");
18        System.out.println("Matrícula:" + func.getMatricula());
19        System.out.println("Nome:" + func.getNome());
20        System.out.println("Cargo:" + func.getCargo());
21        System.out.println("Salário:" + func.getSalario());
22        func.ajustarSalario(10);
23        System.out.println("Salário reajustado:" + func.getSalario());
24        System.out.print("\n");
25     }
26  }
```

A linha 4 declara um objeto chamado *ler* da classe *Scanner*, necessário para acessar os métodos de entrada de dados. A linha 6 instancia o objeto.

A linha 5 declara um objeto *func* da classe *Funcionario.java*. A linha 7 instancia o objeto. As linhas 9, 12, 14 e 16, aguardam a entrada dos dados do funcionário, por meio de chamadas de métodos da classe *Scanner* para esta finalidade. Note que antes de cada pedido é exibida uma mensagem solicitando o dado a ser digitado (linhas 8, 11, 13 e 15). Os dados digitados em cada entrada são passados como parâmetro aos métodos *setter* correspondentes da classe *Funcionario.java*.

A linha 17 apenas quebra uma linha por meio da impressão do caracter "*n*".

As linhas de 18 a 21 exibem na tela o conteúdo dos atributos do objeto *func* da classe *Funcionario.java*. Note que para obter os dados armazenados no objeto

UNINOVE – USO EXCLUSIVO PARA ALUNO

foram feitas chamadas aos métodos *getter*. O sinal de mais (+) em cada linha concatena um rótulo *String* ao conteúdo obtido do atributo do objeto.

A linha 22 chama o método *ajustarSalario* e passa o valor 10, que se refere ao percentual no qual se quer aumentar o salário. Este método recebe o valor como parâmetro e aumenta o salário trocando o valor do atributo *salario* pelo valor reajustado.

A linha 23 obtém o salário reajustado por meio de uma chamada ao método *getSalario* da classe *Funcionario.java* e exibe na tela.

Observe na classe apresentada que para cada informação que se deseja pedir ao usuário, são usadas duas linhas, uma para exibir o pedido da informação, e outra para receber a informação e armazenar no atributo do objeto *func* da classe *Funcionario.java*. Veja por exemplo a entrada da matrícula (linhas 8 e 9):

```
System.out.println("Informe a matrícula:");
func.setMatricula(ler.nextInt());
```

Note que *ler.nextInt()* aguarda a digitação de um valor inteiro. O valor digitado é passado automaticamente como parâmetro para o método *setMatricula* da classe *Funcionario.java*, armazenando este valor no atributo *matricula*.

Após compilar e executar a classe *Teste.java* por meio do pressionamento da tecla *F6*, e fornecer os dados solicitados, será exibida a tela mostrada na Figura 3.10:

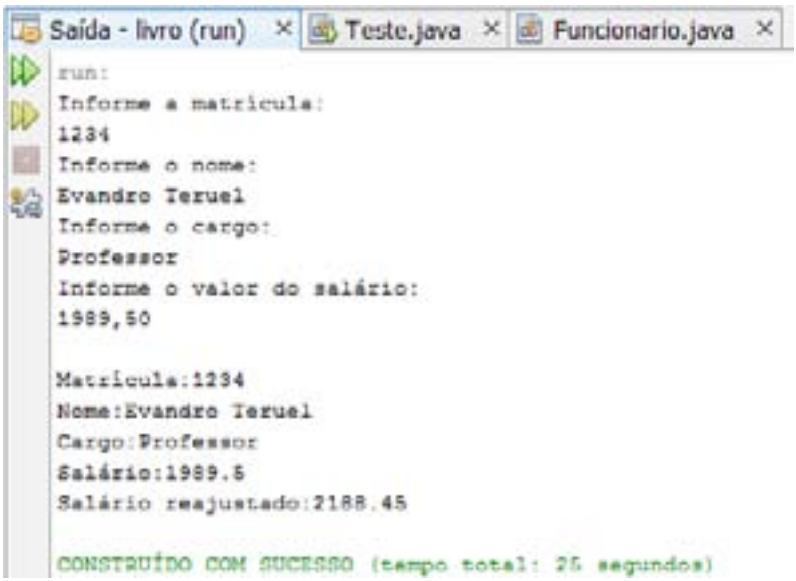


Figura 3.10: Saida da execução da classe Teste.java.

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA O método *nextDouble* da classe *Scanner* requer a digitação de um valor *double* utilizando vírgula para separar casas decimais, caso seja necessário.

3.5.2 ENTRADAS DO USUÁRIO COM A CLASSE JOPTIONPANE

A classe *JOptionPane* do pacote *javax.swing* é utilizada para gerar na tela janelas *pop-up* para receber e exibir dados. O método *showInputDialog* desta classe gera janelas *pop-up* para entrada de dados. Já o método *showMessageDialog* gera uma janela *pop-up* apenas para exibir dados.

Veja a seguir o exemplo com entrada por interface gráfica usando a classe *JOptionPane*.

```
1   import javax.swing.JOptionPane;
2   public class Teste {
3     public static void main(String[] args) {
4       Funcionario func;
5       func = new Funcionario();
6       func.setMatricula(Integer.parseInt(JOptionPane.showInputDialog("Digite a
        matrícula: ")));
7       func.setNome(JOptionPane.showInputDialog("Digite o nome: "));
8       func.setCargo(JOptionPane.showInputDialog("Digite o cargo: "));
9       func.setSalario(Double.parseDouble(JOptionPane.showInputDialog("Digite o
        salário: ")));
10      JOptionPane.showMessageDialog(null,
        "Matrícula:" + func.getMatricula() + "\n"
        + "Nome:" + func.getNome() + "\n"
        + "Cargo:" + func.getCargo() + "\n"
        + "Salário:" + func.getSalario() + "\n");
11      func.ajustarSalario(Integer.parseInt(JOptionPane.showInputDialog("Digite o
        percentual de reajuste para o salário: ")));
12      JOptionPane.showMessageDialog(null, "Salário reajustado: " + func.
        getSalario());
13    }
14  }
```

A linha 4 declara o objeto *func* da classe *Funcionario.java* e a linha 5 instancia este objeto. As linhas de 6 a 9 pedem ao usuário a digitação dos dados dos funcionários e passam estes dados para os métodos *setter* correspondentes da classe *Funcionario.java*. A linha 10 exibe em uma única janela *pop-up* os dados contidos no objeto *func*, por meio de chamadas aos métodos *getter* da

UNINOVE – USO EXCLUSIVO PARA ALUNO

classe *Funcionario.java*. Note que o caracter "\n" serve para quebrar linha na janela *pop-up* que exibe os dados.

A linha 11 pede o percentual de reajuste do salário do funcionário e passa como parâmetro para o método *ajustarSalario* da classe *Funcionario.java*, que altera o valor contido no atributo *salario* para o valor do salário reajustado. A linha 12 exibe o conteúdo do atributo *salario* (que contém o salário reajustado), por meio de uma chamada ao método *getSalario* da classe *Funcionario.java*.

Observe que a entrada de dados ocorrerá por meio de janelas *pop-up* geradas pelo método *showInputDialog* da classe *JOptionPane*. Estas janelas aparecerão uma por uma, ao compilar e executar a classe utilizando a tecla *F6*. Veja na Figura 3.11 como serão estas janelas:



Figura 3.11: Janelas pop-up para entrada de dados de usuário.

Após a digitação dos dados, o método *showMessageDialog* da classe *JOptionPane* exibirá uma janela *pop-up* com os dados que foram armazenados no objeto *func* da classe *Funcionario.java* (linha 10).

Veja na Figura 3.12 como os dados serão exibidos na tela:



Figura 3.12: Exibição de dados em uma janela *pop-up*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Após a exibição dos dados aparecerá a janela *pop-up* mostrada na Figura 3.13, para a digitação do percentual de aumento do salário (linha 11).



Figura 3.13: Entrada do percentual de aumento do salário em uma janela *pop-up*.

Após a digitação do percentual, por exemplo, o valor 10, aparecerá a janela *pop-up* mostrada na Figura 3.14, que exibe o salário reajustado (linha 12).



Figura 3.14: Janela *pop-up* para exibição do salário reajustado.

NOTA

Todo valor digitado em campos de entrada, no Java e na maioria das linguagens, é considerado um valor do tipo *String*, mesmo que a entrada seja um número. Por esse motivo, se você quer manipular o valor que entrou como um valor numérico, deve fazer a conversão para o tipo numérico do seu interesse. As operações de conversão de tipo *String* para tipos numéricos é chamada ***parse***. Assim, veja algumas formas de conversão:

```
byte idade=Byte.parseByte(JOptionPane.showInputDialog("Digite a idade:");
short idade=Short.parseShort(JOptionPane.showInputDialog("Digite a idade:");
int idade=Integer.parseInt(JOptionPane.showInputDialog("Digite a idade:");
long idade=Long.parseLong(JOptionPane.showInputDialog("Digite a idade:");
float salario=Float.parseFloat(JOptionPane.showInputDialog("Digite o salário:");
double  salario=Double.parseDouble(JOptionPane.showInputDialog("Digite
o salário:");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.5.3 Comandos encadeados

Em Java é comum digitar em uma única linha diversos comandos aninhados, que poderiam ser desmembrados em diversas linhas, uma para cada operação. Para quem está iniciando o aprendizado do Java, pode ser difícil lidar com comandos aninhados.

Vamos analisar, por exemplo, a linha abaixo, apresentada na classe *Teste.java*, do tópico anterior:

```
func.setMatricula(Integer.parseInt(JOptionPane.showInputDialog("Digite a matrícula: ")));
```

Há, nesta linha, três operações:

– Solicitação do valor da matrícula por meio do comando *JOptionPane. showInputDialog*.

– Conversão do valor digitado para a matrícula de *String* para *int*, por meio do comando *Integer.parseInt*.

– Passagem do valor convertido para inteiro para o método *setMatricula*, para ser incluído no atributo *matricula* do objeto *func*.

A linha apresentada pode ser desmembrada em três, uma para cada operação, da seguinte forma:

```
String m = JOptionPane.showInputDialog("Digite a matrícula: ");
int  mat = Integer.parseInt(m);
func.setMatricula(mat);
```

Que tal, gostou desta forma?

Vejamos outras linhas que digitamos em exemplos anteriores:

```
func.setSalario(ler.nextDouble());
```

Veja que são executadas duas operações na linha acima:

– Entrada de um valor *double*.

– Passagem do valor *double* digitado para o método *setSalario*, para ser armazenado no atributo *salario* do objeto *func*.

Esta linha poderia ser desmembrada em duas, da seguinte forma:

```
double valor = ler.nextDouble();
func.setSalario(valor);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

DICA Procure ver nas linhas com instruções aninhadas as diversas operações que estão sendo realizadas e procure desmembrá-las em uma instrução para cada operação. Vai facilitar o seu aprendizado.

3.6 CLASSES SEM ATRIBUTOS

Pode parecer, devido à explicação da aplicação desenvolvida nos tópicos anteriores com as classes *Teste.java* e *Funcionario.java*, que toda classe precisa ter atributo, o que não é verdade. Você pode criar classes apenas para concentrar métodos que precisa utilizar com frequência na aplicação. Sempre que precisar utilizar estes métodos, basta instanciar um objeto da classe e chamar o método adequado.

Veja o exemplo a seguir onde será criada a classe *Calculo.java* para concentrar métodos que executam operações aritméticas. Será criada também uma classe *Teste.java*, de onde estes métodos serão chamados.

Para digitar os códigos-fonte que serão apresentados, execute os procedimentos a seguir:

– Crie um projeto chamado "ProjetoTeste".
– Crie um pacote chamado *app*.
– Cria, dentro do pacote *app*, as classes *Calculo.java* e *Teste.java*.

Após a realização dos procedimentos acima, a pasta do projeto deve estar como mostra a Figura 3.15:



Figura 3.15: Criação do Projeto ProjetoTeste no NetBeans.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Na classe *Calculo.java*, digite o seguinte código-fonte:

```
package app;
public class Calculo {
   public Calculo(){
   }
   public double media(double valor1, double valor2) {
      return (valor1 + valor2) / 2;
   }
   public double media(double valor1, double valor2, double valor3) {
      return (valor1 + valor2 + valor3) / 3;
   }
   public int soma(int valor1, int valor2) {
      return (valor1 + valor2);
   }
   public double soma(int valor1, int valor2, double valor3) {
      return (valor1 + valor2 + valor3);
   }
   public double raizQuadrada(int valor) {
      return (Math.sqrt(valor));
   }
}
```

Note que há dois métodos chamados *media*, um que recebe dois valores *double*, calcula a média entre estes valores e retorna o resultado como *double* e um que recebe três valores *double*, calcula a média entre estes valores e retorna o resultado como *double*.

Há também dois métodos chamados *soma*, um que recebe dois valores inteiros, soma estes valores e retorna o resultado como *int* e um que recebe dois valores inteiros e um valor *double*, soma estes valores e retorna o resultado como *double*.

O método chamado *raizQuadrada* recebe um valor *int*, calcula a raiz quadrada deste valor e retorna o resultado como *double*. Para calcular a raiz quadrada foi utilizado o método *sqrt* da classe *Math*, instalada com o Java. A classe *Math* permite realizar diversos tipos de operações matemáticas e trigonométricas.

Perceba que há mais de um método com o mesmo nome na classe, porém, estes métodos de mesmo nome recebem parâmetros diferentes. Isto é chamado na orientação a objetos de sobrecarga de métodos.

O diagrama de classes da UML para a classe acima é mostrado na Figura 3.16.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Calculo
+ media(double valor1, double valor2): double
+ media(double valor1, double valor2, double valor3) : double
+ soma(int valor1, int valor2): int
+ soma(int valor1, int valor2, double valor3): double
+ raizQuadrada(int valor): double

Figura 3.16: Diagrama de classes mostrando a classe Calculo.

Observe que a divisão destinada aos atributos está vazia, já que a classe não possui atributos.

Para interagir com a classe *Calculo.java*, chamando seus métodos, digite na classe principal, *Teste.java*, o seguinte código-fonte:

```
package app;
public class Teste {
   public static void main(String[] args) {
   Calculo calc = new Calculo();
   double med1 = calc.media(10.5, 13.6);
   double med2 = calc.media(10.5, 13.6, 45.6);
   int soma1 = calc.soma(14, 34);
   double soma2 = calc.soma(14, 34, 56.7);
   double r = calc.raizQuadrada(16);
   System.out.println(med1);
   System.out.println(med2);
   System.out.println(soma1);
   System.out.println(soma2);
   System.out.println(r);
   }
}
```

Veja que um objeto chamado *calc*, da classe *Calculo.java* foi instanciado, para poder acessar os métodos da classe. Por meio deste objeto são chamados os métodos e são passados os parâmetros que os métodos requerem. O valor resultante retornado dos métodos é armazenado em variáveis. Note que se o método retorna um valor do tipo *double*, a variável que recebe o retorno também deve ser do tipo *double*. Note também que como há mais de um método com o mesmo nome, o compilador sabe qual deles está sendo chamado pelos parâmetros que são passados. Por exemplo, se forem passados para o método *soma* dois valores inteiros e um valor real, será chamado o método *soma* que está esperando dois valores *int* e um *double*.

As instruções *System.out.println* exibem o conteúdo de cada variável que recebeu valor dos métodos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto, por meio do pressionamento da tecla *F6*, será exibido na tela os valores a seguir, resultantes da chamada aos métodos:

```
12.05
23.233333333333334
48
104.7
4.0
```

O código-fonte da classe *Teste.java* também poderia ser escrito da forma apresentada a seguir, sem alterar o resultado apresentado:

```
package app;
public class Teste {
   public static void main(String[] args) {
   Calculo calc = new Calculo();
   System.out.println(calc.media(10.5, 13.6));
   System.out.println(calc.media(10.5, 13.6, 45.6));
   System.out.println(calc.soma(14, 34));
   System.out.println(calc.soma(14, 34, 56.7));
   System.out.println(calc.raizQuadrada(16));
   }
}
```

Note que, neste caso, são usadas operações aninhadas que chamam os métodos, passam os parâmetros e já exibem os valores retornados, poupando a criação de variáveis.

Como você deve ter percebido, pode criar uma classe apenas com métodos de apoio para operações que irá utilizar na aplicação.

3.7 *Array* de objetos

Neste tópico vamos misturar o o uso de *arrays* visto no capítulo 2, com orientação a objetos. Vamos criar uma *array* unidimensional e armazenar três objetos da classe *Funcionario.java* nesta *array*, um em cada posição (índice), como mostra a representação na Figura 3.17.

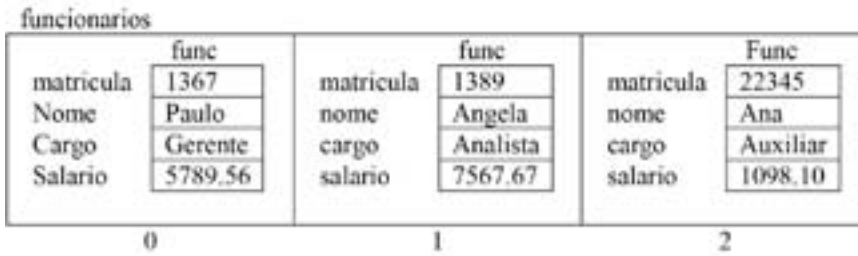


Figura 3.17: Array de objetos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Os dados armazenados serão fornecidos pelo usuário.

Para realizar este trabalho, crie um projeto chamado *ProjetoTeste* e as classes *Teste.java* e *Funcionario.java*, como mostra a Figura 3.18.

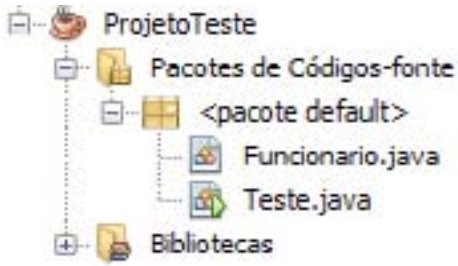


Figura 3.18: Representação do projeto criado no NetBeans.

A classe *Funcionario.java* é a mesma apresentada anteriormente nesse Capítulo, portanto, digite o código-fonte desta classe ou copie-o, caso já tenha digitado.

Na classe principal *Teste.java*, digite o código a seguir:

```
1   import javax.swing.JOptionPane;
2   public class Teste {
3     public static void main(String[] args) {
4       Funcionario funcionarios[];
5       funcionarios = new Funcionario[3];
6       Funcionario func;
7       for (int indice = 0; indice < funcionarios.length; indice++) {
8           func = new Funcionario();
9           func.setMatricula(Integer.parseInt(JOptionPane.showInputDialog("Digi
            te a matrícula: ")));
10          func.setNome(JOptionPane.showInputDialog("Digite o nome: "));
11          func.setCargo(JOptionPane.showInputDialog("Digite o cargo: "));
12          func.setSalario(Double.parseDouble(JOptionPane.showInputDialog("Di
            gite o salário: ")));
13          funcionarios[indice] = func;
14      }
15      for (int indice = 0; indice < funcionarios.length; indice++) {
16          func = funcionarios[indice];
17          JOptionPane.showMessageDialog(null,
            "Matrícula:" + func.getMatricula() + "\n"
            + "Nome:" + func.getNome() + "\n"
            + "Cargo:" + func.getCargo() + "\n"
            + "Salário:" + func.getSalario() + "\n");
18      }
19    }
20  }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Basicamente, o que esta classe faz é pedir dados de funcionários, colocar estes dados em objetos da classe *Funcionario.java* e incluir cada um destes objetos nas posições de uma *array*. Para isso, é utilizado um laço de repetição para passar por cada posição da *array* (linhas de 7 à 14). Em seguida, o conteúdo de cada objeto das posições da *array* tem seu conteúdo exibido. Para percorrer a *array*, um laço de repetição é útilizado (linhas 15 à 18).

Para entender melhor todo o processo, segue uma explicação mais detalhada.

A linha 4 declara a *array funcionarios* e a linha 5 instancia a *array* com três posições, sempre numeradas a partir de 0 (neste caso, de 0 à 2).

A linha 6 declara o objeto *func* da classe *Funcionario.java*.

A linha 5 inicia um laço de repetição que vai de 0 ao tamanho da *array* funcionários -1, incrementando a variável *indice* em 1 a cada passagem pelo interior do laço. Note que a variável *indice* é iniciada com valor 0. O laço de repetição tem seu fechamento na linha 14.

No interior do laço (linha 8), o objeto *func* da classe *Funcionario.java* é instanciado vazio. Na linha 9, é pedido o valor da matrícula do funcionário, convertido em inteiro e armazenado no atributo *matricula* do objeto *func*.

Na linha 10, é pedido o nome do funcionário e armazenado no atributo *nome* do objeto *func*.

Na linha 11, é pedido o cargo do funcionário e armazenado no atributo *cargo* do objeto *func*.

Na linha 12, é pedido o valor do salário do funcionário, convertido em *double* e armazenado no atributo *salario* do objeto *func*.

Na linha 13, o objeto *func* é adicionado à posição 0 da *array funcionarios* e o laço retorna a execução para a linha 7, onde a variável *indice*, que representa as posições da *array* é incrementada em 1, passando a valer 1. Como a condição do laço ainda é verdadeira, o interior do laço é executado novamente.

Note que a linha 8 instancia um novo objeto *func* da classe *Funcionario.java*. As linhas de 9 à 12 são executadas, solicitando novamente valores e armazenando no novo objeto *func*. Na linha 13, o objeto *func*, que já contém os novos dados, é armazenado na posição 1 da *array funcionarios*. Veja que a linha 14 retorna para a linha 7, linha de abertura do laço. Novamente a variável *indice* é incrementada em 1, passando a valer 2. Como a condição do laço ainda é verdadeira, o interior do laço é executado novamente.

Note que a linha 8 instancia um novo objeto da classe *Funcionario.java*. As linhas de 9 à 12 são executadas, solicitando novamente valores e armazenando no novo objeto *func*. Na linha 13, o objeto *func*, que já contém os novos dados, é armazenado na posição 2 da *array funcionarios*. A linha 14 retorna para a linha 7 onde a variável

UNINOVE – USO EXCLUSIVO PARA ALUNO

indice é incrementada em 1 passando a valer 3, o que torna a condição do laço falsa, encerrando sua execução e desviando para a linha 15, que inicia um novo laço, este, para exibir os dados dos objetos em cada posição da *array funcionarios*.

O laço da linha 15 é idêntico ao laço anterior, contando de 0 a 2, ou seja, de 0 ao tamanho da *array funcionarios* -1. A variável *indice* é iniciada em 0 e usada para contar as execuções do interior do laço.

Note na linha 16, que o conteúdo da *array funcionarios* na posição *indice*, que vale 0, é colocado em um objeto *func* da classe *Funcionario.java*. Basicamente esta linha pega o primeiro objeto da posição 0 da *array funcionarios* e coloca em um objeto *func* da classe *Funcionario.java*. A linha 17 exibe os dados deste objeto e a linha 18 retorna ao início do laço, incrementando a variável *indice* em 1, fazendo com que esta variável passe a valer 1. Como condição do laço é verdadeira, o interior do laço é executado novamente. Na linha 16, o objeto da classe *Funcionario.java* contido na posição 1 da *array funcionarios* é armazenado em um objeto *func* da classe *Funcionario.java*. A linha 17 exibe o conteúdo deste objeto e a linha 18 retorna para o início do laço (linha 15). A variável *indice* é incrementada em 1, passando a valer 2. Como a condição ainda é verdadeira, o laço é executado novamente. A linha 16 pega o objeto da classe *Funcionario.java* contido na posição 2 da *array* e armazena em um objeto *func* da classe *Funcionario.java*. A linha 17 exibe o conteúdo deste objeto. A linha 18 retorna para a linha de abertura do laço, onde a variável *indice* é incrementada em 1, passando a valer 3. Como a condição agora se tornou falsa, o laço é encerrado e as linhas 19 e 20 são executadas, terminando a execução da classe.

UNINOVE – USO EXCLUSIVO PARA ALUNO

3.7.1 QUANDO USAR *ARRAYS* DE OBJETOS

Muitas vezes você precisa passar um conjunto de objetos para um método, como parâmetro. Passar um a um pode ser muito trabalhoso, pois necessitaria de muitas chamadas ao método. Neste caso, é mais fácil colocar o conjunto de objetos em uma *array* e passar a *array* como parâmetro em uma única chamada de método.

Quando se quer armazenar um conjunto de objetos no banco de dados, em alguns casos, também é mais fácil armazenar estes objetos em uma *array* e enviar esta *array* para alguma API ou método que gerencie o envio para a tabela do banco de dados.

Há muitos usos para *arrays* de objetos e com o tempo certamente você vai precisar utilizar uma.

3.8 Resumo

Os conceitos fundamentais da orientação a objetos são classes, atributos, métodos, construtores e objetos.

Uma classe é um arquivo de programa Java com algumas características especiais para atender ao paradigma orientado a objetos. Assim, deve possuir uma declaração e um construtor (explícito ou implícito). Além destas estruturas, pode possuir declaração de atributos e/ou constantes e métodos. Classes com características comuns normalmente são agrupadas em contêineres (espécie de pastas), chamados pacotes. As bibliotecas de classes que fazem parte da linguagem Java normalmente vêm agrupadas em pacotes, como *java.swing*, *java.util* etc. Para modelar (ou desenhar) uma classe utiliza-se a notação do diagrama de classes da UML. Para modelar um pacote, utiliza-se a notação do diagrama de pacotes.

As classes normalmente representam abstrações do mundo real para categorização ou classificação de algo. Por exemplo, as pessoas em uma empresa podem ser classificadas em clientes e funcionários. A classe dos clientes pode possuir os atributos nome, endereço, telefone etc. A classe dos funcionários pode possuir nome, cargo, salário etc. Sobre o salário podem ser executadas operações (métodos) para reajustar o salário, calcular o imposto de renda etc.

Um atributo, na programação, é uma variável da classe, e um método, uma operação que pode ser realizada sobre os valores contidos nos atributos e/ou sobre valores recebidos como parâmetro.

Um objeto é uma variável de referência a uma classe que permite incluir valores nos seus atributos (por meio de métodos *setter*) ou obter valores destes atributos (por meio de métodos *getter*), além de permitir, normalmente, acessar seus métodos visíveis. Para instanciar ("criar") um objeto de uma classe, é necessário chamar o construtor da classe utilizando a instrução *new* seguida do nome do construtor, que deve ter o mesmo nome da classe. O construtor pode estar vazio e não receber parâmetros. Neste caso, o objeto será criado com os atributos vazios. Se o construtor receber parâmetros, estes normalmente são adicionados aos atributos da classe no corpo do construtor. Uma classe pode ter vários construtores, desde que recebam parâmetros diferentes. Neste caso, dizemos que há sobrecarga de construtor. Pode-se criar diversos objetos da mesma classe com os atributos em cada objeto contendo valores diferentes.

Um método *setter* é um método que normalmente inicia pela palavra *set*, seguida do nome do atributo com letra inicial maiúscula. Este tipo de método recebe um valor como parâmetro que é incluído no atributo do objeto.

Um método *getter* é um método que normalmente inicia pela palavra *get*, seguida do nome do atributo com letra inicial maiúscula. Este tipo de método normalmente não recebe parâmetro e apenas retorna o valor armazenado no

UNINOVE – USO EXCLUSIVO PARA ALUNO

atributo. Além dos métodos *setter* e *getter*, uma classe pode ter qualquer outro método que pode atuar sobre os valores dos atributos ou não.

A técnica de ocultar de um objeto detalhes internos do funcionamento dos métodos de uma classe, de forma que o conhecimento interno do corpo dos métodos não seja necessário para chamá-los, é conhecida como encapsulamento.

Há classes que são criadas sem atributos, apenas com métodos, para executar operações específicas na aplicação, que normalmente são chamadas diversas vezes. Métodos em uma mesma classe podem ter o mesmo nome, desde que recebam parâmetros diferentes. Neste caso, dizemos que há sobrecarga de métodos.

A entrada de dados do usuário em uma aplicação Java SE pode ser feita via *prompt* de comando, utilizando a classe *Scanner*; via interface gráfica, utilizando a classe *JOptionPane;* ou via formulário, utilizando outras classes do pacote *javax.swing*.

3.9 EXERCÍCIOS

A seguir você encontrará exercícios práticos, onde terá que criar projetos no *NetBeans* e digitar o código-fonte de classes, e também exercícios para fixar os conceitos aprendidos neste Capítulo. Não deixe de fazer estes exercícios.

3.9.1 EXERCÍCIOS PRÁTICOS

1) Crie um projeto chamado *Projeto1* no *NetBeans* com um pacote chamado *teste* contendo uma classe principal chamada *Teste.java* e uma classe chamada *Aluno.java.*

O desenho da classe *Aluno.java* é apresentado a seguir:

Aluno
- matricula:long - nome:String - disciplina:String - numeroAulas:int - numeroFaltas:int - notaAv1:double - notaAv2:double - notaAv3:double
+ calcularMedia():double + calcularLimiteFaltas():double +verificarSituacao(media:double, limiteFaltas:double):String

UNINOVE – USO EXCLUSIVO PARA ALUNO

Instruções para a criação da classe *Aluno.java*:

Na classe *Aluno.java*, declare os atributos, um construtor vazio, os métodos *getter* e *setter* e os métodos descritos *calcularMedia*, *calcularLimiteFaltas* e *verificarSituacao*, descritos a seguir.

O método *calcularMedia* deve descartar a menor das três notas e calcular a média das duas maiores, retornando o valor resultante.

O método *calcularLimiteFaltas* deve retornar o limite de faltas que é 25% do número de aulas. Esse limite de faltas representa o número máximo de faltas que o aluno pode ter em determinada disciplina.

O método *verificarSituacao* deve verificar se o aluno foi aprovado ou não. Para isto, esse método recebe a média do aluno e o limite de faltas que ele pode ter. Para ser aprovado, o aluno deve ter média maior ou igual a 6.00 e o número de faltas menor do que o limite de faltas permitido. Se aprovado, esse método deve retornar a *String* "Aprovado", senão, deve retornar a *String* "Reprovado".

Instruções para a criação da classe *Teste.java*:

Na classe principal *Teste.java*, os dados do aluno devem ser solicitados utilizando interface gráfica por meio da classe *JOptionPane* e armazenados em um objeto da classe *Aluno.java*. Em seguida, os dados armazenados neste objeto devem ser exibidos em uma única caixa de diálogo. Após a exibição dos dados, deve ser exibida a média e a situação do aluno ("Aprovado" ou "Reprovado") em uma nova caixa de diálogo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

2) Crie um projeto chamado *Projeto2* que contenha em um pacote chamado *classes.teste* uma classe principal chamada *Principal.java* e uma classe chamada *Produto.java*. O desenho da classe *Produto.java* é apresentado a seguir:

Produto
- id:long - nome:String - marca:String - preco:double
+ diminuirPreco(percentual:double):void + aumentarPreco(percentual:double):void

Instruções para construir a classe *Produto.java*:

A classe *Produto.java* deve possuir os atributos, um construtor vazio e um construtor recebendo todas as informações do produto como parâmetro. Deve também possuir métodos *getter* e *setter* para os atributos declarados.

O método *diminuirPreco* deve receber um valor *double* e reduzir o preço do produto neste percentual.

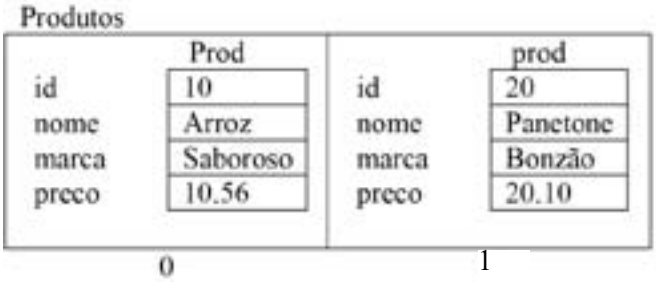
O método *aumentarPreco* deve receber um valor *double* e reduzir o preço do produto neste percentual.

Instruções para construir a classe *Principal.java*:

Devem ser instanciados dois objetos da classe *Produto.java* e cadastrados todos os dados de dois produtos a partir da entrada de dados do usuário por meio de campos gerados com a utilização da classe *Scanner*. Após o cadastro dos dois produtos, o primeiro, deve ter seu preço diminuído em um percentual solicitado ao usuário. O segundo, deve ter seu preço aumentado em um percentual também solicitado ao usuário. Em seguida, todos os dados dos dois produtos devem ser exibidos na tela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

3) Utilize as mesmas classes da questão 2, alterando a classe principal *Teste.java*, de tal forma que os dois objetos da classe *Produto.java* sejam armazenados em uma *array* unidimensional de produtos, como a representada a seguir:



3.9.2 EXERCÍCIOS CONCEITUAIS

1) Defina:

A) Classe.

B) Atributo.

C) Método.

D) Métodos *setter*.

E) Métodos *getter*.

F) Construtor.

G) Objeto.

H) Pacote.

2) Quais são as boas práticas para nomeação de variáveis, classes e pacotes em Java?

3) O que é encapsulamento? Qual é a vantagem proporcionada pelo encapsulamento em aplicações orientadas a objetos?

4) O que é sobrecarga de método e sobrecarga de construtor?

5) Que instrução na declaração do método indica que ele não retorna nada?

6) Quantos valores um método pode retornar de cada vez? Justifique sua resposta.

UNINOVE – USO EXCLUSIVO PARA ALUNO

4. Orientação a objetos – conceitos avançados

Ao chegar neste ponto do livro você precisa ter os conhecimentos básicos da orientação a objetos apresentados no Capítulo anterior. Precisa ser capaz de criar classes, com atributos e métodos, instanciar objetos destas classes e manipular dados nestes objetos. Precisa também entender a função do construtor de uma classe, sobrecarga de construtores e sobrecarga de métodos.

Além dos conceitos básicos da orientação a objetos, precisa estar adaptado ao uso do *NetBeans,* tendo condições de criar projetos com pacotes e classes e compilar e executar estes projetos adequadamente.

Neste Capítulo vamos além dos conceitos básicos. Você vai aprender o conceito de herança em suas várias formas, que podem ocorrer em graus diferentes de maneira simples, por meio do uso de interfaces e por meio do uso de classes abstratas.

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.1 Herança

Herança é um conceito proveniente da observação das relações humanas, onde os filhos herdam características dos pais; e da modelagem de dados, onde tabelas especializadas herdam atributos de tabelas generalizadas.

Você vai usar herança quando possuir um conjunto de atributos e/ou métodos que é comum a um conjunto de classes e possuir alguns deles que são específicos de cada classe. Nesse caso, você centraliza os que são comuns em uma superclasse e os específicos coloca nas respectivas subclasses, de tal forma que estas subclasses herdem os atributos e/ou métodos comuns da superclasse.

Segundo Horstmann (2009, p. 400),

> a herança é um mecanismo para estender as classes existentes adicionando novos métodos e campos. A classe mais geral chama-se superclasse e a classe mais especializada, que herda da superclasse chama-se subclasse.

Veja um exemplo de diagrama de classe representando uma relação de herança entre as classes *Conta*, *ContaCorrente* e *ContaPoupanca*.

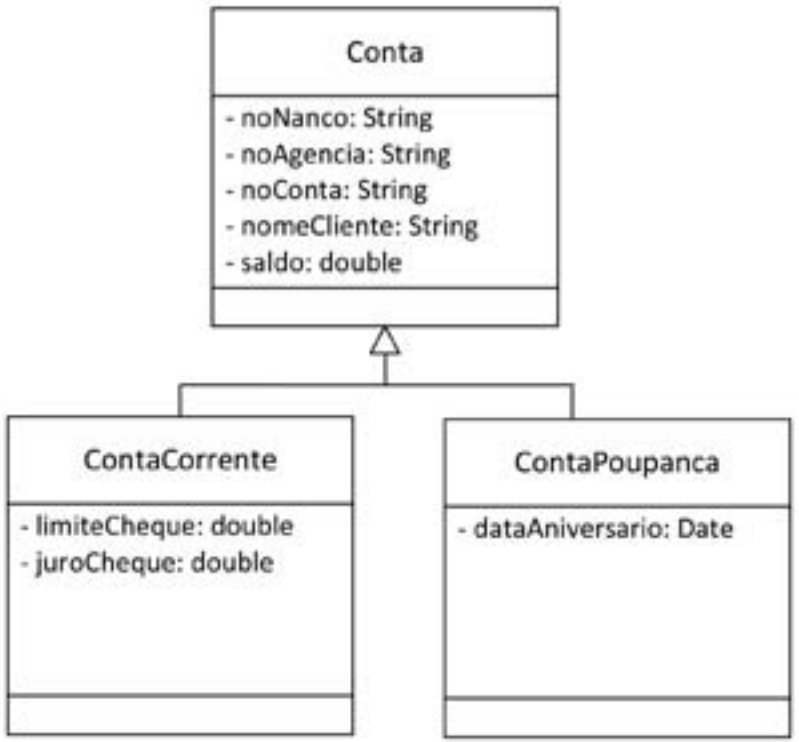


Figura 4.1: Diagrama de classe representando herança.

Note que os atributos *noBanco*, *noAgencia*, *noConta*, *nomeCliente* e *saldo* são comuns a todos os tipos de conta (corrente e poupança), por isso são centralizados em uma classe pai (superclasse) chamada *Conta*. Já os atributos específicos de conta corrente são centralizados na classe filha (subclasse) *ContaCorrente* e os de conta poupança são centralizados na classe filha (subclasse) *ContaPoupanca*.

Na realidade, as subclasses *ContaCorrente* e *ContaPoupanca* possuem todos os atributos e métodos da superclasse *Conta* e também seus atributos e métodos específicos.

NOTA Em uma relação de herança, a classe pai é chamada de superclasse e as classes filhas, de subclasses. Os atributos e métodos comuns são centralizados na superclasse e os específicos, nas subclasses. As subclasses herdam todos os atributos e métodos da superclasse.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Em Java, várias subclasses diferentes podem herdar de uma mesma superclasse, mas uma subclasse não pode herdar de mais de uma superclasse, pois não existe herança múltipla em Java. Logo, no exemplo acima, a subclasse *ContaCorrente*, por exemplo, poderá herdar da superclasse *Conta*, mas não poderia herdar de nenhuma outra superclasse, caso existissem outras superclasses na aplicação.

Para indicar que uma subclasse herda de uma superclasse utiliza-se a palavra reservada *extends*. Por exemplo, veja como fica a declaração da classe *ContaCorrente*.

```
public class ContaCorrente extends Conta {
 //O conteúdo da classe deve ser digitado aqui
}
```

Nos tópicos seguintes, as classes *Conta*, *ContaCorrente* e *ContaPoupanca* serão implementadas em um projeto Java criado no *NetBeans*.

4.1.1 EXEMPLO DE APLICAÇÃO COM HERANÇA

Para implementar as classes do diagrama apresentado no tópico anterior, crie um projeto Java no *NetBeans* e nomeie-o de *ProjetoTeste*. Neste projeto, crie quatro classes: *Conta.java*, *ContaCorrente.java*, *ContaPoupanca.java* e *Principal.java*. A classe *Principal.java* será utilizada para instanciarmos objetos das classes da relação de herança.

A Figura 4.2 mostra como deve ficar o projeto após a criação destas classes.

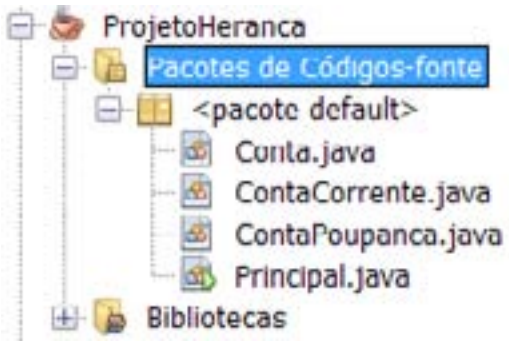


Figura 4.2: Projeto com uso de herança simples.

Nos tópicos seguintes serão apresentados os códigos-fonte das classes criadas.

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.1.1.1 Implementação da superclasse Conta

A superclasse *Conta.java* é uma classe comum, como criada no Capítulo anterior, onde são declarados os atributos, definidos os construtores (um vazio e um recebendo parâmetros) e os métodos *getter* e *setter* para os atributos. Veja como ficará o código-fonte.

```
1   public class Conta {
2     private String noBanco;
3     private String noAgencia;
4     private String noConta;
5     private String nomeCliente;
6     private double saldo;
7
8     public Conta(String noBanco, String noAgencia, String noConta, String
      nomeCliente, double saldo) {
9       this.noBanco = noBanco;
10      this.noAgencia = noAgencia;
11      this.noConta = noConta;
12      this.nomeCliente = nomeCliente;
13      this.saldo = saldo;
14    }
15
16    public Conta() {
17    }
18
19    public String getNoBanco() {
20      return noBanco;
21    }
22
23    public void setNoBanco(String noBanco) {
24      this.noBanco = noBanco;
25    }
26
27    public String getNoAgencia() {
28      return noAgencia;
29    }
30
31    public void setNoAgencia(String noAgencia) {
32      this.noAgencia = noAgencia;
33    }
34
35    public String getNoConta() {
36      return noConta;
37    }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
38
39      public void setNoConta(String noConta) {
40        this.noConta = noConta;
41      }
42
43      public String getNomeCliente() {
44        return nomeCliente;
45      }
46
47      public void setNomeCliente(String nomeCliente) {
48        this.nomeCliente = nomeCliente;
49      }
50
51      public double getSaldo() {
52        return saldo;
53      }
54
55      public void setSaldo(double saldo) {
56        this.saldo = saldo;
57      }
58
59  }
```

NOTA Os métodos da superclasse *Conta.java*, para serem acessados a partir das subclasses *ContaCorrente.java* e *ContaPoupanca.java* devem ser públicos (*public*) ou protegidos (*protected*). Métodos *protected* podem ser acessados a partir de subclasses em uma relação de herança. Já métodos *public* podem ser acessados a partir de qualquer classe.

4.1.1.2 Implementação da subclasse ContaCorrente

A subclasse *ContaCorrente.java* herda todo o conteúdo da classe *Conta.java*, o que pode ser percebido pelo uso da instrução *extends* na sua declaração. Veja a seguir o código-fonte desta subclasse:

```
1   public class ContaCorrente extends Conta {
2     private double limiteCheque;
3     private double juroCheque;
4
5     public ContaCorrente(double limiteCheque, double juroCheque, String noBanco,
      String noAgencia, String noConta, String nomeCliente, double saldo) {
6       super(noBanco, noAgencia, noConta, nomeCliente, saldo);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
7           this.limiteCheque = limiteCheque;
8           this.juroCheque = juroCheque;
9       }
10
11      public ContaCorrente() {
12      }
13
14      public double getLimiteCheque() {
15          return limiteCheque;
16      }
17
18      public void setLimiteCheque(double limiteCheque) {
19          this.limiteCheque = limiteCheque;
20      }
21
22      public double getJuroCheque() {
23          return juroCheque;
24      }
25
26      public void setJuroCheque(double juroCheque) {
27          this.juroCheque = juroCheque;
28      }
29  }
```

A instrução *extends* na linha 1 indica que a subclasse *ContaCorrente.java* herda o conteúdo da classe *Conta.java*.

Note, nas linhas de 5 à 9, o construtor da subclasse *ContaCorrente.java*. Ele recebe como parâmetro os valores necessários para os atributos desta subclasse (*limiteCheque* e *juroCheque*) e também valores para os atributos da superclasse *Conta.java* (*noBanco*, *noAgencia, noConta, nomeCliente* e *saldo*). Os valores para os atributos desta subclasse são incluídos nos atributos nas linhas 7 e 8. Já os valores para os atributos da superclasse são enviados ao construtor da superclasse por meio da instrução *super*, na linha 6.

Nas linhas 11 e 12 há um segundo construtor, que não recebe parâmetros. Este construtor poderá ser usado para criar um objeto desta subclasse vazio e depois, se necessário, incluir valores por meio dos métodos *setter*.

NOTA Em uma subclasse, sempre que você quiser acessar métodos da superclasse, basta digitar a palavra *super* seguida do ponto final e do nome do método. Por exemplo: *super.getNomeCliente();* Vale lembrar que o método deve permitir acesso, ou seja, deve ser público (*public*) ou protegido (*protected*).

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.1.1.3 IMPLEMENTAÇÃO DA SUBCLASSE CONTAPOUPANCA

A subclasse *ContaPoupanca.java* herda todo o conteúdo da classe *Conta.java*, o que também pode ser percebido pelo uso da instrução *extends* na sua declaração. Veja a seguir o código-fonte desta subclasse:

```
1   import java.util.Date;
2
3   public class ContaPoupanca extends Conta {
4     private Date dataAniversario;
5
6     public ContaPoupanca(Date dataAniversario, String noBanco, String noAgencia,
      String noConta, String nomeCliente, double saldo) {
7       super(noBanco, noAgencia, noConta, nomeCliente, saldo);
8       this.dataAniversario = dataAniversario;
9     }
10
11    public ContaPoupanca() {
12    }
13
14    public Date getDataAniversario() {
15      return dataAniversario;
16    }
17
18    public void setDataAniversario(Date dataAniversario) {
19      this.dataAniversario = dataAniversario;
20    }
21  }
```

As linhas de 6 à 9 correspondem ao construtor da subclasse *ContaPoupanca.java*. Ele recebe como parâmetro o valor necessário para o atributo desta subclasse (*dataAniversario*) e também valores para os atributos da superclasse *Conta.java* (*noBanco, noAgencia, noConta, nomeCliente* e *saldo*). O valor para o atributo desta subclasse é incluido no atributo *dataAniversario*. Já os valores para os atributos da superclasse são enviados ao construtor da superclasse por meio da instrução *super*, na linha 7.

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.1.1.4 Implementação da classe Principal

A classe *Principal.java* é por onde instanciaremos objetos das classes envolvidas na relação de herança.

Vamos instanciar, em um primeiro momento, apenas um objeto da subclasse *ContaCorrente.java* e entender como ocorre a instância. Para isso, digite na classe *Principal.java*, o seguinte código-fonte:

```
1    public class Principal {
2       public static void main(String[] args) {
3          ContaCorrente ctCor2;
4          ctCor2 = new ContaCorrente(10000.00, 6.5, "035", "143-2", "01435-9",
           "Evandro", 32789.50);
5       }
6    }
```

Veja na classe *ContaCorrente.java*, que há um construtor que recebe como parâmetro o *limiteCheque*, o *juroCheque*, o *noBanco*, o *noAgencia*, o *noConta*, o *nomeCliente* e o *saldo*. A linha 4 instancia o objeto *ctCor2* por meio de uma chamada a este construtor feita pela instrução *new*.

Na instância do objeto *ctCor2* foram passados como parâmetro diversos valores ao construtor da classe *ContaCorrente.java*. O valor 10000.00 foi passado para a variável *limiteCheque*; o valor 6.5 foi passado para a variável *juroCheque*; o valor "035" foi passado para a variável *noBanco*; o valor "143-2" foi passado para a variável *noAgencia*; o valor "01435-9" foi passado para a variável *noConta*; o valor "Evandro" foi passado para a variável *nomeCliente*; e o valor 32789.50 foi passado para a variável *saldo*.

No construtor da classe *ContaCorrente.java* o conteúdo das variáveis *noBanco, noAgencia, noConta, nomeCliente* e *saldo* são passados para o construtor da superclasse *Conta.java*, por meio da instrução *super*. Já o conteúdo das variáveis *limiteCheque* e *juroCheque* são armazenados nos atributos da classe *ContaCorrente.java*. A Figura 4.3 representa a passagem dos parâmetros na instância do objeto.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 4.3: Instância de objetos da subclasse em uma relação de herança simples.

Basicamente, quando você instancia um objeto da subclasse você passa um conjunto de parâmetros para o construtor desta subclasse. Os parâmetros que são da superclasse são passados ao construtor da superclasse por meio da instrução *super* e os parâmetros que são da subclasse são incluidos nos atributos da subclasse. Os parâmetros que foram passados ao construtor da superclasse são incluídos nos atributos da superclasse no corpo do construtor. O objeto *ctCor2* terá, após sua instância, um formato que pode ser representado pela Figura 4.4:

UNINOVE – USO EXCLUSIVO PARA ALUNO

ctCor2

limiteCheque	10000.00
juroCheque	6.5
noBanco	035
noAgencia	143-2
noConta	01435-9
nomeCliente	Evandro
saldo	32789.50
	setLimiteCheque(double limiteCheque): void
	getLimiteCheque(): double
	setJuroCheque(double juroCheque): void
	getJuroCheque(): double
	setNoBanco(String noBanco): void
	getNoBanco(): String
	setNoAgencia(String noAgencia): void
	getNoAgencia(): String
	setNoConta(String noConta): void
	getNoConta(): String
	setNomeCliente(String nomeCliente): void
	getNomeCliente(): String
	setSaldo(double saldo): void
	getSaldo(): double

Figura 4.4: Formato do objeto ctCor2.

Vamos alterar o código-fonte da classe *Principal.java* instanciando, além do objeto *ctCor2* da subclasse *ContaCorrente.java*, um objeto *ctPoup* da subclasse *ContaPoupanca.java*. Em seguida, vamos exibir o conteúdo dos dois objetos por meio de chamadas aos métodos *getter*.

```
1   import java.util.Date;
2   public class Principal {
3      public static void main(String[] args) {
4         ContaCorrente ctCor2;
5         ContaPoupanca ctPoup;
6         Date data;
7         data = new Date("07/10/2014");
8         ctCor2 = new ContaCorrente(10000.00, 6.5, "035", "143-2", "01435-9",
          "Evandro", 32789.50);
9         ctPoup = new ContaPoupanca(data, "022", "347-2", "003491-1", "Ana Luiza",
          18976.40);
10        System.out.println("Banco:" + ctCor2.getNoBanco() + "\n"
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
                + "Agência:" + ctCor2.getNoAgencia() + "\n"
                + "Conta:" + ctCor2.getNoConta() + "\n"
                + "Cliente:" + ctCor2.getNomeCliente() + "\n"
                + "Saldo:" + ctCor2.getSaldo() + "\n"
                + "Limite:" + ctCor2.getLimiteCheque() + "\n"
                + "Juros do Cheque especial:" + ctCor2.getJuroCheque() + "\n");
11          System.out.println("Banco:" + ctPoup.getNoBanco() + "\n"
                + "Agência:" + ctPoup.getNoAgencia() + "\n"
                + "Conta:" + ctPoup.getNoConta() + "\n"
                + "Cliente:" + ctPoup.getNomeCliente() + "\n"
                + "Saldo:" + ctPoup.getSaldo() + "\n"
                + "Data de aniversário da conta:" + ctPoup.getDataAniversario());
12      }
13  }
```

Observe na linha 7 que foi instanciado um objeto da classe *Date* onde foi armazenada a data de "07/10/2014" neste objeto. Está será a data de aniversário da conta poupança.

Na linha 8 foi instanciado um objeto da subclasse *ContaCorrente.java*, que já estudamos anteriormente, e na linha 9 foi instanciado um objeto da classe *ContaPoupanca.java*.

Na linha 10 o conteúdo dos atributos do objeto *ctCor2* é obtido por meio de chamadas aos métodos *getter* correspondentes e exibidos na tela. Já na linha 11, o conteúdo dos atributos do objeto *ctPoup* também é obtido por meio de chamadas aos métodos *getter* e exibidos na tela.

Quando você compilar e executar o projeto por meio do pressionamento da tecla *F6*, será apresentado na tela o seguinte resultado:

```
Banco:035
Agência:143-2
Conta:01435-9
Cliente:Evandro
Saldo:32789.5
Limite:10000.0
Juros do Cheque especial:6.5

Banco:022
Agência:347-2
Conta:003491-1
Cliente:Ana Luiza
Saldo:18976.4
Data de aniversário da conta:Thu Jul 10 00:00:00 BRT 2014
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Para alterar o conteúdo dos objetos podem ser feitas chamadas aos respectivos métodos *setter*. Veja, por exemplo, como poderiam ser incluídos novos valores no objeto *ctCor2* após sua criação na classe *Principal.java*.

```
ctCor2.setNoBanco("546");
ctCor2.setNoConta("6758-9");
ctCor2.setNoAgencia("01564-2");
ctCor2.setNomeCliente("Marco Antônio");
ctCor2.setSaldo(23456.65);
ctCor2.setJuroCheque(5.9);
ctCor2.setLimiteCheque(1200.5);
```

Para inserir valores no objeto *ctPoup* você pode chamar os métodos *setter* correspondentes de forma semelhante à inclusão que fizemos no objeto *ctCor2*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.1.2 Superclasses e subclasses nas relações de herança

Subclasses são classes que herdam da superclasse. A herança é estabelecida pela instrução *extends* na declaração das subclasses. As subclasses herdam das superclasses atributos e métodos como se estivessem dentro das próprias subclasses.

No exemplo que estamos utilizando, *ContaCorrente.java* e *ContaPoupanca.java* são tipos de *Conta.java*, logo, objetos do tipo *ContaCorrente.java* e *ContaPoupanca.java* são também objetos do tipo *Conta.java*.

Veja o exemplo da instância de dois objetos *ctCor3* e *ctCor4*:

```
Conta ctCor3 = new ContaCorrente(10000.00, 6.5, "035","143-2","01435-9",
"Evandro",32789.50);

ContaCorrente ctCor4 = new ContaCorrente(10000.00, 6.5, "035","143-2","01435-9",
"Evandro",32789.50);
```

Note que *ctCor3* é um objeto da superclasse *Conta.java* criado a partir de uma chamada ao construtor da subclasse *ContaCorrente.java*. Isso é possível porque um objeto da superclasse também é um objeto de sua subclasse. Já *ctCor4* é um objeto da subclasse *ContaCorrente.java* criado a partir de uma chamada ao construtor desta subclasse. A diferença entre os dois objetos criados é que por meio do objeto *ctCor3* não será possível acessar os métodos da subclasse *ContaCorrente.java*, já que ele foi declarado explicitamente como um objeto da superclasse *Conta.java* e só poderá acessar os métodos desta superclasse. Já o objeto *ctCor3*, por ser declarado como objeto da subclasse *ContaCorrente.java*, permitirá o acesso aos métodos da classe *Conta.java* e da subclasse *ContaCorrente.java*.

4.1.3 HERANÇA EM CADEIA

Uma subclasse pode ser superclasse de outras subclasses, criando relações de herança em cadeia. A Figura 4.5 mostra o diagrama de classes de uma relação de herança em cadeia.

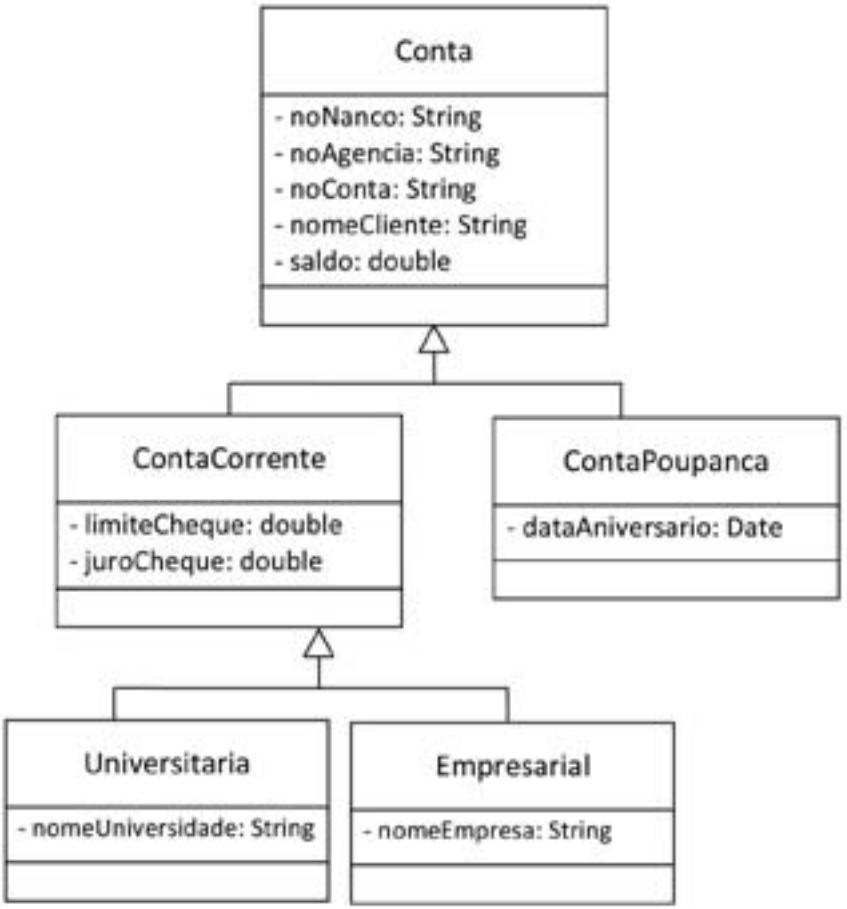


Figura 4.5: Diagrama de classes representando cadeia de herança.

Note, neste caso, que *ContaCorrente.java* é subclasse da superclasse *Conta.java* e superclasse das subclasses *Universitaria.java* e *Empresarial.java*.

As subclasses *Universitaria.java* e *Empresarial.java* herdam toda a cadeia de herança, ou seja, herdam os atributos e métodos de *ContaCorrente* e também de *Conta.java*. Veja como é feita a declaração destas classes:

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
public class Conta{

}

public class ContaCorrente extends Conta{

}

public class Universitaria extends ContaCorrente {

}

public class Empresarial extends ContaCorrente {

}

public class ContaPoupanca Extends Conta {

}
```

Você ainda poderia ter mais subclasses herdando de *Empresarial.java* e *Universitaria.java*.

4.1.4 Conseguindo polimorfismo com herança

Objetos das subclasses em uma relação de herança são também considerados objetos da superclasse. Quando por meio destes objetos se acessa um mesmo método em subclasses diferentes e se obtém resultados diferentes, podemos dizer que estes objetos possuem formas diferentes, o que nos remete ao conceito de polimorfismo.

Este é um conceito complexo na orientação a objetos, que se obtém apenas quando se utiliza herança, interfaces ou classes abstratas.

Para entender como ocorre o polimorfismo nas relações de herança, vamos criar uma pequena aplicação Java cujas classes são representadas no diagrama mostrado na Figura 4.6.

UNINOVE – USO EXCLUSIVO PARA ALUNO

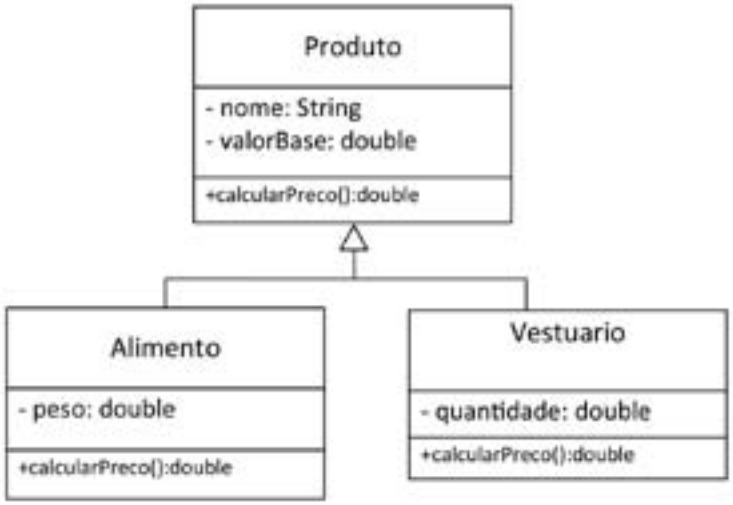


Figura 4.6: Diagrama de classes de aplicação exemplo.

Note que as subclasses *Alimento.java* e *Vestuario.java* herdam os atributos e métodos da superclasse *Produto.java*. Apesar de herdarem o método *calcularPreco* da superclasse, as subclasses reescrevem este método para que possam calcular o preço de maneira especializada, ou seja, o cálculo do preço dos alimentos será diferente do calculo do preço dos produtos de vestuário. Aqui ocorre o que conhecemos como sobrescrita de métodos, pois um método da superclasse é reescrito com a mesma assinatura nas subclasses, porém, com corpo diferente.

Vamos criar um novo projeto no *NetBeans* para implementar esta aplicação.

Crie um novo projeto chamado *ProjetoPoli* e, nele, crie as classes mostradas na Figura 4.7.

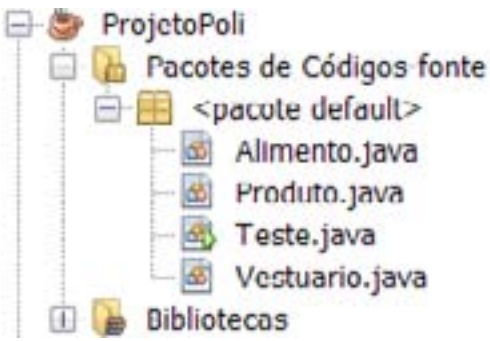


Figura 4.7: representação dos componentes do "ProjetoPoli" no NetBeans.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Escreva o seguinte código-fonte para a superclasse *Produto.java*:

```
1   public class Produto {
2
3       private String nome;
4       private double valorBase;
5
6       public Produto(String nome, double valorBase) {
7           this.nome = nome;
8           this.valorBase = valorBase;
9       }
10
11      public String getNome() {
12          return nome;
13      }
14
15      public void setNome(String nome) {
16          this.nome = nome;
17      }
18
19      public double getValorBase() {
20          return valorBase;
21      }
22
23      public void setValorBase(double valorBase) {
24          this.valorBase = valorBase;
25      }
26
27      public double calcularPreco() {
28          return getValorBase();
29      }
30  }
```

Note que nesta classe o método *calcularPreco* apenas retorna o conteúdo do atributo *valorBase*, por meio de uma chamada ao método *getValorBase*.

Para a subclasse *Alimento.java*, escreva o código-fonte a seguir:

```
1   public class Alimento extends Produto {
2
3       private double peso;
4
5       public Alimento(double peso, String nome, double valorBase) {
6           super(nome, valorBase);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
7               this.peso = peso;
8           }
9
10          public double getPeso() {
11              return peso;
12          }
13
14          public void setPeso(double peso) {
15              this.peso = peso;
16          }
17
18          @Override
19          public double calcularPreco() {
20              return super.getValorBase() * this.peso;
21          }
22      }
```

O método *calcularPreco* é herdado da superclasse *Produto.java*, porém, é reescrito nas linhas de 18 à 21, retornando o conteúdo do atributo *valorBase* a partir de uma chamada ao método *getValorBase* da superclasse *Produto.java* multiplicado pelo conteúdo do atributo *peso* desta subclasse. Apesar do método ter o mesmo nome do método da superclasse *Produto.java*, sua implementação nesta subclasse é diferente. A indicação que o método foi sobrescrito ocorre pela presença da anotação *@Override*, que será vista em detalhes no final deste Capítulo.

Para a subclasse *Vestuario.java*, escreva o código-fonte a seguir:

```
1   public class Vestuario extends Produto{
2       private double quantidade;
3
4       public Vestuario(double quantidade, String nome, double valorBase) {
5           super(nome, valorBase);
6           this.quantidade = quantidade;
7       }
8
9       public double getQuantidade() {
10          return quantidade;
11      }
12
13      public void setQuantidade(double quantidade) {
14          this.quantidade = quantidade;
15      }
16
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
17          @Override
18              public double calcularPreco(){
19              return (super.getValorBase() * this.quantidade) – super.getValorBase() *
                20/100;
20          }
21
22      }
```

O método *calcularPreco* é herdado da superclasse, porém, é reescrito nas linhas de 17 à 20, retornando o conteúdo do atributo *valorBase* multiplicado pelo conteúdo do atributo quantidade menos 20% do *valorBase*. Apesar do método ter o mesmo nome do método da superclasse *Produto.java*, sua implementação nesta superclasse é diferente.

Para iniciar (startar) a aplicação e manipular objetos, precisamos da classe *Teste.java*, cujo código-fonte é mostrado a seguir:

```
1       public class Teste {
2           public static void main(String[] args) {
3               Produto p1 = new Alimento(10.0, "Arroz", 10.5);
4               Produto p2 = new Vestuario(10.0, "Camisa", 10.5);
5               System.out.print("Preço de " + p1.getNome() + " é igual a " +
                p1.calcularPreco() + "\n");
6               System.out.print("Preço de " + p2.getNome() + " é igual a " +
                p2.calcularPreco());
7           }
8       }
```

Ao compilar e executar este projeto, o que pode ser feito pelo pressionamento da tecla *F6*, será exibido na tela os seguintes valores:

```
Preço de Arroz é igual a 105.0
Preço de Camisa é igual a 102.9
```

Note que foram instanciados dois objetos *p1* e *p2* (linhas 3 e 4), ambos do tipo *Produto.java*, porém, com chamadas ao construtor de subclasses diferentes. O objeto *p1* foi instanciado por meio de uma chamada ao construtor da subclasse *Alimento.java* e o objeto *p2* por meio uma chamada ao construtor da subclasse *Vestuario.java*. Em ambas as instâncias, foram passados os mesmos valores numéricos utilizados para calcular o preço do produto.

Na linha 5 é mostrado o nome do produto contido no atributo *nome* do objeto *p1* e o preço, calculado por meio de uma chamada ao método *calcularPreco*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Na linha 6 é mostrado o nome do produto contido no atributo *nome* do objeto *p2* e o preço, calculado também por meio de uma chamada ao método *calcularPreco*.

Note que apesar dos dois objetos serem do tipo da superclasse *Produto.java* e chamarem o método *calcularPreco*, foram retornados valores diferentes, já que a operação contida no método em cada uma das subclasses é diferente. Logo, podemos concluir que os objetos *p1* e *p2*, apesar de serem do mesmo tipo, apresentam formas diferentes. Esta situação, que pode ocorrer em aplicações que utilizam herança, é conhecida como polimorfismo, onde *poli* significa muitas e *morfirmo* significa formas.

4.2 Interfaces

Uma interface, na orientação a objetos, é um tipo de classe que contém apenas métodos abstratos, ou seja, métodos que possuem apenas a assinatura, sem corpo. Podem também possuir constantes. A implementação (construção) dos membros é feita por uma classe concreta que implementa a interface.

Segundo Horstmann (2009, p. 361), "um tipo interface em Java declara um conjunto de métodos e suas assinaturas e permite tornar o código da aplicação mais reutilizável".

Para declarar uma classe como interface utiliza-se no lugar da instrução *class* a instrução *interface*. Veja o exemplo a seguir:

```
public interface Operacoes{

}
```

Nas interfaces, os métodos não podem ter corpo, ou seja, devem ter apenas uma assinatura (declaração).

Veja o exemplo da interface chamada *Operacoes.java* que possui a assinatura do método *calcularMedia*.

```
public interface Operacoes{
   public abstract double calcularMedia(double n1, double n2);
}
```

O método não tem corpo entre chaves e termina com ponto e vírgula. Esta forma de declaração é chamada de assinatura do método. A assinatura do método mostra de onde o método pode ser acessado, o que o método retorna, o nome do método e os parâmetros que eventualmente o método recebe, sem dizer o que o método faz. Ela mostra apenas o que é necessário saber para acessar o método.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Um método em uma interface é implicitamente público (*public*) e abstrato (*abstract*). Não podem ser privados (*private*) nem protegidos (*protected*).

Métodos abstratos são métodos sem corpo que obrigatoriamente terão que ser implementados em uma classe concreta que implementa a interface. Métodos públicos são aqueles que podem ser acessados de qualquer outra classe da aplicação. Como implicitamente todos os métodos em uma interface são públicos e abstratos, então não é necessário escrever isso explicitamente. Logo, a declaração de método acima pode ser escrita apenas da seguinte forma:

```
public interface Operacoes{
  double calcularMedia(double n1, double n2);
}
```

Ao contrário das classes simples, uma interface nunca poderá ser instanciada, por isso, não pode conter um construtor, já que o construtor é utilizado para instância de objetos. Você instancia a classe que implementa a interface, não a interface em si.

Para uma classe implementar a interface, ela deve ter, na linha da sua declaração, a instrução *implements*. Veja o exemplo:

```
public class Professor implements Operacoes {
  @Override
  public double calcularMedia(double n1, double n2) {
    return (n1+n2)/2;
  }
}
```

Veja que por implementar a classe *Operacoes.java*, a classe *Professor.java* foi OBRIGADA a implementar o método *calcularMedia*, ou seja, a construir o corpo do método.

Acima da declaração do método implementado, aparece a instrução *@Override*, que indica que o método foi sobrescrito, ou seja, sua aparição na classe concreta sobrepõe a existência do método sem corpo assinado na interface.

NOTA

Sobrescrita de método ocorre em duas situações:
1. Quando um método é escrito em uma interface ou classe abstrata e reescrito nas classes que implementam a interface ou que herdam a classe abstrata.
2. Quando um método é escrito em uma superclasse nas relações de herança simples e reescrito nas subclasses, normalmente para realizar operações diferentes.

A sobrescrita de métodos é necessária para se conseguir polimorfismo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

O diagrama de classes que mostra a relação entre a classe *Professor.java* e a interface *Operacoes.java* é mostrado na Figura 4.8:

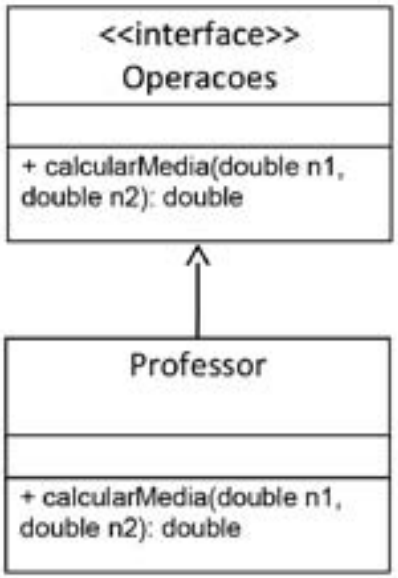


Figura 4.8: Diagrama de classe com interfaces.

Note que, para indicar que a classe implementa a interface, utilizou-se uma linha tracejada com seta apontando para a interface. Acima do nome da interface foi utilizado o estereótipo <<*interface*>>. Um estereótipo, neste caso, é uma palavra que identifica uma categoria especial de classe.

As interfaces podem ser implementadas por diversas classes. Aliás, esta é uma das grandes finalidades das interfaces, centralizar operações obrigando diversas classes que necessitam destas operações a implementá-las. Por exemplo, você pode centralizar a operação de *login* de um sistema em uma interface *Login.java* e obrigar as classes *Funcionario.java*, *Cliente.java* e *Fornecedor.java* a implementar a operação de *login* assinada na interface.

Uma classe pode implementar diversas interfaces. Por exemplo, se a classe *Professor.java* deseja implementar as interfaces *Operacoes.java* e *Aulas.java*, sua declaração deve ser:

```
public class Professor implements Operacoes, Aulas {

}
```

Neste caso, no corpo da classe *Professor.java* devem ser implementados todos os métodos assinados nas interfaces *Operacoes.java* e *Aulas.java*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA Uma interface pode definir uma série de métodos, mas nunca conter implementação deles. Ela só expõe o que seus métodos devem fazer, e não como eles fazem. Como eles fazem será definido em uma implementação dessa interface.

Além de assinaturas de métodos, interfaces também podem conter declaração de constantes, apesar de não ser muito comum. Veja um exemplo:

```
public interface NewInterface {
 public final double MED=6;
 public abstract double calcularMedia(double n1, double n2);
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.2.1 Quando utilizar interfaces

Você normalmente utiliza interfaces em duas situações:

1ª Quando deseja obrigar que um conjunto de classes de diferentes origens implemente um ou mais métodos de maneiras diferentes;

2ª Quando uma subclasse que já está herdando métodos e atributos de uma superclasse em relações de herança precisa herdar (e implementar) métodos de outro lugar. Como Java não permite herança múltipla nas versões anteriores á 8 (na versão 8 já permite), o uso de interfaces, neste caso, pode ser uma boa alternativa.

4.2.2 Conseguindo polimorfismo com utilização de interfaces

Polimorfismo é um conceito da orientação a objetos conseguido quando objetos de um mesmo tipo apresentam formas diferentes, ou seja, comportam-se de maneiras diferentes, mesmo executando a mesma operação (método).

Segundo Horstmann (2009, p. 369), "o polimorfismo indica o princípio de que o comportamento pode variar com base no tipo real de um objeto".

Se você pode invocar um método de uma interface por meio de objetos de classes concretas diferentes, isso significa que o comportamento desta execução pode variar com base no tipo real do objeto que está invocando o método.

Neste tópico, vamos montar a aplicação representada no diagrama de classes mostrado na Figura 4.9 e observar nela o conceito de polimorfismo.

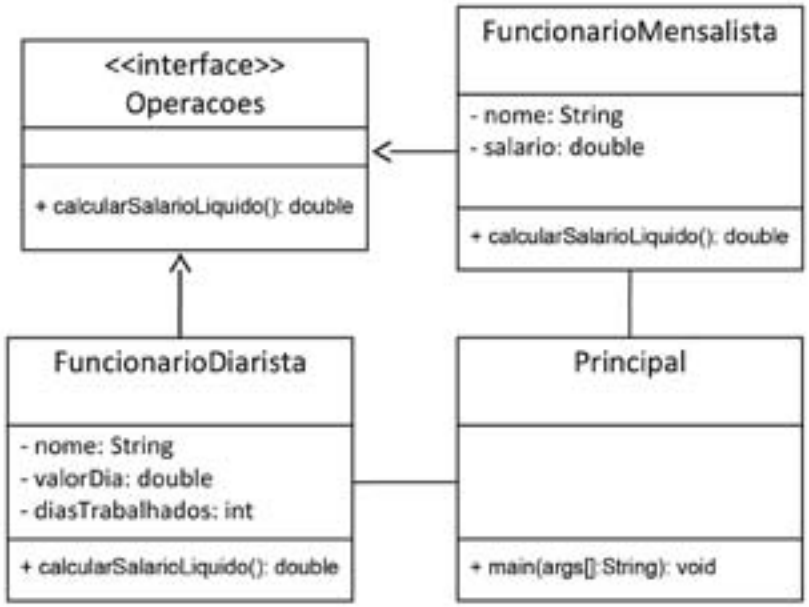


Figura 4.9: Diagrama de classes - conseguindo polimorfismo com interfaces.

Para implementarmos a aplicação mostrada na Figura 4.8, crie um novo projeto no *NetBeans* chamado *ProjetoInterface*. Neste projeto, crie uma interface chamada *Operacoes.java* e as classes *FuncionarioDiarista.java*, *FuncionarioMensalista.java* e *Principal.java*.

Se você não se recorda como cria classes e interfaces no projeto do *NetBeans*, basta clicar com o botão direito do mouse sobre o nome do projeto, selecionar a opção "Novo" e, em seguida, "Classe Java" ou "Interface Java".

Após a criação, seu projeto deve aparecer como mostra a Figura 4.10.

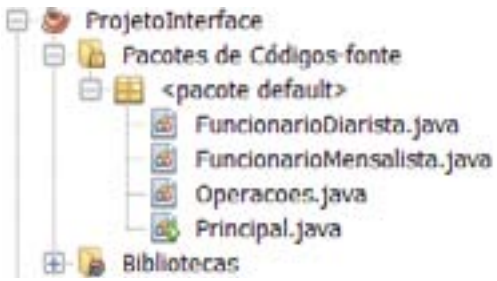


Figura 4.10: Projeto "ProjetoInterface" criado no NetBeans.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Digite o seguinte código-fonte para a interface *Operacoes.java*:

```
1   public interface Operacoes {
2     public double calcularSalarioLiquido();
3   }
```

Note que nesta interface há a assinatura de apenas um método, apesar de uma interface poder concentrar a assinatura de diversos métodos.

Digite o seguinte código-fonte para a classe *FuncionarioMensalista.java*:

```
1   public class FuncionarioMensalista implements Operacoes {
2   private String nome;
3   private double salario;
4
5       public FuncionarioMensalista(String nome, double salario) {
6           this.nome = nome;
7           this.salario = salario;
8       }
9
10      public String getNome() {
11          return nome;
12      }
13
14      public void setNome(String nome) {
15          this.nome = nome;
16      }
17
18      public double getSalario() {
19          return salario;
20      }
21
22      public void setSalario(double salario) {
23          this.salario = salario;
24      }
25
26      @Override
27      public double calcularSalarioLiquido() {
28          return this.salario – this.salario*27.5/100;
29      }
30
31  }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja na linha 1 que esta classe implementa a interface *Operacoes.java*. Isso faz com que o método *calcularSalarioLiquido* tenha que ser obrigatoriamente implementado, ou seja, o método terá que ser reescrito, mas dessa vez, com corpo. A implementação ocorre nas linhas de 26 à 29. Este método retorna o salário diminuído do percentual de 27.5%. Note na linha 26 a instrução *@ Override,* que indica que o método existe na interface e está sendo sobrescrito na classe que implementa a interface.

Os demais aspectos desta classe são comuns. Há atributos declarados, um construtor que recebe parâmetros e métodos *setter* e *getter* para os atributos.

Digite o seguinte código-fonte para a classe *FuncionarioDiarista.java*:

```
1   public class FuncionarioDiarista implements Operacoes{
2     private String nome;
3     private double valorDia;
4     private int diasTrabalhados;
5
6   public FuncionarioDiarista(String nome, double valorDia, int diasTrabalhados) {
7         this.nome = nome;
8         this.valorDia = valorDia;
9         this.diasTrabalhados = diasTrabalhados;
10      }
11
12      public String getNome() {
13         return nome;
14      }
15
16      public void setNome(String nome) {
17         this.nome = nome;
18      }
19
20      public double getValorDia() {
21         return valorDia;
22      }
23
24      public void setValorDia(double valorDia) {
25         this.valorDia = valorDia;
26      }
27
28      public int getDiasTrabalhados() {
29         return diasTrabalhados;
30      }
31
32      public void setDiasTrabalhados(int diasTrabalhados) {
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
33          this.diasTrabalhados = diasTrabalhados;
34      }
35
36      @Override
37      public double calcularSalarioLiquido() {
38          return valorDia*diasTrabalhados;
39      }
40
41  }
```

Note na linha 1 que esta classe implementa a interface *Operacoes.java*. Isso obriga a reescrita do método *calcularSalarioLiquido*, agora com corpo, o que é feito nas linhas de 36 à 39. Este método retorna o valor pago pela diária multiplicado pelo número de dias trabalhados pelo funcionário.

Os demais aspectos da classe são comuns. Veja que são declarados os atributos nas linhas de 2 à 4, definido um construtor que recebe parâmetros (linhas 6 à 10) e definidos os métodos *setter* e *getter* para os atributos (linhas 12 à 34).

Agora que temos as classes implementando a interface, precisamos manipular dados dos funcionários por meio de objetos das classes da aplicação. Isto será feito a partir da classe principal de inicialização da aplicação, chamada *Principal.java*, cujo código-fonte é apresentado a seguir:

```
1       import javax.swing.JOptionPane;
2
3       public class Principal {
4
5           public static void main(String[] args) {
6               String nom;
7               double sal, valDia, valLiquido;
8               int diasTrab;
9               nom = JOptionPane.showInputDialog("Digite o nome do funcionário
                mensalista");
10              sal = Double.parseDouble(JOptionPane.showInputDialog("Digite o valor do
                salário"));
11              FuncionarioMensalista fm = new FuncionarioMensalista(nom, sal);
12              valLiquido = fm.calcularSalarioLiquido();
13              System.out.println("Funcionário: " + fm.getNome() + "\nValor líquido a
                receber: " + valLiquido);
14              nom = JOptionPane.showInputDialog("Digite o nome do funcionário
                Diarista");
15              valDia = Double.parseDouble(JOptionPane.showInputDialog("Digite o valor
                do dia trabalhado"));
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
16              diasTrab = Integer.parseInt(JOptionPane.showInputDialog("Digite o número
                de dias trabalhados no mês"));
17              FuncionarioDiarista fd = new FuncionarioDiarista(nom, valDia, diasTrab);
18              valLiquido = fd.calcularSalarioLiquido();
19              System.out.println("Funcionário: " + fd.getNome() + "\nValor líquido a
                receber: " + valLiquido);
20          }
21      }
```

Note nas linhas 9 e 10 que são pedidos ao usuário o nome e o salário e estes valores são armazenados, respectivamente, nas variáveis *nom* e *sal*. Na linha 11 é instanciado um objeto chamado *fm* do tipo *FuncionarioMensalista.java*, por meio de uma chamada ao construtor desta classe. Veja que este objeto conterá em seus atributos o nome e o salário do funcionário. Digamos que para nome seja digitado "Maria" e para salário seja digitado 1000.00, este objeto terá o formato mostrado na Figura 4.11.

UNINOVE – USO EXCLUSIVO PARA ALUNO

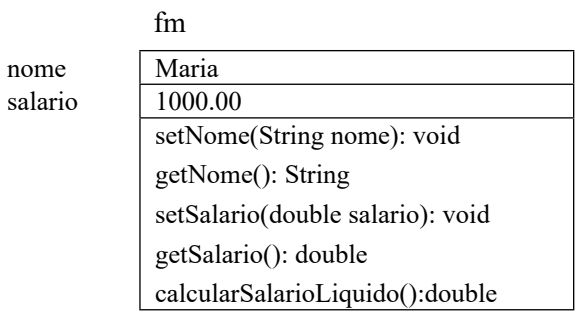


Figura 4.11: Representação do formato do objeto *fm*.

Na linha 12 o método *calcularSalarioLiquido* é chamado por meio do objeto *fm*. O retorno deste método é armazenado na variável *valLiquido* que é exibida na tela na linha 13. A variável *valLiquido* receberá o valor 725.0, pois o método *calcularSalarioLiquido* retornará o salário digitado reduzido de 27.5%.

As linhas de 14 à 16 solicitam o nome do funcionário diarista, o valor pago pelo dia trabalhado e o número de dias trabalhados no mês. Os valores digitados são armazenados, respectivamente, nas variáveis *nom*, *valDia* e *diasTrab*.

Na linha 17 é instanciado um objeto chamado *fd* do tipo *FuncionarioDiarista.java*, por meio de uma chamada ao construtor desta classe, onde os valores armazenados nas variáveis *nom*, *valDia* e *diasTrab* são passados como parâmetro. Digamos que sejam digitados "João" para nome, 100.00 para valor

do dia trabalhado e 10 para dias trabalhados no mês, este objeto terá o formato mostrado na Figura 4.12.

fd

nome	João
valorDia	100.00
diasTrabalhados	10
	setNome(String nome): void
	getNome(): String
	setValorDia(double valorDia): void
	getValorDia(): double
	setDiasTrabalhados(int diasTrabalhados): void
	getDiasTrabalhados(): int
	calcularSalarioLiquido():double

Figura 4.12: representação do formato do objeto *fd*.

Na linha 18 é chamado o método *calcularSalarioLiquido* por meio do objeto *fd*. O resultado retornado é armazenado na variável *valLiquido* e exibido na linha 19. Veja que a variável *valLiquido*, neste caso, receberá o valor 1000.00, pois o método *calcularSalarioLiquido* retorna o conteúdo do atributo *valorDia* multiplicado pelo conteúdo do atributo *diasTrabalhados*.

Após compilar e executar a aplicação, o que pode ser feito por meio do pressionamento da tecla *F6*, serão exibidos na tela os seguintes dados:

```
Funcionário: Maria
Valor líquido a receber: 725.0
Funcionário: João
Valor líquido a receber: 1000.0
```

Observe que os objetos *fm* da classe *FuncionarioMensalista.java* e *fd* da classe *FuncionarioDiarista.java* são também objetos dos tipos da interface *Operacoes.java*, pois estas classes implementam tal interface. Assim, podemos dizer que *fm* e *fd* são objetos do mesmo tipo de interface, o tipo *Operacoes.java*. Estes objetos, ao chamarem o método *calcularSalarioLiquido,* obtiveram como retorno valores diferentes, pois este método executa operações diferentes em cada uma das classes. Quando objetos do mesmo tipo de interface executam uma mesma operação (método) e obtém resultados diferentes, dizemos que houve ***polimorfismo*** (*poli*=muitas; *morfismo*=formas). Os objetos do mesmo tipo de interface, neste caso, apresentaram mais de uma forma diferente.

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA Polimorfismo é o conceito da orientação a objetos que ocorre quando objetos do mesmo tipo de interface ou de subclasses da mesma superclasse nas relações de herança executam uma mesma operação e obtêm resultados diferentes, pois as operações (métodos) possuem formas diferentes. Isto significa que polimorfismo é conseguido apenas com o uso de herança, implementação de interfaces ou de classes abstratas.

4.2.3 IMPLEMENTAÇÃO DE MAIS DE UMA INTERFACE

Uma classe pode implementar diversas interfaces, desde que escreva o corpo dos métodos assinados de todas estas interfaces. Para demostrar como isso é possível, vamos criar uma pequena aplicação com as classes mostradas no diagrama a seguir:

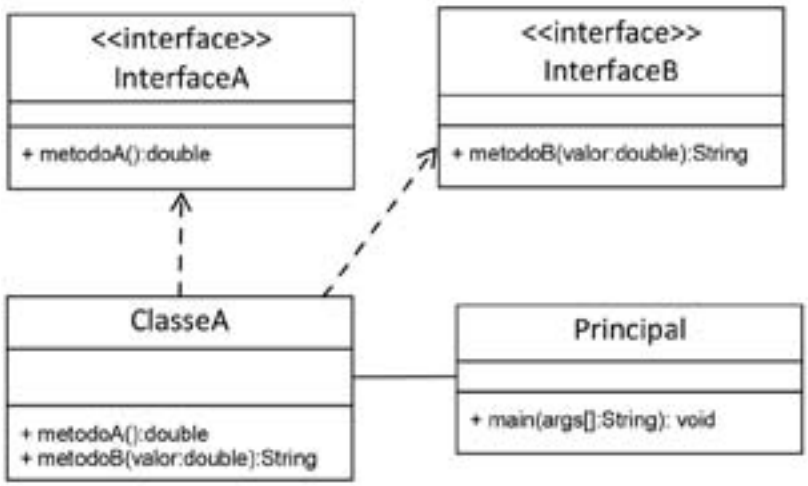


Figura 4.13: Implementação de múltiplas interfaces.

Crie um novo projeto Java no *NetBeans* chamado *ProjetoNovo*. Crie neste projeto as interfaces *InterfaceA.java* e *InterfaceB.java*. Crie também as classes *ClasseA.java* e *Teste.java* (classe principal). A Figura 4.14 mostra como deve estar o projeto após a criação das classes e interfaces.

UNINOVE – USO EXCLUSIVO PARA ALUNO

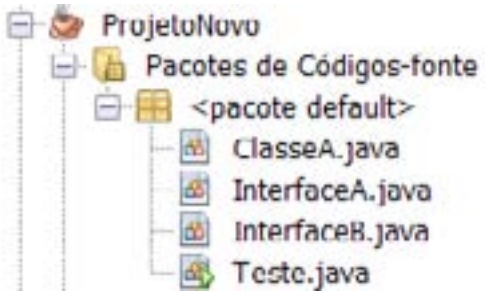


Figura 4.14: Projeto no NetBeans utilizando classes e interfaces.

Digite o seguinte código-fonte para a interface *InterfaceA.java*:

```
1   public interface InterfaceA {
2       public double metodoA();
3   }
```

Digite o seguinte código-fonte para a InterfaceB.java:

```
1       public interface InterfaceB {
2           public String metodoB(double valor);
3       }
```

Digite o seguinte código-fonte para a ClasseA.java:

```
1   public class ClasseA implements InterfaceA, InterfaceB {
2       @Override
3       public double metodoA() {
4           return 10.5;
5       }
6
7       @Override
8       public String metodoB(double valor) {
9           if (valor <= 10) {
10              return "Valor válido";
11          } else {
12              return "Valor inválido";
13          }
14      }
15  }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Note que a *ClasseA.java* implementou as interfaces *InterfaceA.java* e *InterfaceB.java*, logo, foi obrigada a reescrever (com corpo) os métodos *metodoA* e *metodoB* de acordo com a assinatura presente nas interfaces.

Digite o seguinte código-fonte para a *Teste.java*:

```
1    public class Teste {
2        public static void main(String[] args) {
3            ClasseA c = new ClasseA();
4            double v = c.metodoA();
5            String x = c.metodoB(10);
6            System.out.println(x);
7            System.out.println(v);
8        }
9    }
```

A classe *Teste.java* é a classe principal, por onde foi instanciado um objeto da *ClasseA.java* (linha 3) e por meio deste objeto foram acessados os métodos *metodoA* (linha 4) e *metodoB* (linha 5). O retorno destes métodos foram armazenados em variáveis e exibido em seguida (linhas 6 e 7).

NOTA Nas relações de herança uma subclasse não pode herdar de mais de uma superclasse. Já nas relações com interfaces, uma classe pode implementar métodos de diversas interfaces, sendo obrigada a implementá-los.

4.3 Classes Abstratas

Diferente das interfaces, que podem conter apenas assinaturas de métodos e constantes, uma classe abstrata pode conter assinaturas de métodos, métodos implementados, constantes, variáveis, construtores e atributos. Métodos que contém apenas assinatura são chamados métodos abstratos e nas classes abstratas precisam ser declarados com a palavra-chave *abstract*. Assim como acontece com as interfaces, uma classe abstrata não pode ser instanciada diretamente, mas apenas suas subclasses. Deve possuir na sua declaração a instrução *abstract*, da seguinte forma:

```
public abstract class Operacoes {
    public abstract void conectar();
    public abstract void desconectar();
    public void sair(){
     System.exit(0);
   }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja neste exemplo que há dois métodos abstratos (que contém apenas a assinatura) e um método concreto (implementado).

Classes abstratas, assim como as interfaces, servem apenas como modelo para classes concretas. Como são modelos de classes, não podem ser instanciadas diretamente com a instrução *new*, mas podem ser herdadas por classes concretas (subclasses). Uma classe pode herdar uma classe abstrata da mesma forma que uma subclasse herda de uma superclasse nas relações de herança simples, utilizando a instrução *extends*. Veja o exemplo a seguir:

```
public class ControleCliente extends Operacoes {
   @Override
   public void conectar() {
      // a implementação do método deve ocorrer aqui
   }

   @Override
   public void desconectar() {
      // a implementação do método deve ocorrer aqui
   }
}
```

Note que os métodos abstratos da classe abstrata *Operacoes.java* tiveram de ser implementados na subclasse *ControleCliente.java*, que herda a classe abstrata.

NOTA Uma classe abstrata é semelhante a uma superclasse nas relações de herança simples, com a grande diferença de poder ter assinaturas de métodos, obrigando as subclasses a implementá-los, o que não ocorre nas relações de herança simples.

4.3.1 Exemplo de aplicação com classes abstratas

Talvez a grande dúvida sobre o uso das classes abstratas seja quando utilizá-las. Imagine uma aplicação onde você precisa de uma classe que contenha atributos e/ou métodos comuns que queira compartilhar com outras classes. Imagine também que tenha métodos que queira obrigar as subclasses a implementar de formas diferentes. Neste caso, os métodos comuns às subclasses são declarados e escritos (implementados) na classe abstrata e também a assinatura dos métodos que você quer que as subclasses sejam obrigadas a implementar. A diferença do uso de classe abstrata em relação à herança simples é que na herança simples você não obriga as subclasses a implementar nada, pois não são permitidos métodos abstratos nas superclasses.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Outra diferença é que na herança simples você pode instanciar objetos da superclasse, o que não pode ocorrer com classes abstratas.

Neste tópico vamos criar uma pequena aplicação, representada no diagrama de classes mostrado na Figura 4.15.

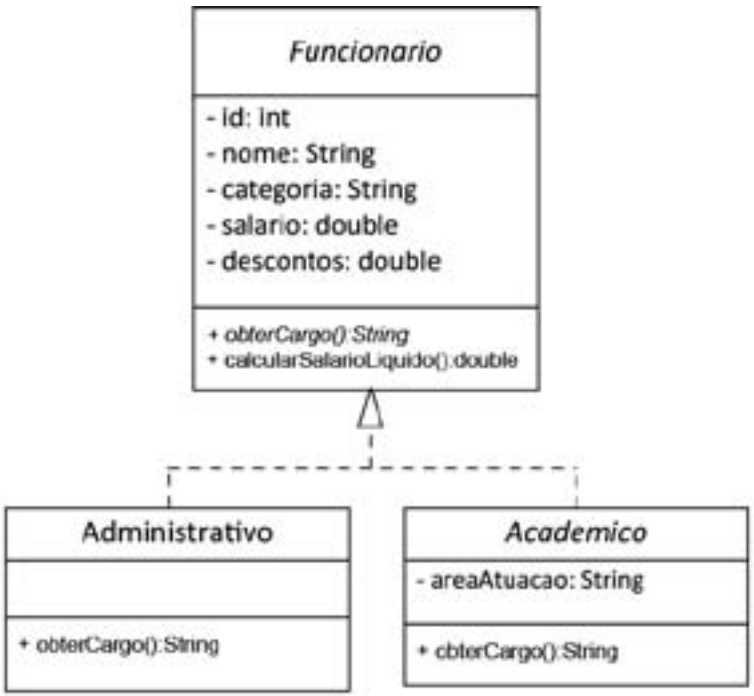


Figura 4.15: Diagrama de classes representando classes abstratas.

Neste exemplo, na classe abstrata *Funcionario.java*, teremos atributos e métodos. O método *calcularSalarioLiquido* é um método cuja implementação é a mesma para as subclasses *Administrativo.java* e *Academico.java*, por isso, será implementado na própria classe abstrata *Funcionario.java*. Já o método *obterCargo.java* terá implementação diferente nas subclasses *Administrativo.java* e *Academico.java*, por isso será abstrato e terá apenas a assinatura na classe *Funcionario.java*, obrigando as subclasses a implementá-lo.

Note que no geral parece uma relação de herança comum, porém há diferenças. Mesmo no digrama de classes, veja que o nome da classe abstrata aparece em itálico, e as linhas que ligam as subclasses à classe abstrata é pontilhada.

Para iniciarmos o exemplo mostrado no diagrama, crie um projeto Java no *NetBeans* chamado *ProjetoAbs* com as classes mostradas na Figura 4.16.

UNINOVE – USO EXCLUSIVO PARA ALUNO

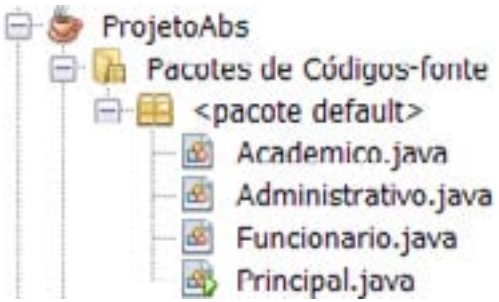


Figura 4.16: Projeto no NetBeans chamado "ProjetoAbs".

Observe que além das classes mostradas no diagrama de classes precisaremos de uma classe principal por onde instanciaremos objetos das subclasses *Administrativo.java* e *Academico.java* para demonstração de uso dos recursos.

Escreva o código-fonte da classe abstrata *Funcionario.java*, mostrado a seguir:

```
1   public abstract class Funcionario {
2
3       private int id;
4       private String nome;
5       private String categoria;
6       private double salario;
7       private double descontos;
8
9       public abstract String obterCargo();
10
11      public Funcionario(int id, String nome, String categoria, double
        salario, double descontos) {
12          this.id = id;
13          this.nome = nome;
14          this.categoria = categoria;
15          this.salario = salario;
16          this.descontos = descontos;
17      }
18
19      public int getId() {
20          return id;
21      }
22
23      public void setId(int id) {
24          this.id = id;
25      }
26
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
27          public String getNome() {
28              return nome;
29          }
30
31          public void setNome(String nome) {
32              this.nome = nome;
33          }
34
35          public String getCategoria() {
36              return categoria;
37          }
38
39          public void setCategoria(String categoria) {
40              this.categoria = categoria;
41          }
42
43          public double getSalario() {
44              return salario;
45          }
46
47          public void setSalario(double salario) {
48              this.salario = salario;
49          }
50
51          public double getDescontos() {
52              return descontos;
53          }
54
55          public void setDescontos(double descontos) {
56              this.descontos = descontos;
57          }
58
59          public double calcularSalarioLiquido() {
60              return salario – descontos;
61          }
62      }
```

Note a declaração dos atributos (linhas de 3 à 7), a assinatura do método abstrato (linha 9), o construtor da classe (linhas 11 à 17), os métodos *setter* e *getter* (linhas de 19 à 57) e o método *calcularSalarioLiquido* (linhas 59 à 61). Como este método será comum às subclasses, ele foi implementado subtraindo-se os descontos do valor do salário.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Escreva agora o código-fonte da subclasse *Administrativo.java*, apresentado a seguir:

```
1   public class Administrativo extends Funcionario {
2
3       public Administrativo(int id, String nome, String categoria, double
        salario, double descontos) {
4           super(id, nome, categoria, salario, descontos);
5       }
6
7       @Override
8       public String obterCargo() {
9           switch (super.getCategoria()) {
10              case "A":
11                  return "Gerente";
12              case "B":
13                  return "Contador";
14               default:
15                  return "Auxiliar";
16          }
17      }
19  }
```

Veja na linha 1 que esta classe herda a classe abstrata *Funcionario.java*. Note que, por isso, ela terá obrigatoriamente um construtor que recebe como parâmetro os valores necessários para os atributos da classe abstrata *Funcionario.java*. Estes valores são passados para o construtor da classe abstrata *Funcionario.java* por meio da instrução *super* (linha 4).

As linhas de 7 à 17 implementam obrigatoriamente o método *obterCargo*, assinado na classe abstrata. Lembre que a instrução *@Override* indica que o método está sendo sobrescrito, só que agora, com corpo.

Escreva agora o código-fonte da subclasse *Academico.java*, apresentado a seguir:

```
1   public class Academico extends Funcionario {
2
3       private String areaAtuacao;
4
5       public Academico(String areaAtuacao, int id, String nome, String
        categoria, double salario, double descontos) {
6           super(id, nome, categoria, salario, descontos);
7           this.areaAtuacao = areaAtuacao;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
8           }
9
10          public String getAreaAtuacao() {
11              return areaAtuacao;
12          }
13
14          public void setAreaAtuacao(String areaAtuacao) {
15              this.areaAtuacao = areaAtuacao;
16          }
17
18          @Override
19          public String obterCargo() {
20              switch (super.getCategoria()) {
21                  case "A":
22                      return "Pesquisador";
23                  case "B":
24                      return "Professor";
25                  default:
26                      return "Tutor";
27              }
28          }
29      }
```

Observe que esta classe herda a classe abstrata *Funcionario.java* (linha 1), o que a obriga a criar um construtor (linhas de 5 a 8) e a implementar o método *obterCargo* (linhas de 18 à 28).

Note que o método *obterCargo*, apesar de ter implementação semelhante nas subclasses *Administrativo.java* e *Academico.java*, retornam valores diferentes, por isso precisam ser implementados em cada uma das subclasses da classe abstrata.

Note também que o construtor é necessário, pois ele precisa receber parâmetros e passá-los para os atributos da classe abstrata *Funcionario.java*, o que é feito pela instrução *super* (linha 6).

Para terminar a aplicação, escreva o código-fonte da classe *Principal.java*, por onde instanciaremos os objetos das subclasse *Administrativo.java* e *Acedemico.java*. O código-fonte da classe *Principal.java* é apresentado a seguir:

```
1       public class Principal {
2
3           public static void main(String[] args) {
4               Administrativo adm = new Administrativo(10, "Paulo Cesar", "A",
                10678.56, 789.50);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
5           Academico aca = new Academico("Matemática", 12, "Maria Paula", "B",
            8432.20, 521.32);
6           System.out.println("Nome: " + adm.getNome() + "\n" + "Cargo: " + adm.
            obterCargo() + "\n" + "Salário Líquido: " + adm.calcularSalarioLiquido() +
            "\n");
7           System.out.println("Nome: " + aca.getNome() + "\n" + "Cargo: " + aca.
            obterCargo() + "\n" + "Salário Líquido: " + aca.calcularSalarioLiquido());
8         }
9
10      }
```

Observe que foram instanciados os objetos *adm*, na subclasse *Administrativo.java* (linha 4) e *aca*, da subclasse *Academico.java* (linha 5). Em seguida foram mostrados por meio de cada objeto o nome (por meio de uma chamada ao método *getNome*), o cargo (por meio de uma chamada ao método *getCargo*) e o salário líquido (por meio de uma chamada ao método *calcularSalarioLiquido*).

Vale lembrar que os objetos *adm* e *aca* são também objetos da classe abstrata *Funcionario.java*, já que os objetos de subclasses são também objetos de sua classe abstrata correspondente. Desta forma, você poderia encontrar a instância destes objetos da seguinte forma:

```
Funcionario adm = new Administrativo(10, "Paulo Cesar", "A", 10678.56, 789.50);
Funcionario aca = new Academico("Matemática", 12, "Maria Paula", "B", 8432.20, 521.32);
```

Veja que a referência é feita à classe abstrata *Funcionario.java*, mas a chamada é feita ao construtor das respectivas subclasses (*new*). Perceba que *adm* e *aca* são objetos da classe abstrata *Funcionario.java* e também das respectivas subclasses apontadas pela chamada ao construtor.

Após concluir a digitação das classes, compile e execute o projeto, por meio da tecla *F6*. Aparecerá na tela o seguinte resultado:

```
Nome: Paulo Cesar
Cargo: Gerente
Salário Líquido: 9889.06

Nome: Maria Paula
Cargo: Professor
Salário Líquido: 7910.880000000001
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA A chamada ao método *obterCargo* (que possui implementação diferente nas subclasses), por meio de dois objetos (*adm* e *aca*) de subclasses do mesmo tipo de classe abstrata, configura polimorfismo, já que estes objetos do mesmo tipo de classe abstrata apresentarão comportamentos diferentes, principalmente nesta chamada de métodos.

4.4 Modificadores de visibilidade

Na declaração de um método, variável ou constante, deve-se indicar de onde podem ser acessados, se apenas de dentro da própria classe (*private*); se de qualquer classe da aplicação (*public*); ou se apenas de dentro do pacote ao qual a classe pertence ou de subclasses ligadas em uma relação de herança (*protected*).

O que define de onde o recurso (variável, método ou constante) poderá ser acessado são os modificadores de visibilidade, que são: *public*, *private* e *protected*.

Veja na Tabela 4.1 um resumo da visibilidade destes modificadores:

Modificador	Mesma classe	Mesmo pacote	Subclasses	Todas as classes
private	x			
public	x	x	x	x
protected	x	x	x	
sem modificador	x	x		

Tabela 4.1: Modificadores de visibilidade.

4.4.1 Modificador public

O modificador *public* indica que o recurso poderá ser acessado a partir de qualquer classe da aplicação, inclusive de dentro da própria classe.

Na UML esse modificador é representado pelo sinal de mais (+).

Vamos a um exemplo.

Crie um novo projeto Java com duas classes: *Cliente.java* e *Principal.java*.

Escreva na classe *Cliente.java*, o código-fonte a seguir:

```
1    public class Cliente {
2        public int id;
3        public String nome;
4        public double renda;
5
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
6           public Cliente() {
7           }
8
9           public double obterRendaAjustada() {
10              return this.renda + obterValor();
11          }
12
13          public double obterValor() {
14              return 1000.00;
15          }
16      }
```

Note que o método *obterRendaAjustada* (linhas 9 à 11) chama o método *obterValor* (linhas 13 à 15), que é público. Veja que um método de uma classe pode chamar outro método público da mesma classe.

Escreva na classe *Principal.java*, o código-fonte a seguir, que acessa recursos da classe *Cliente.java*:

```
1       public class Principal {
2           public static void main(String[] args) {
3               Cliente cli = new Cliente();
4               cli.id = 10;
5               cli.nome = "Ana Paula";
6               cli.renda = 2000.00;
7               System.out.println ("Renda do cliente: " + cli.renda);
8               System.out.println ("Renda ajustada: " + cli.obterRendaAjustada());
9           }
10      }
```

Observe que como os atributos *id*, *nome* e *renda* da classe *Cliente.java* são públicos, não há necessidade de métodos *setter* para acessá-los. Bastou instanciar o objeto *cli* da classe *Cliente* (linha 3) e acessar os atributos incluindo valores neles (linhas 4 à 6).

Para exibir a renda (linha 7), bastou indicar o atributo do objeto onde estava armazenado o valor da renda, sem a necessidade de obter o valor por meio de um método *getter*. Onde teríamos que digitar *cli.getRenda()* se o atributo *renda* fosse privado e tivesse um método *getter* para ele, bastou digitar *cli.renda* (linha 7). O acesso é direto, pois renda é um atributo público na classe *Cliente.java*.

Para exibir a renda ajustada, o método *obterRendaAjustada* da classe *Cliente.java* foi chamado por meio do objeto *cli* (linha 8). Este método pôde ser chamado a partir da classe *Principal.java* porque é público na classe *Cliente.java*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao executar a aplicação, pressionando a tecla *F6*, aparecerá na tela o seguinte resultado:

```
Renda do cliente: 2000.0
Renda ajustada: 3000.0
```

NOTA Apesar de você poder declarar os atributos de uma classe como públicos, como viu no exemplo apresentado, isso não é comum nem indicado, pois outros objetos poderiam, em algumas circunstâncias, ter acesso ao conteúdo dos atributos, já que são públicos. O ideal é sempre declarar os atributos da classe como privados e criar métodos *getter* e *setter* públicos para eles.

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.4.2 MODIFICADOR PRIVATE

O modificador *private* permite que o recurso seja acessado somente de dentro da própria classe. Na UML esse modificador é representado pelo sinal de menos (-).

Para testar o modificador *private,* crie um novo projeto no *NetBeans* com duas classes: *Calculo.java* e *Teste.java*.

Na classe *Calculo.java*, digite o seguinte código-fonte:

```
1   import javax.swing.JOptionPane;
2
3   public class Calculo {
4       private final double IR=27.5;
5
6       public Calculo() {
7       }
8
9       private double obterPercentual(){
10          return Double.parseDouble (JOptionPane.showInputDialog ("Digite o valor do
            percentual: "));
11      }
12
13      public double ajustarSalario(double salario){
14          return (salario + salario * obterPercentual()/100) - (salario * IR/100);
15      }
16  }
```

Veja que na linha 4 foi criada a constante privada *IR* do tipo *double* contendo o valor 27.5. Como sabe, não será possível, na aplicação, mudar o valor armazenado, pois trata-se de uma constante. A palavra *final* é quem define que se trata de uma constante.

Nas linhas de 9 à 11 foi criado o método privado *obterPercentual*. Por ser privado, este método só poderá ser chamado de dentro desta classe. A chamada a este método ocorre da linha 14, do interior do método *ajustarSalario*. O método *obterPercentual* solicita um percentual, converte em *double* e retorna.

Veja que o método *ajustarSalario* é público, assim, poderá ser chamado a partir de outras classes da aplicação. Este método recebe um valor de salário do tipo *double*, aplica o percentual de aumento obtido na chamada ao método *obterPercentual*, subtrai o desconto de imposto contido na constante *IR* e retorna o resultado.

Na classe principal *Teste.java*, digite o seguinte código-fonte:

```
1    import javax.swing.JOptionPane;
2    public class Teste {
3       public static void main(String[] args) {
4          double salario = Double.parseDouble (JOptionPane.showInputDialog
           ("Digite o valor do percentual: "));
5          Calculo c = new Calculo();
6          System.out.print ("Salário Líquido: " + c.ajustarSalario(salario));
7       }
8    }
```

Na linha 4 é pedido um valor de salário, convertido em *double* e armazenado na variável *salario*.

Na linha 5 é instanciado um objeto da classe *Calculo.java* chamado *c*. Por meio deste objeto, na linha 6, é chamado o método público *ajustarSalario* da classe *Calculo.java* e passado como parâmetro o valor do salário. Veja que só é possível chamar o método *ajustarSalario* a partir da classe *Teste.java* porque ele é público. Você não conseguiria, por exemplo, chamar o método *obterPercentual*, pois ele é privado. Nem poderia acessar a constante *IR*, pois ela também é privada.

4.4.3 Modificador protected

O modificador *protected* permite que o recurso seja acessado de dentro da mesma classe, de qualquer classe que esteja no mesmo pacote da classe que contém o recurso e de subclasses da classe que contém o recurso em uma relação de herança. Na UML, esse modificador é representado pelo sinal de cerquilha (#). Este modificador será explicado em mais detalhes no próximo

UNINOVE – USO EXCLUSIVO PARA ALUNO

capítulo, quando serão apresentados exemplos de uso de herança de classes. Apenas para você entender como se utiliza este modificador, segue um exemplo de um método declarado como *protected*:

```
protected int obterCodigo() {
  return 10;
}
```

Você também pode declarar atributos protegidos. Veja o exemplo:

```
protected String nome;
```

> **NOTA** No Java, um recurso protegido (*protected*) pode ser acessado de classes do mesmo pacote em que está a classe que contém o recurso e de subclasses em uma relação de herança. Porém, na maioria das demais linguagens orientadas a objeto, um recurso protegido só poderá ser acessado de subclasses em uma relação de herança.

4.4.4 Sem modificador

Na ausência de um modificador, o recurso poderá ser acessado apenas de dentro da própria classe e de classes que estão no mesmo pacote. Veja como se declara um método sem modificador:

```
double soma( double a, double b){
  return a+b;
}
```

Veja como se declara um atributo sem modificador:

```
String nome;
double salario;
```

4.5 Garbage collector

Quando você instala o Java, integrado na *Java Runtime Environment* (JRE), existe um componente chamado *Garbage Collector*, que é responsável por coletar da memória objetos e dados que estão sendo mais usados. Assim, em Java, o programador não precisa se preocupar em alocar e desalocar memória quando lida com objetos, como acontece na linguagem C ou C++. Quando não precisamos mais de um objeto, basta anular sua referência elegendo-o a ser coletado (liberado da memória) pelo *Garbage Collector*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja o exemplo a seguir:

```
Funcionário func;
func = new Funcionário(1367, "Paulo Henrique", "Gerente", 789.56);
func=null;
```

Quando o objeto *func* torna-se nulo, ele vira candidato a ser desalocado da memória pelo *Garbage Collector*.

NOTA Tipos primitivos (*char, byte, short, int, long, float, double* e *boolean*) alocados dentro de um método não são liberados da memória pelo *Garbage Collector*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

4.6 Anotações

Anotações (ou *annotations*, em inglês) são instruções iniciadas pelo caractere arroba (@) que podem fornecer informações sobre o código Java (seja sobre um método ou uma classe) e, apesar de muitas vezes se assemelharem a comentários, podem ser interpretadas pelos compiladores.

Algumas anotações comuns utilizadas em Java são *@Deprecated* e *@Override*.

@Deprecated é uma anotação colocada na linha anterior à declaração de um método para indicar que seu uso é desencorajado por ser perigoso ou por ter uma alternativa melhor já desenvolvida.

@Override é uma anotação colocada na linha anterior à declaração de um método para indicar que ele está sendo sobrescrito, ou seja, que existe este método em uma superclasse e ele está sendo escrito de outra forma na subclasse. Esta anotação é normalmente utilizada em aplicações que utilizam classes relacionadas por herança onde se deseja obter polimorfismo.

Existem inúmeras outras anotações relacionadas a tecnologias Java específicas, principalmente sobre Java EE (*Enterprise Edition*).

4.7 Resumo

As relações de herança na programação orientada a objetos podem ocorrer de várias maneiras e utilizando vários recursos, como herança simples, interfaces e classes abstratas.

Você deve utilizar herança na sua forma pura e simples quando tem um conjunto de atributos e/ou métodos comuns que deseja que várias subclasses compartilhem. Para não repetir essa parte comum nas subclasses da relação de herança, centralize-os na superclasse e herde-os nas subclasses. As subclasses

poderão ter, além dos recursos herdados, seus próprios recursos (atributos, métodos etc.). Uma subclasse pode herdar de apenas uma superclasse. Não há herança múltipla em Java antes da versão 8, porém, a versão 8 já permite herança múltipla. As subclasses também podem ser superclasses de outras subclasses, em cadeias de herança.

Você deve utilizar interfaces quando deseja obrigar que um conjunto de classes implementem os mesmos métodos, porém, de maneiras diferentes. Estas classes também poderão implementar seus próprios métodos específicos.

As interfaces estabelecem um contrato em que as classes que implementam as interfaces devem aceitar, implementando os métodos assinados nas interfaces. Os métodos nas interfaces são, por padrão, públicos e abstratos, não podendo ser declarados de outra forma.

Você deve utilizar classes abstratas quando tem um conjunto de atributos e/ou métodos comuns que deseja que várias subclasses compartilhem (como nas relações de herança), e também quer que estas subclasses sejam obrigadas a implementar certos métodos de maneiras diferentes. As classes abstratas representam uma mistura entre os recursos providos pelas relações de herança simples e pelas interfaces. Classes abstratas podem ter variáveis, métodos abstratos, métodos implementados, construtores, atributos e constantes.

Tanto as interfaces quanto as classes abstratas não podem ser instanciadas diretamente. Você deve instanciar sempre as subclasses ligadas a elas. Nas relações de herança simples, com interfaces e com classes abstratas, normalmente você tem um mesmo método em várias subclasses implementado de maneiras diferentes. Objetos dessas subclasses que chamam estes métodos normalmente obtém resultados diferentes. Esta é uma das características que permite obter o que chamamos na orientação a objetos de polimorfismo (objetos do mesmo tipo, mas com formas/comportamentos diferentes).

Em uma aplicação, normalmente criamos diversas classes que se comunicam normalmente por meio da criação de objetos umas das outras. A partir de um objeto de uma classe, é possível acessar seus métodos, desde que o modificador de visibilidade permita. Os modificadores de visibilidade definem de onde métodos, variáveis e constantes (recursos) de uma classe podem ser acessados. Se o recurso possuir o modificador *public*, o recurso poderá ser acessado de qualquer classe da aplicação. Se possuir o modificador *private*, só poderá ser acessado do interior da própria classe. Se possuir o modificador *protected*, só poderá ser acessado a partir de classes que estão no mesmo pacote da classe que contém o recurso ou de subclasses de uma classe em uma relação de herança. Se não possuir modificador, poderá ser acessado somente de dentro da própria classe e de classes que estejam no mesmo pacote da classe que contém o recurso.

UNINOVE – USO EXCLUSIVO PARA ALUNO

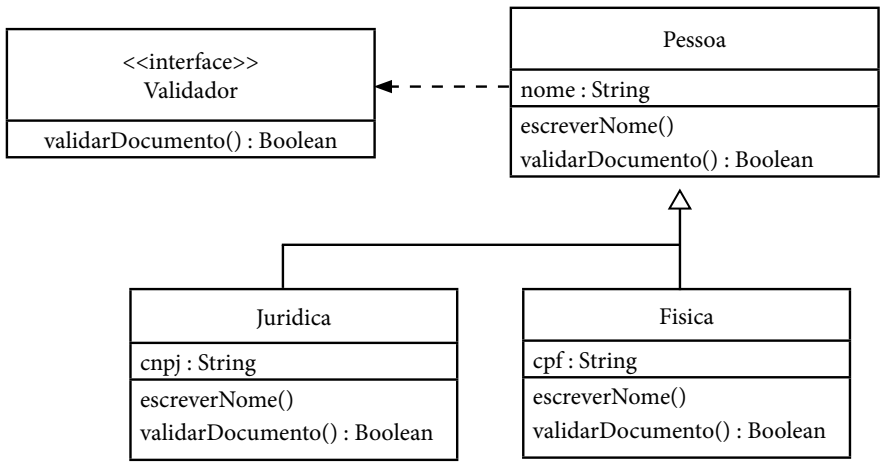
4.8 Exercícios

Você encontrará a seguir exercícios práticos, onde precisará criar projetos e escrever o código-fonte de várias classes e exercícios para fixar os conceitos teóricos do Capítulo. Não deixe de resolvê-los e esclarecer dúvidas com seu professor, caso você seja um estudante de ensino técnico ou superior.

4.8.1 Exercícios práticos

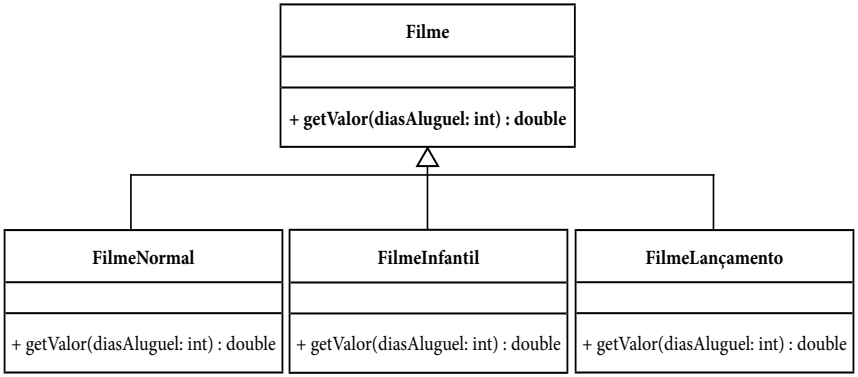
1) Crie um projeto Java e implemente as classes mostradas no diagrama de classes a seguir. Implemente os métodos *setter* e *getter* para os atributos, e o método *validarDocumento* em cada uma das classes. Este método deve apenas retornar *true* ou *false*. Implemente também o método *escreverNome* na classe *Pessoa.java*. Esse método deve apenas imprimir o nome da pessoa em uma caixa de diálogo.

Crie uma classe principal, cadastre uma pessoa física e exiba o nome dela por meio do método *escreverNome*.



2) Implemente as classes mostradas no diagrama de classes a seguir. Para cada uma das classes você deve definir pelo menos um atributo e gerar os métodos *getter* e *setter* para eles. Implemente o método *getValor* em cada uma das classes. Nas subclasses, este método deve calcular o valor do aluguel do filme de maneiras diferentes. Crie uma classe principal e cadastre um filme de cada categoria (normal, infantil e lançamento), calculando o valor do aluguel e exibindo na tela.

UNINOVE – USO EXCLUSIVO PARA ALUNO



4.8.2 Exercícios conceituais

1) Um desenvolvedor Java tem três classes declaradas em uma aplicação: *A*, *B* e *C*. Na classe *A* deseja colocar os atributos que são comuns às classes *B* e *C* e também implementar os métodos que são comuns a estas classes. Porém, deseja colocar na classe *A* um método que não deseja implementar, mas que quer obrigar as classe *B* e *C* a implementá-lo. Baseado nestes dados, qual o melhor recurso a utilizar, herança simples, interface ou classe abstrata? Justifique sua resposta.

2) Em uma aplicação de acesso a banco de dados, os métodos *salvar*, *consultar*, *excluir* e *alterar* deverão ser implementados no módulo de clientes, de produtos e de funcionários, porém, de maneiras diferentes. Qual o melhor recurso a utilizar, uma interface, uma classe abstrata ou herança simples? Justifique sua resposta.

3) Quando em uma aplicação poderá ocorrer polimorfismo? Que tipo de recurso deve haver nessa aplicação para que seja conseguido polimorfismo?

4) Qual a diferença entre sobrescrita de métodos e sobrecarga de métodos?

5) O que pode e o que não pode conter uma classe abstrata?

6) O que é permitido em uma interface?

7) Qual a diferença entre classe abstrata e interface?

UNINOVE – USO EXCLUSIVO PARA ALUNO

8) Podemos considerar que o uso de classe abstrata ou interface provê herança na aplicação? Justifique sua resposta.

9) É possível implementar herança múltipla em Java? Justifique sua resposta.

10) O que são anotações?

11) Qual a finalidade do *Garbage Collector*?

12) O que significa a anotação *@Override* na linha anterior à declaração de um método?

13) Qual o papel do comando *super* em uma subclasse em aplicações que contém relações de herança?

14) Quais instruções são utilizadas para indicar que uma classe herda outra e que implementa os métodos abstratos de outra?

15) Considere que a *C* herda a classe *A* e ambas possuem um construtor vazio padrão. Nestas condições, a instrução abaixo, contida no método *main* de uma classe principal da aplicação está correta? Justifique sua resposta.

```
A obj = new C();
```

16) Uma classe pode implementar mais de uma interface? Justifique sua resposta.

17) Quando uma classe implementa uma interface, o que ela será obrigada a fazer?

18) O que é um método abstrato e em que tipos de classes eles podem aparecer?

19) Explique as diferenças entre os modificadores de visibilidade *public*, *private*, *protected*?

UNINOVE – USO EXCLUSIVO PARA ALUNO

5. Métodos estáticos e tratamento de exceções

Para acessar os métodos públicos de uma classe é necessário instanciar um objeto desta classe, correto?

Está correto desde que o método não seja estático. Métodos estáticos podem ser acessados diretamente a partir do nome da classe, sem precisar de instância de objetos desta classe.

Programar usando métodos estáticos, de certa forma, faz perder a característica principal da orientação a objetos, que é acessar recursos das classes por meio de objetos destas classes. O uso de métodos estáticos e tipos de dados primitivos fazem com que Java não seja considerada por muitos profissionais uma linguagem totalmente orientada a objetos. Em uma linguagem totalmente orientada a objetos, os recursos (métodos, atributos, variáveis, constantes) só podem ser acessados a partir da instância de um objeto da classe que contém o recurso.

Esse assunto renderia um capítulo a parte, por isso, vamos deixar estas discussões de lado e vamos entender como podemos aproveitar os métodos estáticos nas nossas aplicações.

O caso principal de uso de métodos estáticos em uma aplicação é para criar bibliotecas de classes de apoio cujos métodos poderão ser acessados diretamente, sem instância de objetos. Assim, de qualquer classe da aplicação você poderá chamar os métodos estáticos das classes incluídas na biblioteca do projeto. Veremos isso em um exemplo que será apresentado no decorrer deste capítulo.

Além de apresentar os métodos estáticos, serão apresentadas ainda as principais formas de realizar tratamento de exceções em Java. Exceções são entradas não esperadas nos métodos, *arrays* e outras estruturas da aplicação. Por exemplo, quando a aplicação está esperando e digitação do valor do salário e o usuário digita seu nome, ocorrerá uma exceção, pois a aplicação está esperando um valor *double*, mas entrou um valor *String*. Outro exemplo comum é quando você tem, por exemplo, uma *array* com 3 posições numeradas de 0 à 2. Se você tentar, por engano, incluir um valor em uma posição que não existe, ocorrerá uma exceção. Existem inúmeros outros casos em que exceções podem causar sérias interrupções na execução do programa e comprometer todo o sistema.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Para que isso não ocorra, as exceções precisam ser tratadas de forma que se ocorrerem o programa possa contorná-las com o menor impacto possível.

No decorrer deste capítulo serão apresentados exemplos de tratamento das principais exceções.

5.1 Métodos Estáticos

Métodos estáticos são métodos que podem ser acessados diretamente por meio do nome da classe, sem precisar instanciar objetos desta classe. Todo método estático possui na sua declaração a palavra *static*.

Existem duas categorias de classes com métodos estáticos: as que vêm instaladas com o Java e as criadas pelo desenvolvedor da aplicação.

As classes de apoio que são instaladas com o Java possuem métodos estáticos que podem ser chamados na aplicação para auxiliar o programador oferecendo diversas operações já implementadas, evitando que ele tenha que reinventar a roda.

Além dessas classes, você pode criar suas próprias classes de apoio com métodos estáticos que necessita chamar de diversas classes da aplicação de forma fácil e rápida. Nos próximos tópicos serão apresentadas estas duas categorias de classes.

5.1.1 Utilizando métodos estáticos de classes das bibliotecas que fazem parte do Java

Quando você instala o Java no computador, são instaladas uma série de pacotes de classes que fazem parte de bibliotecas do Java que você pode utilizar como apoio para desenvolver suas aplicações.

Usamos com frequência métodos estáticos de classes contidas nestes pacotes. Por exemplo, usamos muito os métodos estáticos da classe *Math.java*, destinados a resolver operações matemáticas e trigonométricas. Veja um exemplo:

```
double b = Math.pow(10,3);
```

Esta operação calcula 10^3, logo, a variável *b* receberá o valor 1000.00. Note que o método *pow* da classe *Math.java* é chamado diretamente por meio do nome da classe. Veja como este método é escrito na classe *Math.java*:

```
public static double pow (double a, double b){
 return StrictMath.pow(a,b);
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Perceba o uso da palavra *static* na declaração do método. Observe ainda que este método ainda aponta para outro, da classe *StrictMath.java*.

No *NetBeans*, no método *main* de uma classe principal, digite *Math* seguido do ponto final, que você verá os métodos e constantes disponíveis na classe *Math.java*. Veja na Figura 5.1 alguns dos métodos estáticos da classe *Math.java*.



Figura 5.1: Métodos estáticos da classe Math.java.

Sempre que digitar o nome de uma classe que possui métodos e/ou constantes acessíveis, estes métodos/constantes aparecerão em uma lista para que você possa escolhê-los, ao invés de digitá-los. Este é um recurso do *NetBeans* para facilitar a vida do desenvolvedor.

Outros exemplos que utilizamos anteriormente foram os métodos estáticos da classe *JOptionPane.java*, que vem instalada no pacote *javax.swing*, que estudaremos no Capítulo 6. Veja um exemplo de uso do método estático *showInputDialog* desta classe:

```
import javax.swing.JOptionPane;
public class Principal {
   public static void main(String[] args) {
      String c = JOptionPane.showInputDialog( "Digite o nome da cidade");
   }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Note que o método *showInputDialog* foi chamado diretamente pelo nome da classe *JOptionPane*, logo, podemos concluir que este método é estático. Veja como este método está escrito na classe *JOptionPane.java*:

```
public static String showInputDialog(Object message)
      throws HeadlessException {
      return showInputDialog(null, message);
   }
```

Veja a seguir outros exemplos para:

1 – Converter um valor *String* para *double*:

```
String sal="12678.56";
double d = Double.parseDouble(sal);
```

Note que foi chamado o método estático *parseDouble* da classe *Double.java*. Este método está escrito na classe *Double.java* da seguinte forma:

```
public static double parseDouble(String s) throws NumberFormatException {
      return FloatingDecimal.readJavaFormatString(s).doubleValue();
}
```

2 – Converter um valor *int* para *String*:

```
int i=10;
String j= String.valueOf(i);
```

Veja que foi chamado o método estático *valueOf* da classe *String.java*. Observe como este método está escrito na classe *String.java*:

```
public static String valueOf(int i) {
      return Integer.toString(i);
}
```

5.1.2 Utilizando métodos estáticos de classes criadas pelo desenvolvedor

Você pode criar classes com métodos estáticos que utiliza constantemente, e utilizar como apoio nas suas aplicações. Vamos a um exemplo prático.

Crie um projeto Java no *NetBeans* chamado *ProjetoStatic*. Crie uma classe chamada *Calculo.java*, em um pacote chamado *calc.mat*. Crie também uma

UNINOVE – USO EXCLUSIVO PARA ALUNO

classe chamada *Principal.java* no pacote padrão. Veja na Figura 5.2 como deve estar seu projeto após realizar estes procedimentos.



Figura 5.2: Projeto "ProjetoStatic" criado no NetBeans.

Na classe *Calculo.java*, digite o seguinte código-fonte:

```
1   package calc.mat;
2
3   public class Calculo {
4
5       public static double soma(double a, double b) {
6           return a + b;
7       }
8
9       public static double soma(double a, double b, double c) {
10          return a + b + c;
11      }
12
13      public static int soma(int a, int b) {
14          return a + b;
15      }
16
17      public static String obterInfo() {
18          return "Esta é uma classe teste";
19      }
20
21      public static double mult(int a, int b) {
22          return a * b;
23      }
24  }
```

Note que há vários métodos, todos estáticos, que executam tarefas específicas. Note que vários destes métodos possuem o mesmo nome, mas recebem parâmetros diferentes. O nome disto é sobrecarga de métodos, lembra? Já discutimos isso em capítulos anteriores.

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA Na mesma classe podem haver métodos estáticos e não estáticos. A diferença será apenas a forma de acesso. Para acessar os métodos estáticos, basta digitar o nome da classe, o ponto final e o nome do método. Para acessar os métodos não estáticos, será necessário instanciar um objeto da classe e fazer o acesso por meio deste objeto.

Para acessarmos os métodos da classe criada, utilizaremos a classe *Principal.java*, cujo código-fonte é apresentado a seguir:

```
1   import calc.mat.Calculo;
2
3   public class Principal {
4
5       public static void main(String[] args) {
6           double a = Calculo.mult(10, 20);
7           System.out.print("Informações sobre a classe:" +
            Calculo.obterInfo());
8           System.out.print(Calculo.soma(10.5, 20.6));
9           double c = Calculo.soma(2.1, 3.4, 10.9);
10          int d = Calculo.soma(3, 4);
11          Calculo calc = new Calculo();
12          int s = calc.soma(10,2);
13          System.out.print(s);
14      }
15  }
```

Veja na linha 1 que a classe *Calculo.java* foi importada do pacote *calc.mat*. Isto foi necessário porque a classe *Principal.java* não está no mesmo pacote da classe *Calculo.java*.

Note que, para acessar os métodos estáticos, digitou-se o nome da classe (*Calculo*), o ponto final e o nome do método, passando os parâmetros necessários. Como há diversos métodos com o mesmo nome na classe *Calculo.java*, ao chamar o método o interpretador sabe qual chamar pelos parâmetros passados.

Note uma particularidade nas linhas 11 e 12. Veja que foi instanciado um objeto da classe *Calculo.java* chamado *calc* (linha 11) e, por meio deste objeto, foi chamado o método estático *soma* (linha 12). Isso não é necessário nem indicado, já que o método *soma* é estático, porém, é possível. O correto mesmo é fazer como na linha 10.

UNINOVE – USO EXCLUSIVO PARA ALUNO

5.1.3 UTILIZANDO MÉTODOS ESTÁTICOS NA CLASSE PRINCIPAL

Você pode criar métodos estáticos dentro da classe principal, e chamá-los do interior do método *main*, que também é estático. Esta forma de uso não é muito comum, já que o método *main* da classe principal normalmente é utilizado para acessar recursos de outras classes. Desta forma, o ideal é colocar os métodos estáticos em outra classe e chamá-los do interior do método *main*, como fizemos no tópico anterior.

Apesar de não ser a forma mais indicada de uso, vamos conferir no exemplo a seguir como acessar do interior do método *main* métodos estáticos criados na própria classe principal.

Crie um projeto no *NetBeans* e crie uma classe principal chamada *Teste.java*. Nessa classe, digite o seguinte código-fonte:

```
1   public class Teste {
2     public static void main(String[] args) {
3       double s = somar(10.5, 30.0);
4       System.out.println ("O resultado da soma é: " + s);
5       double m = multiplicar(5.0, 4.0);
6       System.out.println ("O resultado da multiplicação é: " + m);
7     }
8
9     public static double somar(double a, double b) {
10      return a + b;
11    }
12
13    public static double multiplicar(double a, double b) {
14      return a * b;
15    }
16  }
```

Execute o projeto pressionando a tecla *F6*.

Veja que na linha 2 é feita uma chamada do método somar que está na linha 9. Na linha 5 é chamado o método multiplicar que está na linha 13. O retorno desses métodos é armazenado em variáveis e exibidos nas linhas 4 e 6.

UNINOVE – USO EXCLUSIVO PARA ALUNO

5.1.4 Criando uma biblioteca de classes com métodos estáticos

O uso mais comum de métodos estáticos ocorre como será mostrado a seguir. Normalmente você cria bibliotecas de classes na forma de um arquivo com extensão *.jar* e coloca nelas métodos estáticos utilizados com muita frequência nas suas aplicações.

Uma biblioteca é criada em um projeto Java do *NetBeans*. O nome da biblioteca será o mesmo nome do projeto.

Para mostrar como funciona a criação de bibliotecas de classe com métodos estáticos, crie um projeto chamado *Matematica* e crie nele duas classes *Multiplica.java* e *Soma.java*, no pacote *calc.mat*, conforme mostra a Figura 5.3.



Figura 5.3: Projeto no netBeans para mostrar a criação de bibliotecas de classes.

O código-fonte da classe *Soma.java* é apresentado a seguir:

```
1   package calc.mat;
2
3   public class Soma {
4
5       public static double soma(double a, double b) {
6           return a + b;
7       }
8
9       public static double soma(double a, double b, double c) {
10          return a + b + c;
11      }
12
13      public static int soma(int a, int b) {
14          return a + b;
15      }
16  }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

O código-fonte da classe *Multiplica.java* é apresentado a seguir:

```
1    package calc.mat;
2
3    public class Multiplica {
4
5        public static double mult(int a, int b) {
6            return a * b;
7        }
8
9        public static double mult(double a, double b) {
10           return a * b;
11       }
12
13       public static int mult(int a, int b, int c) {
14           return a * b;
15       }
16   }
```

Agora que você já digitou o código-fonte das duas classes, vamos gerar a biblioteca. Para isso, terá que compilar a aplicação e gerar um arquivo de distribuição que terá o mesmo nome do projeto, mas com a extensão *.jar*. Clique no menu "Executar" e na opção "Limpar e Construir Projeto (Matematica)". Será gerado o arquivo *Matematica.jar* na subpasta *dist*, contida na pasta do projeto. O caminho provavelmente será "*\Matematica\dist\Matematica.jar*". Nossa biblioteca é o arquivo *Matematica.jar*.

Para usar a biblioteca criada, vamos iniciar um novo projeto Java chamado *NovoProjeto*. Nesse projeto, crie uma classe principal chamada *Principal.java*.

Clique na pasta "Biblioteca" do projeto com o botão direito do mouse e selecione a opção "Adicionar Jar/Pasta", como mostra a Figura 5.4.

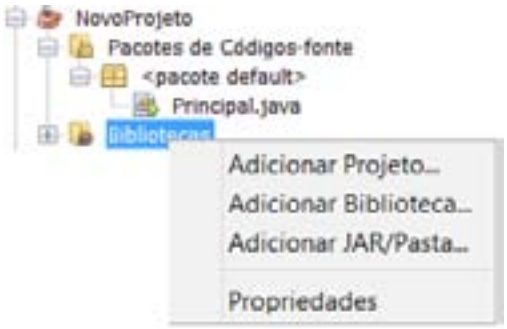


Figura 5.4: Adicionando uma biblioteca JAR na pasta "Bibliotecas" do projeto Java criado com o *NetBeans*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Selecione a biblioteca *Matematica.jar* na pasta *dist* do projeto da biblioteca (\ *Matematica\dist\Matematica.jar*) e clique no botão "Abrir".

Clique no sinal de mais à esquerda da pasta "Bibliotecas", no sinal de mais à esquerda da biblioteca *Matematica.jar* e no sinal de mais à esquerda do pacote *calc.mat*. Você verá as classes da biblioteca, conforme mostra a Figura 5.5.

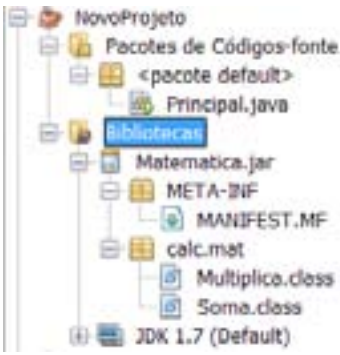


Figura 5.5: Exibição das bibliotecas de classes do projeto criado com o uso do NetBeans.

Na classe *Principal.java* da nossa aplicação, vamos chamar os métodos estáticos das classes *Multiplica.java* e *Soma.java* da nossa biblioteca. Veja na Figura 5.5 que estas classes, compiladas, aparecem com extensão *.class*.

Na classe *Principal.java*, digite o seguinte código-fonte:

```
1   import calc.mat.Multiplica;
2   import calc.mat.Soma;
3
4   public class Principal {
5
6       public static void main(String[] args) {
7           double m = Multiplica.mult(10.5, 2.3);
8           double n = Soma.soma(2.1, 3.0, 4.3);
9           System.out.print("Resultado da multiplicação: " + m);
10          System.out.print("Resultado da soma: " + n);
11      }
12  }
```

Note nas linhas 7 e 8 que são chamados os métodos das classes que estão na biblioteca do projeto, diretamente pelo nome da classe, afinal, os métodos são estáticos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

5.2 TRATAMENTO DE EXCEÇÕES

Uma exceção é um comportamento inesperado do programa causado normalmente por uma entrada não esperada do usuário ou por falha do programador na passagem de parâmetros para um método, na referência a recursos não disponíveis etc. Para evitar que as exceções causem paradas no sistema, é necessário tratá-las, para que, se ocorrerem, causem o menor transtorno possível. Em Java é possível tratar as exceções utilizando as estruturas *try...catch* e *throws*. Estas estruturas serão apresentadas nos próximos tópicos.

5.2.1 TRATAMENTO DE EXCEÇÕES COM TRY...CATCH

O tratamento de exceções com *try...catch* trata o problema, sem passar a responsabilidade pela resolução a quem chamou o método onde a exceção ocorreu. Você utiliza *try...catch* quando sabe como tratar a exceção. Se não sabe, utilize o *throws*, que passa a responsabilidade pelo tratamento a quem chamou o método que gerou a exceção. A sintaxe da estrutura *try...catch* é apresentada a seguir:

```
try {
  // linha(s) que podem causar a exceção
}
catch (Classe_que_vai_tratar_a_exceção objeto_da_classe)
{
  //O que deve acontecer se a exceção ocorrer
}
```

O interpretador tenta executar as linhas que estão no interior das chaves do *try*. Se não conseguir devido a uma exceção, desvia para o *catch*, executando as linhas que estão no interior das chaves.

Vamos a um exemplo prático para entender melhor o uso do *try...catch*.

Crie um projeto no *NetBeans* e uma classe *Principal.java* nesse projeto.

Digite o código-fonte a seguir na classe *Principal.java*:

```
public class Principal {
   public static void main(String[] args) {
      int a=2;
      int b=0;
      int c=a/b;
   }
}
```

O código vai gerar uma exceção, você sabe qual é?

É isso mesmo, não é possível dividir um número inteiro por zero.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Execute o projeto pressionando *F6* que será apresentada a mensagem de erro mostrada na Figura 5.6.



Figura 5.6: Exceção ArithmeticException.

Veja na mensagem que já aparece o nome da classe que deve ser utilizada no *catch* para tratar a exceção, a classe *ArithmeticException*, e também a descrição do erro "*/by zero*".

Para evitarmos que a exceção cause o erro mostrado na Figura 5.6, altere o código-fonte da classe *Principal.java* para:

```
public class Principal {
   public static void main(String[] args) {
      int a = 2;
      int b = 0;
      try {
         int c = a / b;
      } catch (ArithmeticException x) {
         System.out.println("Não é possível dividir por zero");
      }
   }
}
```

Execute o projeto novamente, pressionando a tecla *F6,* que você verá a mensagem "**Não é possível dividir por zero**".

Este tratamento de exceção poderia estar em um método, da seguinte forma:

```
public void dividir(int a, int b){
      try {
         int c = a / b;
      } catch (ArithmeticException x) {
         System.out.println("Não é possível dividir por zero");
      }
   }
}
```

Neste exemplo, a cláusula *catch* recebe como parâmetro um objeto da classe *ArithmeticException* contendo os dados do erro.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Quando uma exceção é gerada, o *catch* recebe um objeto da classe referente à exceção, contendo dados sobre a exceção. No exemplo acima, o objeto *v* possui informações sobre a exceção. Se você quiser, dentro do *catch*, ver a mensagem de erro, basta digitar:

```
System.out.println(v.getMessage());
```

Outros dados do erro podem ser obtidos por meio deste objeto.

Vamos a outro exemplo prático.

Crie um novo projeto Java no *NetBeans* e digite o seguinte código-fonte na classe *Principal.java*:

```
import javax.swing.JOptionPane;
public class Principal {
   public static void main(String[] args) {
      int idade = Integer.parseInt(JOptionPane.showInputDialog("Digite sua
      idade"));
   }
}
```

Suponhamos que o usuário digite o valor mostrado na Figura 5.7, quando o programa for executado:



Figura 5.7: Entrada de valor que gerará a exceção *NumberFormatException.*

Você sabe que exceção será gerada ao clicar no botão OK?

Será gerada a exceção mostrada na Figura 5.8, devido a tentativa de converter a *String* digitada em valor inteiro:

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 5.8: Exceção NumberFormatException.

Veja que já aparece o nome da classe *NumberFormatException* que será usada no *catch* para o tratamento da exceção, e também a mensagem de erro "*For input String : 30 anos*".

A exceção foi gerada porque houve uma tentativa de converter o valor *String* digitado ("30 anos") para inteiro.

Para tratar a exceção, ajuste o código-fonte da classe *Principal.java* para ficar da seguinte forma:

```
import javax.swing.JOptionPane;
public class Principal {
   public static void main(String[] args) {
      try{
         int idade = Integer.parseInt(JOptionPane.showInputDialog("Digite sua
         idade"));
      } catch(NumberFormatException e){
         JOptionPane.showMessageDialog(null, "Digite apenas números no
         campo idade");
      }
   }
}
```

Se você executar novamente o projeto e digitar no campo o mesmo valor ("30 anos") digitado anteriormente, verá uma caixa de diálogo com a mensagem "Digite apenas números no campo idade".

Vamos a outro exemplo prático, apresentado na classe *Principal.java*, a seguir:

```
public class Principal {
   public static void main(String[] args) {
      String[] nomes = new String[2];
      nomes[0] = "Ana";
      nomes[1] = "Paulo";
      nomes[2] = "Pedro";
   }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao executar o projeto, será gerada uma exceção, você sabe qual é?

Veja que se está tentando colocar o nome "Pedro" em uma posição da *array* que não existe, pois as posições vão de 0 à 1, já que a *array* tem duas posições apenas. Aparecerá na tela a mensagem mostrada na Figura 5.9.



Figura 5.9: Exceção ArrayIndexOutOfBoundsException.

Veja que a classe que deve ser utilizada no *catch* para o tratamento da exceção é a classe *ArrayIndexOutOfBoundsException*.

Para resolver este problema, você pode alterar a classe da seguinte forma:

```
0     public class Principal {
1         public static void main(String[] args) {
2             String[] nomes = new String[2];
3             try {
4                 nomes[0] = "Ana";
5                 nomes[1] = "Paulo";
6                 nomes[2] = "Pedro";
7             } catch (ArrayIndexOutOfBoundsException v) {
8                 System.out.println("Você está armazenando valor em uma
                  posição da array que não existe");
9             }
10        }
11    }
```

Se executar o projeto novamente verá a mensagem "Você está armazenando valor em uma posição da *array* que não existe", pois ao tentar executar a linha 6, será feito um desvio automático para o *catch* da linha 7 e executada a linha 8, que mostra a mensagem.

É possível também que um grupo de linhas que se deseja executar possa gerar mais de uma exceção. Nesse caso, todas as exceções terão que ser tratadas por quantos *catch* forem necessários.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja o exemplo a seguir de um método de uma classe de conexão com um banco de dados:

```
1       public int conectar(){
2           Class.forName("com.mysql.jdbc.Driver");
3           Connection conn = DriverManager.getConnection ("jdbc:mysql://
            localhost:3306/banco", "usuario", "senha");
4           return 1;
5       }
```

As linhas 2 e 3 podem gerar exceções.

A linha 2 passa para o método *forName* da classe *Class.java* um parâmetro solicitando o carregamento da classe *Driver.class* do pacote *com.mysql.jdbc* que deve estar na pasta "Bibliotecas" do projeto. Caso esta classe não esteja na biblioteca do projeto, será gerada uma exceção do tipo *ClassNotFoundException*.

A linha 3 passa para o método *getConnection* da classe *DriverManager.java* três parâmetros, o URL e o nome do banco de dados, o nome do usuário do banco de dados e a senha deste usuário. Se um destes parâmetros estiver incorreto será gerada uma exceção do tipo *SQLException*. Para tratar estas duas exceções, deve-se mudar o código-fonte do método para:

```
1       public int conectar(){
2          try {
3              Class.forName("com.mysql.jdbc.Driver");
4              Connection conn = DriverManager.getConnection ("jdbc:mysql://
               localhost:3306/banco","usuario","senha");
5              return 1;
6          } catch (ClassNotFoundException ex) {
7              return 2;
8          } catch (SQLException x){
9              return 3;
10         }
11      }
```

Veja que a linha 3 pode gerar uma exceção de falta da classe *Driver.class* e desviar para a linha 6, onde os dados do erro ficarão no objeto *ex* da classe *ClassNotFoundException,* e será retornado para quem chamou o método o valor 2.

A linha 4 pode gerar uma exceção por encontrar dados incorretos do banco de dados. Os dados do erro ficarão no objeto *x* da classe *SQLException* e será retornado o valor 3 para que chamou o método.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Note que o método acima retorna um valor inteiro para quem chamá-lo. Se as linhas 3 e 4 forem executadas sem erro, será retornado o valor 1. Se a linha 2 gerar a exceção de *ClassNotFoundException*, será retornado o valor 2. Se a linha 4 gerar uma exceção do tipo *SQLException*, será retornado o valor 3. Com os valores retornados quem chamou o método saberá o que ocorreu na sua execução. Os métodos de acesso a bancos de dados serão melhor estudados no Capítulo 7 deste livro.

NOTA Toda linha de código que executa comando SQL ou de conexão com banco de dados pode gerar uma exceção do tipo *SQLException*, por isso, este tratamento de exceção é comum em aplicações Java que acessam banco de dados.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Caso você não tenha certeza de qual exceção será gerada, pode usar uma forma de tratamento de exceção genérica, por meio da classe *Exception*. Veja a seguir o método apresentado anteriormente tratado desta forma:

```
1       public int conectar(){
2           try {
3               Class.forName("com.mysql.jdbc.Driver");
4               Connection conn = DriverManager.getConnection ("jdbc:mysql://
                localhost:3306/banco","usuario","senha");
5               return 1;
6           } catch (Exception ex) {
7               return 2;
8           }
9       }
```

Note que qualquer tipo de exceção que ocorrer nas linhas do interior das chaves do *try* irá desviar para o *catch*, que receberá em um objeto da classe *Exception* informações da exceção e retornará o valor 2. Nesse caso, duas exceções diferentes podem ocorrer, uma na linha 3 e outra na linha 4. O desvio para um único *catch* não permite diferenciar o tratamento das duas, mas apresentar um tratamento genérico. Este é o motivo pelo qual esta forma de tratamento de exceções não é recomendada.

5.2.2 Tratamento de Exceções com throws

Quando você não quer tratar a exceção, ou não tem ideia clara de como resolver o problema que pode ser causado na execução de uma linha ou trecho de código de um método, deve utilizar a cláusula *throws*, que passa a responsabilidade pelo tratamento da exceção a que chamou o método. Com o *throws* o método

declara que pode lançar o erro, e se este ocorrer, o método que o chamou deve tratar a exceção.

Se você colocar *throws* na declaração do método, você está dizendo algo como:

"Olha, tem certas operações que eu executo mas que não tenho a mínima ideia de como resolver, resolva você".

Por exemplo:

```
public void imprimeArquivo(File arquivo) throws FileNotFoundException{
  //Implementação do corpo do método
}
```

Neste método, o desenvolvedor não sabe o que fazer se o arquivo recebido como parâmetro não for encontrado, por isso, o *throws* passa a responsabilidade de tratar a exceção *FileNotFoundException* para quem chamar o método, que terá que tratá-lo com *try...catch*.

Veja o exemplo abaixo, do método *conectar* da classe *Dao.java*, que faz uma conexão com um banco de dados:

```
1   import java.sql.Connection;
2   import java.sql.DriverManager;
3   import java.sql.SQLException;
4   public class Dao {
5       public int conectar(String driver, String url, String usuario, String senha)
        throws ClassNotFoundException, SQLException{
6           Class.forName(driver);
7           Connection conn = DriverManager.getConnection (url, usuario, senha);
8           return 1;
9       }
10  }
```

No tópico anterior tratamos este método (conectar) com o *try...catch*, mas agora, da forma que ele foi implementado, o tratamento das exceções *ClassNotFoundException* e *SQLException* será responsabilidade de quem chamar o método. Isso pode ser percebido pelo uso da cláusula *throws* na declaração do método. Quem chamar o método *conectar* terá que tratar com *try...catch* estas exceções.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja a exceção tratada na chamada ao método *conectar,* a partir de uma classe chamada *Principal.java*.

```
1   import java.sql.SQLException;
2   public class Principal {
3      public static void main(String[] args) {
4         Dao d = new Dao();
5         try {
6            int r = d.conectar();
7         } catch (ClassNotFoundException ex) {
8            System.out.print("A classe do driver de acesso ao banco de dados não está
             dispponível");
9         } catch (SQLException ex) {
10           System.out.print("Há um erro nos dados de conexão com o banco de dados");
11        }
12     }
13  }
```

Observe na linha 4 que foi instanciado um objeto *d* da classe *Dao.java*. Por meio deste objeto, na linha 6, foi chamado o método *conectar*. Note que essa chamada de método obrigatoriamente terá que estar no interior de um *try*, pois o método tem a cláusula *throws* que obriga quem chamá-lo a tratar as exceções lançadas, que, neste caso, são *ClassNotFoundException* e *SQLException*.

Caso no método *conectar* tenha ocorrido uma exceção do tipo *ClassNotFoundException* (linha 7), será exibida a mensagem da linha 8. Caso tenha ocorrido uma exceção do tipo *SQLException* (linha 9), será exibida a mensagem da linha 10.

NOTA A cláusula *throws,* na declaração de um método, passa a responsabilidade pelo tratamento das exceções que possam ocorrer no método a quem chamá-lo.

As formas mais comuns de lidar com exceções foram apresentadas neste capítulo, mas há muito mais sobre exceções. Você pode conferir no *link: <http://blog.caelum.com.br/lidando-com-exceptions/>* um bom artigo intitulado "*Lidando com Exceptions*". Se deseja informações mais técnicas, acesse o *link*:

<http://www.caelum.com.br/apostila-java-orientacao-objetos/excecoes-e-controle-de-erros/#11-11-exercicios-excecoes>.

UNINOVE – USO EXCLUSIVO PARA ALUNO

5.3 Resumo

Este capítulo tratou de dois assuntos: uso de métodos estáticos e tratamento de exceções.

Métodos estáticos podem ser chamados diretamente por meio do nome da classe, sem precisar instanciar objeto. Aqueles métodos de apoio que utilizamos com frequência em diversas classes e/ou aplicações, normalmente declaramos como estáticos em uma classe e incluímos na biblioteca do projeto. O Java traz um conjunto de classes com métodos estáticos para facilitar o trabalho do programador, como a classe *Math*, para operações matemáticas e trigonométricas; e a classe *JOptionPane*, para gerar caixas de diálogo etc. Uma classe pode ter métodos estáticos e não estáticos. Para declarar um método como estático, utiliza-se a instrução *static* na declaração do método. Utilize métodos estáticos para criar bibliotecas e reutilizá-las em vários projetos.

Uma exceção em Java ocorre por meio da entrada não esperada de dados do usuário, por parâmetros incorretos para um método, pela falta de um recurso requerido etc. Tratar a exceção permite minimizar o problema, evitando a interrupção da execução do programa e a exibição daquelas mensagens de erro que assustam o usuário.

O tratamento de exceções em Java pode ocorrer por meio de duas estruturas fundamentais: *try...catch* e *throws*.

Utilize *throws* em um método quando você não quer ou não sabe como tratar a exceção e deseja deixar o tratamento para quem chamar o método. Nesse caso, dizemos que o *throws* lança a exceção para ser tratada por quem chamou o método.

Utilize *try...catch* para tratar a exceção de fato, quando você sabe qual é a exceção e como ela deve ser tratada. O *try* tenta executar uma linha ou bloco de linhas. Se ocorrer algum problema, automaticamente é feito um desvio para o *catch*, que executa a ação necessária relacionada a resolução do problema.

5.4 Exercícios

1) Analise o código abaixo e preencha corretamente a lacuna:

```
public static void metodo() {
  try {
    new java.io.FileInputStream("arquivo.txt");
  } catch (_____________________ e) {
    System.out.println("Nao foi possível abrir o arquivo para leitura");
  }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

2) Considere o código a seguir:

```
public class Teste {
   public static void main(String[] args) {
      String f1 = null;
      String f2 = null;
      f2 = f1.toUpperCase();
      System.out.println("Frase antiga: " + f1);
      System.out.println("Frase nova: " + f2);
   }
}
```

Ao executar o código foi exibida a seguinte mensagem:

Exception in thread "main" java.lang.NullPointerException at Teste.main(Teste.java:5)

Reescreva o código tratando a exceção gerada com *try...catch*.

3) Considere o código do método abaixo:

```
private static void aumentarLetras() {
      String f1 = null;
      String f2 = null;
      f2 = f1.toUpperCase();
      System.out.println("Frase antiga: " + f1);
      System.out.println("Frase nova: " + f2);
}
```

Este código gerará uma exceção do tipo *NullPointerException*. Como você não deseja tratar a exceção, mas sim deixar que ela seja tratada por quem chamar o método, reescreva o código de forma a permitir isso.

4) Considere o código a seguir:

```
import javax.swing.JOptionPane;
public class Teste {
   public static void main(String[] args) {
      int a[] = new int[3];
      for (int cont = 0; cont <= a.length; cont++) {
         a[cont] = Integer.parseInt(JOptionPane.showInputDialog("Digite um
         valor inteiro"));
      }
   }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao executar a classe acima aparece a seguinte mensagem:

Exception in thread “main” java.lang.ArrayIndexOutOfBoundsException: 3
at Teste.main(Teste.java:9)

Reescreva o código tratando esta exceção com *try...catch*.

5) Considere o código a seguir:

```
import javax.swing.JOptionPane;
public class Teste {
   public static void main(String[] args) {
      int a, b, c;
      a = Integer.parseInt(JOptionPane.showInputDialog("Digite um valor
      inteiro"));
      b = Integer.parseInt(JOptionPane.showInputDialog("Digite outro
      valor inteiro"));
      c = a / b;
   }
}
```

A) Trate a exceção que será gerada se for digitado na primeira caixa de diálogo o valor abaixo:



B) Trate a exceção que será gerada se for digitado na segunda caixa de diálogo o valor abaixo:



UNINOVE – USO EXCLUSIVO PARA ALUNO

6) O método abaixo tem o objetivo de incluir dados em uma tabela de um banco de dados. Reescreva-o direcionando a exceção adequada para quem chamar o método.

```
public int salvar(Cliente cli){
   String sql = "insert into cliente values ('" + cli.getCodigo() + "', '" +
   cli.getNome() + "', '" + cli.getCargo() + "', '" + cli.getTelefone() + "',
   '" +  cli.getEmail()+ "', "+ cli.getRenda()+ ")";
   return statement.executeUpdate(sql);
}
```

7) Crie uma classe em um projeto Java com um método estático para calcular a média entre três notas, descartando a menor delas, calculando a média das duas maiores e retornando o resultado. Construa o arquivo *.JAR* do projeto. Crie um novo projeto e adicione o arquivo *.JAR* à biblioteca. Crie uma classe principal nesse projeto, leia três notas e utilize o método estático da classe que está na biblioteca do projeto para calcular a média.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6. Criação de interface gráfica

Uma das partes mais importantes de um *software* é a interface do usuário. A interface do usuário é composta por todas as telas que o usuário tem contato quando interage com o *software*. Criar telas com visual agradável e que consumam pouco recurso computacional não é uma tarefa fácil.

A linguagem Java possui duas APIs (*Application Programming Interface*) com pacotes de classes específicos para a criação de interfaces do usuário, que poupa o programador de parte da complexidade da programação. Trata-se das APIs *swing* e *awt*, bastante conhecidas pelos desenvolvedores.

A API *awt* foi a primeira utilizada na construção de interface gráfica de usuário (GUI – *Graphical User Interface*) da plataforma Java, mas como usava muitas rotinas do sistema operacional (tendo diferenças em cada um deles), foi aos poucos perdendo espaço para a API *swing*, que pode ser utilizado da mesma maneira em qualquer sistema operacional. Podemos dizer que a API *swing* é a evolução da API *awt*, e vem sendo constantemente atualizada, sendo a Implementação de Referência (IR) para a criação de interface gráfica do usuário em aplicações *desktop* construídas com Java.

Neste capítulo, vamos focar no estudo da API *swing*, mas utilizaremos também alguns recursos da API *awt*, quando necessário.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.1 API swing

A API *swing* é a implementação de referência oficial para o desenvolvimento de interface gráfica do usuário (GUI) em aplicações *desktop* Java.

Ela é basicamente voltada à criação de janelas, normalmente formulários, que podem ter barras de menus, botões e outros elementos necessários em uma aplicação. A API *swing* é composta por diversos pacotes de classes, sendo que os mais utilizados são *javax.swing* e *javax.swing.event*. O foco desse capítulo serão as classes destes dois pacotes principais.

6.1.1 Contêineres swing

Para criar um componente de interface do usuário, por exemplo, uma janela, uma barra de menu e campos de formulário, precisamos escolher um contêiner

que receberá esses componentes. Nesse caso, o contêiner mais indicado é a classe *JFrame*, do pacote *javax.swing*. Criamos uma classe que estende (herda) a classe *JFrame* e que gera uma tela e adicionamos os componentes nela.

Na hierarquia da classes da API *swing*, os contêineres estão no topo. Um contêiner pode conter e controlar a exibição de qualquer outro componente. A Figura 6.1 mostra uma visão simplificada da relação de herança entre as principais classes da API *swing* e *awt*, incluindo os contêineres.

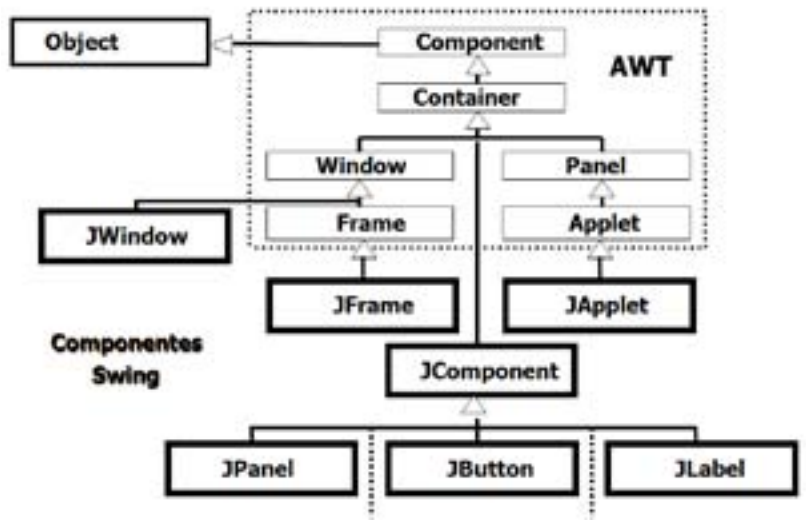


Figura 6.1: Principais classes das APIs *swing* e *awt*.

A forma mais comum de se construir componentes gráficos utilizando a API *swing* é especializá-los, ou seja, criar uma classe que estende o comportamento de outra. Dessa forma, todos os métodos da superclasse serão herdados, facilitando a construção do código-fonte. Se nossa classe estender (herdar) a classe *JFrame*, poderemos utilizar os métodos da classe *JFrame* e de suas subclasses para definir as propriedades do contêiner gerado.

NOTA Além do contêiner *JFrame* há outros tipos de contêiner na API *swing*, como *JPanel* e *JApplet*. Neste capítulo focaremos nos conteineres principais, que são *JFrame e JPanel*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.1.2 FORMAS DE CRIAÇÃO DE *LAYOUT* UTILIZANDO A API *SWING*

O *layout* de uma janela e de seus componentes, criado com as classes dos pacotes da API *swing,* pode ser definido de duas maneiras:

1. Posicionando-se estaticamente cada elemento na tela.

2. Utilizando gerenciadores de *layout*.

No posicionamento estático você tem controle total sobre o *layout* que está sendo criado, mas precisa fazer normalmente um conjunto de cálculos para que os elementos não sejam posicionados uns sobre os outros.

Quando se utiliza os gerenciadores de *layout* (que são classes e métodos específicos para lidar com o *layout* dos contêineres) o alinhamento, posicionamento e dimensionamento dos componentes é feito automaticamente. A vantagem é que você não precisa fazer cálculos para posicionar componentes. A desvantagem é que você tem que se contentar com os *layouts* pré-definidos, tendo pouca liberdade de modificação e personalização.

Os gerenciadores de *layout* são classes contidas no pacote *java.awt*. Estas classes são: *FlowLayout*, *GridLayout*, *BorderLayout* e *GridBagLayout*. Você pode ainda, se desejar, misturar o uso dos gerenciadores de *layout*, caso necessário.

6.1.3 APLICAÇÃO EXEMPLO CRIANDO *LAYOUT* COM POSICIONAMENTO ESTÁTICO

O posicionamento estático consiste no programador definir a posição de todos os elementos do *layout*, desde a janela até dos componentes em seu interior. Apesar de este modo permitir a personalização e controle absoluto sobre o *layout*, é necessário um conjunto de cálculos para posicionar adequadamente os elementos sem que uns sobreponham outros no todo ou em parte.

Para mostrar como esta forma de criação de *layout* funciona, crie um projeto Java no *NetBeans* chamado *ControleCliente* e, neste projeto, crie as classes *Principal.java* e *TelaCliente.java*. Após este procedimento seu projeto deve estar como mostra a Figura 6.2.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 6.2: Projeto "ControleCliente" criado no NetBeans.

Basicamente a interface do usuário será gerada na classe *TelaCliente.java*, que será chamada a partir da classe *Principal.java*.

Digite o seguinte código-fonte para a classe *TelaCliente.java*:

```
1   import javax.swing.JButton;
2   import javax.swing.JFrame;
3   import javax.swing.JLabel;
4   import javax.swing.JTextField;
5
6   public class TelaCliente extends JFrame{
7   private JLabel lblNome, lblRenda;
8   private JTextField txtNome, txtRenda;
9   private JButton btnSalvar, btnSair;
10
11      public TelaCliente() {
12          // definindo propriedades do formulário
13          setLayout(null);
14          setDefaultCloseOperation(EXIT_ON_CLOSE);
15          setSize(600, 200);
16          setLocation(200, 300);
17          setTitle("Controle de Clientes");
18
19          //Definindo propriedades dos campos do formulário
20          lblNome = new JLabel("Nome:");
21          lblNome.setSize(50,30);
22          lblNome.setLocation(30,10);
23          add(lblNome);
24
25          txtNome = new JTextField();
26          txtNome.setSize(480,30);
27          txtNome.setLocation(80,10);
28          add(txtNome);
29
30          lblRenda = new JLabel("Renda:");
31          lblRenda.setSize(50,30);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
32          lblRenda.setLocation(30,45);
33          add(lblRenda);
34
35          txtRenda = new JTextField();
36          txtRenda.setSize(100,30);
37          txtRenda.setLocation(80,45);
38          add(txtRenda);
39
40          btnSalvar = new JButton("Salvar");
41          btnSalvar.setSize(80,30);
42          btnSalvar.setLocation(30,90);
43          add(btnSalvar);
44
45          btnSair = new JButton("Sair");
46          btnSair.setSize(80,30);
47          btnSair.setLocation(120,90);
48          add(btnSair);
49      }
50  }
```

Para construir o *layout*, são usadas as classes *JFrame*, *JButton*, *JLabel* e *JTextField*, do pacote *javax.swing*. Estas classes são importadas das linhas 1 à 4.

Observe que a classe *TelaCliente.java* herda a classe *JFrame* (linha 6). Isso significa que do interior desta classe será possível acessar métodos da classe *JFrame* e de suas subclasses.

Como serão necessários rótulos, campos de texto e botões na tela que será gerada, foram declarados objetos das classes *JLabel*, *JTextField* e *JButton* nas linhas de 7 à 9.

Note ainda que a maioria da programação que gera a tela está no interior do construtor da classe *TelaCliente.java* (linhas de 11 à 49). Isso significa que quando um objeto desta classe for instanciado, todo este bloco de código será executado.

Nas linhas de 13 à 17 são definidas propriedades da janela por meio de chamadas aos métodos da classe *JFrame* e de suas subclasses.

Note que como não utilizamos gerenciadores de *layout*, a linha 13 passa o valor *null* para o método *setLayout* da classe *JFrame*, definindo que não será utilizado nenhum gerenciador de *layout*.

A linha 14 passa o valor *EXIT_ON_CLOSE* para o método *setDefaultCloseOperation* da classe *JFrame*, indicando que ao clicar no "X" vermelho do canto superior direito da janela a aplicação será encerrada.

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas 15 e 16 definem, respectivamente, o tamanho da janela e a posição desta na tela. Veja na Figura 6.3 que o método *setSize* define que a janela terá 600 *pixels* de largura por 200 *pixels* de altura (linha 15). Veja também que o método *setLocation* define que a janela aparecerá a partir da coluna de *pixel* 300 (eixo x) e da linha de *pixel* 200 (eixo y).

```
setSize(600, 200);
setLocation(200, 300);
```



Figura 6.3: Posicionamento da janela da aplicação.

A linha 17 define um título que aparecerá na barra de título da janela.

As linhas a partir da 20 definem as propriedades dos rótulos, campos e botões que são colocados na janela. Perceba que para todos os elementos há uma instância definida pelo comando *new*, um tamanho definido pelo método *setSize*, uma posição definida pelo método *setLocation* e por fim, o elemento é adicionado à janela por meio do método *add*. Veja a seguir a criação de cada elemento, onde é mostrado o tamanho e posicionamento do elemento.

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas a seguir criam o rótulo “Nome:” por meio de um objeto *lblNome* da classe *JLabel* (linhas 20 a 23).

```
lblNome = new JLabel("Nome:");
lblNome.setSize(50,30);
lblNome.setLocation(30,10);
add(lblNome);
```



Figura 6.4: Posicionamento do rótulo “Nome” na janela.

As linhas a seguir criam o campo vazio para a digitação do nome, por meio de um objeto *txtNome* da classe *JTextField* (linhas 25 à 28). Veja na Figura 6.5 como este campo aparecerá na janela.

```
txtNome = new JTextField();
txtNome.setSize(480,30);
txtNome.setLocation(80,10);
add(txtNome);
```



Figura 6.5: Posicionamento do campo nome na janela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas a seguir criam o rótulo "Renda:" por meio de um objeto *lblRenda* da classe *JLabel* (linhas 30 à 33). Veja na Figura 6.6 como este rótulo aparecerá na janela.

```
lblRenda = new JLabel("Renda:");
lblRenda.setSize(50,30);
lblRenda.setLocation(30,45);
add(lblRenda);
```



Figura 6.6: Posicionamento do rótulo "Renda" na janela.

As linhas a seguir criam o campo vazio para a digitação da renda por meio de um objeto *txtRenda* da classe *JTextField* (linhas 35 à 38). Veja na Figura 6.7 como este campo aparecerá na janela.

```
txtRenda = new JTextField();
txtRenda.setSize(100,30);
txtRenda.setLocation(80,45);
add(txtRenda);
```



Figura 6.7: Posicionamento do campo renda na janela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas a seguir criam o botão “Salvar”, por meio de um objeto *btnSalvar* da classe *JButton* (linhas 40 à 43). Veja na Figura 6.8 como este botão aparecerá na janela.

```
btnSalvar = new JButton("Salvar");
btnSalvar.setSize(80,30);
btnSalvar.setLocation(30,90);
add(btnSalvar);
```



Figura 6.8: Posicionamento do botão “Salvar” na janela.

As linhas a seguir criam o botão “Sair”, por meio de um objeto *btnSair* da classe *JButton* (linhas 45 à 48). Veja na Figura 6.9 como este botão aparecerá na janela.

```
btnSair = new JButton("Sair");
btnSair.setSize(80,30);
btnSair.setLocation(120,90);
add(btnSair);
```



Figura 6.9: Posicionamento do botão “Sair” na janela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Quando você chama um método a partir de um objeto, este método pode ser da classe que instanciou o objeto ou de qualquer uma de suas subclasses, quando esta classe herda outras classes. Veja por exemplo a seguinte chamada ao método *setSize*:

```
btnSair = new JButton("Sair");
btnSair.setSize(80,30);
```

Apesar do objeto *btnSair* ser da classe *JButton*, o método *setSize* é da classe *Component*. Isso ocorre porque a classe *JButton* herda a classe *AbstractButton*, que herda a classe *JComponente*, que herda a classe *Container*, que herda a classe *Component*.

NOTA Quando chamamos um método por meio de um objeto de uma classe que faz parte de uma cadeia de herança, o método pode ser de qualquer uma das classes da cadeia de herança.

Para gerar a tela programada na classe *TelaCliente.java*, é necessário instanciar um objeto desta classe, o que é feito na classe *Principal.java*, cujo código-fonte é apresentado a seguir:

```
1    public class Principal {
2        public static void main(String[] args) {
3            TelaCliente tela = new TelaCliente();
4            tela.setVisible(true);
5        }
6    }
```

Veja que a linha 3 instancia um objeto da classe *TelaCliente.java* e, por meio deste objeto, é feita uma chamada ao método *setVisible* da classe *Window* (subclasse da cadeia de herança de *JFrame*), passando como parâmetro o valor *true*, que torna a janela visível.

Para compilar e executar o projeto, pressione a tecla *F6* ou clique no menu "Executar" e na opção "Executar Projeto". Ao fazer isso você verá a janela mostrada na Figura 6.10.

UNINOVE – USO EXCLUSIVO PARA ALUNO

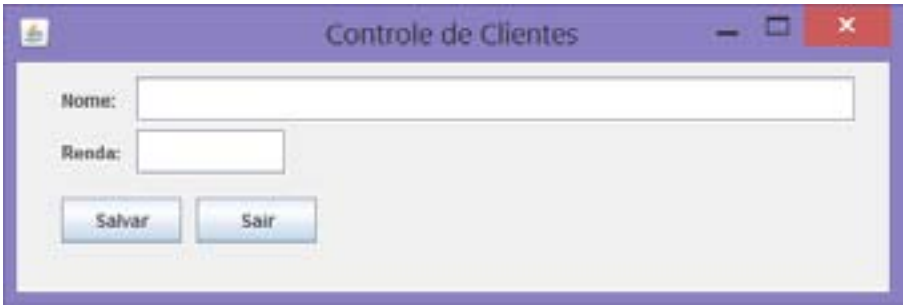


Figura 6.10: Janela do formulário de cadastro.

Você deve ter observado que utilizamos duas classes para criar a aplicação: *TelaCliente.java* e *Principal.java*.

Poderíamos ter conseguido o mesmo resultado apenas com a classe *Principal.java*. Para isso, o código-fonte desta classe deveria ficar da seguinte forma:

```
1   import javax.swing.JButton;
2   import javax.swing.JFrame;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4   import javax.swing.JLabel;
5   import javax.swing.JTextField;
6
7   public class Principal extends JFrame {
8
9       private JLabel lblNome, lblRenda;
10      private JTextField txtNome, txtRenda;
11      private JButton btnSalvar, btnSair;
12
13      public Principal() {
14          // definindo propriedades do formulário
15          setLayout(null);
16          setDefaultCloseOperation(EXIT_ON_CLOSE);
17          setSize(600, 200);
18          setLocation(200, 300);
19          setTitle("Controle de Clientes");
20
21          //Definindo propriedades dos campos do formulário
22          lblNome = new JLabel("Nome:");
23          lblNome.setSize(50, 30);
24          lblNome.setLocation(30, 10);
25          add(lblNome);
26
27          txtNome = new JTextField();
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
28              txtNome.setSize(480, 30);
29              txtNome.setLocation(80, 10);
30              add(txtNome);
31
32              lblRenda = new JLabel("Renda:");
33              lblRenda.setSize(50, 30);
34              lblRenda.setLocation(30, 45);
35              add(lblRenda);
36
37              txtRenda = new JTextField();
38              txtRenda.setSize(100, 30);
39              txtRenda.setLocation(80, 45);
40              add(txtRenda);
41
42              btnSalvar = new JButton("Salvar");
43              btnSalvar.setSize(80, 30);
44              btnSalvar.setLocation(30, 90);
45              add(btnSalvar);
46
47              btnSair = new JButton("Sair");
48              btnSair.setSize(80, 30);
49              btnSair.setLocation(120, 90);
50              add(btnSair);
51          }
52
53          public static void main(String[] args) {
54              Principal tela = new Principal();
55              tela.setVisible(true);
56          }
57      }
```

Note que na linha 54 foi instanciado um objeto da própria classe *Principal.java* por meio do comando *new*, que chama o construtor desta classe. O construtor compreende as linhas de 13 à 51. Quando o construtor é executado, a janela é gerada na memória. Para visualizar esta janela, o método *setVisible* é chamado passando como parâmetro o valor *true*, que torna a janela visível (linha 55).

Note que a classe *Principal.java* agora herda a classe *JFrame* (linha 7).

UNINOVE – USO EXCLUSIVO PARA ALUNO

NOTA Além de rótulos, campos de formulário e botões, é possível criar barras de menus, áreas de texto, caixas de combinação e de listagem, painéis, abas, campos formatados, enfim, qualquer componente comum de interface de usuário. A forma de criação e uso é semelhante ao que foi visto no exemplo apresentado. Instancia-se o objeto referente ao elemento, define-se a posição e o dimensionamento e adiciona-o à tela. No decorrer deste capítulo você aprenderá como trabalhar com estes elementos.

6.2 Criação de interface com gerenciadores de *layout*

Vimos no tópico anterior como posicionar e dimensionar os componentes na área da interface utilizando posicionamento estático. Apesar do controle que se tem sobre a criação da tela, este método requer cálculos para posicionar corretamente os componentes, o que demanda um pouco mais de tempo.

Além do posicionamento estático, podemos utilizar gerenciadores de *layout* que fazem todo trabalho de cálculo para alinhamento, posicionamento e dimensionamento dos componentes. Os gerenciadores de *layout* são a seguintes classes do pacote *java.awt*: *FlowLayout*, *GridLayout*, *BorderLayout* e *GridBagLayout*.

Estes gerenciadores de *layout* serão apresentados a seguir neste capítulo.

6.2.1 FlowLayout

Este gerenciador de *layout* é o mais simples de todos e possui as seguintes características:

– Faz o alinhamento padrão ao centro e acima, com deslocamento em linhas (*flow* = fluxo). Quando você adiciona o primeiro elemento, ele é posicionado ao centro e na parte de cima do contêiner. Quando você adiciona novos elementos, eles vão sendo posicionados em linha, à direita do anterior. Se acabar a área disponível na linha, o elemento cai automaticamente para a próxima linha. Apesar dos elementos aparecerem por padrão ao centro, eles podem ser posicionados à esquerda ou à direita.

– Tem a capacidade de manter o tamanho original dos componentes durante o redimensionamento (mantém o *prefered size* dos componentes). Se você redimensiona a janela, o tamanho dos componentes não é modificado, mantendo os valores originais.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.1.1 Exemplo simples com FlowLayout

Veja a seguir a mesma aplicação que desenvolvemos anteriormente, mas agora utilizando o gerenciador de *layout FlowLayout*. Crie um projeto Java no *NetBeans* chamado *ProjetoFlow* e nele crie uma classe chamada *Principal.java*. A Figura 6.11 mostra como deve estar o projeto após sua criação.

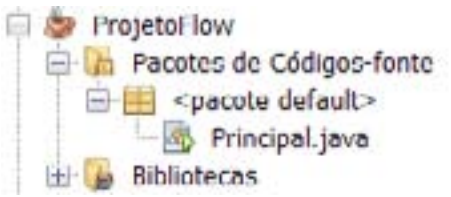


Figura 6.11: Projeto utilizando o gerenciador de layout FlowLayout.

Digite o seguinte código-fonte para a classe *Principal.java*:

```
1   import java.awt.FlowLayout;
2   import javax.swing.*;
3   public class Principal extends JFrame {
4      private JLabel lblNome, lblRenda;
5      private JTextField txtNome, txtRenda;
6      private JButton btnSalvar, btnSair;
7
8      public Principal() {
9        // Propriedades da janela
10       setTitle("Exemplo de Flow layout");
11       setLayout(new FlowLayout());
12       setDefaultCloseOperation(EXIT_ON_CLOSE);
13       setSize(600, 200);
14       setLocation(200, 300);
15
16       // definição dos componentes da janela
17       lblNome = new JLabel("Nome:");
18       add(lblNome);
19       txtNome = new JTextField(45);
20       add(txtNome);
21       lblRenda = new JLabel("Renda:");
22       add(lblRenda);
23       txtRenda = new JTextField(15);
24       add(txtRenda);
25       btnSalvar = new JButton("Salvar");
26       add(btnSalvar);
27       btnSair = new JButton("Sair");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
28              add(btnSair);
29          }
30
31          public static void main(String args[]) {
32              Principal tela = new Principal();
33              tela.setVisible(true);
34          }
35      }
```

A linha 11 define que o *layout* da janela será *FlowLayout* utilizando o posicionamento padrão ao centro. Caso você queira que o alinhamento seja à esquerda, precisa digitar a seguinte instrução:

```
setLayout(new FlowLayout(FlowLayout.LEFT));
```

Observe nas linhas 13 e 14 que o tamanho e posição da janela devem ser definidos, porém, das linhas 17 à 28 os objetos referentes a cada elemento são apenas instanciados e adicionados ao contêiner (janela), sem a necessidade de definição da posição e do tamanho. A linha 32 instancia um objeto da própria classe e por meio desse objeto chama o método *setVisible* passando o parâmetro *true*, para exibir a janela.

Ao compilar e executar o projeto pressionando a tecla *F6*, você verá a tela mostrada na Figura 6.12.

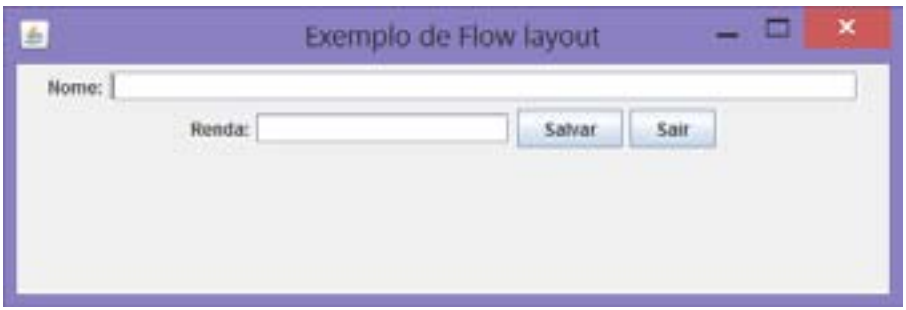


Figura 6.12: Tela gerada utilizando o gerenciador de layout FlowLayout.

Veja que o campo "Nome" foi inserido na tela, ocupando praticamente toda a primeira linha. Em seguida foi incluído o campo "Renda". Como este campo não coube na primeira linha do contêiner, ele foi deslocado automaticamente para a segunda linha. Em seguida foram inseridos os botões "Salvar" e "Sair". Como estes botões couberam na mesma linha do campo "Renda", foram colocados à direita. Note que o conteúdo das duas linhas se mantém centralizado e na parte superior da janela. Se fossem inserido novos

UNINOVE – USO EXCLUSIVO PARA ALUNO

componentes eles apareceriam à direita do botão “Sair”, mudando de linha caso não coubessem.

Caso você aumente a largura da janela, os elementos que estavam na segunda linha sobem para a primeira linha, sempre à direita do elemento anterior, até preencher toda a linha, como mostra a Figura 6.13.



Figura 6.13: Redimensionamento da janela que utiliza o gerenciador de layout FlowLayout.

6.2.2 BorderLayout

O gerenciador de *layout BorderLayout* divide o contêiner em cinco áreas: norte, sul, leste, oeste e centro. Além de não manter o tamanho original dos componentes durante o redimensionamento (já que eles são aumentados de acordo com a tela), o contêiner pode receber somente um componente por área. A Figura 6.14 mostra como o gerenciador de *layout BorderLayout* divide a janela.

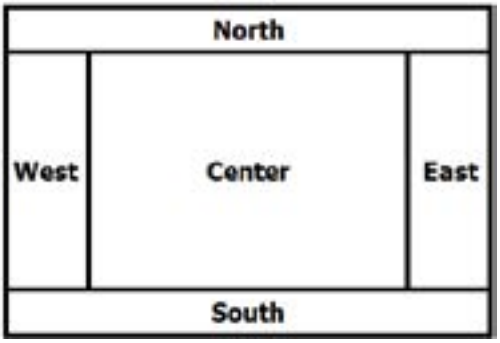


Figura 6.14: Áreas do *layout BorderLayout*.

O componente colocado em uma área usa todo o espaço dessa área.

NOTA O gerenciador de *layout* padrão dos contêineres é o *BorderLayout*. Isso significa que se você não definir explicitamente o gerenciador de *layout* por meio do método *setLayout*, o *BorderLayout* será adotado por padrão.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.2.1 EXEMPLO SIMPLES COM BORDERLAYOUT

Para testar o uso do gerenciador de *layout BorderLayout*, crie um projeto chamado *ProjetoBorder* com uma classe principal chamada *Tela.java*, conforme mostra a Figura 6.15.

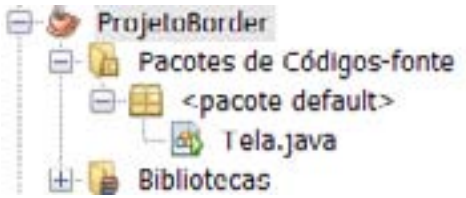


Figura 6.15: Projeto "ProjetoBorder" usando Borderlayout.

Na classe *Tela.java*, digite o seguinte código-fonte:

```
1   import java.awt.BorderLayout;
2   import javax.swing.*;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4
5   public class Tela extends JFrame {
6
7       private JButton btnCentro, btnLeste, btnOeste, btnNorte, btnSul;
8
9       public Tela() {
10          setTitle("Exemplo com Border Layout");
11          setLayout(new BorderLayout());
12          setDefaultCloseOperation(EXIT_ON_CLOSE);
13          setSize(700, 200);
14          setLocation(300, 200);
15          btnCentro = new JButton("Centro");
16          add(btnCentro, BorderLayout.CENTER);
17          btnLeste = new JButton("Leste");
18          add(btnLeste, BorderLayout.EAST);
19          btnSul = new JButton("Sul");
20          add(btnSul, BorderLayout.SOUTH);
21          btnNorte = new JButton("Norte");
22          add(btnNorte, BorderLayout.NORTH);
23          btnOeste = new JButton("Oeste");
24          add(btnOeste, BorderLayout.WEST);
25      }
26
27      public static void main(String args[]) {
28          Tela tela = new Tela();
29          tela.setVisible(true);
30      }
31  }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Na linha 11 o *layout* da janela foi definido como *BorderLayout*.

Note nas linhas de 15 a 24 que cada elemento é instanciado e, em seguida, adicionado ao contêiner (janela) indicando a posição em que será colocado. Veja, por exemplo, as linhas 15 e 16. Na linha 15 um objeto da classe *JButton* é instanciado e na linha 16 este objeto é adicionado ao contêiner por meio de uma chamada ao método *add* da classe *Container*. Este método receberá como parâmetro o nome do objeto que será adicionado e também a posição do objeto. O comando *BorderLayout.CENTER* obtêm o valor da constante chamada *CENTER* da classe *BorderLayout*. Lá na classe *BorderLayout* esta constante é declarada da seguinte forma:

```
public static final String CENTER = "Center";
```

Como a constante *CENTER* é pública e estática, pode ser acessada do interior da nossa classe *Tela.java*. As demais linhas seguem o mesmo padrão, posicionando os demais elementos na janela.

Ao compilar e executar o projeto pressionando a tecla *F6* você verá a tela mostrada na Figura 6.16.

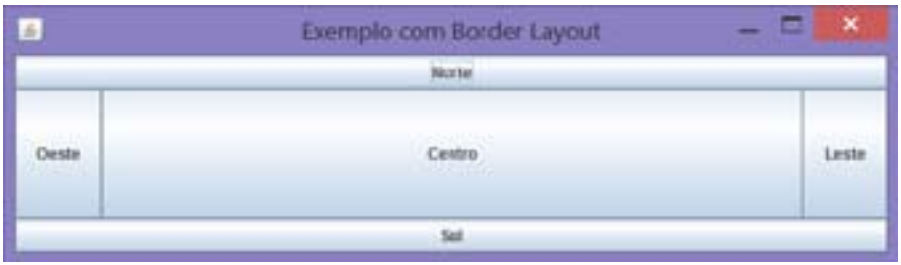


Figura 6.16: Tela gerada com BorderLayout.

Veja que foi colocado um botão em cada região do *layout* – já que só é permitido um elemento por área – e eles ocuparam todo o espaço daquela área. Se você aumentar ou diminuir a janela, o tamanho dos botões aumenta ou diminui proporcionalmente.

NOTA

Este tipo de *layout* é utilizado para que cada região da janela receba novos contêineres (normalmente *JPanel*) e estes contêineres menores agrupem elementos como botões, rótulos, campos texto etc. Veremos como fazer isso mais adiante neste capítulo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.3 GRIDLAYOUT

O gerenciador de *layout GridLayout* divide o contêiner em linhas e colunas como uma tabela com células (*grid*). Se você aumentar ou diminuir a janela, o tamanho dos componentes acompanharão proporcionalmente as novas dimensões da janela, ou seja, se você aumentar a janela, o tamanho dos componentes aumenta, se diminuir, o tamanho dos componentes diminui. Em cada célula do *grid*, ou seja, em cada posição do *layout*, você pode colocar apenas um elemento.

6.2.3.1 EXEMPLO SIMPLES COM GRIDLAYOUT

Para entender melhor como este gerenciador de *layout* funciona, crie um novo projeto Java no *NetBeans* chamado *ProjetoGrid* com uma classe principal chamada *Tela.java*, conforme mostra a Figura 6.17.



Figura 6.17: Projeto "ProjetoGrid" usando GridLayout.

Na classe *Tela.java*, digite o seguinte código-fonte:

```
1   import java.awt.GridLayout;
2   import javax.swing.*;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4
5   public class Tela extends JFrame {
6
7       private JLabel lblNome, lblRenda;
8       private JTextField txtNome, txtRenda;
9       private JButton btnSalvar, btnSair;
10      private JButton sim, nao, cancelar;
11      private JLabel mensagem;
12
13      public Tela() {
14          setTitle("Exemplo com GridLayout");
15          setDefaultCloseOperation(EXIT_ON_CLOSE);
16          setLayout(new GridLayout(3, 2));
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
17            setSize(600, 200);
18            setLocation(300, 200);
19            lblNome = new JLabel("Nome:");
20            add(lblNome);
21            txtNome = new JTextField();
22            add(txtNome);
23            lblRenda = new JLabel("Renda:");
24            add(lblRenda);
25            txtRenda = new JTextField();
26            add(txtRenda);
27            btnSalvar = new JButton("Salvar");
28            add(btnSalvar);
29            btnSair = new JButton("Sair");
30            add(btnSair);
31        }
32
33        public static void main(String args[]) {
34            Tela tela = new Tela();
35            tela.setVisible(true);
36        }
37    }
```

Na linha 16 o *layout* da janela é definido como *GridLayout* com três linhas e duas colunas, totalizando 6 células. As linhas 17 e 18 definem, respectivamente, tamanho e posição da janela na tela.

Das linhas 19 à 30 os objetos são declarados e adicionados na janela. Veja que os objetos vão sendo colocados nas células da *grid* da esquerda para a direita e de cima para baixo, de forma sequencial.

NOTA Todas as células da *grid* possuem o mesmo tamanho, que pode ser alterado automaticamente na medida em que a janela é aumentada ou diminuída. Não há como definir tamanho diferenciado para cada coluna, linha ou célula. Se deseja tamanhos diferentes, utilize o gerenciador de *layout GridBagLayout*, que será visto mais a frente neste capítulo.

Note que a linha 34 instancia um objeto da própria classe *Tela.java* por meio do comando *new*, que chama o construtor da classe (linhas de 13 à 31). Nessa operação, todo o conteúdo do construtor é executado, gerando a tela. A linha 35 exibe a janela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto pressionando a tecla *F6* você verá a tela apresentada na Figura 6.18.



Figura 6.18: Tela gerada com layout GridLayout.

Observe a divisão da tela em três linhas e duas colunas e o posicionamento dos elementos na ordem em que foram adicionados. Cada elemento ocupa toda a área da célula da *grid*.

6.2.4 GridBagLayout

O gerenciador de *layout GridBagLayout* é o mais flexível de todos, porém, é o mais complexo, pois cada elemento deve conter uma posição inicial, uma posição final, um tamanho, uma escala, um alinhamento e um preenchimento. Ele divide o contêiner em linhas e colunas como uma tabela com células (*grid*). Dependendo do alinhamento dentro de uma célula, pode ou não manter o tamanho original dos componentes. Em cada célula, pode ser colocado apenas um componente e cada célula pode ser expandida para ocupar o espaço de uma ou mais células adjacentes usando regras específicas de alinhamento e posicionamento.

Quando se utiliza o gerenciador de *layout GridBagLayout*, um objeto da classe *GridBagConstraints* é normalmente utilizado para determinar o posicionamento do elemento na *grid*.

6.2.4.1 Exemplo simples com GridBagLayout

Para entender melhor como funciona o gerenciador de *layout GridBagLayout*, vamos criar a tela cuja divisão é mostrada na Figura 6.19.

UNINOVE – USO EXCLUSIVO PARA ALUNO

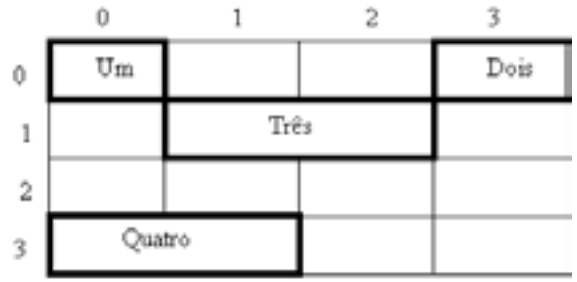


Figura 6.19: Exemplo de layout GridBagLayout.

Note que as células possuem posicionamento específico e algumas possuem tamanhos diferentes.

Crie um novo projeto Java no *NetBeans* chamado *ProjetoGridBag* com a classe *Tela.java*, conforme mostra a Figura 6.20.



Figura 6.20: Projeto "ProjetoGridBag" utilizando GridBagLayout.

Na classe *Tela.java*, digite o seguinte código-fonte:

```
1   import java.awt.*;
2   import javax.swing.*;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4
5   public class Tela extends JFrame {
6
7       private JButton btn1, btn2, btn3, btn4;
8
9       public Tela() {
10          setTitle("Exemplo com GridBagLayout");
11          setDefaultCloseOperation(EXIT_ON_CLOSE);
12          setSize(500, 200);
13          setLayout(new GridBagLayout());
14          GridBagConstraints gridCons = new GridBagConstraints();
15
16          btn1 = new JButton("Um");
17          btn2 = new JButton("Dois");
18          btn3 = new JButton("Três");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
19            btn4 = new JButton("Quatro");
20
21            gridCons.gridx = 0;
22            gridCons.gridy = 0;
23            add(btn1, gridCons);
24
25            gridCons.gridx = 3;
26            add(btn2, gridCons);
27
28            gridCons.gridx = 1;
29            gridCons.gridy = 1;
30            gridCons.gridwidth = 2;
31            add(btn3, gridCons);
32
33            gridCons.gridx = 0;
34            gridCons.gridy = 3;
35            add(btn4, gridCons);
36        }
37
38        public static void main(String args[]) {
39            Tela tela = new Tela();
40            tela.setVisible(true);
41        }
42    }
```

Observe na linha 13 a indicação de que a janela utilizará o *layout GridBagLayout*. Na linha 14 é instanciado um objeto da classe *GridBagConstraints* por meio do qual os elementos serão posicionados. Nas linhas de 16 à 19 os botões que serão inseridos na tela são criados.

Nas linhas 21 e 22 definem-se, respectivamente, a posição no eixo *x* e *y* (coluna e linha) para o botão 1 (*btn1*), inserido na tela na linha 23. Veja que a inserção na tela é feita pelo método *add* da classe *Container*, que precisa de dois parâmetros: o objeto a ser inserido e o posicionamento. Note que o botão *btn1* foi inserido na célula 0,0, ou seja, na coluna 0 e linha 0 da *grid*.

Na linha 25 é definida a posição do botão 2 (*btn2*). Veja que ele será colocado na terceira coluna. Como a linha não foi definida, por padrão será 0, ou seja, o botão será colocado na posição 3,0 (coluna 3 e linha 0). Na linha 26 o botão é adicionado à janela com o respectivo posicionamento.

Nas linhas 28 e 29 define-se que o botão 3 (*btn3*) será colocado na célula de posição 1,1, ou seja, coluna 1 e linha 1. Na linha 30 define-se que esta célula ocupará duas colunas, ou seja, terá a largura de duas colunas. Esse efeito é semelhante à mesclagem de células do Excel. Na linha 31, o botão 3 é adicionado à janela com o respectivo posicionamento.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Nas linhas 33 e 34 é definido o posicionamento do botão 4 (*btn4*). Este botão será posicionado na coluna 0 da linha 3, ou seja, na célula de posição 0,3. Na linha 35 o botão 4 é adicionado à janela com o respectivo posicionamento.

Ao compilar e executar o projeto pressionando-se a tecla *F6,* você verá a tela mostrada na Figura 6.21.



Figura 6.21:Janela usando GridBagLayout.

Apesar de existir uma grade, ela é utilizada apenas para o posicionamento do conteúdo, mas não é visualizada.

6.2.4.2 Definição das propriedades do GridBagLayout

No exemplo apresentado você percebeu o uso das propriedades *gridx*, *gridy* e *gridwidth*, acessadas por meio de um objeto da classe *GridBagConstraints*, para definir o posicionamento e dimensionamento das células do *layout* definido como *GridBagLayout*. Há, porém, muitas outras propriedades que podem ser definidas. Estas propriedades são apresentadas a seguir:

gridx e gridy – Especificam a posição x e y para adicionar o componente. A propriedade *gridx* permite especificar qual a posição absoluta horizontal onde o componente será posicionado. A propriedade *gridy* permite especificar qual a posição absoluta vertical onde o componente será posicionado.

gridheight e gridwidth – Especificam o tamanho do componente baseado em células. É com essa propriedade que você indicará o número de células que um elemento ocupará. A variável *gridwidth* especifica o número de células que o componente ocupará na horizontal. A variável *gridheight* especifica o número de células que o componente ocupará na vertical. Valor padrão é 1 para as duas propriedades.

weightx e weighty – Essas propriedades especificam um percentual de crescimento da célula, não do componente, quando a mesma precisar ser redimensionada. O valor padrão é 0 e os valores percentuais são dados em casas decimais, por exemplo: 0.03 refere-se a 3%, 1.00 refere-se a 100% etc.

UNINOVE – USO EXCLUSIVO PARA ALUNO

ipadx e ipady – Determinam as bordas internas do componente. O tamanho mínimo será o *ipad* + o valor mínimo do componente.

fill – Determina como redimensionar o componente. Os valores possíveis para esta propriedade são:

NONE – Não permite redimensionar o componente.

HORIZONTAL – Permite redimensionar horizontalmente, fazendo com que o componente ocupe toda a área horizontal na célula.

VERTICAL – Permite redimensionar verticalmente, fazendo com que o componente ocupe toda a área vertical na célula.

BOTH – Permite redimensionar o componente para ambas as direções.

O valor padrão é *NONE*.

Por exemplo, para que cada componente ocupe todo o espaço da célula digita-se a instrução seguinte:

```
gridCons.fill = GridBagConstraints.BOTH;
```

Onde *gridCons* é um objeto da classe *GridBagConstraints*.

anchor – Essa propriedade é utilizada somente quando o tamanho do componente é menor que a área que lhe foi concedida. Sendo assim, o *layout* precisa saber em que posição da célula deixará o componente. Os valores para essa propriedade são de dois tipos: Absolutos (*CENTER, NORTH, NORTHEAST, EAST, SOUTHEAST, SOUTH, SOUTHWEST, WEST,* e *NORTHWEST*) e Relativos (*PAGE_START, PAGE_END, LINE_START, LINE_END, FIRST_LINE_START, FIRST_LINE_END, LAST_LINE_START* e *LAST_LINE_END*). O valor padrão é *CENTER*.

insets – Determina o espaço vazio em volta do componente, como uma margem externa. Por exemplo, para deixar espaço de 5 *pixel* em volta de um componente em todos os lados, utiliza-se a instrução seguinte:

```
gridCons.insets = new Insets(5, 5, 5, 5);
```

Onde *gridCons* é um objeto da classe *GridBagConstraints*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.4.3 Exemplo com GridBagLayout utilizando diversas propriedades

Vamos construir um exemplo um pouco mais complexo, que utiliza mais propriedades na definição do *layout* mostradas na Figura 6.22.

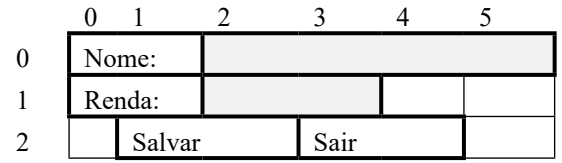


Figura 6.22: Exemplo de layout GridBagLayout.

Note que:

– O rótulo "Nome:" está na posição *gridx=0* e *gridy=0*, ocupando duas colunas (*gridwidth=2*).

– O campo para o preenchimento do nome está na posição *gridx=2* e *gridy=0*, ocupando quatro colunas (*gridwidth=4*).

– O rótulo "Renda:" está na posição (*gridx=0* e *gridy=1*), ocupando duas colunas (*gridwidth=2*).

– O campo para o preenchimento da renda está na posição *gridx=2* e *gridy=1*, ocupando duas colunas (*gridwidth=2*).

– O botão "Salvar" está na posição (*gridx=1* e *gridy=2*), ocupando duas colunas (*gridwidth=2*).

– O botão "Sair" está na posição (*gridx=3* e *gridy=2*), ocupando duas colunas (*gridwidth=2*).

Para gerar a tela apresentada, crie um novo projeto chamado *ProjetoGridBagNovo* e uma classe principal chamada *Tela.java*, conforme mostra a Figura 6.23.

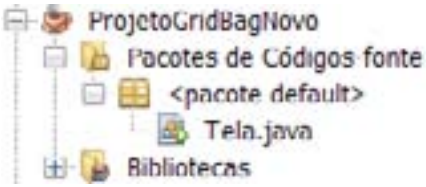


Figura 6.23: Projeto "ProjetoGridBagNovo".

UNINOVE – USO EXCLUSIVO PARA ALUNO

Na classe *Tela.java*, digite o seguinte código-fonte:

```
1   import java.awt.*;
2   import javax.swing.*;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4
5   public class Tela extends JFrame {
6
7       private JLabel lblNome, lblRenda;
8       private JTextField txtNome, txtRenda;
9       private JButton btnSalvar, btnSair;
10
11      public Tela() {
12          setTitle("Exemplo com GridBagLayout");
13          setDefaultCloseOperation(EXIT_ON_CLOSE);
14          setSize(700, 200);
15          setLayout(new GridBagLayout());
16          GridBagConstraints gridCons = new GridBagConstraints();
17
18          lblNome = new JLabel("Nome:");
19          txtNome = new JTextField(45);
20          lblRenda = new JLabel("Renda:");
21          txtRenda = new JTextField(15);
22          btnSalvar = new JButton("Salvar");
23          btnSair = new JButton("Sair");
24
25          gridCons.gridx = 0;
26          gridCons.gridy = 0;
27          gridCons.gridwidth = 2;
28          add(lblNome, gridCons);
29
30          gridCons.gridx = 2;
31          gridCons.gridy = 0;
32          gridCons.gridwidth = 4;
33          add(txtNome, gridCons);
34
35          gridCons.gridx = 0;
36          gridCons.gridy = 1;
37          gridCons.gridwidth = 2;
38          gridCons.insets = new Insets(15, 0, 8, 0);
39          add(lblRenda, gridCons);
40
41          gridCons.gridx = 2;
42          gridCons.gridy = 1;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
43            gridCons.gridwidth = 2;
44            gridCons.anchor = GridBagConstraints.WEST;
45            add(txtRenda, gridCons);
46
47            gridCons.gridx = 1;
48            gridCons.gridy = 2;
49            gridCons.gridwidth = 2;
50            gridCons.insets = new Insets(5, 5, 5, 5);
51            gridCons.anchor = GridBagConstraints.CENTER;
52            add(btnSalvar, gridCons);
53
54            gridCons.gridx = 3;
55            gridCons.gridy = 2;
56            gridCons.gridwidth = 2;
57            gridCons.anchor = GridBagConstraints.CENTER;
58            add(btnSair, gridCons);
59        }
60
61        public static void main(String args[]) {
62            Tela tela = new Tela();
63            tela.setVisible(true);
64        }
65    }
```

Neste exemplo será comentado apenas o que é novo em relação aos exemplos anteriores. Veja que o rótulo *lblNome*, criado na linha 18, será posicionado na coluna 0 e linha 0 e terá duas colunas de largura.

Já o campo *txtNome*, criado na linha 19, será posicionado na coluna 2 e linha 0 e terá 4 colunas de largura.

O rótulo *lblRenda*, criado na linha 20, será posicionado na coluna 0 e linha 1 e terá duas colunas de largura. Note que este rótulo terá 15 *pixels* de margem superior e 8 *pixel* de margem inferior. Isso foi definido pela propriedade *insets* (linha 38) com objetivo de desgrudar o conteúdo desta linha das demais.

O campo *txtRenda*, criado na linha 21, será posicionado na coluna 2 e linha 1 e terá 2 colunas de largura. Como o tamanho deste campo não chega a preencher as duas colunas, ele foi alinhado à esquerda pela propriedade *anchor* da linha 44.

O botão *btnSalvar*, criado na linha 22, será posicionado na coluna 1 e linha 2 e terá 2 colunas de largura. Este botão terá uma margem de 5 *pixel* em todos os seus lados (linha 50) e será centralizado (linha 51).

O botão *btnSair*, criado na linha 23, será posicionado na coluna 3 e linha 2 e terá 2 colunas de largura. Este botão será centralizado (linha 57).

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6,* será exibida a tela mostrada na Figura 6.24.

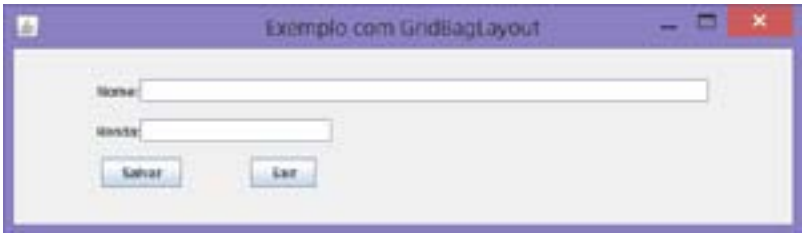


Figura 6.24: Tela criada com GridBagLayout.

Apesar de não termos utilizado nos exemplos apresentados a propriedade *gridheight*, ela é muito importante para fazer a mesclagem de linhas. Veja o *layout* mostrado na Figura 6.25.

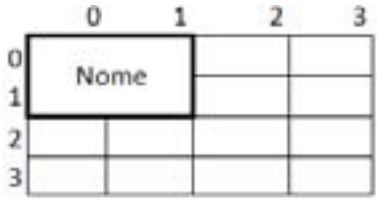


Figura 6.25: Exemplo de Layout que utiliza Grid BagLayout com mesclagem de células.

Para gerar o rótulo "Nome" serão necessárias as linhas abaixo, considerando *lblNome* do tipo *JLabel* e *gridCons* um objeto da classe *GridBagConstraints*. Veja como ficaria o bloco de código para gerar o rótulo "Nome":

```
lblNome = new JLabel("Nome:");
gridCons.gridx = 0;
gridCons.gridy = 0;
gridCons.gridwidth = 2;
gridCons.gridheight = 2;
gridCons.anchor = GridBagConstraints.CENTER;
add(lblNome, gridCons);
```

Criar uma tela com *GridBagLayout* permite um bom nível de personalização, porém este é o mais complexo dos gerenciadores de *layout*, dando quase tanto trabalho quanto criar o *layout* utilizando posicionamento estático.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.5 Utilizando gerenciadores de layout compostos

Em muitos casos a criação do *layout* necessita do uso de mais de um gerenciador de *layout* na mesma tela. Normalmente isso ocorre quando você só pode colocar um elemento em uma área do *layout* e dentro deste elemento contêiner (que pode ser, por exemplo, um *JPanel*), deseja colocar outros elementos. Nesse caso, a janela pode utilizar um gerenciador de *layout* e o elemento contêiner colocado em uma posição da janela pode utilizar outro. A decisão de qual e quantos gerenciadores de *layout* precisarão ser utilizados depende do projeto, mas você precisa saber quando usar cada um dos gerenciadores de *layout* vistos anteriormente.

DICA

Para colocar mais de um componente dentro de uma área de um contêiner devemos usar o gerenciador de *layout FlowLayout*. Se quisermos dividir uma área em células padrão, devemos usar o *GridLayout.* Se estas células precisam ter tamanhos e posicionamentos personalizados, utilizamos o *GridBagLayout*. Se quisermos dividir um contêiner em norte, sul, leste, oeste e centro, devemos usar o *BorderLayout*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.5.1 Exemplo utilizando gerenciadores de layout compostos

Neste tópico vamos criar uma tela semelhante a de um editor de texto simples, como o Bloco de Notas do Windows, utilizando dois gerenciadores de *layout*, *FlowLayout* e *BorderLayout*. Para isso, crie um novo projeto Java chamado ProjetoLayoutComposto e com uma classe principal chamada *Tela.java*, como mostra a Figura 6.26.

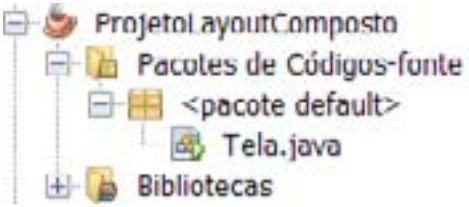


Figura 6.26: Projeto utilizando gerenciadores de layout compostos.

Na classe *Tela.java*, digite o seguinte código-fonte:

```
1   import java.awt.*;
2   import javax.swing.*;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4   import java.util.Date;
5   public class Tela extends JFrame {
6
7       private JPanel pnlBotoes;
8       private JPanel pnlBarraStatus;
9       private JScrollPane pnlScrTexto;
10      private JButton btnNovo, btnLimpar, btnSalvar, btnSair;
11      private JLabel lblMensagem, lblHora;
12      private JTextArea txtAreaTexto;
13
14      public Tela() {
15
16          // Propriedades da janela (JFrame)
17          setTitle("Layout composto");
18          setDefaultCloseOperation(EXIT_ON_CLOSE);
19          setLocation(200, 130);
20          setSize(640, 480);
21          setLayout(new BorderLayout());
22
23          // Criação dos elementos
24          btnNovo = new JButton("Novo");
25          btnSalvar = new JButton("Salvar");
26          btnLimpar = new JButton("Limpar");
27          btnSair = new JButton("Sair");
28          lblMensagem = new JLabel("Mensagem: ");
29          lblHora = new JLabel("Data e Hora: " + new Date().toString());
30          txtAreaTexto = new JTextArea("Digite seu texto aqui: ", 20, 40);
31          pnlScrTexto = new JScrollPane(txtAreaTexto,
            JScrollPane.VERTICAL_SCROLLBAR_ALWAYS,
            JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
32
33          // Criação do painel para colocar os botões e adição dos botões a
            este painel
34          pnlBotoes = new JPanel(new FlowLayout());
35          pnlBotoes.add(btnNovo);
36          pnlBotoes.add(btnSalvar);
37          pnlBotoes.add(btnLimpar);
38          pnlBotoes.add(btnSair);
39
40          // Criação do painel para a barra de status e adição dos botões a
            este painel
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
41            pnlBarraStatus = new JPanel(new FlowLayout());
42            pnlBarraStatus.add(lblMensagem);
43            pnlBarraStatus.add(lblHora);
44
45            // Adição dos paineis à Janela (JFrame)
46            add(pnlBotoes, BorderLayout.NORTH);
47            add(pnlScrTexto, BorderLayout.CENTER);
48            add(pnlBarraStatus, BorderLayout.SOUTH);
49
50        }
51
52        public static void main(String args[]) {
53            Tela tela = new Tela();
54            tela.setVisible(true);
55        }
56    }
```

Em uma janela (*JFrame*), que é considerada um contêiner, podem ser adicionados diversos elementos, inclusive outros contêineres, como *JScrollPane* e *JPanel*. Contêineres criados pela classe *JPanel* podem conter a maioria dos outros elementos de tela, como caixas de texto, rótulos, caixas de combinação etc. Contêineres criados pela classe *JScrollPane* podem conter caixas de texto dimensionadas com colunas e linhas definidas e já permitem apresentar barra de rolagem.

Para entender o código apresentado, segue a explicação das principais linhas.

Note que esta aplicação é composta por quatro painéis:

– A janela *JFrame*.

– O painel para colocar os botões (*pnlBotoes*).

– O painel com a caixa de texto (*pnlScrTexto*).

– O painel para compor uma barra de *status* (*pnlBarraStatus*).

A janela (*JFrame*) foi configurada para utilizar o *layout BorderLayout* (linha 21).

O painel para inserir os botões (*pnlBotoes*) foi configurado para usar o gerenciador de *layout FlowLayout* (linha 34).

O painel para inserir a barra de *status* (pnlBarraStatus) foi configurado para usar o gerenciador de *layout FlowLayout* (linha 41).

Para a área de texto (*pnlScrTexto*) não foi definido gerenciador de *layout* (linhas 30 e 31).

UNINOVE – USO EXCLUSIVO PARA ALUNO

Note que os painéis *pnlBotoes*, *pnlScrTexto* e *pnlBarraStatus* são criados, elementos são adicionados a eles e, por fim, eles são adicionados à janela *JFrame* (linhas de 46 à 48).

Note na linha 29, que o rótulo *lblHora* é criado com a data atual que é convertida do tipo *Date* para *String* por meio da instrução *new Date().toString()*, que já instancia um objeto da classe *Date*, que obtém a data atual e converte esta data para *String* por meio do método *toString*.

Na linha 30 a caixa de texto é criada com um texto padrão e com 20 linhas e 40 colunas. Está caixa de texto é adicionada ao painel (*pnlScrTexto*) que gera barras de rolagem horizontais e verticais (linha 31), que ficarão sempre visíveis (*ALWAYS*).

Nas linhas de 34 à 38 o painel *pnlBotoes* é criado e os botões são adicionados a ele.

Nas linhas de 41 à 43 o painel *pnlBarraStatus* é criado e os rótulos são adicionados a ele.

Na linha 46 o painel *pnlBotoes* é adicionado à janela ao norte, já que a janela usa o gerenciador de *layout BorderLayout*.

Na linha 47 o painel *pnlSrcTexto*, que contém a caixa de texto com barras de rolagem, é inserido na janela ao centro.

Na linha 48 o painel *pnlBarraStatus* é adicionado à janela ao sul.

Após compilar e executar o projeto pressionando a tecla *F6* será exibida a janela mostrada na Figura 6.27.



Figura 6.27: Tela com layout composto.

Os botões ainda não respondem ao evento de clique, mas veremos como se programa isto no decorrer deste capítulo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.5.2 Aspectos visuais da aplicação

Os componentes visuais da API *swing* podem sofrer a ação de um conjunto de métodos que nos permite controlar aspectos visuais dos elementos de tela, como tipo da fonte, cor da fonte, cor do fundo, cursor, borda etc. Para mostrar como estes aspectos visuais podem ser modificados, utilizaremos exemplos nos próximos tópicos considerando os elementos criados no exemplo do tópico anterior.

6.2.5.2.1 Mudando o aspecto do cursor

Considerando o último exemplo apresentado, veja as linhas que permitirá mudar o cursor quando o ponteiro do mouse for posicionado sobre os painéis *pnlBotoes*, *pnlBarraStatus* e sobre a área de texto *txtAreaTexto*:

```
Cursor hand = new Cursor(Cursor.HAND_CURSOR);
pnlBotoes.setCursor(hand);
Cursor text = new Cursor(Cursor.TEXT_CURSOR);
txtAreaTexto.setCursor(text);
Cursor cross = new Cursor(Cursor.CROSSHAIR_CURSOR);
pnlBarraStatus.setCursor(cross);
```

No painel *pnlBotoes* o cursor aparecerá no formato de uma mão com o dedo indicador apontando um clique (*Cursor.HAND_CURSOR*). Na caixa de texto *txtAreaTexto* aparecerá um cursor comum em áreas de texto (*Cursor.TEXT_CURSOR*). No painel *pnlBarraStatus* aparecerá um cursor no formato de uma cruz preta (*Cursor.CROSSHAIR_CURSOR*).

Note que em ambos os casos um objeto da classe *Cursor* é instanciado, passando para o construtor da classe, como parâmetro, uma constante que indica o cursor desejado. Em seguida, este objeto é passado como parâmetro a um método *setter* da classe que instanciou o elemento, fazendo com que o cursor escolhido seja aplicado ao elemento.

6.2.5.2.2 Mudando o tipo, efeito e tamanho da fonte

Para mudar o tipo da fonte de um elemento será necessário um objeto da classe *Font*. Veja o bloco de código a seguir:

```
Font arialBold = new Font("Calibri", Font.BOLD, 16);
txtAreaTexto.setFont(arialBold);
Font arial = new Font("Arial", Font.ITALIC, 10);
lblMensagem.setFont(arial);
```

Note que o tipo, efeito e tamanho da fonte é definido em um objeto da classe *Font* que é aplicado ao elemento por meio do método *setFont*. Na instância do

UNINOVE – USO EXCLUSIVO PARA ALUNO

objeto da classe *Font* são passados como parâmetro para o construtor o tipo da fonte, o efeito desejado e o tamanho da fonte.

6.2.5.2.3 MUDANDO A COR DA FONTE E DO PLANO DE FUNDO

Para definir a cor da letra de um elemento basta chamar o método *setForeground* a partir do objeto referente ao elemento e passar a cor como parâmetro. Isso pode ser feito de duas maneiras: passando uma constante da classe *Color* para indicar a cor ou passando um objeto da classe *Color* como parâmetro.

Veja a definição da cor da fonte por meio de uma constante da classe *Color*:

```
txtAreaTexto.setForeground(Color.blue);
```

Veja a definição da cor da fonte por meio de um objeto da classe *Color*:

```
Color cor = new Color(255,0,126);
txtAreaTexto.setForeground(cor);
```

Ou ainda:

```
txtAreaTexto.setForeground(new Color(255,0,126));
```

Para definir a cor do plano de fundo do elemento, utiliza-se o método *setBackground*, que funciona de forma semelhante ao método *setForeground* apresentado acima.

```
pnlBotoes.setBackground(new Color(0,255,0));
```

Ou ainda:

```
Color cor = new Color(0,255,0);
txtAreaTexto.setForeground(cor);
```

Ou ainda:

```
txtAreaTexto.setForeground(Color.green);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.5.2.4 Apresentando dicas sobre o elemento

As dicas são muito úteis para indicar formato de preenchimento de campos ou para descrever botões em um formulário. Elas aparecem ao posicionar o ponteiro do mouse sobre o elemento, apresentando o texto referente à dica. Veja um exemplo:

```
btnSalvar.setToolTipText("Clique neste botão para salvar o documento");
```

Ao posicionar o mouse sobre o botão "Salvar" (*btnSalvar*) o texto referente à dica será mostrado.

6.2.5.2.5 Definindo bordas para o elemento

Para colocar borda em um elemento é necessário um objeto da classe *Border*, instanciado a partir do construtor da subclasse *LineBorder*, passando como parâmetro a cor e a espessura da borda. Veja um exemplo:

```
Border borda = new LineBorder(Color.RED, 2);
pnlBarraStatus.setBorder(borda);
```

Ou ainda:

```
pnlBarraStatus.setBorder( new LineBorder(Color.RED, 2) );
```

Note que a aplicação no elemento da borda definida ocorre por meio do método *setBorder* da classe do elemento em que se quer aplicar a borda. Para que estas instruções funcionem terá que importar as classes *Border* e *LineBorder*, por meio das instruções:

```
import javax.swing.border.Border;
import javax.swing.border.LineBorder;
```

6.2.5.2.6 Centralizando a janela

Para que a janela apareça centralizada quando aplicação for executada, basta trocar as linhas:

```
setSize(800, 700);
setLocation(300, 200);
```

Por:

```
Toolkit tk = Toolkit.getDefaultToolkit();
Dimension d = tk.getScreenSize();
int largura = d.width/2;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
int altura = d.height/;
setSize(largura, altura);
setLocation(largura/2, altura/2);
```

Note que o tamanho da tela é obtido por meio de uma chamada ao método *getScreenSize* da classe *Toolkit* e armazenado em um objeto *d* da classe *Dimension*. Por meio desse objeto é possível obter a largura (*width*) e a altura (*height*) da tela. A divisão por 2 é utilizada para achar o ponto central da tela (eixos x e y). A janela terá a largura e a altura da medida deste ponto central, o que é definido na passagem do parâmetro para o método *setSize*. O ponto de posição inicial da janela será a medida do ponto central dividida por dois, o que é definido no parâmetro passado ao método *setLocation*.

6.2.5.2.7 Dimensionando elementos

Para definir a largura e a altura de um elemento (tamanho), utiliza-se o método *setSize* da classe referente ao elemento. Para definir a posição da tela em que o elemento aparecerá, utiliza-se o método *setLocation* da classe referente ao elemento. Veja alguns exemplos:

```
setSize(800, 700);
setLocation(300, 200);
btnSalvar.setSize(100, 30);
btnSalvar.setLocation(10, 20);
```

Em lugar destas instruções existe o método *setBounds*, que define a posição e o tamanho do elemento em uma única instrução. Veja como fica os exemplos anteriores utilizando o método *setBounds*:

```
setBounds(300, 200, 800, 700);
btnSalvar.setBounds(10, 20,100, 30);
```

Os dois primeiros parâmetros referem-se ao posicionamento do elemento nos eixos *x* e *y*. Os dois últimos parâmetros referem-se à largura e altura do elemento.

6.2.6 Criação de interfaces gráficas ricas utilizando a API *swing*

Até agora utilizamos classes para a criação de componentes visuais mais básicos da API *swing*, como janelas (classe *JFrame*), botões (classe *JButton*), rótulos (classe *JLabel*), campos de texto (classe *JTextField*), área de texto (classe *JTextArea*), painéis de rolagem (classe *JScrollPane*) e painel de uso geral (classe *JPanel*).

UNINOVE – USO EXCLUSIVO PARA ALUNO

Neste tópico, vamos utilizar outros componentes que nos permitirá criar interfaces mais ricas, com menus (classe *JMenuBar*), caixas de diálogo (classe *JOptionPane*), combos (classe *JComboBox*), listas (classe *JList*) e tabelas (classe *JTable*).

6.2.6.1 Criando menus

Para construir um menu você precisará criar uma barra de menus, as opções do menu principal e os itens de cada opção deste menu. Para isso terá que utilizar, respectivamente, as classes *JMenuBar*, *JMenu* e *JMenuItem*.

Primeiro você cria os itens de menu, os menus e a barra de menus. Depois, adiciona os itens de menu aos menus, adiciona os menus na barra de menus e, por último, adiciona a barra de menu à janela.

Vamos criar um exemplo para gerar a tela mostrada na Figura 6.28.

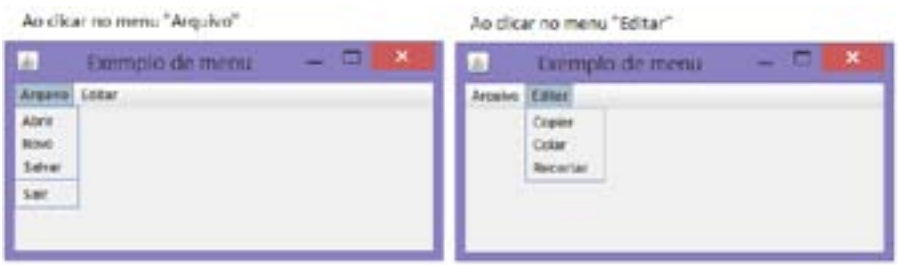


Figura 6.28: Criação de menus.

Crie um projeto chamado *ProjetoMenu* com uma classe principal chamada *TelaMenu.java*, conforme mostra a Figura 6.29.



Figura 6.29: Projeto para criação de menus.

Na classe *TelaMenu.java*, digite o seguinte código-fonte:

```
1    import javax.swing.*;
2    import static javax.swing.JFrame.EXIT_ON_CLOSE;
3
4    public class TelaMenu extends JFrame {
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
5
6       private JMenuItem mnuItmAbrir, mnuItmNovo, mnuItmSalvar, mnuItmSair,
        mnuItmCopiar, mnuItmColar, mnuItmRecortar;
7       private JMenu mnuArquivo, mnuEditar;
8       private JMenuBar mnuBar;
9
10      public TelaMenu() {
11          setTitle("Exemplo de menu");
12          setDefaultCloseOperation(EXIT_ON_CLOSE);
13          setSize(400, 200);
14          setLocation(300, 200);
15          // instanciando os objetos referentes aos elementos do menu
16          mnuItmAbrir = new JMenuItem("Abrir");
17          mnuItmNovo = new JMenuItem("Novo");
18          mnuItmSalvar = new JMenuItem("Salvar");
19          mnuItmSair = new JMenuItem("Sair");
20          mnuItmCopiar = new JMenuItem("Copiar");
21          mnuItmColar = new JMenuItem("Colar");
22          mnuItmRecortar = new JMenuItem("Recortar");
23          mnuArquivo = new JMenu("Arquivo");
24          mnuEditar = new JMenu("Editar");
25          // Adicionando os itens de menu ao menu Arquivo
26          mnuArquivo.add(mnuItmAbrir);
27          mnuArquivo.add(mnuItmNovo);
28          mnuArquivo.add(mnuItmSalvar);
29          mnuArquivo.addSeparator();
30          mnuArquivo.add(mnuItmSair);
31          // Adicionando os itens de menu ao menu Editar
32          mnuEditar.add(mnuItmCopiar);
33          mnuEditar.add(mnuItmColar);
34          mnuEditar.add(mnuItmRecortar);
35          // Adicionando os menus à barra de menus
36          mnuBar = new JMenuBar();
37          mnuBar.add(mnuArquivo);
38          mnuBar.add(mnuEditar);
39          // Adicionando a barra de menus à janela
40          setJMenuBar(mnuBar);
41      }
42
43      public static void main(String args[]) {
44          TelaMenu tela = new TelaMenu();
45          tela.setVisible(true);
46      }
47  }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas de 6 à 8 declaram os objetos que serão utilizados nos menus. Estes objetos são instanciados nas linhas de 16 à 24.

As linhas de 26 à 30 constroem o menu "Arquivo", ou seja, adiciona (ou agrega) as opções "Abrir", "Novo", "Salvar" e "Sair" ao menu "Arquivo". Note, na linha 29, que o método *addSeparator* adiciona uma linha de separação entre as opções "Salvar" e "Sair".

As linhas de 32 à 34 constroem o menu "Editar", ou seja, adiciona as opções "Copiar", "Colar" e "Recortar" ao menu "Editar".

As linhas 37 e 38 adicionam o menu "Arquivo" e o menu "Editar" à barra de menus.

A linha 40 adiciona a barra de menus à janela. Note que não foi necessário definir um gerenciador de *layout* para a barra de menus, pois ela é posicionada, por padrão, na parte superior da tela.

Se desejasse você poderia adicionar outros elementos na tela abaixo da barra de menus. Observe que as ações de clique das opções do menu não estão funcionando, pois ainda não foram programadas. Você vai aprender como programar botões e opções de menu mais à frente, ainda nesse capítulo.

6.2.6.2 Trabalhando com caixas de diálogo

A classe *JOptionPane* da API *swing* traz um conjunto de caixas de diálogo prontas que podem ser chamadas pelos métodos *showConfirmDialog*, *showInputDialog*, *showMessageDialog* e *showOptionDialog*. Estes métodos são descritos a seguir:

– *showConfirmDialog* – confirmação de ação.
– *showInputDialog* – entrada de dados.
– *showMessageDialog* – exibição de mensagem.
– *showOptionDialog* – caixa personalizada.

NOTA Os métodos da classe *JOptionPane* são públicos e estáticos, por isso podem ser chamados diretamente a partir do nome da classe. Estes métodos também são sobrecarregados, ou seja, são escritos na classe várias vezes, porém, recebendo parâmetros diferentes.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.6.2.1 Método showMessageDialog

O método *showMessageDialog* da classe *JOptionPane* permite mostrar diversos tipos de mensagens em uma caixa de diálogo.

A forma mais simples de uso desse método é definindo apenas a mensagem:

```
JOptionPane.showMessageDialog(null, "Erro na aplicação");
```

Note que apenas dois parâmetros são passados para o método *showMessageDialog*: *null* e a mensagem "Erro na aplicação".

Veja a tela que será gerada na Figura 6.30.

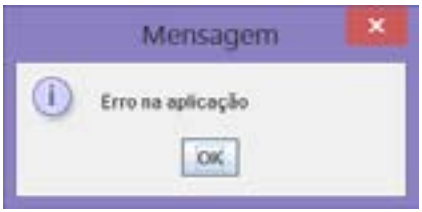


Figura 6.30: Caixa de diálogo de mensagem.

O parâmetro *null* refere-se ao nome nome do objeto em que se quer relacionar a caixa de diálogo. Se for, por exemplo, um botão *JButton* chamado *btnSalvar*, você poderia escrever a linha da seguinte forma:

```
JOptionPane.showMessageDialog(btnSalvar, "Erro na aplicação");
```

Nesse caso, a caixa de diálogo será relacionada ao botão *btnSalvar* e aparecerá, consequentemente, sobre este botão na tela.

Veja outro exemplo onde outras propriedades da caixa de diálogo são definidas, como título, formato do ícone e mensagem:

```
JOptionPane.showMessageDialog(null, "Erro na aplicação", "Falha no aplicativo", JOptionPane.
ERROR_MESSAGE);
```

Nesse caso, o método *showMessageDialog* recebe como parâmetro o nome do objeto ao qual a caixa de diálogo estará relacionada (neste caso, *null* significa que não estará relacionada a nenhum objeto), a mensagem, o título da caixa de diálogo e o formato do ícone que ficará à esquerda da mensagem. Veja a janela que será gerada na Figura 6.31.

UNINOVE – USO EXCLUSIVO PARA ALUNO

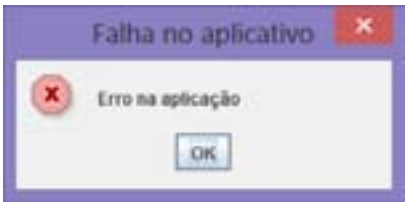


Figura 6.31: Caixa de diálogo indicando erro.

6.2.6.2.2 Método showInputDialog

O método *showInputDialog* abre uma caixa de diálogo onde se pode entrar com um valor. O valor digitado será sempre do tipo *String*, independente de ser número ou não. Veja um exemplo:

```
String nome = JOptionPane.showInputDialog("Qual seu nome?:");
```

Veja na Figura 6.32 o que aparecerá na tela ao executar a linha acima:

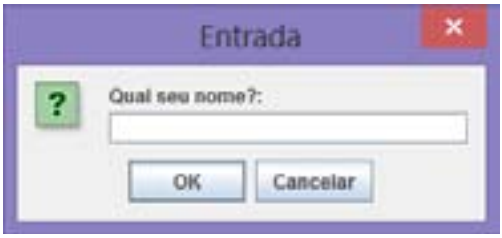


Figura 6.32: Caixa de diálogo para entrada de dados.

Você também pode pedir a informação colocando título na janela e personalizando o ícone que aparece à esquerda. Veja como na linha abaixo:

```
String nome = JOptionPane.showInputDialog(null, "Qual seu nome?: ", "Cadastro", JOptionPane.
DEFAULT_OPTION);
```

Veja na Figura 6.33 o que aparece na tela ao executar a linha acima:



Figura 6.33: Caixa de diálogo de entrada com título.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Como visto nos capítulos anteriores, para receber o valor digitado em uma variável de tipo numérico primitivo você deve converter o valor digitado de *String* para o tipo numérico desejado.

Veja os exemplos a seguir:

```
byte idade = Byte.parseByte(JOptionPane.showInputDialog("Qual a sua idade"));
int cod = Integer.parseInt(JOptionPane.showInputDialog("Digite o código do produto"));
long codBarra = Long.parseLong(JOptionPane.showInputDialog("Digite o código de barras"));
double sal = Double.parseDouble(JOptionPane.showInputDialog("Digite o salário"));
```

6.2.6.2.3 MÉTODO SHOWCONFIRMDIALOG

O método *showConfirmDialog* gera uma janela para confirmação de alguma ação, onde o usuário pode clicar em uma das opções disponíveis, normalmente, "Sim" ou "Não". Veja um exemplo:

```
int op = JOptionPane.showConfirmDialog(null, "Deseja encerrar?", "sair do Sistema",
JOptionPane.YES_NO_OPTION);
if (op == 0) {
  JOptionPane.showMessageDialog(null, "Você clicou na opção Sim");
} else if (op == 1) {
  JOptionPane.showMessageDialog(null, "Você clicou na opção Não");
}
```

Ao executar o código acima, veja na Figura 6.34 como aparecerá a caixa de diálogo:

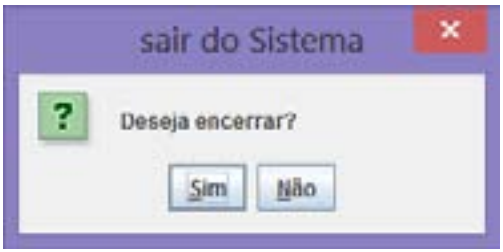


Figura 6.34: Caixa de diálogo de confirmação.

Se clicar na opção "Sim", este método retorna o valor 0. Se clicar na opção "Não", este método retorna o valor 1.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.6.2.4 Método showOptionDialog

O método *showOptionDialog* permite criar caixas de diálogo personalizadas a partir de opções presentes em uma *array* unidimensional. Veja o exemplo:

```
Object[] opt = {"Sim", "Não", "Não sei"};
 int op = JOptionPane.showOptionDialog(null, "Clique OK para continuar", "Aviso", JOptionPane.DEFAULT_OPTION, JOptionPane.INFORMATION_MESSAGE, null, opt, opt[0]);
 if (op == 0) {
  JOptionPane.showMessageDialog(null, "Você clicou na opção Sim");
 } else if (op == 1) {
  JOptionPane.showMessageDialog(null, "Você clicou na opção Não");
 } else if (op == 2) {
  JOptionPane.showMessageDialog(null, "Você clicou na opção Não Sei");
 }
```

Veja na Figura 6.35 a caixa de diálogo gerada:



Figura 6.35: caixa de diálogo de confirmação personalizada.

Se clicar no botão "Sim", o método *showOptionDialog* retornará o valor 0. Se clicar no botão "Não", este método retornará o valor 1. Se clicar no botão "Não Sei", este método retornará o valor 2.

6.2.6.3 Trabalhando por botões

Na API *swing* há várias classes para criar os mais variados tipos de botões. Já utilizamos a classe *JButton* para criar botões padrão.

Existe ainda a classe *JRadioButton*, para criar botões circulares agrupados pela classe *ButtonGroup,* de forma que apenas um dos botões do grupo possa ser selecionado. É ideal para escolha de sexo, estado civil etc.

A classe *JCheckBox* permite criar botões quadrados e pequenos e permite que mais de um esteja selecionado ao mesmo tempo. É ideal para seleção de produtos, por exemplo.

A classe *JToogleButton* permite criar botões que podem ser ativados e desativados, alternado a cada clique.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Para testar a criação de botões, crie um projeto chamado *ProjetoBotoes* com uma classe chamada *Tela.java*, conforme mostra a Figura .



Figura 6.36: Projeto utilizando botões variados.

Digite o seguinte código-fonte na classe *Tela.java*:

```
1   import javax.swing.*;
2   import java.awt.*;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4
5   public class Tela extends JFrame {
6
7       private JButton btnSalvar, btnAbrir;
8       private JCheckBox chkTeatro, chkCinema, chkTv;
9       private JRadioButton rdoMasculino, rdoFeminino;
10      private JToggleButton tglAlarme;
11      private JPanel pnlBotoes1, pnlBotoes2, pnlBotoes3, pnlBotoes4;
12
13      public Tela() {
14          // Propriedades dajanela JFrame
15          setTitle("Exemplo com botões");
16          setDefaultCloseOperation(EXIT_ON_CLOSE);
17          setLayout(new GridLayout(4, 1));
18          setSize(300, 200);
19          setLocation(300, 200);
20
21          // Criação (instância) dos botões
22          btnSalvar = new JButton("Salvar");
23          btnAbrir = new JButton("Abrir");
24          chkTeatro = new JCheckBox("Teatro");
25          chkCinema = new JCheckBox("Cinema");
26          chkTv = new JCheckBox("TV");
27          rdoMasculino = new JRadioButton("Masculino");
28          rdoFeminino = new JRadioButton("Feminino");
29          tglAlarme = new JToggleButton("Alarme");
30
31          // Criação do painel 1 e adição de botões
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
32          pnlBotoes1 = new JPanel();
33          pnlBotoes1.add(btnSalvar);
34          pnlBotoes1.add(btnAbrir);
35
36          // Criação do painel 2 e adição de botões
37          pnlBotoes2 = new JPanel();
38          pnlBotoes2.add(chkTeatro);
39          pnlBotoes2.add(chkCinema);
40          pnlBotoes2.add(chkTv);
41
42          // Criação do grupo ao qual são adicionados os botões radio
43          ButtonGroup gruSexo = new ButtonGroup();
44          gruSexo.add(rdoMasculino);
45          gruSexo.add(rdoFeminino);
46
47          // Criação do painel 3 e adição de botões
48          pnlBotoes3 = new JPanel();
49          pnlBotoes3.add(rdoMasculino);
50          pnlBotoes3.add(rdoFeminino);
51
52          // Criação do painel 4 e adição dobotão
53          pnlBotoes4 = new JPanel();
54          pnlBotoes4.add(tglAlarme);
55
56          // Adição dos botões à janela
57          add(pnlBotoes1);
58          add(pnlBotoes2);
59          add(pnlBotoes3);
60          add(pnlBotoes4);
61      }
62
63      public static void main(String arg[]) {
64          Tela tela = new Tela();
65          tela.setVisible(true);
66      }
67  }
```

A tela é dividida em 4 células de tamanhos iguais pelo gerenciador de *layout GridLayout* (linha 17), ou seja, este gerenciador define que a janela terá 4 linhas e uma coluna.

Note que são criados 4 painéis (*pnlBotoes1*, *pnlBotoes2*, *pnlBotoes3* e *pnlBotoes4*). Em cada um são adicionados os botões de tipos diferentes (linhas 32 à 34, 37 à 40, 48 à 50, 53 e 54) e depois esses painéis são adicionados à janela (linhas 57 à 60).

UNINOVE – USO EXCLUSIVO PARA ALUNO

Perceba nas linhas de 43 à 45 que os botões do tipo *JRadioButton* são adicionados a um mesmo grupo do tipo *ButtonGroup*. Isso é necessário para que não seja permitido selecionar mais de um botão do grupo.

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6* será exibida a janela mostrada na Figura



Figura 6.37: Janela com botões diversificados.

Observe que os botões de cada linha estão em um painel diferente. Note que podem ser selecionados vários botões *check* da segunda linha, porém na terceira linha é permitido selecionar apenas um sexo por meio dos botões *radio*. Na terceira linha o botão "Alarme" pode ser ativado com um clique e desativado com outro. Veja na Figura 6.38 como ficará a janela após as algumas seleções de botões.



Figura 6.38: janela com botões selecionados.

Ainda não estamos programando as ações dos botões, ou seja, o que deve ocorrer quando o botão for clicado, mas vamos fazer isso mais a frente no tópico sobre tratamento de eventos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.6.3.1 Métodos para trabalhar com botões

Há um conjunto de métodos para se trabalhar com botões que pode incluir ou obter o rótulo dos botões, verificar se o botão foi selecionado, selecionar o botão automaticamente etc. Estes métodos se aplicam a todos os botões vistos no exemplo anterior, ou seja, a botões das classes *JCheckBox*, *JButton*, *JRadioButton* e *JToggleButton*. Veja a seguir os principais métodos:

String teatro = chkTeatro.getText();

Obtém o rótulo do botão e armazena em uma variável *String*.

chkTeatro.setText("Shows");

Troca o rótulo do botão pela palavra "Shows".

chkTeatro.setSelected(true);

Seleciona automaticamente o botão.

chkTeatro.setVisible(true);

Torna o botão visível. Para ocultá-lo, mude o parâmetro para *false*.

if(chkTeatro.isSelected()){}

Verifica se o botão está selecionado.

6.2.6.4 Trabalhando com campos texto

Em uma janela a entrada de dados é feita normalmente por meio de campos texto. Há várias classes na API *swing* que permitem a entrada de texto, cada uma com características específicas.

A classe *JTextField* já foi utilizada em projetos anteriores nesse livro. Ela permite a criação de uma caixa para entrada de texto com uma linha, sem formatação.

A classe *JTextArea* permite a criação de caixas para entrada de texto com múltiplas linhas, sem formatação.

A classe *JPasswordField* permite a criação de campos para entrada de senhas, com uma linha, sem formatação, onde os caracteres digitados não aparecem de forma clara.

A classe *JFormattedTextField* permite a criação de campos formatados, onde se pode definir máscaras de entrada por meio da classe *MaskFormatter* e outros tipos de formatação.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Para testar o uso dessas classes, crie um novo projeto chamado *ProjetoCampos,* com uma classe chamada *Tela.java*, como mostra a Figura 6.39.



Figura 6.39: projeto utilizando campos texto.

Digite o seguinte código-fonte para a classe *Tela.java*:

```
1   import javax.swing.*;
2   import java.awt.*;
3   import java.text.*;
4   import static javax.swing.JFrame.EXIT_ON_CLOSE;
5   import javax.swing.text.MaskFormatter;
6
7   public class Tela extends JFrame {
8
9      private JLabel lblNome, lblNascimento, lblTelefone, lblCpf, lblCep, lblEndereco;
10     private JTextField txtNome;
11     private JFormattedTextField txtNascimento, txtCep, txtTelefone, txtCpf;
12     private JTextArea txtEndereco;
13     MaskFormatter mskCep, mskTelefone, mskCpf, mskNascimento;
14
15     public Tela() {
16        setLayout(new GridLayout(6, 1));
17        FlowLayout esquerda = new FlowLayout(FlowLayout.LEFT);
18        setDefaultCloseOperation(EXIT_ON_CLOSE);
19        setSize(600, 450);
20        setLocation(300, 200);
21        // Criação dos rótulos dos campos
22        lblNome = new JLabel("Nome");
23        lblNascimento = new JLabel("Nascimento");
24        lblTelefone = new JLabel("Telefone");
25        lblCpf = new JLabel("CPF");
26        lblCep = new JLabel("CEP");
27        lblEndereco = new JLabel("Endereço");
28        // Criação dos campos e definição das máscaras quando pertinente
29        txtNome = new JTextField(40);
30        try {
31           mskNascimento = new MaskFormatter("##/##/####");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
32              mskNascimento.setPlaceholderCharacter('_');
33              mskTelefone = new MaskFormatter("(##)####-####");
34              mskTelefone.setPlaceholderCharacter('_');
35              mskCpf = new MaskFormatter("#########-##");
36              mskCpf.setPlaceholderCharacter('_');
37              mskCep = new MaskFormatter("#####-###");
38              mskCep.setPlaceholderCharacter('_');
39          } catch (ParseException ex) {
40              JOptionPane.showMessageDialog (null,"Erro na definição das
                máscaras dos campos");
41          }
42          txtNascimento = new JFormattedTextField(mskNascimento);
43          txtTelefone = new JFormattedTextField(mskTelefone);
44          txtCpf = new JFormattedTextField(mskCpf);
45          txtCep = new JFormattedTextField(mskCep);
46          txtEndereco = new JTextArea("", 3, 40);
47          JScrollPane scrollEndereco = new JScrollPane(txtEndereco,
            JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED,
            JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED);
48          // Criação do painel para inserção do rótulo e do campo nome
49          JPanel pnlNome = new JPanel(esquerda);
50          pnlNome.add(lblNome);
51          pnlNome.add(txtNome);
52          // Criação do painel para inserção do rótulo e do campo nascimento
53          JPanel pnlNascimento = new JPanel(esquerda);
54          pnlNascimento.add(lblNascimento);
55          pnlNascimento.add(txtNascimento);
56          // Criação do painel para inserção do rótulo e do campo telefone
57          JPanel pnlTelefone = new JPanel(esquerda);
58          pnlTelefone.add(lblTelefone);
59          pnlTelefone.add(txtTelefone);
60          // Criação do painel para inserção do rótulo e do campo CPF
61          JPanel pnlCpf = new JPanel(esquerda);
62          pnlCpf.add(lblCpf);
63          pnlCpf.add(txtCpf);
64          // Criação do painel para inserção do rótulo e do campo CEP
65          JPanel pnlCep = new JPanel(esquerda);
66          pnlCep.add(lblCep);
67          pnlCep.add(txtCep);
68          // Criação do painel para inserção do rótulo e do campo endereço
69          JPanel pnlEndereco = new JPanel(esquerda);
70          pnlEndereco.add(lblEndereco);
71          pnlEndereco.add(scrollEndereco);
72          // Adição dos paineis à janela
73          add(pnlNome);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
74              add(pnlNascimento);
75              add(pnlTelefone);
76              add(pnlCpf);
77              add(pnlCep);
78              add(pnlEndereco);
79
80          }
81
82          public static void main(String arg[]) {
83              Tela tela = new Tela();
84              tela.setVisible(true);
85          }
86      }
```

Esta classe apresenta um conjunto de campos de entrada na tela, desde campos simples até campos formatados e caixas de texto. Cada campo é colocado em um painel e este painel é colocado na janela, um abaixo do outro, sequencialmente.

Temos uma forma um pouco diferente de aplicação de gerenciadores de *layout*. Veja na linha 16 que a janela (*JFrame*) usa o gerenciador de *layout GridLayout* com 6 linhas e uma coluna, ou seja, com 6 células, uma abaixo da outra. Em cada célula será colocado um painel com um campo diferente.

Como se deseja colocar os painéis alinhados à esquerda, instanciou-se um objeto chamado *esquerda*, da classe *FlowLayout*, na linha 17, utilizando o parâmetro *FlowLayout.LEFT* que define o alinhamento do conteúdo do contêiner que aplicar esse *layout* à esquerda. Note que este gerenciador de *layout* representado pelo objeto *esquerda* é usado nos painéis instanciados nas linhas 49, 53, 57 61, 65 e 69. Isso faz com que estes painéis tenham seu conteúdo alinhado à esquerda.

Nas linhas de 31 à 38 foram definidas as máscaras de entrada para diversos campos. Perceba que para cada campo é definida uma máscara por meio de um objeto da classe *MaskFormatter* e o caracter que deve aparecer no lugar do cerquilha (#) da máscara, neste caso, é o *underline* (_).

Perceba que as máscaras são aplicadas aos campos nas linhas de 42 à 45. Note também que a criação das máscaras ocorre no interior de uma estrutura de tratamento de exceção *try*. Isso é necessário porque na criação da máscara é instanciado um objeto da classe *MaskFormatter* passando a máscara como parâmetro *String*. Esse parâmetro pode estar incorreto e, nesse caso, quando ele for convertido de *String* para uma máscara válida pode ocorrer uma exceção do tipo *ParseException*, caindo no bloco *catch* e exibindo uma mensagem. Se as máscaras estiverem corretas, o bloco *catch* jamais será executado.

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas 46 e 47 criam uma caixa de texto para o endereço. Veja na linha 46 que o objeto da caixa de texto (*JTextArea*) é instanciado vazio, com 3 linhas e 40 colunas. Como caixas de texto podem conter mais texto do que a parte visível comporta, a linha 47 adiciona barras de rolagem à caixa de texto que aparecerão somente se o texto exceder o tamanho da caixa.

As linhas de 49 à 51 criam um painel e incluem o campo *nome* nele. As linhas de 53 à 55 criam outro painel e incluem o *endereço*. As demais linhas, na sequência, criam os painéis para os demais campos e incluem os respectivos campos neles.

As linhas de 73 à 78 incluem os painéis na janela, lembrando que eles usam o gerenciador de *layout FlowLayout* e a janela usa o gerenciador de *layout GridLayout*.

DICA Use sempre que possível painéis para agrupar campos ou para agrupar rótulo e campo. Painéis podem ser configurados e formatados facilmente, podendo receber cores, bordas etc.

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6* aparecerá a tela mostrada na Figura 6.40.

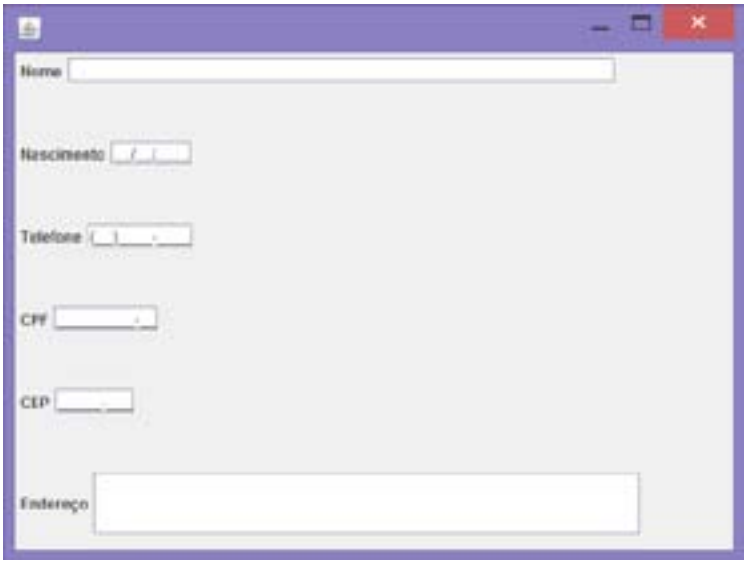


Figura 6.40: Janela com campos.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja na Figura 6.41 como fica a tela após o preenchimento dos campos.

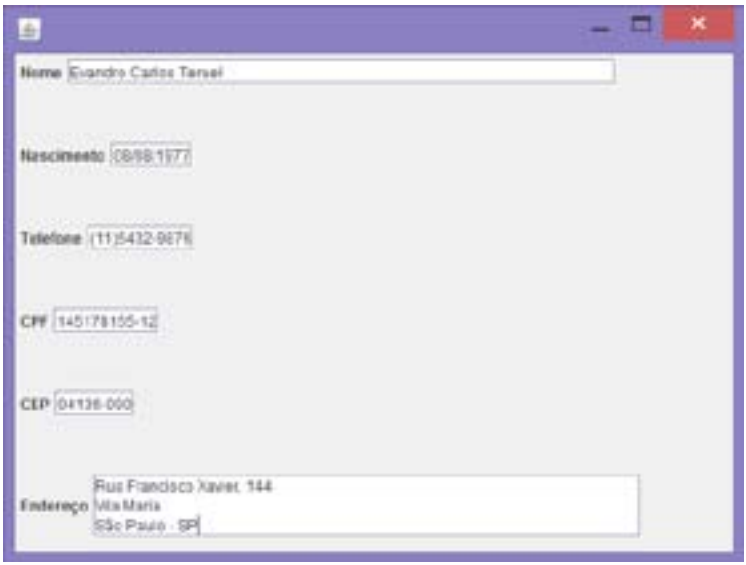


Figura 6.41: janela com campos preenchidos.

6.2.6.4.1 MÉTODOS PARA TRABALHAR COM CAMPOS TEXTO

Há um conjunto de métodos que pode manipular valores em campos texto e permitem, entre outras coisas, adicionar valores aos campos, obter os valores digitados, posicionar o cursor no campo etc.

Os exemplos apresentados a seguir aplicam-se a todos os tipos de campos texto vistos até agora, ou seja, a campos das classes *JFormattedTextField*, *JTextField* e *JTextArea*.

String cep = txtCep.getText();

Obtém o texto digitado no campo e armazena em uma variável do tipo *String*.

txtCep.requestFocus();

Posiciona o cursor no campo.

txtCep.setForeground(Color.BLUE);

Define a cor de letra para o campo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

txtCep.setBackground(Color.red);

Define a cor de fundo para o campo.

txtCep.setFont(new Font("Arial", Font.BOLD, 14));

Define uma configuração de fonte para o campo.

txtCep.setText("04135-090");

Inclui o valor entre aspas no campo.

6.2.6.5 Trabalhando com listas

Para trabalhar com listas utilizando a API *swing* normalmente você utiliza duas classes principais: *JList* e *DefaultListModel*.

A classe *JList* cria a lista. Se quisermos utilizar um modelo de lista já existente (o que é indicado) utilizamos a classe *DefaultListModel*.

Para entender como trabalhar com listas, crie um projeto chamado *ProjetoLista* com uma classe principal chamada *TelaLista.java*, conforme mostra a Figura 6.42.

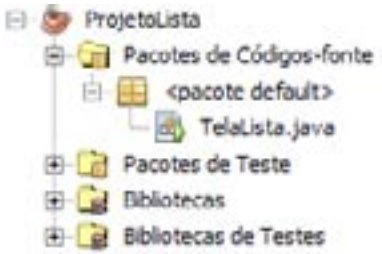


Figura 6.42: Projeto para trabalhar com listas.

Na classe *TelaLista.java*, digite o seguinte código-fonte:

```
1    import javax.swing.*;
2    import java.awt.*;
3
4    public class TelaLista extends JFrame {
5
6        private JList lstCidades;
7        private DefaultListModel lstCidadesModel;
8
9
10       public TelaLista() {
11
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
12          setTitle("Exemplo com lista");
13          setDefaultCloseOperation(EXIT_ON_CLOSE);
14          setSize(200, 150);
15          setLocation(300, 200);
16
17
18          lstCidadesModel = new DefaultListModel();
19          lstCidades = new JList(lstCidadesModel);
20
21
22          String cidade1 = "São Paulo";
23          String cidade2 = "Rio de Janeiro";
24          String cidade3 = "Campinas";
25
26          lstCidadesModel.addElement(cidade1);
27          lstCidadesModel.addElement(cidade2);
28          lstCidadesModel.addElement(cidade3);
29
30          add(lstCidades, BorderLayout.CENTER);
31
32          JLabel lblCidade = new JLabel("Escolha uma cidade");
33          add(lblCidade, BorderLayout.NORTH);
34      }
35
36      public static void main(String arg[]) {
37          TelaLista tela = new TelaLista();
38          tela.setVisible(true);
39      }
40  }
```

Para criar uma lista foram utilizados objetos de duas classes: *DefaultListModel* e *JList*. Quando foi instanciado um objeto da classe *JList* que efetivamente criou a lista, foi preciso passar como parâmetro o objeto da classe *DefaultListModel*, indicando que a lista utiliza um modelo padrão (linha 19).

Note nas linhas de 22 à 24 que são criadas três variáveis do tipo *String* como nomes de cidades que serão colocadas na lista nas linhas de 26 à 28 por meio dos métodos *addElement* da classe *DefaultListModel*. As opções da lista recebem um número de índice iniciado por 0, na ordem que são inseridas na lista.

Na linha 30 a lista é adicionada à janela (*JFrame*) utilizando o gerenciador de *layout BorderLayout*. Veja que a lista é posicionada na área "*CENTER*", ou seja, na área central do *layout*.

Para que a lista tenha um rótulo como título, um objeto da classe *JLabel* foi instanciado na linha 32 e adicionado à janela, ao norte.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6*, aparecerá a tela mostrada na Figura 6.43.

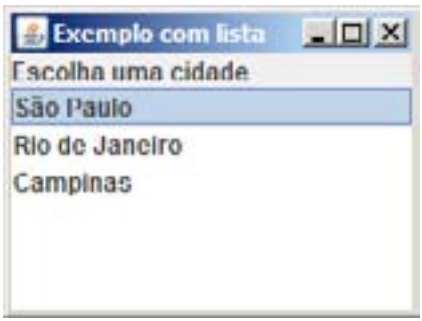


Figura 6.43: janela com listas.

Veja que foi clicado na primeira opção da lista e ela ficou selecionada. Você pode rolar o cursor sobre os elementos da lista utilizando as teclas de direção do teclado ou clicar na opção que deseja selecionar.

6.2.6.5.1 Métodos para trabalhar com as listas

Na classe apresentada, por meio do objeto *lstCidadesModel* da classe *DefaultListModel* é possível acessar um conjunto de métodos úteis para manipular opções na lista, dentre eles:

lstCidadesModel.add(2, "Franca");

Adiciona o elemento "Franca" na terceira posição da lista de cima para baixo, ou seja, na posição de índice 2.

lstCidadesModel.clear();

Remove todos os elementos da lista.

if(lstCidadesModel.contains("Campinas")){ }

Verifica se a lista contém a opção "Campinas".

if(lstCidadesModel.isEmpty()){}

Verifica se a lista está vazia.

UNINOVE – USO EXCLUSIVO PARA ALUNO

String cidade = (String) lstCidadesModel.elementAt(1);

Obtém o elemento de índice 1 da lista, converte em String e armazena na variável String cidade.

String cidade = (String) lstCidadesModel.firstElement();

Obtém o primeiro elemento da lista, converte em String e armazena na variável String cidade.

String cidade = (String) lstCidadesModel.lastElement();

Obtém o último elemento da lista, converte em String e armazena na variável String cidade.

String cidade = (String) lstCidadesModel.get(1);

Obtém o elemento de índice 1 da lista, converte em String e armazena na variável String cidade.

int total = lstCidadesModel.getSize();

Obtém o número de elementos da lista.

lstCidadesModel.insertElementAt("Itu", 2);

Insere o elemento "Itu" na posição de índice 2 da lista.

lstCidadesModel.remove(2);

Remove o elemento de índice número 2 da lista, ou seja, a terceira opção da lista de cima para baixo.

lstCidadesModel.removeAllElements();

Remove todos os elementos da lista e define o tamanho da lista como 0.

lstCidadesModel.removeElement("São Paulo");

Remove o elemento "São Paulo" da lista.

UNINOVE – USO EXCLUSIVO PARA ALUNO

lstCidadesModel.removeElementAt(2);

Remove o elemento de índice 2.

lstCidadesModel.set(2, "Votuporanga");

Substitui o item de índice 2 da lista por "Votuporanga".

lstCidadesModel.setSize(2);

Define o número de elementos visíveis na lista.

int tamanho = lstCidadesModel.size();

Obtém o tamanho da lista.

Você percebeu que as opções foram adicionadas à lista (*JList*) por meio do seu modelo (*DefaultListModel*). Se você quiser, pode dispensar o modelo e adicionar as opções diretamente na lista, da seguinte forma:

```
String[] cidades = new String[]{"São Paulo", "Rio de Janeiro", "Campinas"};
JList lstCidades = new JList(cidades);
```

Neste caso você poderia utilizar no objeto *lstCidades* da classe *JList* para acessar diversos métodos importantes, dentre eles:

int indiceSelecionado = lstCidades.getSelectedIndex();

Obtém o número de índice da opção selecionada na lista.

lstCidades.setForeground(Color.BLUE);

Define uma cor de letra para as opções da lista.

lstCidades.setFont(new Font("Arial", Font.BOLD, 14));

Define uma configuração de fonte para as opções da lista.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.2.6.6 TRABALHANDO COM CAIXAS DE COMBINAÇÃO

De forma semelhante à criação de listas, a criação de caixas de combinação (*combo boxes*) necessita de uma classe para a criação do combo (*JComboBox*) e uma classe para definir um modelo padrão a ser utilizado (*DefaultComboBoxModel*).

Para entender como funciona a criação de caixas de combinação, crie um novo projeto Java chamado *ProjetoComboBox* com uma classe chamada *TelaCombo.java*, conforme mostra Figura 6.44.

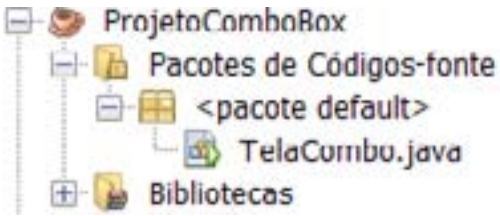


Figura 6.44: projeto com combo boxes.

Na classe *TelaCombo.java* digite o seguinte código-fonte:

```
1   import javax.swing.*;
2   import java.awt.*;
3   import static javax.swing.JFrame.EXIT_ON_CLOSE;
4
5   public class TelaCombo extends JFrame {
6
7       private JComboBox cmbCidades;
8       private DefaultComboBoxModel cmbCidadesModel;
9
10      public TelaCombo() {
11
12          setTitle("Exemplo ComoBox");
13          setDefaultCloseOperation(EXIT_ON_CLOSE);
14          setSize(300, 200);
15          setLocation(300, 200);
16
17          cmbCidadesModel = new DefaultComboBoxModel();
18          cmbCidades = new JComboBox(cmbCidadesModel);
19
20          String cidade1 = "São Paulo";
21          String cidade2 = "Rio de Janeiro";
22          String cidade3 = "Campinas";
23
24          cmbCidadesModel.addElement(cidade1);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
25              cmbCidadesModel.addElement(cidade2);
26              cmbCidadesModel.addElement(cidade3);
27              cmbCidades.setEditable(true);
28
29              JPanel auxNomes = new JPanel();
30              auxNomes.add(cmbCidades);
31              add(auxNomes, BorderLayout.WEST);
32
33              JLabel lblTituloLista = new JLabel("Escolha uma opção");
34              add(lblTituloLista, BorderLayout.NORTH);
35          }
36
37          public static void main(String arg[]) {
38              TelaCombo tela = new TelaCombo();
39              tela.setVisible(true);
40          }
41      }
```

Note na linha 17 que foi instanciado um objeto da classe *DefaultComboBoxModel.* Este objeto foi passado como parâmetro na instância de um objeto da classe *JComboBox* na linha 18. Isso significa que a caixa de combinação usará o modelo de caixa de combinação padrão definido pela classe *DefaultComboBoxModel*.

Nas linhas de 20 à 22 foram armazenados em variáveis *String* o nome das cidades que foram adicionadas à caixa de combinação nas linhas de 24 à 26. Perceba que os nomes das cidades são passados como parâmetro ao método *addElement* da classe *DefaultComboBoxModel*. Cada opção na caixa de combinação recebe um número de índice, na ordem em que foram inseridos, começando por 0. Logo "São Paulo" tem índice 0, "Rio de Janeiro" tem índice 1 e "Campinas" tem índice 2.

Para tornar a caixa de combinação editável foi usado o método *setEditable* da classe *JComboBox,* passando como parâmetro o valor *true* (linha 27). Dessa forma a caixa de combinação permitirá a entrada de dados via linha editável.

Nas linhas de 29 e 30 foi criado um painel e a lista foi adicionada a ele. Em seguida, na linha 31, este painel foi adicionado à janela usando o gerenciador de *layout BorderLayout*, que o posicionou à esquerda da janela (*WEST*).

Na linha 33 foi instanciado um objeto da classe *JLabel,* usado como rótulo, que foi posicionado acima da caixa de combinação. Este objeto foi adicionado à janela na linha 34, na posição norte do *layout*, ou seja, acima da caixa de combinação.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6* será mostrada a janela à esquerda na Figura 6.45.

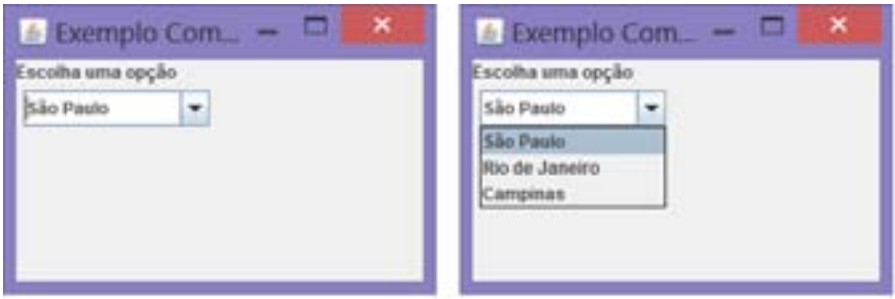


Figura 6.45: Janela com combo boxes.

Ao clicar na caixa de combinação ela se abre, como mostra a janela da direita.

6.2.6.6.1 MÉTODOS PARA TRABALHAR COM AS CAIXAS DE COMBINAÇÃO

Após criar uma caixa de combinação, podemos utilizar um conjunto de métodos por meio do objeto da classe *DefaultComboBoxModel* para adicionar novos elementos, remover elementos, obter elementos e executar outras operações na caixa de combinação. Alguns dos principais métodos para executar estas operações são apesentados a seguir:

String cidade = (String) cmbCidadesModel.getElementAt(1);

Obtém o elemento de índice 1 da lista, converte em String e armazena na variável cidade.

String cidade = (String) cmbCidadesModel.getSelectedItem();

Obtém o elemento selecionado na lista, converte em String e armazena na variável cidade.

int tamanho = cmbCidadesModel.getSize();

Obtém o tamanho da lista e armazena na variável tamanho.

cmbCidadesModel.insertElementAt("Franca", 2);

Insere o elemento "Franca" na posição de índice 2 da lista.

UNINOVE – USO EXCLUSIVO PARA ALUNO

cmbCidadesModel.removeAllElements();

Remove todos os elementos da lista.

cmbCidadesModel.removeElement("Campinas");

Remove o elemento "Campinas" da lista.

cmbCidadesModel.removeElementAt(1);

Remove o elemento de índice 1 da lista.

cmbCidadesModel.setSelectedItem("Itu");

Muda o item selecionado da lista para "Itu".

Você pode perceber que na classe apresentada adicionamos os itens à caixa de combinação (*JComboBox*) por meio de seu modelo (*DefaultComboBoxModel*) utilizando o método *addElement* (linhas de 24 à 26). Você poderia adicionar os elementos diretamente à caixa de combinação, dispensando o modelo, da seguinte forma:

```
String[] cidades = new String[]{"São Paulo", "Rio de Janeiro", "Campinas"};
JComboBox  cmbCidades = new JComboBox(cidades);
```

Ou ainda:

```
JComboBox  cmbCidades = new JComboBox();
cmbCidades.addItem("São Paulo");
cmbCidades.addItem("Rio de Janeiro");
cmbCidades.addItem("Campinas");
```

Veja alguns métodos que podem ser aplicados por meio do objeto *cmbCidades* da classe *JComboBox*, sem utilizar o *DefaultComboBoxModel*:

cmbCidades.removeItem("Rio de Janeiro");

Remove o item "Rio de Janeiro" da caixa de combinação.

cmbCidades.removeItemAt(1);

Remove o segundo item da caixa de combinação, de cima para baixo, ou seja, o item "Rio de Janeiro", que tem índice 1.

UNINOVE – USO EXCLUSIVO PARA ALUNO

cmbCidades.removeAllItems();

Remove todos os itens da caixa de combinação.

cmbCidades.setSelectedItem("Campinas");

A opção selecionada e visível na caixa de combinação será "Campinas".

cmbCidades.setSelectedIndex(2);

A opção selecionada e visível na caixa de combinação será a terceira opção de cima para baixo (opção que tem índice 2), ou seja, "Campinas".

String cidadeSelecionada = (String) cmbCidades.getSelectedItem();

Obtém a cidade selecionada na caixa de combinação, converte para String e armazena na variável String cidadeSelecionada.

int indiceSelecionado = cmbCidades.getSelectedIndex();

Obtém o número de índice da opção selecionada na caixa de combinação e armazena na variável indiceSelecionado.

String cidade = (String) cmbCidades.getItemAt(2);

Obtém a cidade de índice 2 na caixa de combinação, converte para String e armazena na variável String cidade. A opção de índice 2 é "Campinas".

int totalCidades = cmbCidades.getItemCount();

Obtém o número de opções que tem na caixa de combinação e armazena na variável totalCidades.

cmbCidades.setForeground(Color.BLUE);

Define a cor azul de fundo para a caixa de combinação.

cmbCidades.setFont(new Font("Arial", Font.BOLD, 14));

Define a fonte arial, com efeito de negrito e de tamanho 14 para a caixa de combinação.

UNINOVE – USO EXCLUSIVO PARA ALUNO

cmbCidades.setMaximumRowCount(2);

Define o número de opções que estará visível quando abrir a caixa de combinação.

6.2.6.7 Trabalhando com tabelas

As tabelas são muito úteis para a exibição de dados, principalmente resultantes de consultas realizadas a bancos de dados. Para criar tabelas podem ser utilizadas duas classes principais: *JTable* e *TableModel*.

A classe *JTable* é utilizada para a criação da tabela. Para aplicar a ela um modelo padrão que nos isenta da responsabilidade de criar um novo modelo de tabela, utilizamos a classe *DefaultTableModel*.

Para testar o uso destas classes, crie um novo projeto chamado *ProjetoTabela* com uma classe principal chamada *TelaTabela.java*, conforme mostra a Figura



Figura 6.46: Projeto para trabalhar com tabelas.

Na classe *TelaTabela.java*, digite o seguinte código-fonte:

```
1   import javax.swing.*;
2   import javax.swing.table.*;
3   import java.awt.*;
4   import static javax.swing.JFrame.EXIT_ON_CLOSE;
5
6   public class TelaTabela extends JFrame {
7
8       private JTable tblClientes;
9       private DefaultTableModel tblClientesModel;
10
11      public TelaTabela() {
12          setTitle("Exemplo de tabela");
13          setDefaultCloseOperation(EXIT_ON_CLOSE);
14          setSize(400, 200);
15          setLocation(300, 200);
16
17          String[] cols = {"Código", "Nome", "E-mail"};
18          tblClientesModel = new DefaultTableModel(cols, 4);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
19          tblClientes = new JTable(tblClientesModel);
20
21          String[] linha1 = {"1", "Fernando Pereira de Souza", "fersouza@ig.com.br"};
22          String[] linha2 = {"2", "Angela Cristina", "acris@ig.com.br"};
23          String[] linha3 = {"3", "Pedro Henrique", "pedro@yahoo.com.br"};
24          String[] linha4 = {"4", "Ana Luiza Benfica", "ana@gmail.com"};
25
26          tblClientesModel.insertRow(0, linha1);
27          tblClientesModel.insertRow(1, linha2);
28          tblClientesModel.insertRow(2, linha3);
29          tblClientesModel.insertRow(3, linha4);
30          tblClientesModel.setNumRows(4);
31
32          JScrollPane scrRolagem = new JScrollPane(tblClientes,
            JScrollPane.VERTICAL_SCROLLBAR_ALWAYS,
            JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
33          add(scrRolagem, BorderLayout.CENTER);
34
35          JLabel lblTitulo = new JLabel("Lista de Clientes");
36          add(lblTitulo, BorderLayout.NORTH);
37
38      }
39
40      public static void main(String arg[]) {
41          TelaTabela tela = new TelaTabela();
42          tela.setVisible(true);
43      }
44  }
```

Note que na linha 17 foi criada uma *array* unidimensional e nela foram armazenados os valores que serão rótulos das colunas da tabela. Esta *array* foi adicionada à tabela na linha 18, juntamente com o número de linhas que a tabela deverá ter, neste caso, 4 linhas.

Na linha 19 foi instanciado um objeto da classe *JTable* aplicando o modelo de tabela instanciado na linha 18.

Nas linhas de 21 à 24 foram criadas *arrays* unidimensionais com os valores que serão adicionados em cada linha da tabela. Nas linhas de 26 à 29, estas *arrays* foram adicionadas à tabela, respectivamente, nas linhas de 0 à 3, pois apesar da tabela ter 4 linhas, a numeração vai de 0 à 3.

Na linha 30 foi definido o número de linhas da tabela, neste caso, 4 linhas.

Na linha 32, a tabela foi adicionada à barra de rolagem, indicando, pelos parâmetros, que estas barras (verticais e horizontais) estarão sempre visíveis, independente do conteúdo da tabela exceder a área disponível ou não. Na linha

UNINOVE – USO EXCLUSIVO PARA ALUNO

33 a barra de rolagem com a tabela é adicionada à janela na posição central do *layout BorderLayout*.

Na linha 35 é criado um rótulo para a tabela. Este rótulo é adicionado à janela na posição norte do *layout Borderlayout*, ou seja, acima da tabela.

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6* aparecerá a tela mostrada na Figura 6.47.



Figura 6.47: Janela com tabela.

Note a presença das barras de rolagem na vertical e na horizontal. Note também o título acima da tabela. Para manipular os valores nesta tabela podem ser utilizados um conjunto de métodos, que serão apresentados a seguir.

6.2.6.7.1 Métodos para trabalhar com tabelas

Você pode querer incluir novos elementos na tabela, excluir elementos, alterá-los, mudar o tamanho das colunas, obter um elemento selecionado etc. Para executar estas operações, você pode utilizar um conjunto de métodos das classes *DefaultTableModel*. Os principais métodos serão apresentados a seguir:

tblClientesModel.addColumn("Telefone");

Adiciona a coluna "Telefone" à direita na tabela.

String[] linha5 = {"4", "Ana Luiza Benfica", "ana@gmail.com", "4567-9087"};

tblClientesModel.addRow(linha5);

Adiciona uma nova linha no final da tabela, contendo os valores presentes na array linha5.

UNINOVE – USO EXCLUSIVO PARA ALUNO

String[] linha6 = {"4", "Ana Luiza Benfica", "ana@gmail.com", "5678-0987"};

tblClientesModel.insertRow(3, linha6);

Adiciona uma nova linha na tabela, na posição 3, ou seja, adiciona a quarta linha, já que o índice de numeração de linhas começa em 0.

int qtdColunas = tblClientesModel.getColumnCount();

Obtém o número de colunas da tabela.

String nomeColuna = tblClientesModel.getColumnName(1);

Obtém o nome (rótulo) da coluna 1 da tabela, ou seja, da segunda coluna, já que a numeração de colunas começa no índice 0.

int numLinhas = tblClientesModel.getRowCount();

Obtém o número de linhas da tabela.

String valor = (String) tblClientesModel.getValueAt(2, 3);

Obtém o valor contido na célula de linha 2 e coluna 3 da tabela e armazena em uma variável String.

tblClientesModel.setNumRows(6);

Determina que a tabela terá 6 linhas.

tblClientesModel.setColumnCount(5);

Determina que a tabela terá 5 colunas.

tblClientesModel.setValueAt("Maria Paula", 1, 1);

Insere na célula de linha 1 e coluna 1 da tabela o valor "Maria Paula".

tblClientesModel.removeRow(3);

Remove a linha de número 3 da tabela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.3 Tratamento de eventos

Eventos são estímulos na aplicação, produzidos ou não pelo usuário, que realizam chamadas a métodos para tratar o evento, ou seja, para indicar o que ocorrerá a partir daquele estímulo. Estes métodos normalmente executam operações para responder ao estímulo com a ação adequada.

Cada tipo de componente em uma janela suporta um conjunto de eventos que não passam de objetos proveniente de classes, que possuem métodos e atributos.

As principais categorias de eventos que podem ocorrer em uma aplicação Java são:

– Eventos de Ação.
– Eventos de Janela.
– Eventos de Foco.
– Eventos de Mouse.

Nos tópicos seguintes serão apresentados exemplos de cada uma dessas categorias.

6.3.1 Tratamento de eventos de ação

Os eventos de ação são tratados normalmente a partir de métodos assinados na interface *ActionListener*. Estes eventos são gerados quando se clica em objetos do formulário como botões, listas, opções de menu etc. São os eventos mais comuns em aplicações que utilizam as APIs *swing* e *awt*.

A maneira mais comum para uma classe tratar os eventos de ação é implementando o método *actionPerformed* da interface *ActionListener*.

Sendo assim, a classe deve implementar a interface *ActionListener* e reescrever obrigatoriamente o método *actionPerformed*. Além disso, o elemento que sofrerá a ação de clique do mouse deverá se auto monitorar constantemente para identificar quando ocorre o clique do mouse. Isso é feito pelo método *addActionListener*.

Veja a seguir a estrutura básica de uma classe preparada para tratar eventos de ação:

```
public class Tela extends JFrame implements ActionListener {

   @Override
   public void actionPerformed(ActionEvent e) {

    }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Para testar o tratamento desta categoria de eventos, vamos criar um projeto Java no *NetBeans* chamado *Aplicativo,* com uma classe principal chamada *Aplicativo.java*, conforme mostra a Figura 6.48.

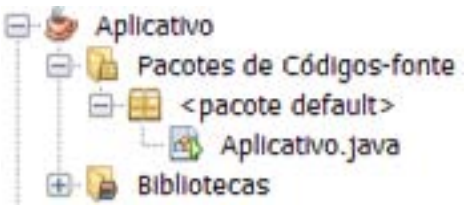


Figura 6.48: Projeto para tratamento de eventos de ação de clique.

A classe *Aplicativo.java* vai gerar uma janela com diversos campos e dois botões: "Adicionar à tabela" e "Limpar campos". Ao preencher os campos e clicar no botão "Adicionar à tabela", o conteúdo dos campos são adicionados à uma tabela na parte inferior da janela. Ao clicar no botão "Limpar campos", todos os campos são limpos.

Veja a seguir o código-fonte da classe *Aplicativo.java*:

```
1   import javax.swing.*;
2   import java.awt.*;
3   import java.awt.event.ActionEvent;
4   import java.awt.event.ActionListener;
5   import java.text.ParseException;
6   import static javax.swing.JFrame.EXIT_ON_CLOSE;
7   import javax.swing.border.TitledBorder;
8   import javax.swing.table.DefaultTableModel;
9   import javax.swing.text.MaskFormatter;
10
11  public class Aplicativo extends JFrame implements ActionListener {
12
13      private JPanel pnlCampos, pnlTabela, pnlBotoes, pnlNome,
        pnlTelefone, pnlCidade;
14      private TitledBorder tituloPnlCampos, tituloPnlTabela, tituloPnlBotoes;
15      private JLabel lblNome, lblTelefone, lblCidade;
16      private JTextField txtNome;
17      private JFormattedTextField txtTelefone;
18      private MaskFormatter mskTelefone;
19      private JTable tblClientes;
20      private DefaultTableModel tblClientesModel;
21      JButton btnAdicionar, btnLimpar;
22      private JComboBox cmbCidades;
23      private DefaultComboBoxModel cmbCidadesModel;
24
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
25          public Aplicativo() {
26              definirJanela();
27
28              pnlCampos = new JPanel(new GridLayout(3, 1));
29              tituloPnlCampos = BorderFactory.createTitledBorder("Cadastro de
                Clientes");
30              pnlCampos.setBorder(tituloPnlCampos);
31
32              pnlBotoes = new JPanel(new FlowLayout());
33              tituloPnlBotoes = BorderFactory.createTitledBorder("Botões");
34              pnlBotoes.setBorder(tituloPnlBotoes);
35
36              pnlTabela = new JPanel(new GridLayout(1, 1));
37              tituloPnlTabela = BorderFactory.createTitledBorder("Tabela de
                exibição de dados");
38              pnlTabela.setBorder(tituloPnlTabela);
39
40              lblNome = new JLabel("Nome");
41              lblTelefone = new JLabel("Telefone");
42              lblCidade = new JLabel("Cidade");
43              txtNome = new JTextField(50);
44              try {
45                  mskTelefone = new MaskFormatter("(##)####-####");
46                  mskTelefone.setPlaceholderCharacter('_');
47              } catch (ParseException ex) {
48                  JOptionPane.showMessageDialog(null, "Máscara incorreta");
49              }
50              txtTelefone = new JFormattedTextField(mskTelefone);
51              cmbCidadesModel = new DefaultComboBoxModel();
52              cmbCidades = new JComboBox(cmbCidadesModel);
53
54              String cidade1 = "São Paulo";
55              String cidade2 = "Rio de Janeiro";
56              String cidade3 = "Campinas";
57              cmbCidadesModel.addElement(cidade1);
58              cmbCidadesModel.addElement(cidade2);
59              cmbCidadesModel.addElement(cidade3);
60
61              pnlNome = new JPanel(new FlowLayout(FlowLayout.LEFT));
62              pnlNome.add(lblNome);
63              pnlNome.add(txtNome);
64              pnlCampos.add(pnlNome);
65
66              pnlTelefone = new JPanel(new FlowLayout(FlowLayout.LEFT));
67              pnlTelefone.add(lblTelefone);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
68          pnlTelefone.add(txtTelefone);
69          pnlCampos.add(pnlTelefone);
70
71          pnlCidade = new JPanel(new FlowLayout(FlowLayout.LEFT));
72          pnlCidade.add(lblCidade);
73          pnlCidade.add(cmbCidades);
74          pnlCampos.add(pnlCidade);
75
76          btnAdicionar = new JButton("Adicionar à tabela");
77          btnAdicionar.addActionListener(this);
78          btnLimpar = new JButton("Limpar campos");
79          btnLimpar.addActionListener(this);
80          pnlBotoes.add(btnAdicionar);
81          pnlBotoes.add(btnLimpar);
82
83          String[] cols = {"Nome", "Telefone", "Cidade"};
84          tblClientesModel = new DefaultTableModel(cols, 3);
85          tblClientes = new JTable(tblClientesModel);
86          tblClientesModel.setNumRows(0);
87
88          JScrollPane scrRolagem = new JScrollPane(tblClientes,
89                  JScrollPane.VERTICAL_SCROLLBAR_ALWAYS,
90                  JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
91          pnlTabela.add(scrRolagem);
92
93          add(pnlCampos, BorderLayout.NORTH);
94          add(pnlBotoes, BorderLayout.CENTER);
95          add(pnlTabela, BorderLayout.SOUTH);
96      }
97
98      private void definirJanela() {
99          setTitle("Aplicação");
100         setDefaultCloseOperation(EXIT_ON_CLOSE);
101         setLayout(new BorderLayout());
102         setSize(800, 700);
103         setLocation(300, 200);
104     }
105
106     public static void main(String[] args) {
107         Aplicativo ap = new Aplicativo();
108         ap.setVisible(true);
109     }
110
111     @Override
112     public void actionPerformed(ActionEvent e) {
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
113             if (e.getSource() == btnAdicionar) {
114                 String[] linha = {txtNome.getText(), txtTelefone.getText(),
                    cmbCidadesModel.getSelectedItem().toString()};
115                 tblClientesModel.addRow(linha);
116             } else if (e.getSource() == btnLimpar) {
117                 txtNome.setText(null);
118                 txtTelefone.setText(null);
119                 cmbCidadesModel.setSelectedItem("São Paulo");
120                 txtNome.requestFocus();
121             }
122         }
123     }
```

Veja na linha 11 que a classe *Aplicativo.java* herda a classe *JFrame* e implementa os métodos abstratos da interface *ActionListener*. Nesse caso há apenas um método abstrato, o método *actionPerformed*, que está implementado obrigatoriamente, nas linhas de 111 à 122.

As linhas de 13 à 23 declaram os objetos referentes aos componentes que serão colocados na tela. Veja que teremos painéis, títulos de painéis, rótulos, campos de texto simples, campo de texto formatado, tabela, botões de ação e caixa de combinação (*combo*).

Note que o contrutor da classe é implementado da linha 25 à linha 96. Toda a criação de componentes e adição à janela ocorre nesse bloco, no interior do construtor.

Neste exemplo, as configurações da janela foram colocadas em um método *definirJanela*, que é chamado na linha 26. Este método é implementado da linha 98 à linha 104. Você vai encontrar as configurações padrão de janela nesse método, com definição de título (linha 99), definição de que ao clicar no "X" vermelho no canto superior direito da janela a aplicação será encerrada (linha 100), definição do gerenciador de *layout* utilizado na janela, nesse caso, *FlowLayout* (linha 101), largura e altura da janela (linha 102) e posição da janela (linha 103).

As linhas de 28 à 38 criam três painéis, um para colocar os campos, um para colocar os botões e um para colocar a tabela. Cada painel é definido para usar um gerenciador de *layout* específico. Veja que o primeiro painel usa *GridLayout* com 3 linhas e 1 coluna (linha 28), o segundo usa *FlowLayout* (linha 32), e o terceiro usa *GridLayout* com 1 linha e 1 coluna (linha 36). Essas definições de *layout* determinam como os elementos que serão colocados nos painéis serão posicionados. Caso tenha dúvidas sobre eles, releia os tópicos anteriores deste Capítulo sobre gerenciadores de *layout*.

Note que cada painel também terá uma moldura visível com um título. Isso é definido nas linhas 29 e 30, 33 e 34, 37 e 38.

UNINOVE – USO EXCLUSIVO PARA ALUNO

As linhas de 40 à 52 instanciam os objetos que serão colocados nos painéis e, consequentemente, na janela. Nesse bloco de linhas, não há novidade em relação ao que vimos nos exemplos anteriores.

As linhas de 54 à 59 definem o conteúdo da caixa de combinação (*combo*).

As linhas de 61 à 74 criam respectivamente painéis para o campo *nome*, *telefone* e *cidade* e adiciona esses campos aos respectivos painéis. Note que cada painel é criado para usar o gerenciador de *layout Flowlayout* com conteúdo alinhado à esquerda. Perceba que cada painel (que contém os campos) é adicionado ao painel *pnlCampos* nas linhas 64, 69 e 74.

As linhas de 76 à 81 criam um painel, adiciona os botões dentro e adiciona este painel ao painel *pnlBotoes*. Aqui tem uma grande novidade. Note o uso do método *addActionListener*. É esse método que faz com que o botão, ao receber um clique, chame automaticamente o método *actionPerformed* da linha 112. Dentro do método *actionPerformed* foi programado o que deve ocorrer quando o botão recebe o clique. Se você esquecer de chamar este método *addActionListener* para o botão, o método *actionPerformed* não será chamado e, consequentemente, nada acontecerá.

Nas linhas de 83 à 91 é criada a tabela e definidos seus parâmetros. Veja que será uma tabela com 3 colunas contendo como título das colunas os rótulos "Nome", "Telefone" e "Cidade" (linhas 83 e 84). Na linha 86 define-se que a tabela terá inicialmente 0 linhas. Na linha 88 a tabela é agregada à barra de rolagem e na linha 91 é adicionada ao painel *pnlTabela*.

Nas linhas de 93 à 95, os painéis que contém os campos, os botões e a tabela são adicionados à janela.

Veja agora o método *actionPerformed* (linha 112), que é acionado ao clicar nos botões "Adicionar à janela" (*btnAdicionar*) e "Limpar campos" (*btnLimpar*). Quando o botão recebe o clique, o método *addActionLister* aciona o método *actionPerformed,* que recebe como parâmetro um objeto da classe *ActionEvent* contendo informações sobre o botão clicado. Se o botão clicado foi o botão *btnAdicionar* (linha 113), então será criada uma *array String* unidimensional com o conteúdo dos campos *nome*, *telefone* e *cidade* (linha 114). Esta *array* é adicionada à tabela na linha 115. Se o botão clicado não foi o botão *btnAdicionar*, mas sim o botão *btnLimpar* (linha 116), os campos são limpos (linhas 17 e 118), a opção "São Paulo" é selecionada como cidade na caixa de combinação (linha 119) e o cursor é posicionado no campo *nome* (linha 120).

Perceba que o que deve ocorrer ao clicar nos botões é programado de forma personalizada no método *actionPerformed*. O objeto *e* da classe *ActionEvent* permite verificar qual foi o botão clicado por meio do método *getSource*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6*, aparecerá a tela mostrada na Figura 6.49.

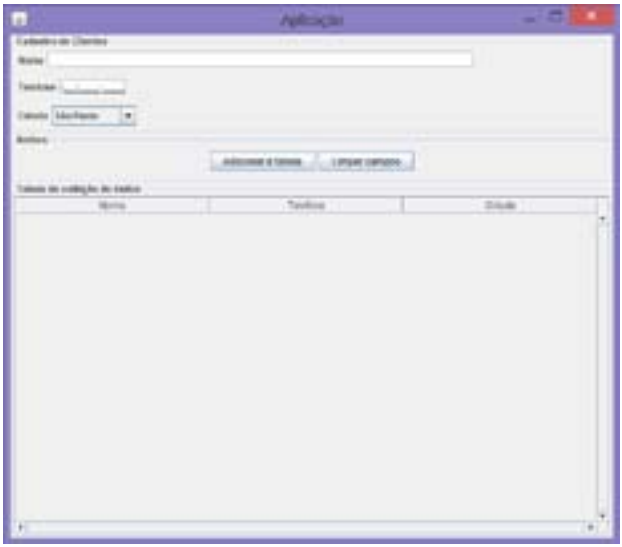


Figura 6.49: Janela com tratamento de evento de clique em botões.

Ao preencher os campos e clicar no botão "Adicionar à tabela", os dados são incluídos na tabela abaixo dos botões, aumentando o número de linhas a cada adição. Após cadastrar alguns nomes e adicionar à tabela, a janela estará como mostra a Figura 6.50.

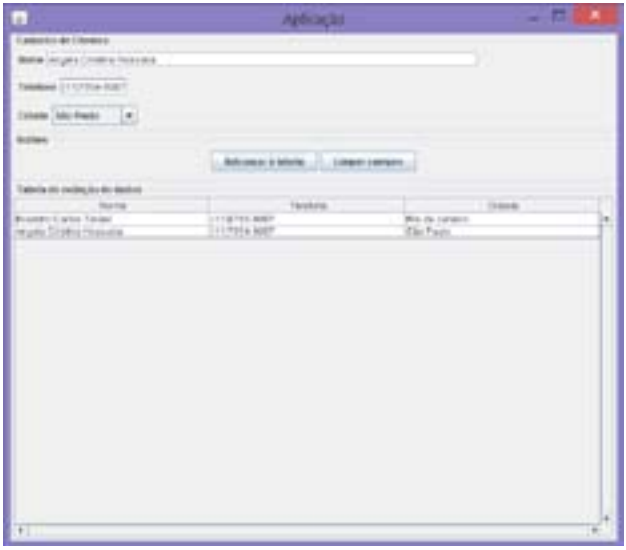


Figura 6.50: Janela com campos preenchidos e tratamento de evento de clique em botões.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Programar os eventos de clique em botões e em opções do menu em aplicações Java é comum e faz parte do dia-a-dia do programador. Há outras formas de tratar esses eventos chegando aos mesmos resultados, mas estas formas não serão abordadas nesse livro. Procure aprender e memorizar bem uma forma de fazer e use-a bem nas suas aplicações. Não estude todas as formas ao mesmo tempo para evitar confusão.

6.3.2 TRATAMENTO DE EVENTOS DE JANELA

Os eventos de janela podem ocorrer quando a janela abre, quando ela fecha, quando ganha ou perde o foco, quando é maximizada ou minimizada etc.

Há um conjunto enorme de eventos de janela, mas, neste livro, vamos tratar de dois deles, o de fechar e o de abrir a janela.

Há várias classes para tratar os eventos de janela, como *WindowEvent*, *WindowListener*, *WindowAdapter*, *WindowFocusEvent*, *WindowFocusListener*, *WindowFocusAdapter*, *WindowStateEvent*, *WindowStateListener* e *WindowStateAdapter*.

No geral, independente das classes utilizadas, se consegue os mesmos resultados, por isso, nesse tópico vamos utilizar apenas as classes *WindowsAdapter* e *WindowEvent*.

Para testar o tratamento de eventos de janela, crie um novo projeto Java, chamado *ProjetoJanela,* e crie nele três classes: *Tela.java*, *EventoClose.java* e *EventoOpen.java*.

A Figura 6.51 mostra a pasta do projeto após sua criação.



Figura 6.51: Projeto para tratamento de eventos de janela.

As classes *EventoOpen.java* e *EventoClose.java* herdarão a classe *WindowsAdapter* e reescreverão, respectivamente, os métodos *windowOpened* e *windowClosing* da superclasse *WindowsAdapter*. O conteúdo destes métodos indicará o que deve ocorrer quando a janela criada pela classe *Tela.java* abrir e quando a janela fechar.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Na classe *EventosOpen.java*, escreva o seguinte código-fonte:

```
1    import java.awt.event.WindowAdapter;
2    import java.awt.event.WindowEvent;
3    import javax.swing.JOptionPane;
4
5    public class EventoOpen extends WindowAdapter {
6
7       @Override
8       public void windowOpened(WindowEvent we) {
9          JOptionPane.showMessageDialog(null, "A janela foi aberta");
10      }
11   }
```

Veja que a classe *EventoOpen*.java herda a classe *WindowAdapter* (linha 5).

Note que o método *windowOpened* existe na classe *WindowAdapter* e está sendo sobrescrito aqui. Isso é indicado pela instrução *@Override* na linha 7. Este método será chamado quando a janela for aberta, e mostrará em uma caixa de diálogo a mensagem contida na linha 9.

Na classe *EventosClose.java*, escreva o seguinte código-fonte:

```
1    import java.awt.event.WindowAdapter;
2    import java.awt.event.WindowEvent;
3    import javax.swing.JOptionPane;
4
5    public class EventoClose extends WindowAdapter {
6
7       @Override
8       public void windowClosing(WindowEvent we) {
9          int resp = JOptionPane.showConfirmDialog(null, "Deseja sair ?");
10         if (resp == JOptionPane.YES_OPTION) {
11            System.exit(0);
12         }
13      }
14   }
```

Veja que a classe *EventoClose.java* herda a classe *WindowAdapter* (linha 5).

Note que o método *windowClosing* existe na classe *WindowAdapter* e está sendo sobrescrito aqui. Este método será chamado quando a janela for fechada, normalmente ao se clicar no "X" vermelho do canto superior direito. Quando isso ocorrer, aparecerá uma caixa de diálogo perguntando se deseja sair da aplicação (linha 9). Esta caixa de diálogo terá três botões: "Sim", "Não" e

UNINOVE – USO EXCLUSIVO PARA ALUNO

"Cancelar". Se for clicado no botão "Sim" (linha 10), a aplicação é finalizada e descarregada da memória (linha 11).

Na classe *Tela.java*, escreva o seguinte código-fonte:

```
1   import java.awt.FlowLayout;
2   import javax.swing.JFrame;
3   import javax.swing.JLabel;
4   import javax.swing.JTextField;
5
6   public class Tela extends JFrame {
7
8       private JLabel lblNome;
9       private JTextField txtNome;
10
11      public Tela() {
12          setTitle("Eventos de Janela");
13          setSize(300, 200);
14          setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
15          setLayout(new FlowLayout());
16          lblNome = new JLabel("Nome");
17          txtNome = new JTextField(20);
18          add(lblNome);
19          add(txtNome);
20          addWindowListener(new EventoClose());
21          addWindowListener(new EventoOpen());
22      }
23
24      public static void main(String[] args) {
25          Tela t = new Tela();
26          t.setVisible(true);
27      }
28  }
```

Esta é uma classe comum, que gera uma janela com um botão para a inclusão do nome. Criamos dezenas de classes como esta nos tópicos anteriores, por isso, vamos à explicação apenas do necessário.

Veja na linha 14 que a finalização automática da aplicação ao se clicar no "X" do canto superior direito da janela foi cancelada. Isso foi necessário porque queremos que ao clicar no "X" seja gerado um evento e chamado o método *windowClosing* da classe *EventoClose.java*.

O método *addWindowListener* da linha 20 monitora a janela até que ela feche. Quando isso ocorrer, um objeto da classe *EventoClose.java* é instanciado

UNINOVE – USO EXCLUSIVO PARA ALUNO

e passado como parâmetro para este método. Isso faz com que o método *windowClosing* da classe *EventoClose.java* seja executado.

O mesmo acontece na linha 21, mas com a classe *EventoOpen.java*, que executa o método *windowOpened*.

Perceba que são as linhas 20 e 21 que "puxam o gatilho", quando o evento ocorre e direcionam para os métodos corretos para tratá-los. Dessa forma, se deseja apenas tratar o evento de fechamento da janela, mantenha a linha 20 e retire a linha 21. Se deseja apenas tratar o evento de abertura da janela, mantenha apenas a linha 21.

As linhas 20 e 21 poderiam ser desmembradas em quatro linhas, mantendo o mesmo resultado, ficando da seguinte forma:

```
EventoClose evtClose = new EventoClose();
addWindowListener(evtClose);
EventoOpen evtOpen = new EventoOpen();
addWindowListener(evtOpen);
```

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6*, aparecerá uma caixa de diálogo e a janela, como mostra a Figura 6.52.



Figura 6.52: Resultado do tratamento de eventos de abertura e fechamento da janela.

Ao clicar no "X" vermelho do canto superior direito da janela aparecerá a caixa de diálogo mostrada na Figura 6.53.



Figura 6.53: Caixa de diálogo para confirmação do fechamento da janela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao clicar no botão “Sim” a aplicação será finalizada.

Apesar de termos criado uma classe para tratar cada evento, poderíamos tratar os eventos dentro da mesma classe que gera a tela. Nesse caso, a aplicação teria apenas uma classe, mas o código ficaria um pouco mais complexo. Veja abaixo como ficará o código-fonte, neste caso:

```
1   import java.awt.FlowLayout;
2   import java.awt.event.WindowAdapter;
3   import java.awt.event.WindowEvent;
4   import javax.swing.JFrame;
5   import javax.swing.JLabel;
6   import javax.swing.JOptionPane;
7   import javax.swing.JTextField;
8
9   public class Tela extends JFrame {
10
11      private JLabel lblNome;
12      private JTextField txtNome;
13
14      public Tela() {
15        setTitle("Eventos de Janela");
16         setSize(300, 200);
17         setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
18         setLayout(new FlowLayout());
19         lblNome = new JLabel("Nome");
20         txtNome = new JTextField(20);
21         add(lblNome);
22         add(txtNome);
23         addWindowListener(new WindowAdapter() {
24            @Override
25            public void windowClosing(WindowEvent we) {
26              int resp = JOptionPane.showConfirmDialog(null, "Deseja sair ?");
27               if (resp == JOptionPane.YES_OPTION) {
28                  System.exit(0);
29               }
30            }
31         });
32         addWindowListener(new WindowAdapter() {
33            @Override
34            public void windowOpened(WindowEvent we) {
35               JOptionPane.showMessageDialog(null, "A janela foi aberta");
36            }
37         });
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
38          }
39
40          public static void main(String[] args) {
41              Tela t = new Tela();
42              t.setVisible(true);
43          }
44      }
```

Veja que o evento de tratamento da janela é tratado das linhas 23 à 31 e o evento de abertura da janela é tratado das linhas 32 à 38. Não haveria diferença com relação a colocar o tratamento de eventos em classes separadas.

6.3.3 Tratamento de eventos de foco

Os eventos de foco são disparados quando o elemento ganha ou perde o foco. Por exemplo, uma caixa de texto ganha o foco quando você clica sobre ela; e perde o foco quando você sai dela.

Podemos usar diversas classes para tratar eventos de foco, como *FocusListener*, *FocusAdapter* e *FocusEvent*. Neste tópico criaremos um exemplo utilizando as duas últimas.

Trataremos os eventos de foco de forma semelhante à que tratamos os eventos de janela, criando classes auxiliares para o tratamento dos eventos.

Para testar o uso destes eventos, crie um projeto Java no *NetBeans* chamado *AplicativoFoco* e nele crie as classes *EventoGanhaFoco.java*, *EventoPerdeFoco.java* e *Tela.java* (classe principal), conforme mostra a Figura 6.54.

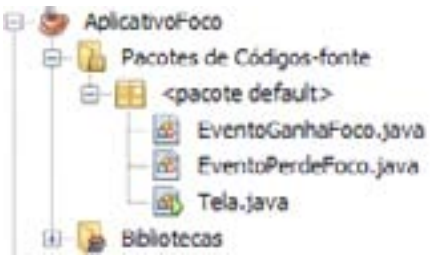


Figura 6.54: Projeto para tratar eventos de foco.

Nesse projeto, será gerada uma janela com dois campos: *nome* e *renda*. Quando o campo *renda* ganhar o foco do cursor, será chamado o método *focusGained* da classe *EventoGanhaFoco.java*. Quando o campo perder o foco, será chamado o método *focusLost* da classe *EventoPerdeFoco.java*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Digite o código-fonte abaixo para a classe *EventoGanhaFoco.java*:

```
1   import java.awt.event.FocusAdapter;
2   import java.awt.event.FocusEvent;
3
4   public class EventoGanhaFoco extends FocusAdapter {
5
6       @Override
7       public void focusGained(FocusEvent e) {
8           System.out.println("O campo Renda ganhou o foco");
9       }
10  }
```

O método *focusGained* (linha 7) existe na classe herdada *FocusAdapter* (linha 4) e é sobrescrito nesta classe. Esse método será chamado quando o campo renda da janela ganhar o foco.

Digite o código-fonte abaixo para a classe *EventoGanhaFoco.java*:

```
1   import java.awt.event.FocusAdapter;
2   import java.awt.event.FocusEvent;
3
4   public class EventoPerdeFoco extends FocusAdapter {
5
6       @Override
7       public void focusLost(FocusEvent e) {
8           System.out.println("O campo Renda perdeu o foco");
9       }
10  }
```

O método *focusLost* (linha 7) existe na classe herdada *FocusAdapter* (linha 4) e é sobrescrito nesta classe. Esse método será chamado quando o campo *renda* da janela perder o foco.

Digite o código-fonte abaixo para a classe *EventoGanhaFoco.java*:

```
1     import java.awt.FlowLayout;
2     import javax.swing.JFrame;
3     import javax.swing.JLabel;
4     import javax.swing.JTextField;
5
6     public class Tela extends JFrame {
7
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
8           private JLabel lblNome, lblRenda;
9           private JTextField txtNome, txtRenda;
10
11          public Tela() {
12              setTitle("Eventos de Foco");
13              setSize(300, 200);
14              setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
15              setLayout(new FlowLayout());
16              lblNome = new JLabel("Nome");
17              txtNome = new JTextField(20);
18              lblRenda = new JLabel("Renda");
19              txtRenda = new JTextField(10);
20              add(lblNome);
21              add(txtNome);
22              add(lblRenda);
23              add(txtRenda);
24              txtRenda.addFocusListener(new EventoGanhaFoco());
25              txtRenda.addFocusListener(new EventoPerdeFoco());
26          }
27
28          public static void main(String[] args) {
29              Tela t = new Tela();
30              t.setVisible(true);
31          }
32      }
```

Perceba na linha 24 que é adicionado um *Listener* ("ouvinte") ao campo *textRenda* de tal forma que quando ele ganhar o foco, um objeto da classe *EventoGanhaFoco.java* é instanciado e passado como parâmetro para o método *addFocusListener*. Isso faz com que o método *focusGained* da classe *EventoGanhaFoco.java* seja executado.

O mesmo ocorre na linha 25. Quando o campo renda perder o foco, um objeto da classe *EventoPerdeFoco.java* é instanciado e passado como parâmetro para o método *addFocusListener*. Isso faz com que o método *focusLost* da classe *EventoPerdeFoco.java* seja executado.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto por meio do pressionamento da tecla F6 aparecerá a janela mostrada na Figura 6.55.



Figura 6.55: Tela para tratamento de evento de foco.

Ao clicar no campo *renda* e ele ganhar o foco, aparecerá no console a mensagem "O campo Renda ganhou o foco". Ao sair do campo *renda*, por exemplo, clicando no campo *nome*, aparecerá no console a mensagem "O campo Renda perdeu o foco".

Console é a área de saída do *NetBeans*, situada geralmente abaixo do código-fonte na tela, conforme mostra a Figura 6.56.



Figura 6.56: Saída da aplicação exemplo para tratamento de eventos de foco.

Caso queira, ao invés de criar classes auxiliares para tratar os eventos de ganho e perda de foco, pode fazer diretamente na classe que gera a janela, ou seja, na classe *Tela.java*, da seguinte forma:

```
1    import java.awt.FlowLayout;
2    import java.awt.event.FocusAdapter;
3    import java.awt.event.FocusEvent;
4    import javax.swing.JFrame;
5    import javax.swing.JLabel;
6    import javax.swing.JTextField;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
7
8       public class Tela extends JFrame {
9
10          private JLabel lblNome, lblRenda;
11          private JTextField txtNome, txtRenda;
12
13          public Tela() {
14              setTitle("Eventos de Foco");
15              setSize(300, 200);
16              setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17              setLayout(new FlowLayout());
18              lblNome = new JLabel("Nome");
19              txtNome = new JTextField(20);
20              lblRenda = new JLabel("Renda");
21              txtRenda = new JTextField(10);
22              add(lblNome);
23              add(txtNome);
24              add(lblRenda);
25              add(txtRenda);
26              txtRenda.addFocusListener(new FocusAdapter() {
27                  @Override
28                  public void focusGained(FocusEvent we) {
29                      System.out.println("O campo Renda ganhou o foco");
30                  }
31              });
32              txtRenda.addFocusListener(new FocusAdapter() {
33                  @Override
34                  public void focusLost(FocusEvent we) {
35                      System.out.println("O campo Renda perdeu o foco");
36                  }
37              });
38          }
39
40          public static void main(String[] args) {
41              Tela t = new Tela();
42              t.setVisible(true);
43          }
44      }
```

Veja que o evento de ganho de foco é tratado das linhas 26 à 31 e o evento de perda de foco é tratado da linha 32 à 37.

A grande vantagem de tratar o evento na mesma classe é que você pode, no interior do método que trata o evento, fazer referência aos elementos da tela, pois estão na mesma classe. Não há diferença, neste caso, em termos de resultado.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.3.4 TRATAMENTO DE EVENTOS DE MOUSE

Os eventos de mouse acontecem quando o mouse interage com o elemento, seja sendo posicionado sobre o elemento, saindo de sobre o elemento, clicando no elemento, movendo-se sobre o elemento etc.

As principais classes utilizadas para tratar eventos de mouse são *MouseListener*, *MouseMotionListener*, *MouseMotionAdapter, MouseAdapter e MouseEvent*. Nesse exemplo utilizaremos as duas últimas.

Ao contrário dos exemplos anteriores, vamos tratar os eventos de mouse em uma única classe, evitando a criação de classes auxiliares.

No exemplo apresentado a seguir, vamos tratar apenas os eventos *mouseEntered* e *mouseExited* que tratam, respectivamente, os eventos de posicionar o mouse sobre a área do elemento e retirar o mouse de sobre a área do elemento. Há ainda diversos outros eventos, como *mouseClicked*, *mousePressed*, *mouseReleased*, *mouseWheelMoved*, *mouseDragged* e *mouseMoved*, mas não serão tratados neste exemplo.

Para testar o uso de eventos de mouse, crie um projeto chamado *AplicativoEventosMouse* com uma classe principal chamada *Tela.java*, conforme mostra a Figura 6.57.



Figura 6.57: projeto para tratamento de eventos de mouse.

A classe *Tela.java* terá um campo nome e um botão "Salvar". Quando o mouse for posicionado sobre o botão, sua cor de fundo mudará para amarelo (evento *mouseEntered*). Quando ele sair da área do botão, sua cor de fundo mudará para vermelho (evento *mouseExited*).

Na classe *Tela.java*, digite o seguinte código-fonte:

```
1  import java.awt.Color;
2  import java.awt.FlowLayout;
3  import java.awt.event.MouseAdapter;
4  import java.awt.event.MouseEvent;
5  import javax.swing.JButton;
6  import javax.swing.JFrame;
7  import javax.swing.JLabel;
8  import javax.swing.JTextField;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
9
10      public class Tela extends JFrame {
11
12          private JLabel lblNome;
13          private JTextField txtNome;
14          private JButton btnSalvar;
15
16          public Tela() {
17              setTitle("Eventos de Mouse");
18              setSize(300, 200);
19              setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
20              setLayout(new FlowLayout());
21              lblNome = new JLabel("Nome");
22              txtNome = new JTextField(20);
23              btnSalvar = new JButton("Salvar");
24
25              add(lblNome);
26              add(txtNome);
27              add(btnSalvar);
28              btnSalvar.addMouseListener(new MouseAdapter() {
29                  @Override
30                  public void mouseEntered(MouseEvent we) {
31                      btnSalvar.setBackground(Color.yellow);
32                  }
33                  @Override
34                  public void mouseExited(MouseEvent evt) {
35                      btnSalvar.setBackground(Color.red);
36                  }
37              });
38          }
39
40          public static void main(String[] args) {
41              Tela t = new Tela();
42              t.setVisible(true);
43          }
44      }
```

Observe que os eventos *mouseEntered* (linha 30) e *mouseExited* (linha 34), são tratados das linhas 28 à 37. Caso queira tratar outros eventos, deve incluir o método referente ao evento imediatamente após a linha 36. Perceba na linha 28 que foi adicionado um *Listener* ("ouvinte") ao botão para monitorá-lo quanto aos eventos de mouse. Toda vez que o ponteiro do mouse entra em contato com a área da superfície do botão, um objeto da classe *MouseAdapter* é instanciado e passado como parâmetro ao método *addMouseListener*. Na instância do

UNINOVE – USO EXCLUSIVO PARA ALUNO

objeto, são implementados (ou sobrescritos) os métodos *mouseEntered* e *mouseExited* (que existem vazios na classe *MouseAdapter*), permitindo tratar estes dois eventos.

Note na implementação do método *mouseEntered* (linhas de 30 à 32) que quando o ponteiro do mouse passar sobre a área do botão, ele ficará amarelo (linha 31). Quando sair da área do botão, será executado o método *mouseExited* (linhas de 34 à 36) e o botão ficará vermelho (linha 35).

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6* aparecerá a tela mostrada na Figura 6.58.



Figura 6.58: Tela com tratamento de eventos de mouse.

Você pode perceber que o tratamento de eventos em Java não é uma tarefa fácil. A maioria dos eventos tratados nos exemplos deste Capítulo foram aplicados apenas a alguns tipos de elementos da janela, mas a forma de tratamento de eventos sobre outros elementos ocorre de forma semelhante.

Mais à frente neste Capítulo você verá que há maneiras mais simples de se trabalhar com a criação de interfaces de usuário utilizando as APIs *swing* e *awt* no *NetBeans*.

6.4 Criação de aplicação com várias janelas ou com abas

Até agora criamos pequenas aplicações que utilizaram apenas uma janela, porém, em aplicações reais criadas nas empresas, muitas janelas podem ser necessárias. Estando em uma janela, o usuário pode acessar outra clicando em um botão, em uma opção de menu etc. Normalmente um evento chama um método que carrega outra janela da aplicação.

Uma alternativa ao uso de janelas é a utilização de abas (guias) na mesma janela. Nesse caso, cada guia pode conter o que estaria em uma janela diferente. Esta pode ser uma ótima alternativa para quem não quer lidar com abertura, fechamento e alternância entre janelas.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Nesse tópico vamos criar uma aplicação simples que utiliza duas janelas e mostrar como se faz para passar de uma para outra, quando necessário. Vamos também criar uma aplicação que utiliza abas na mesma janela e mostrar como se alterna entre elas.

6.4.1 Exemplo de aplicação com várias janelas

Criar uma aplicação que utiliza várias janelas não é uma tarefa difícil. Difícil mesmo é ajustar os elementos nas janelas de maneira adequada. O exemplo que será apresentado a seguir é composto por uma única classe que cria três janelas *JFrame* e alterna entre elas por meio de clique em botões. Não teremos a preocupação em colocar outros elementos nas janelas, pois o objetivo é explicar as operações de alternância entre janelas. Vamos então ao exemplo.

Crie um novo projeto Java no *NetBeans* chamado *AplicativoMultijanelas* com uma classe principal chamada *Menu.java*, conforme mostra a Figura 6.59.

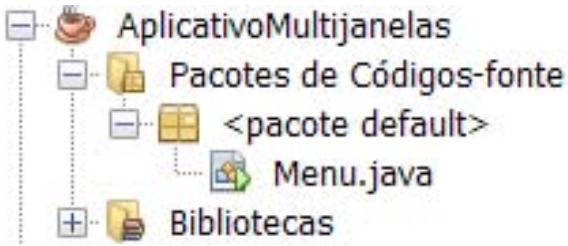


Figura 6.59: Projeto com múltiplas janelas.

Na classe *Menu.java*, digite o seguinte código-fonte:

```
1   import java.awt.*;
2   import java.awt.event.*;
3   import javax.swing.*;
4
5   public class Menu extends JFrame implements ActionListener {
6
7       JFrame frmJanela1, frmJanela2;
8       JButton btnJanela1, btnJanela2, btnVolta1, btnVolta2;
9
10      public Menu() {
11
12          //Programação da janela principal
13          setTitle("Janela Principal");
14          setDefaultCloseOperation(EXIT_ON_CLOSE);
15          setLayout(new FlowLayout());
16          btnJanela1 = new JButton("Chama Janela 1");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
17          btnJanela1.addActionListener(this);
18          btnJanela2 = new JButton("Chama Janela 2");
19          btnJanela2.addActionListener(this);
20          add(btnJanela1);
21          add(btnJanela2);
22          setBounds(100, 100, 300, 200);
23
24          //Programação da janela 1
25          frmJanela1 = new JFrame("Janela 1");
26          frmJanela1.setDefaultCloseOperation(EXIT_ON_CLOSE);
27          frmJanela1.setBounds(100, 100, 300, 200);
28          frmJanela1.getContentPane().setBackground(Color.yellow);
29          btnVolta1 = new JButton("Voltar");
30          frmJanela1.add(btnVolta1, "South");
31          btnVolta1.addActionListener(this);
32
33          //Programação da janela 2
34          frmJanela2 = new JFrame("Janela 2");
35          frmJanela2.setDefaultCloseOperation(EXIT_ON_CLOSE);
36          frmJanela2.setBounds(100, 100, 300, 200);
37          frmJanela2.getContentPane().setBackground(Color.green);
38          btnVolta2 = new JButton("Voltar");
39          frmJanela2.add(btnVolta2, "South");
40          btnVolta2.addActionListener(this);
41
42      }
43
44      @Override
45      public void actionPerformed(ActionEvent e) {
46          Object btn = e.getSource();
47          if (btn == btnJanela1) {
48              setVisible(false);
49              frmJanela1.setVisible(true);
50          }
51          if (btn == btnJanela2) {
52              setVisible(false);
53              frmJanela2.setVisible(true);
54          }
55          if (btn == btnVolta1) {
56              setVisible(true);
57              frmJanela1.setVisible(false);
58          }
59          if (btn == btnVolta2) {
60              setVisible(true);
61              frmJanela2.setVisible(false);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
62              }
63          }
64
65          public static void main(String[] args) {
66              Menu m = new Menu();
67              m.setVisible(true);
68          }
69      }
```

A linha 5 indica que a classe *Menu.java* herda a classe *JFrame* e implementa o método abstrato *actionPerformed* da interface *ActionListener*. Caso você não se lembre, a classe *JFrame* é utilizada para criar janelas *swing* e a interface *ActionListener* para que possam ser tratados eventos de ação, como cliques de mouse em botões.

As linhas 7 e 8 declaram os elementos que serão utilizados na aplicação. Atenção à declaração dos objetos *frmJanela1* e *frmJanela2* da classe *JFrame* na linha 7. Estes objetos serão usados para criar duas janelas.

Dentro do construtor da classe *Menu*.java, que vai da linha 10 à 42, são criadas as janelas e todos os elementos que serão incluídos nelas.

As linhas de 12 à 22 criam a janela principal, que aparecerá na tela quando a aplicação for compilada e executada. Além de criar a janela, estas linhas incluem nela os botões *btnJanela1* ("Chama Janela 1") e *btnJanela2* ("Chama Janela 2"). Note que esta janela utilizará o gerenciador de *layout FlowLayout* (linha 15) e que finalizará a aplicação ao clicar no "X" do canto superior direito da janela (linha 14). Perceba nas linhas 17 e 19 que um *Listener* ("ouvinte"), ficará monitorando os botões. Quando eles receberem um clique, o método *actionPerformed* (linha 45) será chamado e um objeto da classe *ActionEvent* permitirá identificar a origem do clique. Na linha 22 foi utilizado o método *setBounds,* que determina a posição e o tamanho da janela em uma única operação. Se você não se recorda, os dois primeiros parâmetros desse método determinam a posição da janela nos eixos *x* e *y* e os dois últimos parâmetros determinam o tamanho da janela (largura e altura).

As linhas de 24 à 31 criam a janela *frmJanela1* que será chamada ao clicar no botão *btnJanela1* da janela principal. Além de criar a janela, estas linhas incluem nela o botão *btnVolta1* ("Voltar"), que ao ser clicado volta para a janela principal. Note que esta janela também poderá encerrar a aplicação se for clicado no "X" do canto superior direito (linha 26). Perceba que o objeto da janela tem nome de *frmJanela1* e por meio desse objeto são adicionados elementos à janela e realizadas configurações de propriedades, como na linha 27, que determina a posição e o tamanho da janela, e na linha 28, que determina a cor de fundo da janela. Esta janela utilizará o gerenciador de *layout* padrão,

UNINOVE – USO EXCLUSIVO PARA ALUNO

que é o *BorderLayout*, por isso, o botão *btnVolta1* foi adicionado na posição sul do *layout* (linha 30). Perceba que a este botão foi adicionado um *Listener* ("ouvinte") para quando ele for clicado chamar o método *actionPerformed* da linha 45, que indica o que deve ser feito após o clique.

As linhas de 33 à 40 criam a janela *frmJanela2* que será chamada ao clicar no botão *btnJanela2* da janela principal. Além de criar a janela, estas linhas incluem nela o botão *btnVolta2* ("Voltar"), que ao ser clicado volta para a janela principal.

Observe que todos os botões possuem um *Listener* ("ouvinte") adicionado pelo método *addActionListener*. Este *Listener* chama automaticamente o método *actionPerformed* da linha 45 quando o botão é clicado. Este método recebe um objeto da classe *ActionEvent* com informações sobre o objeto de origem do clique.

Na linha 46, um objeto da classe *Object* recebe o nome e informações sobre o botão clicado. Por meio deste objeto, são feitas comparações para indicar o que deve ocorrer de acordo com o clique em cada um dos botões das janelas.

Se o evento de clique foi gerado no botão *btnJanela1* (linha 47), a janela principal fica oculta (linha 48) e a janela *frmJanela1* se torna visível na tela.

Se o evento de clique foi gerado no botão *btnJanela2* (linha 51), a janela principal fica oculta (linha 52) e a janela *frmJanela2* se torna visível na tela.

Se o evento de clique foi gerado no botão *btnVolta1* (linha 55), a janela principal se torna visível (linha 56) e a janela *frmJanela1* se torna oculta (linha 57).

Se o evento de clique foi gerado no botão *btnVolta2* (linha 59), a janela principal se torna visível (linha 60) e a janela *frmJanela2* se torna oculta (linha 61).

As linhas de 65 à 68 iniciam a aplicação quando ela for compilada e executada. A linha 66 instancia um objeto da classe Menu por meio de uma chamada ao construtor da classe utilizando o comando *new*. O construtor então é executado, gerando as janelas na memória. A linha 67 exibe apenas a janela principal. As demais ficam na memória, mas só serão visualizadas quando os botões forem clicados.

NOTA

Toda e qualquer classe em Java é subclasse da classe *Object*. Mesmo quando não há *extends* na declaração da classe, o Java automaticamente faz a classe criada herdar a classe *Object*. Como as classes que criamos são subclasses da classe *Object*, elas também herdam os métodos da classe *Object*, como *getClass*, *toString*, *equals* etc. Muitos métodos que chamamos de classes instaladas com o Java retornam um objeto da classe *Object*, como o método *getSource* da classe *ActionEvent*. Por ser genérico, este objeto normalmente pode ser convertido em outros objetos, de acordo com a necessidade de quem recebe.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6*, aparecerá a janela mostrada na Figura 6.60.



Figura 6.60: Tela principal da aplicação com várias janelas.

Ao clicar no botão "Chama Janela 1", a janela principal ficará oculta e a janela mostrada à esquerda na Figura 6.61 será apresentada na tela. Se for clicado no botão "Chama Janela 2", a janela principal ficará oculta e a janela mostrada à direita na Figura 6.61 será apresentada na tela.



Figura 6.61: Janelas da aplicação abertas a partir da janela principal.

Se for clicado no botão "Voltar", a janela atual ficará oculta e a janela principal será mostrada na tela novamente.

Você deve ter observado que trabalhar com janelas em uma única classe não é uma tarefa difícil, porém, dependendo da quantidade de elementos que forem colocados na janela, o código-fonte pode ficar muito grande e difícil de gerenciar e manter. Nesse caso, uma alternativa é criar uma classe que herda a classe *JFrame* para cada janela. Quando um botão de uma janela for clicado, você instancia um objeto da classe referente a outra janela e a torna visível.

NOTA Quando você criar aplicação com várias janelas, alterne entre elas tornando visível a que você deseja, e invisível a que não quer ver. Isto é feito por meio do método *setVisible* do objeto referente à janela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.4.2 EXEMPLO DE APLICAÇÃO COM ABAS NA MESMA JANELA

Utilizar abas é tão fácil como trabalhar com diversas janelas e oferece a facilidade de ter o conteúdo de várias janelas em uma só, separado por abas (ou guias). Você pode alternar entre as abas facilmente, clicando no título da aba.

Para testar o trabalho com abas, crie um novo projeto Java no *NetBeans* chamado *ProjetoAbas,* e nele crie a classe principal *TelaAbas.java*, como mostra a Figura 6.62.



Figura 6.62: Projeto da aplicação com janelas com abas.

Para trabalhar com abas você deve criar um painel que suporta abas, utilizando a classe *JTabbedPane*. Além desse painel, você deve criar vários painéis comuns, utilizando a classe *JPanel*. Nesses painéis comuns você deve colocar os elementos que cada aba terá. No final, cada painel *JPanel* será então adicionado como uma aba do painel *JTabbedPane*.

Na classe *TelaAbas.java*, digite o seguinte código-fonte:

```
1   import java.awt.*;
2   import javax.swing.*;
3
4   public class TelaAbas extends JFrame {
5
6       private JTabbedPane pnlComGuias;
7       private JPanel pnl1, pnl2;
8       private JLabel lblUsuario, lblSenha, lblNome, lblEndereco, lblCidade;
9       private JTextField txtUsuario, txtNome, txtEndereco, txtCidade;
10      private JPasswordField txtSenha;
11
12      public TelaAbas() {
13
14          setTitle("Aplicação de painel com guia");
15          setSize(300, 200);
16          setLocation(300,300);
17          setDefaultCloseOperation(EXIT_ON_CLOSE);
18          setBackground(Color.gray);
19
20          //Cria o painel pnl2 que será incluído na primeira aba
21          pnl1 = new JPanel();
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
22          pnl1.setLayout(null);
23          lblUsuario = new JLabel("Username:");
24          lblUsuario.setBounds(10, 15, 150, 20);
25          pnl1.add(lblUsuario);
26          txtUsuario = new JTextField();
27          txtUsuario.setBounds(10, 35, 150, 20);
28          pnl1.add(txtUsuario);
29          lblSenha = new JLabel("Senha:");
30          lblSenha.setBounds(10, 60, 150, 20);
31          pnl1.add(lblSenha);
32          txtSenha = new JPasswordField();
33          txtSenha.setBounds(10, 80, 150, 20);
34          pnl1.add(txtSenha);
35
36          //Cria o painel pnl2 que será incluído na segunda aba
37          pnl2 = new JPanel();
38          pnl2.setLayout(new GridLayout(3, 2));
39          lblNome = new JLabel("Nome:");
40          pnl2.add(lblNome);
41          txtNome = new JTextField();
42          pnl2.add(txtNome);
43          lblEndereco = new JLabel("Endereço:");
44          pnl2.add(lblEndereco);
45          txtEndereco = new JTextField();
46          pnl2.add(txtEndereco);
47          lblCidade = new JLabel("Cidade:");
48          pnl2.add(lblCidade);
49          txtCidade = new JTextField();
50          pnl2.add(txtCidade);
51
52          // Cria o painel com abas e adiciona os paineis pnl1 e pnl2 a ele
53          pnlComGuias = new JTabbedPane();
54          pnlComGuias.addTab("Login", pnl1);
55          pnlComGuias.addTab("Cadastro", pnl2);
56          add(pnlComGuias, BorderLayout.CENTER);
57      }
58
59      public static void main(String args[]) {
60          TelaAbas tela = new TelaAbas();
61          tela.setVisible(true);
62      }
63  }
```

Nas linhas de 6 à 10 são declarados os objetos que serão colocados na janela. Veja que são declarados dois painéis *JPanel* (linha 7) e um painel *JTabbedPane* (linha 6). Os dois painéis *JPanel*, depois de construídos, serão adicionados ao painel que gera as abas, o painel *JTabbedPane* (linhas 54 e 55).

UNINOVE – USO EXCLUSIVO PARA ALUNO

O construtor da classe contém as linhas que geram os elementos e os adicinam em painéis e na tela. Ele vai da linha 12 à linha 57.

Nas linhas de 14 à 18 são definidas as propriedades da janela, como em projetos anteriores.

Nas linhas de 21 à 34 é criado o painel *pnl1* (linha 21), com *layout* nulo (linha 22), o que significa que os elementos terão a posição e o tamanho definidos manualmente por meio do método *setBounds*. Veja que diversos elementos são criados e adicionados a este painel.

A linha 23 cria o rótulo "*Username*", que tem sua posição e tamanho definidos na linha 24 e é adicionado ao painel *pnl1* na linha 25. O mesmo procedimento ocorre com o campo texto para entrada do nome do usuário, das linhas 26 à 28; com o rótulo "Senha", das linhas 29 à 31; e com o campo texto para a entrada da senha, das linhas 32 à 34. Veja que o campo para a entrada de senha é criado utilizando a classe *JPasswordField* (linha 32).

Das linhas 36 à 50 é criado o painel *pnl2* (linha 37) com *layout GridLayout* com 3 linhas e 2 colunas (linha 38). Nas linhas seguintes, de 39 à 50, são criados rótulos e campos texto e adicionados a este painel.

Na linha 53 é criado o painel que suporta guias, o *JTabbedPane*. Na linha 54 o painel *pnl1* é adicionado na primeira guia deste painel com o título de guia "Login". Na linha 55 o painel *pnl2* é adicionado a este painel com o título de guia "Cadastro". Na linha 56 o painel com as guias (*pnlComGuias*) é adicionado à janela na posição central do *layout BorderLayout*.

A linha 60 cria um objeto desta classe (*TelaAbas.java*), o que executa o construtor da classe, criando a janela e os objetos da janela na memória. A linha 61 torna tudo isso visível ao usuário.

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6*, a janela da esquerda na Figura 6.63 é apresentada na tela.

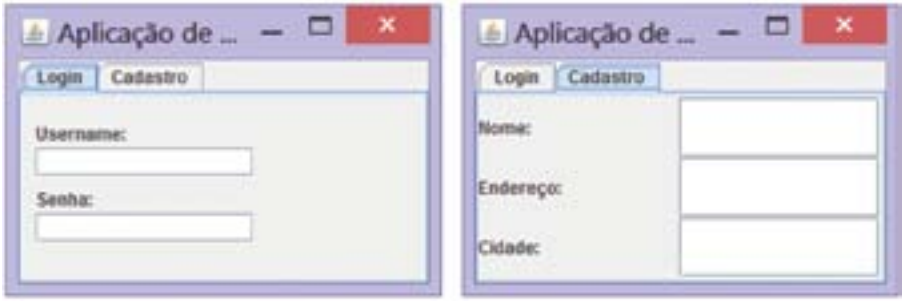


Figura 6.63: Janela com abas.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Perceba na Figura 6.63 que a janela da esquerda mostra a aba “Login” aberta, quando a aplicação é executada. Já a janela da direita mostra a aba “Cadastro” aberta, quando o usuário clicar nesta aba. Você pode alternar entre as abas facilmente, bastando para isso clicar sobre o título dela.

As abas possuem um índice numérico começado por 0 que vai aumentando na medida em que vão sendo adicionadas à janela. Assim, a primeira aba terá índice 0, a segunda terá índice 1, e assim por diante.

A seguir são apresentados os principais métodos utilizados para executar operações com as abas.

pnlComGuias.setSelectedIndex(1);

Ao executar a aplicação, a aba ativa será a segunda (aba 1).

pnlComGuias.setBackgroundAt(0, Color.BLUE);

O plano de fundo do título da primeira aba (aba 0) ficará azul.

pnlComGuias.setEnabledAt(0, false);

A primeira aba (aba 0) será desativada. Ao clicar, não será aberta.

pnlComGuias.setTitleAt(1, "Controle");

O título da segunda aba (aba 1) será "Controle".

int indice = pnlComGuias.getSelectedIndex();

Obtém o número de índice da aba ativa e armazena na variável do tipo int.

String tituloAba = pnlComGuias.getTitleAt(1);

Obtém o título da segunda aba (aba 1) e armazena na variável String.

if(pnlComGuias.isEnabledAt(1)){};

Verifica se a segunda aba (aba 1) está ativa.

pnlComGuias.remove(1);

Remove a segunda aba (aba 1).

UNINOVE – USO EXCLUSIVO PARA ALUNO

6.5 Criação de interfaces do usuário utilizando recursos de arrastar/soltar com o *NetBeans*

Provavelmente nesta altura do Capítulo você deve estar comparando o desenvolvimento de interfaces do usuário em Java com o desenvolvimento em outras plataformas, como com o *Visual Studio* da *Microsoft*. Desenvolvendo, por exemplo, em C# ou *Visual Basic* com o *Visual Studio*, você cria rapidamente interfaces gráficas de usuário (GUIs) com recursos de arrastar/soltar. É possível criar uma interface de usuário em minutos. Porém, você não tem acesso ao código-fonte que é gerado pelos componentes que você coloca na janela. Eles são, neste caso, uma enorme caixa-preta. Na plataforma Java, dá trabalho, demora, mas você cria cada componente e tem mais controle sobre eles.

Apesar disso, as empresas querem agilidade no desenvolvimento de software, tanto na modelagem e projeto, quanto na programação. Por isso, acabam optando, quando possível, por ferramentas que permitam desenvolver o software mais rapidamente.

Para não ficar atrás de outras ferramentas, a *Sun Microsystem* (hoje de propriedade da *Oracle Corporation*) adicionou ao *NetBeans* recursos que permitem criar interfaces do usuário de forma semelhante ao *Visual Studio* da Microsoft, com recursos de arrastar/soltar.

Neste tópico, vamos explorar estes recursos e criar interfaces de usuário ricas em pouco tempo.

6.5.1 Exemplo de criação de interface gráfica simples utilizando os recursos de arrastar/soltar do *NetBeans*

Neste tópico vamos criar um projeto de uma interface de usuário simples utilizando as APIs *swing* e *awt* e os recursos de arrastar/soltar do *NetBeans*.

Para criar um novo projeto, execute os passos a seguir:

– Clique no menu “Arquivo” e na opção “Novo projeto”.

– Na divisão “Categorias”, selecione Java; na divisão “Projetos”, selecione Aplicativo Java e clique no botão “Próximo”.

– No campo “Nome do Projeto:” digite ArrastarSoltar e desmarque a caixa de seleção “Criar Classe Principal”, pois não iremos criar nenhuma classe principal. Faremos isso mais adiante.

– Clique no botão “Finalizar”.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Ao terminar este processo, o projeto estará como mostra a Figura 6.64.

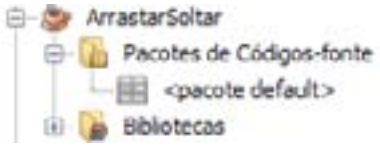


Figura 6.64: Projeto com recursos de arrastar/soltar.

Agora vamos criar a classe principal, que será um formulário *JFrame*. Execute, para isso, os seguintes procedimentos:

– Clique com o botão direito sobre o nome do projeto (ArrastarSoltar), selecione a opção "Novo" e a opção "*Form JFrame*", como mostra a Figura 6.65.

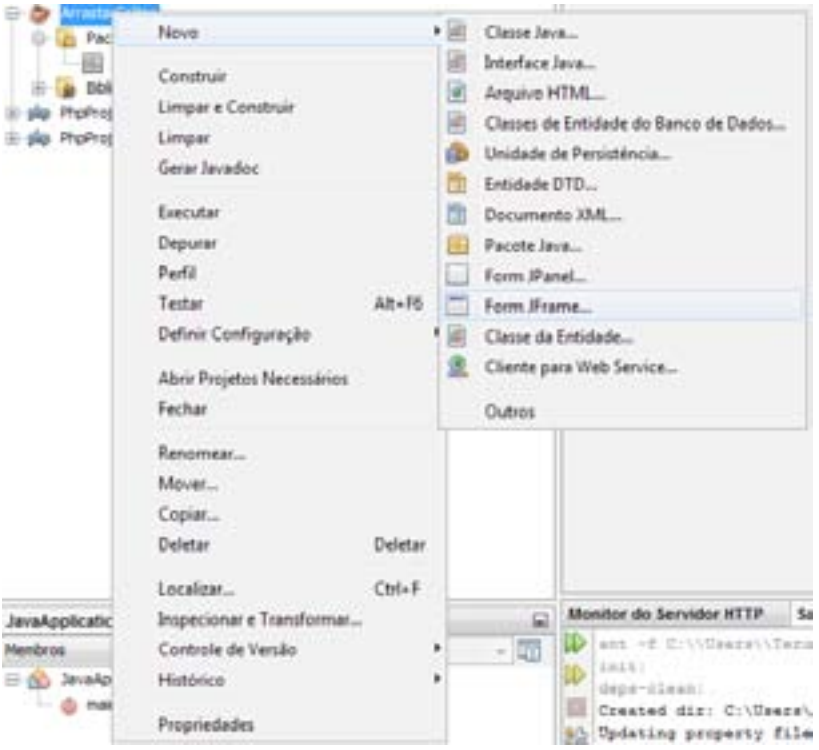


Figura 6.65: Criação de formulário JFrame.

– No campo "Nome da Classe", digite *Tela* e clique no botão "Finalizar".

Aparecerá na área de trabalho do *NetBeans*, no centro da tela, um formulário vazio e à direita, uma paleta com os elementos que podem ser arrastados para este formulário, como mostra a Figura 6.66.

UNINOVE – USO EXCLUSIVO PARA ALUNO

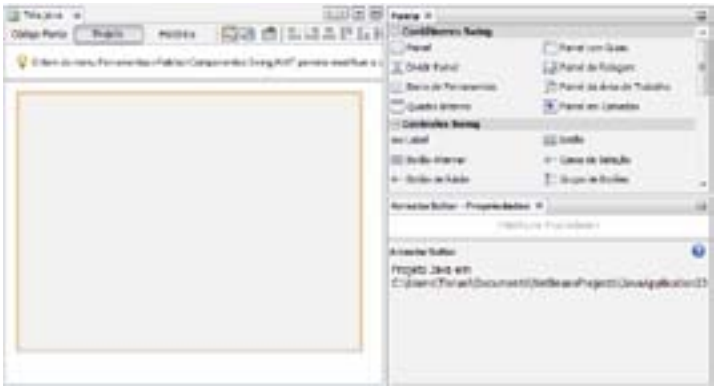


Figura 6.66: Paleta de elementos que podem ser adicionados ao formulário.

Veja que na paleta (painel) da direita existe o elemento *Label* (*JLabel*), Botão (*JButton*), Painel (*JPanel*) etc. Há uma barra de rolagem nessa paleta que permite visualizar os demais elementos. A Figura 6.67 mostra os elementos disponíveis nesta paleta.



Figura 6.67: Paleta de elementos que podem ser arrastados para o formulários.

Clique no formulário no centro da tela para selecioná-lo. Se você posicionar o ponteiro do mouse sobre suas laterais, poderá redimensioná-lo. Observe que ao clicar no formulário, do lado inferior direito, aparece um painel com as propriedades que podem ser modificadas no formulário. A Figura 6.68 mostra algumas destas propriedades.

UNINOVE – USO EXCLUSIVO PARA ALUNO

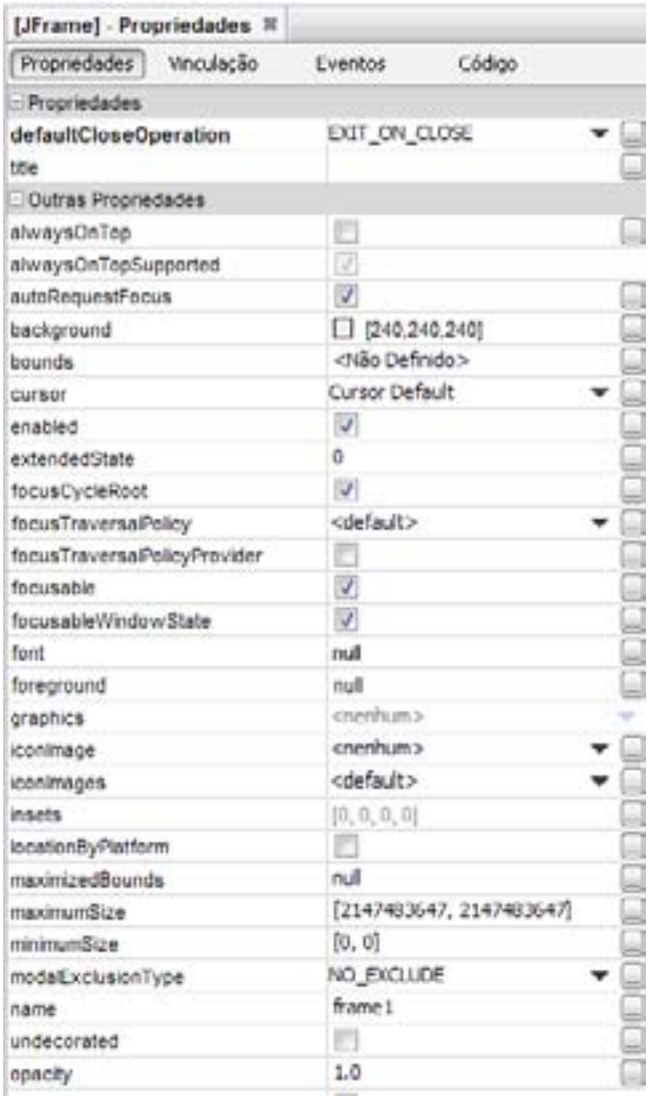


Figura 6.68: painel para mudar as propriedades dos elementos de tela.

Para mudar as propriedades de qualquer elemento, basta clicar sobre ele que as propriedades passíveis de modificação serão mostradas.

No canto inferior esquerdo da tela há um painel chamado "Navegador", onde são exibidos o nome do formulário *JFrame* e os nomes dos elementos incluídos no formulário. Clicando com o botão direito do mouse sobre "*JFrame*", você poderá definir algumas propriedades do formulário, como o gerenciador de *layout* que ele deve utilizar. A Figura 6.69 mostra este procedimento.

UNINOVE – USO EXCLUSIVO PARA ALUNO

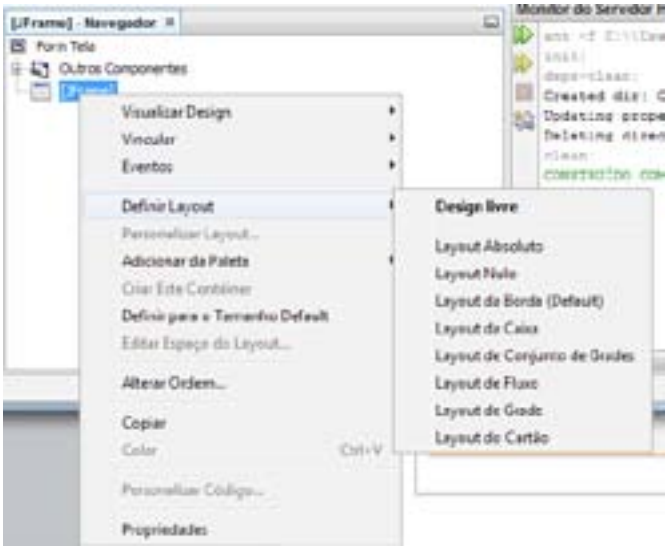


Figura 6.69: Painel Navegador – mudando o layout do formulário.

Veja que o *layout* padrão do formulário é o *BorderLayout* ("*Layout* de Borda"), mas sugiro mudar para "*Layout* nulo", para você ter mais liberdade para arrastar elementos para o formulário. Todos os elementos colocados no formulário aparecerão nesse painel "Navegador", abaixo de "*JFrame*".

Arraste da paleta para o formulário dois painéis e os redimensione, como mostra a Figura 6.70. No painel da parte superior, coloque três rótulos (*JLabel*) e três campos de texto (*JTextField*). No painel da parte inferior, coloque três botões (*JButton*). Selecione os painéis e mude a propriedade "*background*", selecionando cor de fundo para eles. Veja na Figura 6.70 como deve ficar o formulário.

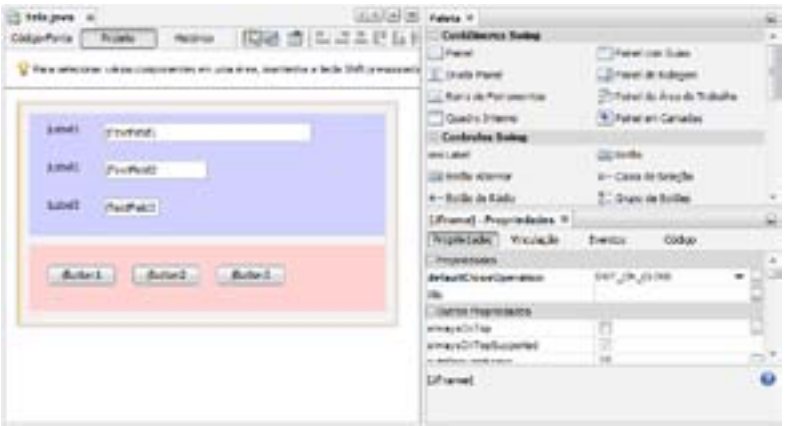


Figura 6.70: Formulário com elementos arrastados.

UNINOVE – USO EXCLUSIVO PARA ALUNO

No painel "Navegador", no canto inferior esquerdo da tela, você verá o nome padrão de todos os elementos do formulário, conforme Figura a seguir:

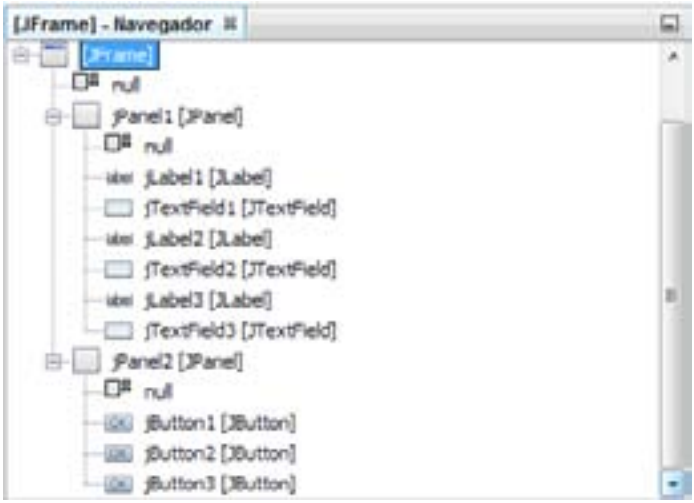


Figura 6.71: Apinel "Navegador", com os nomes dos elementos do formulário.

Fica difícil programar os botões utilizando os nomes padrão dos elementos, por isso, temos que renomeá-los. Precisamos também mudar o texto padrão que aparece nesses elementos. Há diversas maneiras de fazer isso. Uma delas é selecionar o elemento e mudar o nome e o texto no painel de propriedades que aparece à direita. Outra forma, é clicar com o botão direito sobre o elemento e selecionar "Editar Texto", para mudar o que está escrito no elemento e "Alterar o Nome da Variável", para mudar o nome do elemento. Veja estas opções na Figura 6.72.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 6.72: Nomeando os elementos e mudando os rótulos.

Ao clicar com o botão direito do mouse sobre a primeita caixa de texto e selecionar a opção "Alterar o Nome da Variável", aparecerá a janela mostrada na Figura 6.73.



Figura 6.73: Nomeação de elementos do formulário.

Mude o nome para *txtNome* e clique no botão "OK".

Mude o nome de todos os elementos utilizando o procedimento acima, de forma que eles fiquem com os nomes mostrados no painel "Navegador" apresentado na Figura 6.74.

UNINOVE – USO EXCLUSIVO PARA ALUNO

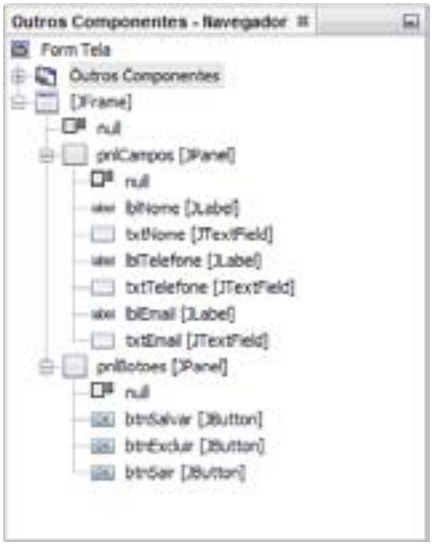


Figura 6.74: Painel "Navegador" com elementos nomeados.

Mude também o texto dos elementos, de forma que o formulário fique com a aparência mostrada na Figura 6.75.

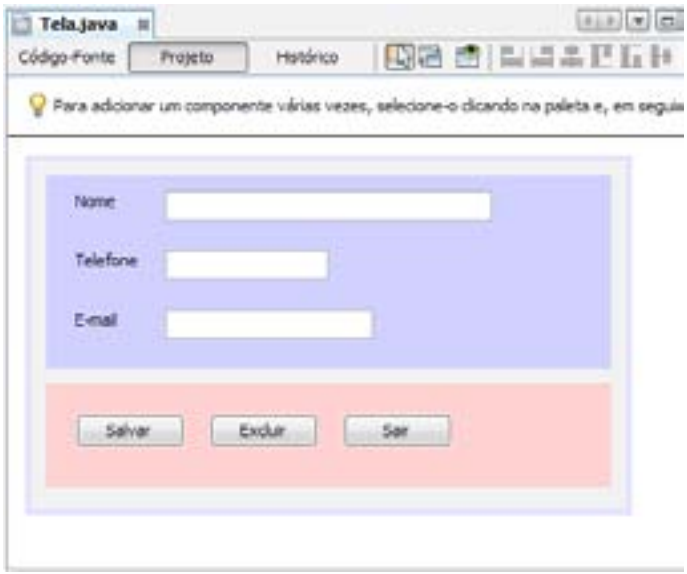


Figura 6.75: Formulário criado e com propriedades definidas.

Para que o formulário apareceça centralizado ao executar e também apareça no tamanho em que foi desenhado, você deve alterar, no painel de propriedades à direita, a política de tamanho de formulário. Para isso, no painel "Propriedades",

UNINOVE – USO EXCLUSIVO PARA ALUNO

clique na opção “Código”, na parte superior do painel e, no campo “Política de Tamanho do Form”, selecione a opção “Gerar Código de Redimensionamento”. Marque ainda a opção “Gerar Centralizado”. Veja a seguir a Figura 6.76, do painel “Propriedades”, onde são realizados estes procedimentos.

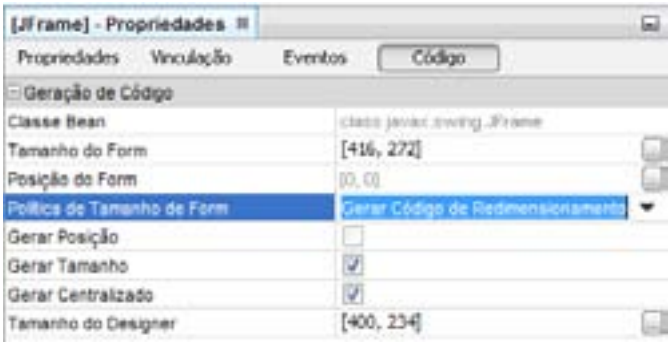


Figura 6.76: definindo a política de tamanho do formulário.

Para compilar e executar o projeto e ver como está o visual da tela, basta pressionar a tecla *F6* a qualquer momento.

Chegou a hora de programar os botões. Para isso, clique com o botão direito do mouse sobre o botão “Salvar”, selecione a opção “Eventos”, em seguida a opção “*Action*” e, por último, a opção “*actionPerformed*”, como mostra a Figura 6.77.

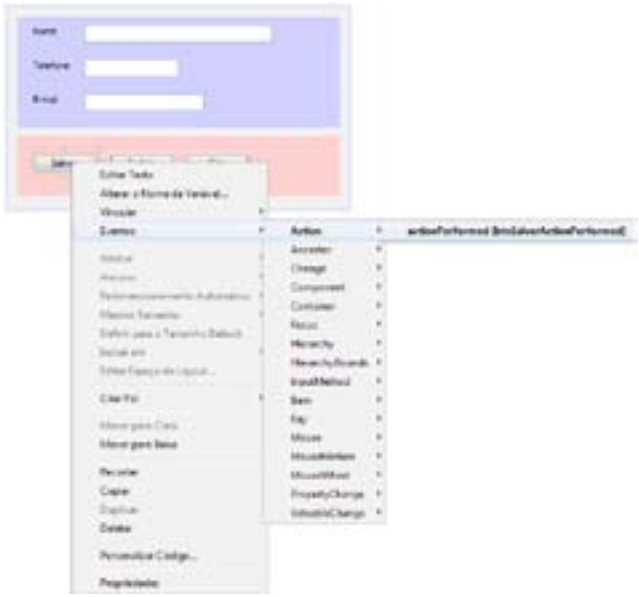


Figura 6.77: Programando os eventos dos botões.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Aparecerá uma área para edição de código-fonte, onde o cursor estará na linha 90 da Figura 6.78, e onde deve ser programado o que deve ocorrer quando o usuário clicar no botão “Salvar”. A Figura 6.78 mostra detalhes desta área de programação.



Figura 6.78: Área de programação dos eventos dos botões.

Na Figura 6.78, alguns detalhes merecem atenção. Veja que por trás daquele formulário para onde você arrasta os elementos, há uma classe gerada automaticamente, de acordo com os elementos que você vai adicionando e com as propriedades que você vai mudando. Na linha 10 você pode ver que a classe foi declarada. Na linha 97 você pode ver a implementação do método *main*, já que esta é a classe principal. Na linha 89 você pode ver o método *btnSalvarActionPerformed*, utilizado para programar o evento de clique no botão “Salvar”. Na linha 93 você pode ver o método *btnExcluirActionPerformed*, utilizado para programar o evento de clique no botão “Excluir”.

Veja que a numeração de linhas do número 16 passa para 88. Isso ocorre porque tem um conjunto de linhas ocultas entre a linha 16 e a linha 88. Para ver essas linhas, clique no sinal de mais (+) que aparece na linha 16. Você verá que as linhas ocultas serão mostradas. Verá que nessas linhas está a programação referente a geração dos elementos arrastados para a tela. Você não pode alterar o código-fonte dessa área, pois ela é gerada na medida em que você interage montando o formulário. Poderá apenas inserir código de programação nos métodos de tratamento de evento.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Perceba na parte superior da tela que há duas opções clicáveis e relevantes: "Código-Fonte" e "Projeto". Se você clicar em "Projeto", será mostrado o formulário para você incluir novos elementos ou modificar elementos já existentes. Se você clicar em "Código-Fonte", aparecerá esta área de edição do código-fonte.

Pressionando *F6* para compilar e executar o projeto, você verá que a janela aparecerá centralizada, como mostra a Figura 6.79.

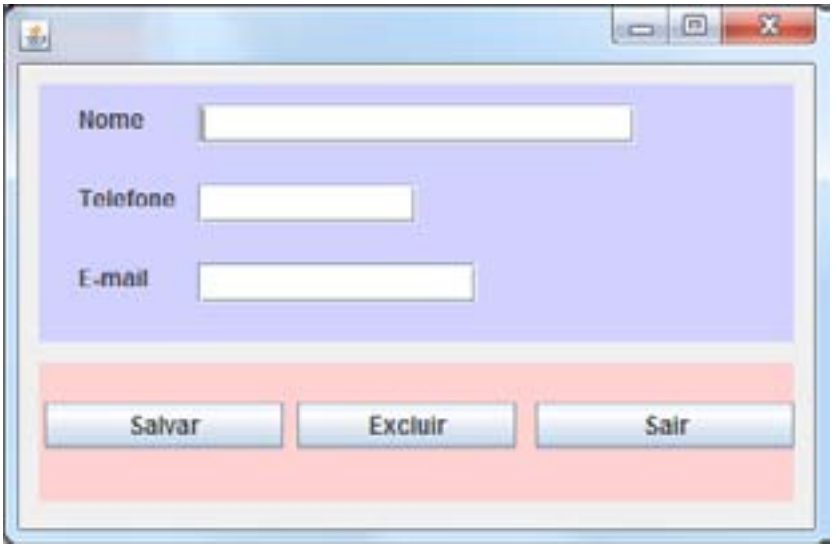


Figura 6.79: Janela gerada com recursos de arrastar/soltar.

Não programamos o botão "Salvar", pois faremos isso no próximo Capítulo, quando trataremos de operações com acesso a banco de dados. Neste tópico o objetivo é mostrar como é fácil criar uma interface do usuário utilizando os recursos de arrastar/soltar do *NetBeans*.

Caso em algum momento algum dos painéis da área de trabalho do *NetBeans* for fechado, você poderá abrí-lo novamente clicando na opção "Janela" do menu, conforme mostra a Figura 6.80.

UNINOVE – USO EXCLUSIVO PARA ALUNO

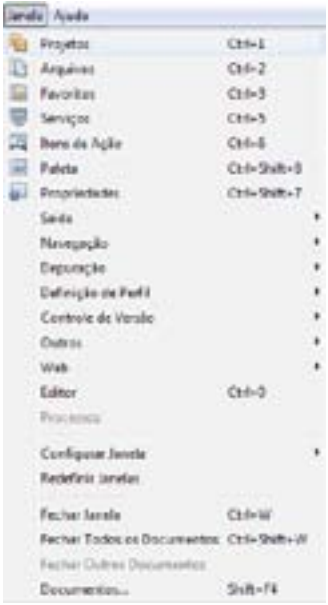


Figura 6.80: menu “Janela” para exibir paineis fechados por engano.

Veja que você pode clicar e exibir os painéis “Projetos”, “Arquivos” e “Serviços”, que aparecem na parte superior esquerda da área de trabalho; o painel “Paleta”, que aparece na parte superior direita da janela com os elementos que podem ser arrastados para o formulário; o painel “Propriedades”, que aparece na parte inferior direita da janela e permite mudar as propriedades dos elementos colocados no formulário etc.

6.6 Resumo

A interface do usuário é uma das partes mais importantes da aplicação, pois representa todas as telas às quais o usuário tem acesso.

Para criar interfaces gráficas de usuário (GUIs) em aplicações para *desktop* com Java utilizam-se as APIs *swing* e *awt*. A API *swing* é a evolução da *awt* e é composta por diversos pacotes de classes, sendo que os mais utilizados são *javax.swing* e *javax.swing.event*. Nesses pacotes existem um conjunto de classes que permitem criar, configurar e apresentar qualquer componente de tela como janelas, painéis, rótulos, caixas de texto, menus, tabelas etc, e tratar eventos que podem ocorrer nesses componentes.

Na criação de uma janela é comum o uso da classe *JFrame*. Para agrupar componentes na janela é comum o uso da classe *JLabel*. Estas classes permitem criar contêineres para agrupar elementos, normalmente, com características comuns.

UNINOVE – USO EXCLUSIVO PARA ALUNO

A criação de janelas que compõem a interface do usuário utilizando as APIs *swing* e *awt* pode ser feita de duas maneiras: posicionando os elementos estaticamente de forma manual ou posicionando os elementos automaticamente utilizando gerenciadores de *layout*.

No posicionamento estático você define a posição e o tamanho do elemento em *pixel*, de acordo com as coordenadas *x* e *y*. Dessa forma você terá mais controle sobre os elementos, porém, precisará fazer cálculos para que uns não sobreponham outros. Os métodos utilizados para posicionar e dimensionar os elementos são *setSize*, *setLocation* e *setBounds*.

Na criação de janelas de forma automática você usa gerenciadores de *layout* que já apresentam uma pré-disposição de *layout* onde você terá apenas que encaixar os elementos. É mais fácil de utilizar, mas você fica preso às características pré-definidas do *layout*.

Os gerenciadores de *layout* são classes contidas no pacote *java.awt* que possuem atributos e métodos como classes convencionais. Estas classes são *FlowLayout*, *GridLayout*, *BorderLayout* e *GridBagLayout*.

O gerenciador de *layout* padrão é *BorderLayout*, que divide a janela em cinco regiões: norte, sul, leste, oeste e centro. Cada região poderá conter apenas um elemento que ocupará toda área disponível.

O gerenciador de *layout* mais simples é o *FlowLayout*, que encaixa os elementos um à direita do outro a partir do topo da janela, quebrando linha automaticamente.

O gerenciador de *layout GridLayout* divide o contêiner em linhas e colunas como uma tabela com células (*grid*). Em cada célula você só poderá colocar um elemento, que ocupará toda a área disponível na célula.

O gerenciador de *layout GridBagLayout* é o mais flexível de todos, porém, é o mais complexo para se criar, pois cada elemento deve conter uma posição inicial, uma posição final, um tamanho, uma escala, um alinhamento e um preenchimento. Ele divide o contêiner em linhas e colunas como uma tabela com células (*grid*). Esse gerenciador de *layout* permite a mesclagem de células e, consequentemente, a criação de células de tamanhos diferentes.

Na maioria das vezes, quando criamos uma aplicação *desktop* para uma empresa, temos que utilizar vários gerenciadores de *layout* em uma mesma janela, normalmente, um para a janela principal e diversos outros para os painéis que a janela terá. A combinação de gerenciadores de *layout* permite mais flexibilidade ao desenvolvedor.

Uma aplicação *desktop* normalmente não é composta de apenas uma janela, mas sim por um conjunto de janelas. Uma alternativa à criação de várias

UNINOVE – USO EXCLUSIVO PARA ALUNO

janelas é a criação de abas (ou guias) na mesma janela. Para criar janelas você utiliza a classe *JFrame*, e para criar abas na janela você pode utilizar as classes *JTabbedPane* e *JPanel*.

Os elementos colocados na janela podem ser formatados, tendo suas propriedades de formatação alteradas. Assim, você poderá definir cores de fonte e de fundo, tipo e tamanho de fonte etc. Cada elemento permite mudar um conjunto de formatações específicas, que podem não ser comuns a todos os elementos.

As janelas criadas nas aplicações *desktop* podem conter menus agrupados em uma barra de menus posicionada na parte superior da tela. Esses menus podem ser criados pelas classes *JMenuBar*, *JMenu* e *JMenuItem*. Além dos menus, nas janelas, você pode inserir diversos tipos de botões por meio de classes como *JButton*, *JRadioButton*, *ButtonGroup*, *JCheckBox* e *JToogleButton*.

As caixas de texto permitem que o usuário entre com informações que podem ser manipuladas na aplicação. Para criar e formatar caixas de texto você utiliza classes como *JTextField*, *JTextArea*, *JPasswordField*, *JFormattedTextField* e *MaskFormatter*.

Além das caixas de texto você pode criar caixas de combinação (combos), por meio das classes *JComboBox* e *DefaultComboBoxModel*; listas, por meio das classes *DefaultListModel* e *JList*; tabelas, por meio das classes *JTable* e *TableModel* etc.

Todos os elementos colocados na janela, incluindo a própria janela, são sucetíveis a eventos como cliques, abertura, fechamento, foco etc. Quando estes eventos acontecem, métodos específicos são chamados para tratá-los.

Existem várias categorias de eventos, sendo que as principais são eventos de ação, eventos de janela, eventos de foco e eventos de mouse.

Os eventos de ação são normalmente disparados quando se clica em um elemento. Eles são tratados utilizando classes como *ActionListener* e *ActionEvent*. Todos os elementos de tela sucetíveis a clique podem tratar eventos deste tipo.

Os eventos de janela ocorrem quando a janela recebe algum estímulo, seja do usuário, ou automaticamente. Ocorrem quando a janela abre, quando ela fecha, quando ganha ou perde o foco, quando é maximizada ou minimizada etc. Para tratar os eventos de janela podem ser utilizadas diversas classes, como *WindowEvent*, *WindowListener*, *WindowAdapter*, *WindowFocusEvent*, *WindowFocusListener*, *WindowFocusAdapter*, *WindowStateEvent*, *WindowStateListener* e *WindowStateAdapter*.

Eventos de foco ocorrem normalmente quando o elemento ganha ou perde o foco. Eles são tratados normalmente pelas classes *FocusListener*, *FocusAdapter* e *FocusEvent*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Eventos de mouse acontecem quando o mouse interage com o elemento, seja sendo posicionado sobre o elemento, saindo de sobre o elemento, clicando no elemento, movendo-se sobre o elemento etc. As principais classes utilizadas para tratar eventos de mouse são *MouseListener*, *MouseMotionListener*, *MouseMotionAdapter*, *MouseAdapter* e *MouseEvent*.

Criar interface gráfica de usuário manualmente e tratar seus eventos utilizando as APIs *swing* e *awt* é uma tarefa difícil e desafiadora, pois demanda muito tempo, tanto na aprendizagem para usar os recursos como na programação em si. A carga de comandos utilizados é muito grande. Pensando em facilitar a vida do programador, a *Sun Microsystem* incluiu no *NetBeans* um conjunto de recursos que permite criar as telas com recursos de arrastar/soltar, como outros ambientes de desenvolvimento, por exemplo, o *Visual Studio* da *Microsoft*. Criar as telas com esses recursos faz com que o desenvolvedor tenha mais tempo para se dedicar à programação dos eventos e das regras de negócio da aplicação. Diversas ferramentas têm sido desenvolvidas para agilizar ou até mesmo automatizar a criação da interface do usuário com o auxílio do desenvolvedor.

6.7 Exercícios

Os exercícios serão divididos em duas partes: os exercícios práticos, que envolvem criação de telas e programação de eventos, e os exercícios conceituais, que desenvolvem o entendimento dos conceitos apresentados neste capítulo.

6.7.1 Exercícios práticos

1) Crie a janela a seguir utilizando posicionamento estático e manual dos elementos de tela. Veja que há dois painéis com título. O painel superior contém os campos e o painel inferior contém os botões. Não utilize recursos de arrastar/soltar.



UNINOVE – USO EXCLUSIVO PARA ALUNO

2) Crie a janela a seguir utilizando gerenciadores de *layout*. Note que a barra de menus contém os menus "Arquivo" e "Ajuda". No menu "Arquivo" coloque os itens de menu "Salvar" e "Sair". No menu "Ajuda", coloque apenas a opção "Obter ajuda". Não utilize recursos de arrastar/soltar.



3) Crie a janela a seguir utilizando gerenciadores de *layout*. Programe os botões "Fatorial" e "Raiz Quadrada". O botão "Fatorial", ao ser clicado, deve calcular o fatorial do número inserido no campo "Valor" e exibir o resultado no campo "Resultado". O botão "Raiz Quadrada", ao ser clicado, deve calcular a raiz quadrada do número inserido no campo "Valor" e exibir o resultado no campo "Resultado". Não utilize recursos de arrastar/soltar.

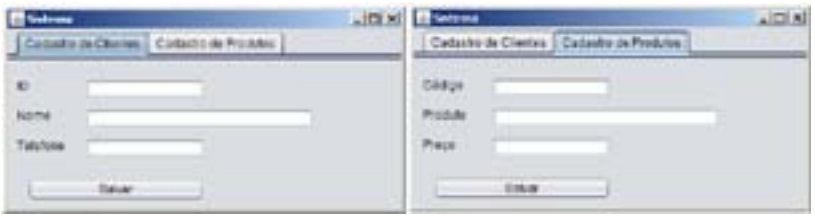


4) Crie uma aplicação Java que gere a janela com o menu principal abaixo. Ao ser clicado na opção "Cadastro de Clientes", a janela principal deve ser fechada e deve ser aberta a janela de cadastro de clientes. Ao ser clicado no botão "Cadastro de Produtos", a janela principal deve ser fechada e deve ser aberta a janela para o cadastro de produtos. Ao clicar no botão "Voltar" das telas secundárias, estas janelas devem ser fechadas e a janela principal deve ser mostrada. Utilize ou não gerenciadores de *layout*. Não utilize recursos de arrastar/soltar.

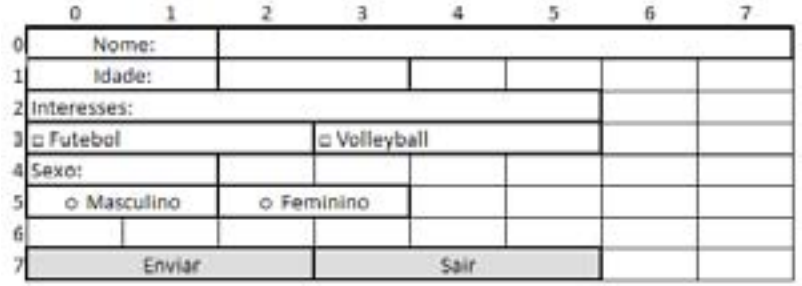
UNINOVE – USO EXCLUSIVO PARA ALUNO



5) Crie uma janela que contenha duas abas: “Cadastro de Clientes” e “Cadastro de Produtos”. Cada aba deve conter um conjunto de campos e um botão, como mostra a Figura a seguir. Utilize ou não gerenciadores de *layout*. Não utilize recursos de arrastar/soltar.

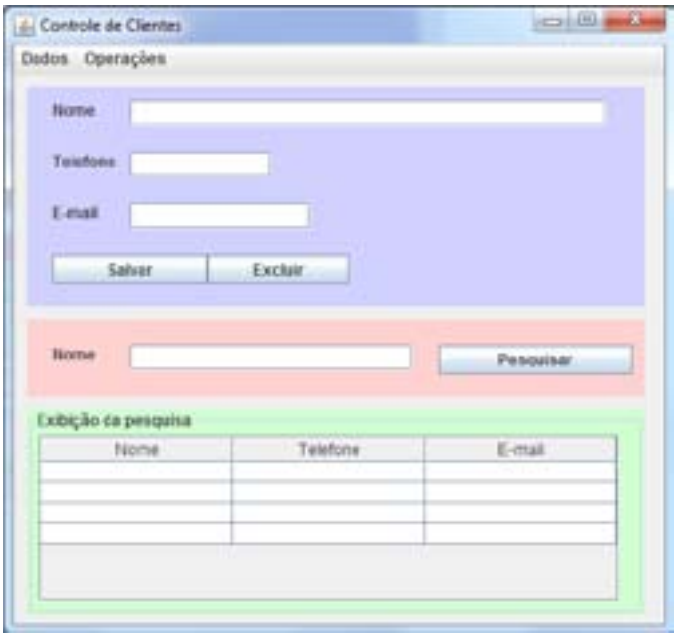


6) Crie a tela cujo *layout* é mostrado a seguir, utilizando o gerenciador de *layout GridBagLayout*. Programe o botão “Sair” de forma que ao ser clicado a aplicação seja finalizada. Não utilize recursos de arrastar/soltar.



UNINOVE – USO EXCLUSIVO PARA ALUNO

7) Crie a janela a seguir, considerando que o menu "Dados" contém a opção "Ver gráfico" e o menu "Operações" contém a opção "Sair". Programe a opção "Sair", de forma que ao ser clicada finalize a aplicação. Não utilize recursos de arrastar/soltar.



8) Crie a mesma janela apresentada na questão anterior utilizando recursos de arrastar/soltar do *NetBeans*.

6.7.2 Exercícios conceituais

1) Quais são as APIs do Java que permitem a criação de interface gráfica de usuário para aplicações *desktop*?

2) Quais são os principais pacotes da API *swing*?

3) Quais são as principais classes utilizadas para criar contêineres em interfaces gráficas de usuário para *desktop* com Java?

4) Quais são os métodos utilizados para determinar a posição e o tamanho de um elemento na janela *JFrame*?

5) Quais são os gerenciadores de *layout* utilizados em Java?

UNINOVE – USO EXCLUSIVO PARA ALUNO

6) Quando utilizar cada um dos gerenciadores de *layout* da API *awt*?

7) Qual classe da API *swing* é utilizada para gerar caixas de diálogo?

8) Que método deve ser utilizado para criar caixas de diálogo de confirmação em uma aplicação?

9) Quais classes da API *swing* são utilizadas para criar menus?

10) Que métodos são utilizados para mudar a cor do plano de fundo e da fonte de um elemento?

11) Quais classes da API *swing* são utilizadas para criar os principais tipos de botões que podem existir em um contêiner?

12) A que os botões *JRadioButton* precisam ser associados para que seja permitido a seleção de apenas um em um conjunto deste tipo de botão?

13) Que método deve ser utilizado para verificar se um botão foi selecionado?

14) Explique o procedimento necessário para definir uma máscara de entrada em um campo formatado.

15) Quais as principais classes utilizadas para criar listas e caixas de combinação (combos)?

16) Qual método deve ser utilizado para se obter a opção selecionada em uma caixa de combinação (combo) ou em uma lista?

17) Como se associa um modelo de tabela *DefaultTableModel* a uma tabela *JTable*?

18) Como se adiciona dados em uma linha de um modelo de tabela *DefaultTableModel*?

UNINOVE – USO EXCLUSIVO PARA ALUNO

19) Como se obtém um valor selecionado em uma célula de uma tabela *JTable*?

20) Quais são os principais tipos de eventos que podem ser tratados em aplicações *desktop* que utilizam as APIs *swing* e *awt*?

21) Quais classes são normalmente utilizadas para lidar com eventos de ação de clique do mouse sobre o elemento?

22) Quais são os principais eventos de mouse?

23) Quais as principais classes que podem ser utilizadas para tratar eventos de janela?

24) Que método(s) precisa(m) ser escritos em classes que implementam a interface *ActionListener*?

25) Para que serve o método *addActionListener* em classes que tratam eventos de ação?

26) Explique o bloco de código a seguir, considerando *btnSalvar* um objeto da classe *JButton*.

```
btnSalvar.addMouseListener(new MouseAdapter() {
  @Override
   public void mouseEntered(MouseEvent we) {
       btnSalvar.setBackground(Color.yellow);
   }    @Override
    public void mouseExited(MouseEvent evt) {
       btnSalvar.setBackground(Color.red);
  }
});
```

27) Qual classe deve ser utilizada para criar um painel que suporta abas (ou guias)? O que precisa ser adicionado a este painel para construir as abas?

28) Qual alternativa os desenvolvedores do *NetBeans* encontraram para agilizar a criação de interface gráfica de usuário?

UNINOVE – USO EXCLUSIVO PARA ALUNO

7. Acesso a banco de dados

Os programas de computador suportam a entrada, o processamento e as saídas de dados, mas estes dados, precisam quase sempre serem guardados em um banco de dados para manipulação posterior.

Você pode conectar a sua aplicação Java a qualquer Sistema Gerenciador de Banco de Dados (SGBD), como MySQL, PostgreSQL, Oracle, SQL Server etc. Para isso precisará de um *driver* produzido pelo fabricante do SGBD que permite tal conexão. O *driver* normalmente é um pacote de classes que você precisará referenciar na aplicação no momento de fazer a conexão com o banco de dados.

Neste Capítulo você aprenderá a realizar conexão com banco de dados e executar operações de cadastro (*insert*), alteração (*update*), consulta (*select*) e exclusão (*delete*). Uniremos tudo que foi visto neste livro até este momento em projetos mais robustos que envolvem criação de interface gráfica de usuário, encapsulamento, interfaces, pacotes e operações em banco de dados.

7.1 Os padrões ODBC e JDBC

Para acessar bancos de dados em aplicações Java existem dois padrões que determinam as regras para acesso de maneira eficiente: *Open Database Connectivity* (ODBC) e *Java Database Connectivity* (JDBC). Para utilizar estes padrões, os fabricantes de SGBDs[1] criam os *drivers* (que são bibliotecas de classes) para seguir as regras impostas por esses padrões. Assim, você precisará de um *driver JDBC* ou um *driver ODBC* do fabricante do SGBD para fazer a ligação da aplicação com o banco de dados.

ODBC é uma interface padrão *Microsoft Windows* que permite a ligação de aplicações feitas em qualquer linguagem de programação com qualquer banco de dados. Já JDBC é uma interface padrão, independente de plataforma, que permite a comunicação entre aplicações baseadas em Java e SGBDs.

JDBC é para aplicações feitas com a linguagem Java, independente de um sistema operacional específico. ODBC é para aplicações feitas em qualquer linguagem de programação, mas fica preso ao sistema operacional Windows.

1 SGBD é a sigla de Sistema Gerenciador de Banco de Dados. Trata-se do *software*, também conhecido como servidor de banco de dados, onde você poderá criar bancos de dados e tabelas e executar operações SQL.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Use ODBC quando precisar de maior desempenho, já que este padrão está mais integrado ao sistema operacional. Quando precisar de independência de plataforma, ou seja, quiser que sua aplicação execute em máquinas com qualquer sistema operacional, use JDBC.

Neste Capítulo utilizaremos o padrão JBDC, cujo *driver* já vem integrado à IDE *NetBeans*. Com este *driver*, poderemos conectar a aplicação a qualquer SGBD, modificando apenas algumas *strings* de conexão.

Existem outras formas para se acessar banco de dados utilizando *frameworks* e bibliotecas de persistência, mas estes recursos apenas abstraem e escondem do programador a complexidade do uso da API JDBC, mas não deixam de utilizá-la.

7.2 Baixando e instalando o MySQL

O MySQL foi criado por uma empresa chamada MySQL AB, comprada posteriormente pela *Sun Microsystems* (criadora do Java), que foi comprada pela *Oracle Corporation*. Logo, o MySQL, o *NetBeans* e o Java são de propriedade da *Oracle Corporation*. Sendo assim, o MySQL funciona muito bem integrado com Java, tanto que o *NetBeans* já traz o *driver* JDBC de conexão (*connector*) incorporado.

Para utilizar o MySQL com o *NetBeans* para desenvolver aplicações para *desktop* em Java que acessam banco de dados, você precisará instalar o MySQL no computador. Para isso, deverá baixar o *MySQL Community Server* e o *MySQL Workbench*. O *MySQL Community Server* é o servidor de banco de dados e o *MySQL Workbench* é uma ferramenta de desenvolvimento que provê um ambiente visual (gráfico) para projetar e modelar bancos de dados, manipular bancos de dados e tabelas por meio de comando SQL e assistentes, administrar bancos de dados (permissões, *backup*, *restore* etc.) e fazer a migração de bancos de dados.

Para baixar estes recursos, siga as orientações a seguir:

– Entre no site <http://dev.mysql.com/downloads/>.

– Clique no *link "MySQL Community Server"*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Aparecerá na página a área mostrada na Figura 7.1.



Figura 7.1: Tela para a seleção de versões do MySQL para baixar.

– Clique no botão "*Download"*.

Aparecerá uma página com a área mostrada na Figura 7.2.



Figura 7.2: seleção da versão do MySQL para Windows.

Note que há duas opções de *download* para a plataforma Windows. A primeira, de 1.4M, vai baixando e já instalando o MySQL em seu computador. A segunda, de 270.0M, permite baixar e salvar os recursos MySQL no computador para fazer a instalação posterior. Sugiro clicar nesta opção, pois caso você tenha algum problema na instalação ou tenha que reinstalar posteriormente, não precisará baixar os recursos novamente.

– Clique no botão "*Download*".

Na página que aparecerá, clique na parte inferior da tela no *link* "*No thanks, just start my download*". Está opção evita que você tenha que criar um usuário no site para acessar a área de *download*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Aparecerá, na parte inferior da tela, a janela mostrada na Figura 7.3.



Figura 7.3: Opções para salvar ou executar o instalador do MySQL.

– Clique no botão "Executar", caso queira que após baixar já seja iniciada a instalação, ou no botão "Salvar", para salvar na pasta configurada para receber os *downloads* em seu computador. Caso queira escolher a pasta, clique na setinha preta à direita do botão "Salvar".
– Após o *download* ser concluído, abra a pasta onde gravou o arquivo *mysql-installer-community-5.6.21.0* e clique sobre ele.

Durante a instalação, basta clicar no botão "*Next*", e em algumas telas, "*Execute*", até chegar à tela mostrada na Figura 7.4, onde será solicitado a senha para o superusuário "*root*", que já é instalado no MySQL.

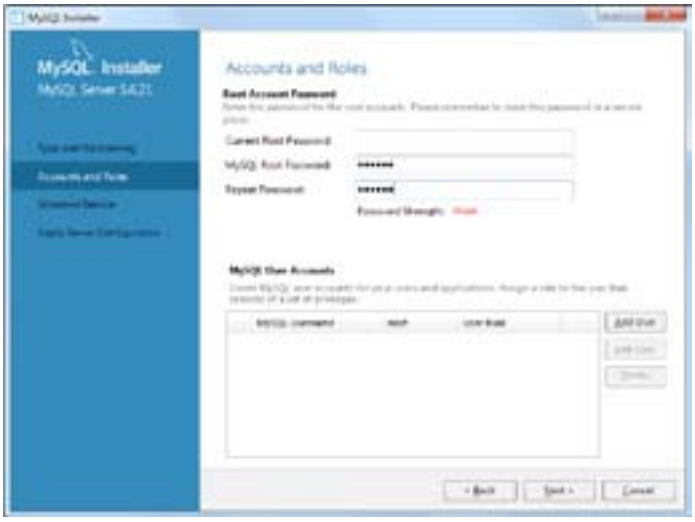


Figura 7.4: Definindo a senha do usuário root durante a instalação do MySQL.

– Após digitar e repetir a senha que você deseja atribuir ao servidor MySQL, clique no botão "*Next*" e prossiga a instalação até terminar.

7.2.1 Criando o banco de dados e a tabela com o MySQL Workbench

Após concluir a instalação, clique no botão "Iniciar" do Windows, em "Todos os programas", na pasta "MySQL" e na opção "*MySQL Workbench 6.1 CE*". Aparecerá a tela mostrada na Figura 7.5.

UNINOVE – USO EXCLUSIVO PARA ALUNO

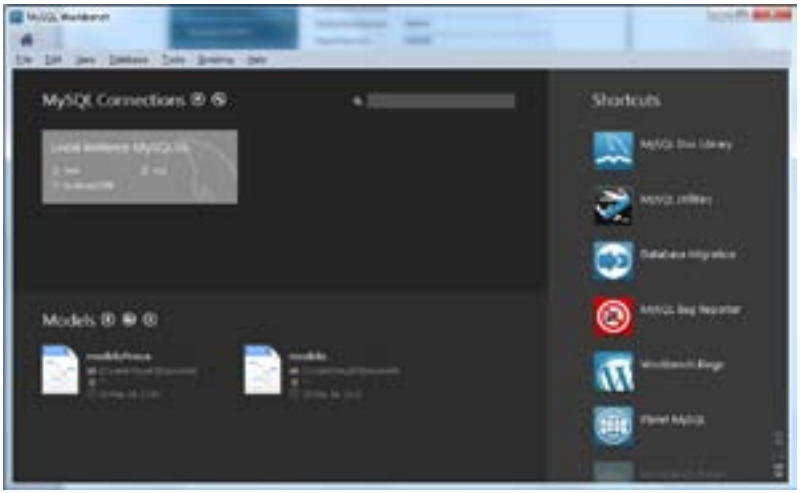


Figura 7.5: Tela inicial do MySQl Workbench.

– Para acessar a área de trabalho do *MySQL Workbench,* clique na opção "*Local Instance MySQL56*", que aparecerá a tela mostrada na Figura 7.6.

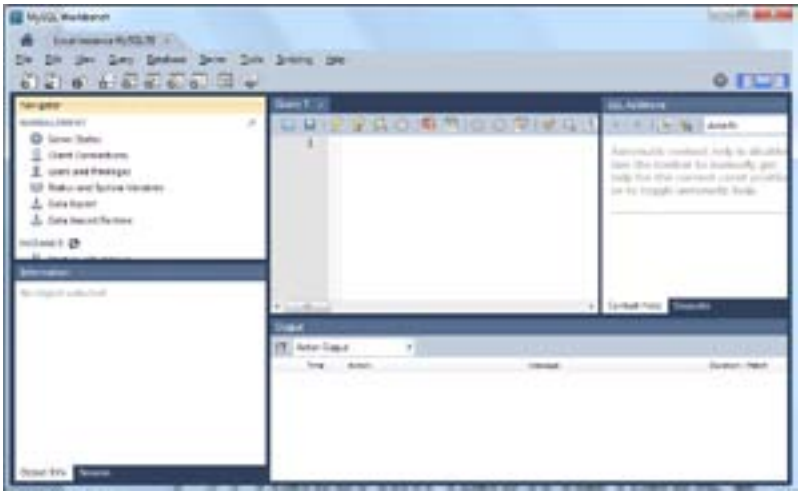


Figura 7.6: Área de trabalho do MySQL Workbench.

Na tela mostrada na Figura 7.6, já podemos criar nosso banco de dados, as tabelas que necessitamos e executar outros comandos SQL, caso seja necessário.

– Para criar um novo banco de dados, clique na ferramenta "*create a new schema in the connected server*", na parte superior da tela. Trata-se de uma ferramenta com desenho de um barril com um sinal de mais no canto inferior esquerdo ( ).

UNINOVE – USO EXCLUSIVO PARA ALUNO

Aparece a tela mostrada na Figura 7.7.

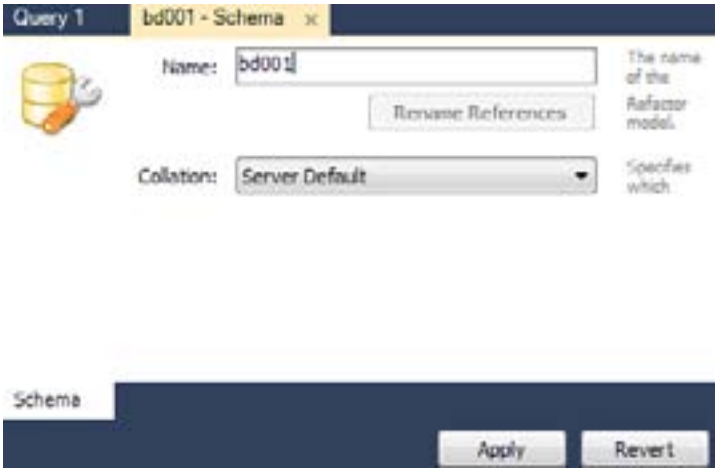


Figura 7.7: Criação do banco de dados – nomeação.

– No campo "*Name:*", digite o nome do banco de dados, neste caso, "bd001".
– Clique no botão "Apply". Aparece a instrução CREATE SCHEMA 'bd001';
– Clique no botão "Apply" novamente. Em seguida, clique no botão "Finish".

Pronto! O Banco de dados "bd001" foi criado.

Do lado esquerdo da tela aparecem o nome dos bancos de dados existentes no computador, como mostra a Figura 7.8.

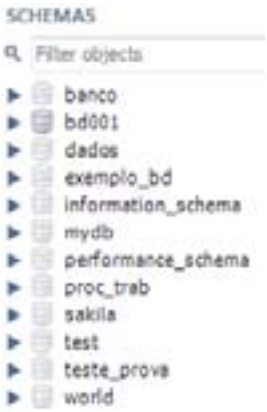


Figura 7.8: Bancos de dados existentes.

Note que nosso banco de dados "bd001" já está na lista de bancos de dados existentes.

UNINOVE – USO EXCLUSIVO PARA ALUNO

– Clique na seta à esquerda do nome do banco de dados e aparecerá o que mostra a Figura 7.9.



Figura 7.9: Opções do banco de dados.

– Clique com o botão direito sobre "*Tables*" e selecione a opção "*Create Table...*", como mostra a Figura 7.10.



Figura 7.10: Criação da tabela.

– Na tela que aparece, no campo "*Table Name:*" digite o nome da tabela, neste caso, "cliente".
– Em seguida, digite os campos que a tabela terá, como mostra a Figura 7.11.



Figura 7.11: Definição dos campos da tabela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

– Clique no botão “Apply”.

Aparecerá uma tela com o código SQL a seguir, que gerará a tabela:

```
CREATE TABLE 'bd001'.'cliente' (
  'id' INT NOT NULL,
  'nome' VARCHAR(50) NULL,
  'telefone' VARCHAR(15) NULL,
  'sexo' VARCHAR(15) NULL,
  'renda' DOUBLE NULL,
  PRIMARY KEY ('id'));
```

– Clique no botão “Apply” novamente e, em seguida, no botão “Finish”.

Pronto! Nossa tabela “cliente” já está criada e aparecerá à esquerda da tela, como mostra a Figura 7.12.



Figura 7.12: Tabela criada.

Não é necessário incluir, alterar, consultar ou excluir dados no banco de dados por meio do *MySQL Workbench*. Só utilizamos essa ferramenta para criar o banco de dados e a tabela. As operações no banco de dados serão executadas na aplicação Java que será criada.

7.2.2 Criando o banco de dados e a tabela com o *NetBeans*

É possível criar o banco de dados e a tabela diretamente no *NetBeans*, dispensando o *MySQL Workbench.* Para isso, siga as orientações abaixo.

Para se conectar ao *MySQL server* e criar o banco de dados e a tabela usando o *NetBeans*, inicie o servidor MySQL clicando em “Iniciar”, “Todos os programas”, “MySQL”, “*MySQL Server 5.6*” e “*MySQl 5.6 Command Line Cliente*”. Caso prefira, pode abrir o *Workbench* apenas para iniciar o *MySQL Server*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Na área de trabalho do *NetBeans*, clique na guia "Serviços", mostrada na Figura 7.13.

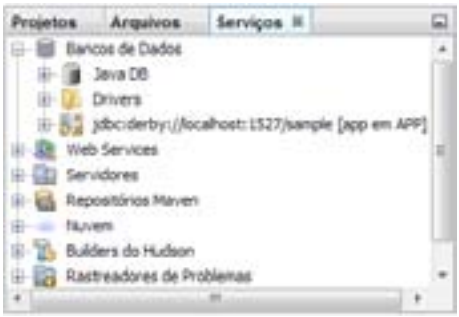


Figura 7.13: Guia serviços do NetBeans.

– Clique com o botão direito do mouse sobre a opção "Banco de Dados". Selecione a opção "Registrar Servidor MySQL".

Digite os dados do servidor, como mostra a Figura 7.14, inclusive a senha que você definiu na instalação do MySQL.

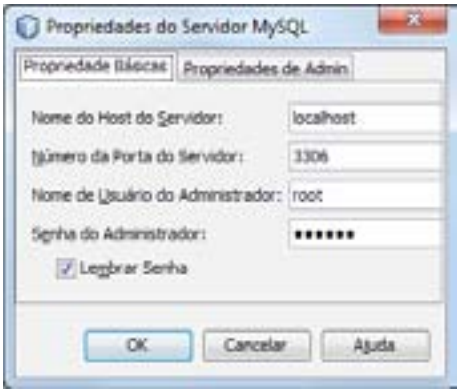


Figura 7.14: Informando os dados do servidor MySQL.

– Clique no botão "OK".
– Na janela ativa aparecerá a opção "*Servidor MySQL em localhost:3306[root]*". Clique com o botão direito do mouse sobre esta opção e selecione "Conectar". Clique no sinal de mais à esquerda desta opção e verá o nome dos bancos de dados existentes no computador.
– Clique com o botão direito do mouse sobre esta opção e selecione a opção "Criar banco de Dados", como mostra a Figura 7.15.

UNINOVE – USO EXCLUSIVO PARA ALUNO

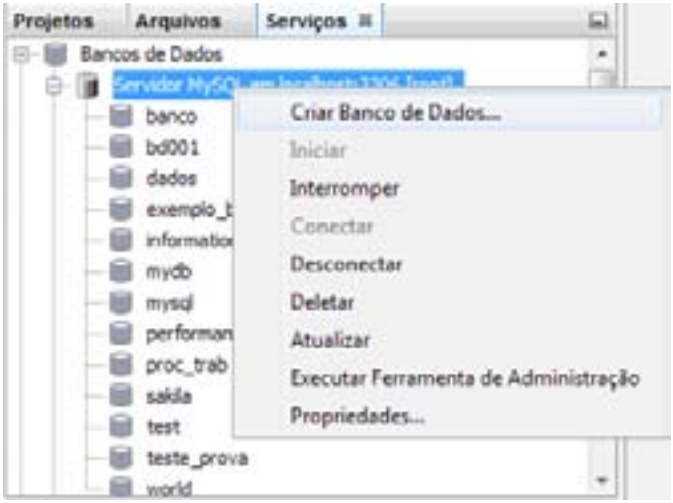


Figura 7.15: Criação do banco de dados.

– Aparecerá uma janela onde você deve digitar o nome do banco de dados, como mostra a Figura 7.16.



Figura 7.16: Nomeando o banco de dados.

– Clique no botão “OK”.

Aparecerá na janela a conexão com o banco de dados criado, com a seguinte descrição:

jdbc:mysql://localhost:3306/bd002?zeroDateTimeBehavior=convertToNull

– Clique no sinal de mais à esquerda desta opção e aparecerá os bancos de dados existentes, como mostra a Figura 7.17.

UNINOVE – USO EXCLUSIVO PARA ALUNO

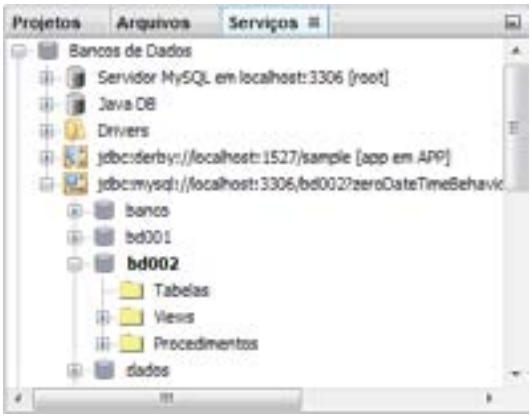


Figura 7.17: Mostrando os bancos de dados exeistentes.

– Clique com o botão direito do mouse sobre a opção “Tabelas” e selecione a opção “Criar Tabela”.
– No campo “Nome da tabela”, digite “cliente”, conforme mostra a Figura 7.18.



Figura 7.18: Criando a tabela.

– Clique no botão “Adicionar Coluna” para criar os campos da tabela.

Aparecerá a Figura 7.19.



Figura 7.19: Criação dos campos da tabela.

UNINOVE – USO EXCLUSIVO PARA ALUNO

– No campo “Nome:”, digite *id*, no campo “Tipo:”, selecione *int*, em “Restrições”, selecione “Chave primária” e clique no botão “OK”.

Repita esse procedimento até criar todos os campos mostrados na Figura 7.20.

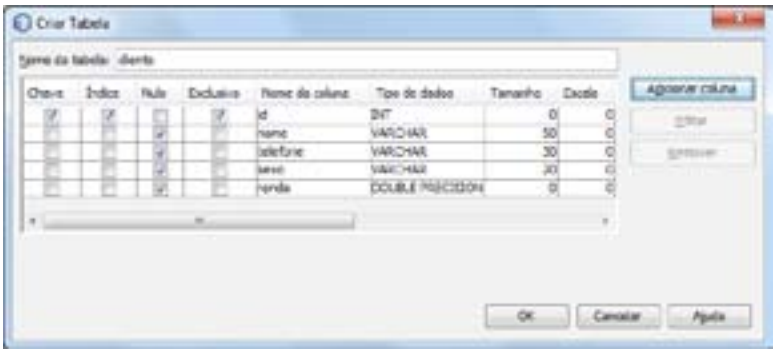


Figura 7.20: Campos da tabela.

– Após terminar, clique no botão “OK”.

Pronto! A tabela está criada, como mostra a Figura 7.21.

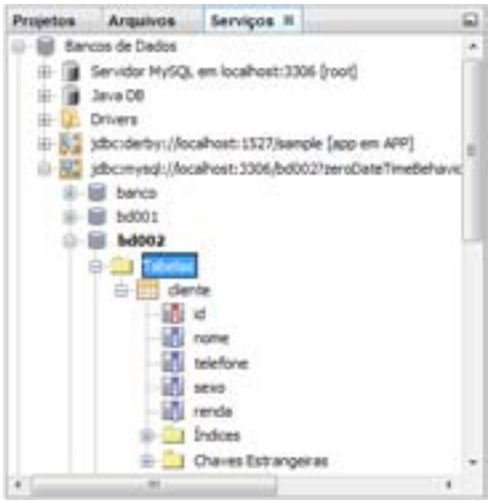


Figura 7.21: Exibindo a tabela criada.

– Clique na guia “Projetos”, à esquerda da guia “Serviços”, para iniciar a aplicação Java que será conectada ao banco de dados criado.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.3 OPERAÇÕES EM BANCO DE DADOS

Agora que você já sabe como criar um banco de dados e uma tabela no MySQL, já podemos começar a executar operações nesta tabela do banco de dados. Para todos os exemplos deste Capítulo, utilizaremos o banco de dados "bd002" e a tabela "cliente" criados no tópico anterior.

Para executar operações em um banco de dados, precisamos nos conectar a ele, executar a operação e desconectar. Muitas vezes mantemos a conexão aberta e a gerenciamos através de um *pool* de conexões, o que diminui consideravelmente a carga de operações no banco de dados.

O *pool* de conexões é um repositório que mantém uma lista de conexões abertas e reutilizáveis. Basicamente é criado por uma classe que gerencia as operações de abertura e fechamento da conexão. Nos exemplos apresentados a seguir não criaremos *pool* de conexão, pois o nível de complexidade é muito alto para o aprendizado inicial pretendido neste Capítulo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.3.1 CLASSES E INTERFACES USADAS EM OPERAÇÕES COM BANCO DE DADOS

Para executar operações em um banco de dados você precisará, além do banco de dados e da tabela:

– De um *driver* de conexão criado pelo fabricante do SGBD que vamos utilizar. O *NetBeans* já traz incorporado *drivers* para diversos SGBDs, como *MySQL*, *Oracle*, *PostgreSQL* etc.

– Do pacote de classes *java.sql*, que traz as interfaces *Connection*, *Statement*, *PreparedStatement* e *ResultSet*, e as classes *DriverManager* e *SQLException*. Todas estas classes e interfaces cumprem um papel específico e importante para realizar operações com um banco de dados. Este papel é apresentado a seguir:

  • ***Connection*** – Estabelece e mantém uma conexão ativa com o banco de dados.

  • ***Statement*** ou ***PreparedStatement*** – Permite executar comandos SQL no banco de dados, como *insert*, *select*, *delete* e *update*. Por meio de métodos assinados nesta interface é possível utilizar estes comandos. O método *executeQuery*, permite executar o comando *select* e o método *executeUpdate* permite executar os comandos *insert*, *update* e *delete*.

  • ***ResultSet*** – Recebe o retorno das consultas realizadas na tabela por meio do comando *select*. Os dados que o comando *select* retornam do banco de dados são recebidos por um objeto da interface *ResultSet*.

● ***SQLException*** – Quando uma exceção relacionada a uma operação no banco de dados ocorre, um objeto desta classe é recebido na cláusula *catch* de um tratamento de exceção iniciado pela instrução *try*. Por meio deste objeto é possível obter informações do erro, como o número (método *getErrorCode*), a mensagem de erro (método *getErrorMessage*) etc.

● ***DriverManager*** – Passa os parâmetros para realizar a conexão com o banco de dados, como o caminho, a porta, o nome do banco de dados, o usuário e a senha, e gerencia o uso do *driver* de conexão fornecido pelo fabricante do SGBD.

Nos tópicos a seguir você aprenderá como utilizar essas classes e interfaces para realizar operações no banco de dados.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.3.2 Operação de cadastro

Neste tópico você verá um pequeno exemplo que faz uma conexão com o banco de dados, executa uma inclusão de registro (cadastro) na tabela e fecha a conexão. O exemplo será apresentado duas vezes, uma utilizando a interface *PreparedStatement* para executar o comando SQL *insert* e outra utilizando a interface *Statement*.

Crie um novo projeto Java chamado *TesteBanco* e uma classe principal chamada *Banco.java*.

Para executar operações no banco de dados, temos que adicionar o *driver* JDBC à biblioteca do projeto. Para isso, clique com o botão direito do mouse sobre a pasta "Bibliotecas" e selecione a opção "Adicionar Biblioteca".

Aparecerá a tela mostrada na Figura 7.22.

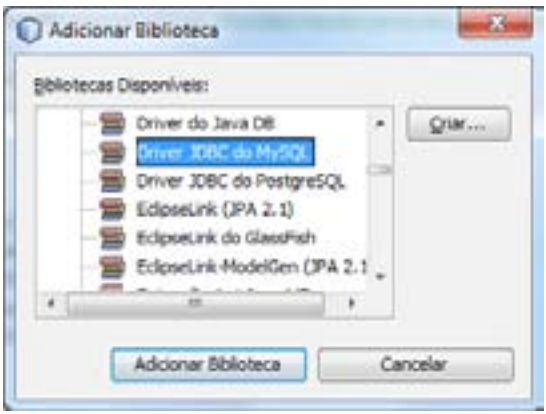


Figura 7.22: Janela para a seleção do driver JDBC.

Selecione, na lista de bibliotecas disponíveis, a opção "*Driver JDBC do MySQL*" e clique na opção "Adicionar Biblioteca".

Ao realizar este procedimento o projeto deverá estar como mostra a Figura 7.23.



Figura 7.23: Projeto para inclusão de dados no banco de dados.

Veja que o *driver* JDBC do MySQL já está na pasta "Bibliotecas" do projeto.

Clique no sinal de mais (+) à esquerda do nome do *driver* ("*Driver JDBC do MySQL – mysql-connector-java-5.1.23-bin.jar*") que você verá o que mostra a Figura 7.24.

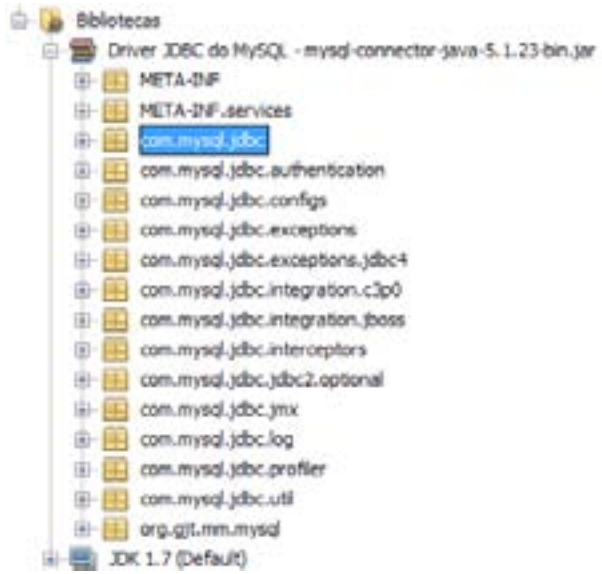


Figura 7.24: Pacotes do driver JDBC do MySQL.

Note que nesta biblioteca de *driver* há um pacote de classes chamado "*com.mysql.jdbc*". Clique no sinal de mais à esquerda do nome deste pacote e você verá a relação de classes que ele contém, como mostra a Figura 7.25.

UNINOVE – USO EXCLUSIVO PARA ALUNO

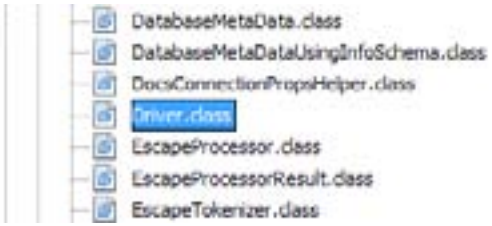


Figura 7.25: Classe do driver na biblioteca do projeto.

Nesta lista de classes, localize a classe "*Driver.class*". Esta classe é especial, pois será utilizada em todas as classes que você criar para realizar a conexão com o banco de dados.

Na classe *Banco.java*, digite o código-fonte a seguir:

```
1   import java.sql.Connection;
2   import java.sql.DriverManager;
3   import java.sql.PreparedStatement;
4   import java.sql.SQLException;
5   import javax.swing.JOptionPane;
6
7   public class Banco {
8
9       public static void main(String[] args) {
10          try {
11              Class.forName("com.mysql.jdbc.Driver");
12              Connection conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
13              PreparedStatement st = conn.prepareStatement("insert into cliente
                (id, nome, telefone, sexo, renda) values (?, ?, ?, ?, ?)");
14              st.setInt(1, 10);
15              st.setString(2, "Marco Antonio");
16              st.setString(3, "(11)5678-9087");
17              st.setString(4, "Masculino");
18              st.setDouble(5, 14567.56);
19              st.executeUpdate();
20              JOptionPane.showMessageDialog(null, "Os dados do cliente foram
                salvos");
21              conn.close();
22          } catch (ClassNotFoundException | SQLException ex) {
23              JOptionPane.showMessageDialog(null, "Erro ao conectar com o
                Banco de Dados");
24          }
25      }
26  }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Veja que as operações no banco de dados estão concentradas dentro de uma instrução *try* (da linha 10 à 21). Isso significa que se algumas destas operações falharem será feito um direcionamento para a linha da instrução *catch* (linha 22). Neste bloco de linhas, poderão ocorrer apenas dois tipos de exceções: *ClassNotFoundException* e *SQLException*.

Uma exceção *ClassNotFoundException* poderá ser gerada na linha 11, se a classe *Driver.class* não for encontrada no pacote *com.mysql.jdbc* da biblioteca do *driver* do projeto. Veja que adicionamos essa biblioteca ao projeto antes de criar esta classe.

Uma exceção *SQLException* poderá ser gerada na linha 19, quando a instrução *insert* definida na linha 13 for executada pelo método *executeUpdate*. Caso esta instrução esteja incorreta, gerará a exceção.

A linha 11 informa a classe do *driver* que será utilizada para fazer a conexão entre a aplicação e o MySQL.

A linha 12 faz a conexão com o banco de dados. Veja que o método estático *getConnection* da classe *DriverManager* é utilizado para passar os parâmetros de conexão com o banco de dados. São passados três parâmetros, apresentados a seguir:

- **jdbc:mysql://localhost:3306/bd002** – indica o IP da máquina em que o MySQL server está instalado (*localhost* indica a máquina local, mas este valor pode ser substituído pelo IP da máquina), a porta onde o MySQL recebe requisições (*3306*) e o nome do banco de dados (*bd002*).
- **root** – indica o nome do usuário que acessará o banco de dados. Deve ser um usuário registrado no MySQL. O valor *root* é de um usuário padrão já instado e configurado automaticamente na instalação do MySQL.
- **teruel** – indica a senha do MySQL. Neste caso, *teruel* é a minha senha. Não deixe de colocar a senha que você definiu na instalação do MySQL em seu computador.

O método *getConnection*, após passar os parâmetros de conexão com o banco de dados, retorna um objeto da interface *Connection*, contendo a conexão estabelecida. Este objeto é recebido e chamado de *conn* (linha 12).

A linha 13 passa para o método *prepareStatement* da interface *Connection* uma instrução *insert* incompleta, pois precisa receber parâmetros nos locais onde estão os pontos de interrogação. Estes parâmetros são fornecidos nas linhas de 14 à 18. Veja na linha 14 que o valor 10 é passado como parâmetro e substituirá o primeiro ponto de interrogação. O nome "Marco Antonio" é passado como parâmetro e substituirá o segundo ponto de interrogação, e assim por diante.

UNINOVE – USO EXCLUSIVO PARA ALUNO

A linha 19 executa a instrução *insert* definida na linha 13, após ela receber todos os parâmetros e se tornar completa. Veja que o método *executeUpdate* da interface *Statement* é responsável por executar o comando *insert* da linguagem SQL. A linha 20 mostra uma mensagem indicando que a inclusão foi realizada e a linha 21 encerra (fecha) a conexão com o banco de dados.

Se ocorrerem as exceções *ClassNotFoundException* ou *SQLException* será mostrada a mensagem contida na linha 23.

Ao compilar e executar o projeto acima por meio do pressionamento da tecla *F6*, será mostrada a mensagem apresentada na Figura 7.26.

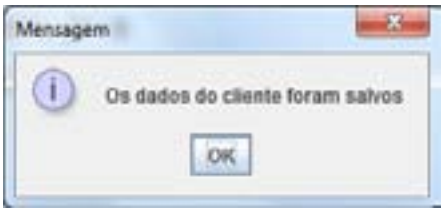


Figura 7.26: Confirmação de cadastro.

NOTA

Na classe apresentada, você poderia solicitar os valores a serem inseridos ao usuário e passar estes valores como parâmetros para a instrução *insert* definida no método *prepareStatement*. A interface *PreparedStatement* permite executar uma operação SQL parametrizada, ou seja, que recebe parâmetros.

Além da interface *PreparedStatement*, a interface *Statement* também pode ser utilizada para executar comandos SQL. A grande diferença é que a instrução SQL executada por métodos da interface *PreparedStatement* pode receber parâmetros e a instrução SQL executada pela interface *Statement*, não.

Veja o mesmo exemplo apresentado anteriormente utilizando a interface *Statement*.

```
1    import java.sql.*;
2    import javax.swing.JOptionPane;
3
4    public class Banco {
5
6        public static void main(String[] args) {
7            try {
8                Class.forName("com.mysql.jdbc.Driver");
9                 Connection conn = DriverManager.getConnection
                 ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
10              Statement st = conn.createStatement();
11              int id = 12;
12              String nome = "Marco Antonio";
13              String telefone = "(11)5678-9087";
14              String sexo = "Masculino";
15               double renda = 14567.56;
16              String sql = "insert into cliente (id, nome, telefone, sexo, renda)
                values (" + id + ", '" + nome + "', '" + telefone + "', '" + sexo + "', " +
                renda + ")";
17               st.executeUpdate(sql);
18              JOptionPane.showMessageDialog(null, "Os dados do cliente foram
                salvos");
19              conn.close();
20          } catch (ClassNotFoundException | SQLException ex) {
21              JOptionPane.showMessageDialog(null, "Erro ao conectar com o
                Banco de Dados");
22          }
23      }
24  }
```

Note no bloco de linhas de 10 à 17 a execução da instrução *insert* utilizando a interface *Statement*. Veja que a instrução *insert* é armazenada na variável SQL (linha 16), concatenando-se as partes da instrução com os valores contidos nas variáveis onde estão os dados a serem inseridos. Perceba como esta linha é complexa. O sinal de + é utilizado para concatenar valores na *String*.

Perceba na linha 17 que a instrução *insert* contida na variável *sql* é executada. Observe com atenção as diferenças entre o uso da interface *Statement* (neste exemplo), e da interface *PreparedStatement* (no exemplo anterior). O uso da interface *PreparedStatement* é mais comum na comunidade de desenvolvedores.

NOTA

Se você deseja se conectar a outro SGBD que não seja o MySQL, deverá apenas trocar o *driver* adicionado à biblioteca para o *driver* fornecido pelo fabricante do SGBD e mudar as seguintes linhas de conexão:

Class.forName("com.mysql.jdbc.Driver");
Connection conn = DriverManager.getConnection ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");

Bastará mudar os parâmetros passados para o método *forName* da classe *Class* e *getConnection* da classe *DriverManager*. Você descobrirá quais são os parâmetros na documentação do *driver* JDBC ou no site do fabricante do SGBD.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.3.3 Operação de consulta

Para realizar uma operação de consulta (busca) em uma tabela do banco de dados é necessário se conectar ao banco de dados, executar a consulta e fechar a conexão. Podem ser utilizados métodos das interfaces *PreparedStatement* ou *Statement* para executar a consulta.

Veja o exemplo da classe *Banco.java* que busca um registro na tabela cujo campo *id* possua o valor 10:

```
1   import java.sql.*;
2
3   public class Banco {
4
5       public static void main(String[] args) {
6           try {
7               Class.forName("com.mysql.jdbc.Driver");
8               Connection conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
9               PreparedStatement st = conn.prepareStatement("select * from
                cliente where id = ? ");
10              st.setInt(1, 10);
11              ResultSet rs = st.executeQuery();
12              if (rs.next()) {
13                  System.out.println("ID: " + rs.getString("id"));
14                  System.out.println("Nome: " + rs.getString("nome"));
15                  System.out.println("Telefone: " + rs.getString("telefone"));
16                  System.out.println("Sexo: " + rs.getString("sexo"));
17                  System.out.println("Renda: " + rs.getString("renda"));
18              } else {
19                  System.out.println("Cliente não encontrado");
20              }
21              conn.close();
22          } catch (ClassNotFoundException | SQLException ex) {
23              System.out.println("Erro ao tentar se conectar ao banco de
                dados");
24          }
25      }
26  }
```

Veja na linha 9 a geração da instrução SQL *select* incompleta, necessitando como parâmetro do *id* para substituir o ponto de interrogação. Este parâmetro, é passado na linha 10.

UNINOVE – USO EXCLUSIVO PARA ALUNO

A linha 11 executa a instrução SQL *select* por meio do método *executeQuery* da interface *PreparedStatement*. O resultado da pesquisa na tabela do banco de dados é armazenado no objeto *rs* da interface *ResultSet*. A linha 12 verifica se este objeto contém dados. Se sim, é porque a consulta à tabela retornou algum registro. Neste caso, as linhas de 13 à 17 exibem os dados retornados.

Veja que para obter o conteúdo de cada campo da tabela, utilizou-se o método *getString* da interface *ResultSet*. A instrução *rs.getString("id")*, por exemplo, retorna o *id* contido no objeto *rs* da interface *ResultSet*.

Caso a consulta não tenha retornado nenhum registro, o que aconteceria se o *id* informado para a pesquisa não existisse no banco de dados, a mensagem da linha 19 seria mostrada.

Ao compilar e executar o projeto, por meio do pressionamento da tecla *F6*, os dados serão apresentados na tela, como mostra a Figura 7.27.

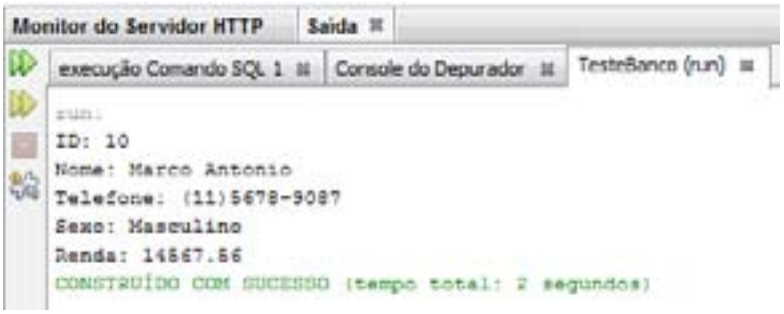


Figura 7.27: Exibição dos dados da consulta.

Veja agora o mesmo exemplo utilizando a interface *Statement*:

```
1   import java.sql.*;
2
3   public class Banco {
4
5       public static void main(String[] args) {
6           try {
7               Class.forName("com.mysql.jdbc.Driver");
8               Connection conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
9               Statement st = conn.createStatement();
10              int id = 12;
11              String sql = "select * from cliente where id = " + id;
12              ResultSet rs = st.executeQuery(sql);
13              if (rs.next()) {
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
14                  System.out.println("ID: " + rs.getString("id"));
15                  System.out.println("Nome: " + rs.getString("nome"));
16                  System.out.println("Telefone: " + rs.getString("telefone"));
17                  System.out.println("Sexo: " + rs.getString("sexo"));
18                  System.out.println("Renda: " + rs.getString("renda"));
19              } else {
20                  System.out.println("Cliente não encontrado");
21              }
22              conn.close();
23          } catch (ClassNotFoundException | SQLException ex) {
24              System.out.println("Erro ao tentar se conectar ao banco de dados");
25          }
26      }
27  }
```

Observe que as linhas responsáveis pela consulta estão no intervalo de linhas de 9 à 13. Note na linha 11 que a instrução *select* é armazenada na variável *String sql*, concatenada como o valor contido na variável *id*. A linha 12 executa a instrução *select* contida na variável *sql* e armazena o registro retornado da tabela em um objeto da interface *ResultSet*. Se a consulta retornou algum valor (linha 13), os dados são exibidos na tela (linhas de 14 à 18).

Note que as diferenças não são muito grandes em relação ao uso da interface *PreparedStatement*.

7.3.4 Operação de consulta de todos os registros

O tópico anterior apresentou uma consulta que retornava apenas um registro. Este tópico apresentará uma consulta que retornará vários registros. O que muda, de fato, é a instrução SQL *select* executada e a forma de exibir os dados, pois se são retornados mais de um registro, será necessário um laço de repetição para percorrê-los e exibi-los.

Veja o exemplo da classe *Banco.java* utilizando a interface *PreparedStatement*:

```
1   import java.sql.*;
2
3   public class Banco {
4
5       public static void main(String[] args) {
6           try {
7               Class.forName("com.mysql.jdbc.Driver");
8               Connection conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
9               PreparedStatement st = conn.prepareStatement("select * from
                cliente");
10               ResultSet rs = st.executeQuery();
11               while (rs.next()) {
12                  System.out.println("ID: " + rs.getString("id"));
13                  System.out.println("Nome: " + rs.getString("nome"));
14                  System.out.println("Telefone: " + rs.getString("telefone"));
15                  System.out.println("Sexo: " + rs.getString("sexo"));
16                  System.out.println("Renda: " + rs.getString("renda"));
17                  System.out.println("---------------------------------------");
18               }
19               conn.close();
20            } catch (ClassNotFoundException | SQLException ex) {
21               System.out.println("Erro ao tentar se conectar ao banco de
                 dados");
22            }
23         }
24      }
```

Note que a instrução SQL *select* é preparada na linha 9 e executada na linha 10. Neste caso, não foi necessário nenhum parâmetro na instrução *select*.

Caso a consulta tenha retornado um ou mais registros, a linha 11 define um laço de repetição que será executado enquanto houver próximo registro na relação de registros retornada na consulta.

Enquanto tiver registro a ser exibido, o laço de repetição das linhas 11 à 18 será executado, exibindo na tela os dados de cada pessoa. Veja que o método *getString* é responsável por obter os dados do objeto *rs* da interface *ResultSet,* como *String*. Você poderia utilizar os métodos *getDouble*, *getInt* etc para obter os dados de outros tipos.

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6*, os dados cadastrados na tebela do banco de dados serão apresentados na tela, como mostra a Figura 7.28.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 7.28: Exibição do resultado da consulta que retorna vários registros.

Veja agora o exemplo da classe *Banco.java* utilizando a interface *Statement*:

```
1   import java.sql.*;
2
3   public class Banco {
4
5       public static void main(String[] args) {
6           try {
7               Class.forName("com.mysql.jdbc.Driver");
8               Connection conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
9               Statement st = conn.createStatement();
10              String sql = "select * from cliente";
11              ResultSet rs = st.executeQuery(sql);
12              while (rs.next()) {
13                  System.out.println("ID: " + rs.getString("id"));
14                  System.out.println("Nome: " + rs.getString("nome"));
15                  System.out.println("Telefone: " + rs.getString("telefone"));
16                  System.out.println("Sexo: " + rs.getString("sexo"));
17                  System.out.println("Renda: " + rs.getString("renda"));
18                  System.out.println("--------------------------------------");
19              }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
20              conn.close();
21          } catch (ClassNotFoundException | SQLException ex) {
22              System.out.println("Erro ao tentar se conectar ao banco de dados");
23          }
24      }
25  }
```

Observe que as linhas responsáveis pela consulta estão no intervalo de 9 à 11. Note na linha 10 que uma instrução *select* é armazenada em uma variável *String* que é passada como parâmetro para o método *executeQuery* da linha 11 para ser executada. O resultado da busca será armazenado no objeto *rs* da interface *ResultSet*. Se foram retornados registros na consulta, eles serão mostrados pelas linhas de 12 à 19.

7.3.5 OPERAÇÃO DE EXCLUSÃO

Para executar uma operação de exclusão de registro na tabela você precisa se conectar ao banco de dados, executar a instrução *SQL delete* e fechar a conexão com o banco de dados. Você pode executar esta operação utilizando as interfaces *PreparedStatement* ou *Statement*.

Veja o exemplo da classe *Banco.java* utilizando a interface *PreparedStatement*:

```
1       import java.sql.*;
2
3       public class Banco {
4
5           public static void main(String[] args) {
6               try {
7                   Class.forName("com.mysql.jdbc.Driver");
8                    Connection conn = DriverManager.getConnection
                    ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
9                   PreparedStatement st = conn.prepareStatement("delete from
                    cliente where id = ? ");
10                   st.setInt(1, 10);
11                   int r = st.executeUpdate();
12                   if (r == 1) {
13                      System.out.println("Cliente excluído com sucesso");
14                  } else {
15                      System.out.println("Cliente não encontrado");
16                  }
17                  conn.close();
18              } catch (ClassNotFoundException | SQLException ex) {
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
19                System.out.println("Erro ao tentar se conectar ao banco de
                  dados");
20             }
21          }
22       }
```

Veja que a instrução SQL *delete* é preparada na linha 9, aguardando o *id* como parâmetro, que será inserido na posição onde está o ponto de interrogação. Este parâmetro é fornecido na linha 10.

A linha 11 executa o comando *delete* (preparado na linha 9) por meio do método *executeUpdate*. Este método retorna o número de linhas da tabela que foram afetadas pela execução do comando *delete*, neste caso, 1, pois apenas um registro foi excluído; ou 0, caso nenhum registro tenha sido excluído. Se o valor retornado for 1 (linha 12), é porque a exclusão ocorreu. Neste caso, será apresentada a mensagem definida na linha 13. Se o retorno foi 0, o que indica que a operação de exclusão não ocorreu, normalmente porque o *id* passado como parâmetro não existe cadastrado na tabela, será mostrada a mensagem definida na linha 15.

Veja agora o exemplo da classe *Banco.java* utilizando a interface *Statement*:

```
1     import java.sql.*;
2
3     public class Banco {
4
5        public static void main(String[] args) {
6           try {
7              Class.forName("com.mysql.jdbc.Driver");
8               Connection conn = DriverManager.getConnection
               ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
9               Statement st = conn.createStatement();
10              int id = 12;
11             String sql = "delete from cliente where id = " + id;
12              int r = st.executeUpdate(sql);
13              if (r == 1) {
14                 System.out.println("Cliente excluído com sucesso");
15              } else {
16                 System.out.println("Cliente não encontrado");
17              }
18              conn.close();
19          } catch (ClassNotFoundException | SQLException ex) {
20             System.out.println("Erro ao tentar se conectar ao banco de dados");
21          }
22       }
23    }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Observe que as linhas responsáveis pela exclusão do registro cujo *id* é igual a 12 estão no intervalo de 9 à 12. Perceba que a instrução *delete* é armazenada em uma variável *String* (linha 11) que é passada como parâmetro para o método *executeUpdate* na linha 12, para ser executada. Se a operação ocorreu, exibe a mensagem presente na linha 14. Se não ocorreu, exibe a mensagem presente na linha 16.

7.3.5.1 Operação de alteração

Para executar uma operação de alteração de registro em uma tabela do banco de dados você deverá se conectar ao banco de dados, executar um comando SQL *update* e fechar a conexão com o banco de dados.

Veja o exemplo da classe *Banco.java* utilizando a interface *PreparedStatement*:

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
1   import java.sql.*;
2
3   public class Banco {
4
5       public static void main(String[] args) {
6           try {
7               Class.forName("com.mysql.jdbc.Driver");
8               Connection conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
9               PreparedStatement st = conn.prepareStatement("update cliente
                set nome=?, telefone=?, sexo=?, renda=? where id=?");
10              st.setString(1, "Anderson Silva");
11              st.setString(2, "(11)4567-9876");
12              st.setString(3, "Masculino");
13              st.setDouble(4, 22987.56);
14              st.setInt(5, 1);
15              st.executeUpdate();
16              System.out.println("Os dados do cliente foram alterados");
17              conn.close();
18          } catch (ClassNotFoundException | SQLException ex) {
19              System.out.println("Erro ao tentar se conectar ao banco de
                dados");
20          }
21      }
22  }
```

Esta classe altera os dados do registro cadastrado cujo conteúdo do campo *id* é igual a 1.

Observe que a instrução SQL *update* é preparada na linha 9, aguardando apenas os parâmetros que serão incluídos nos lugares onde estão os pontos de interrogação. Estes parâmetros são passados em ordem, das linhas 10 à 14. A linha 15 executa a operação de alteração e exibe a mensagem da linha 16. Note que o método da interface *PreparedStatement* responsável por executar um comando *update* é o método *executeUpdate*.

Na linha 17 a conexão com o banco de dados é finalizada.

Ao compilar e executar o projeto por meio do pressionamento da tecla *F6* aparecerá a mensagem mostrada na Figura 7.29, caso você tenha um registro contendo o valor 1 no campo *id*.

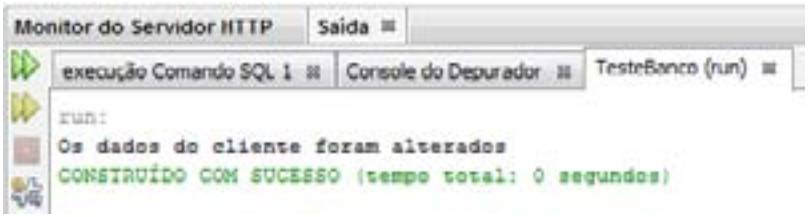


Figura 7.29: Confirmação da alteração de um registro no banco de dados.

Veja agora o exemplo da classe *Banco.java* utilizando a interface *Statement*:

```
1   import java.sql.*;
2
3   public class Banco {
4
5       public static void main(String[] args) {
6           try {
7               Class.forName("com.mysql.jdbc.Driver");
8               Connection conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
9               int id = 3;
10              String nome = "João de Deus";
11              String telefone = "(11)7865-9087";
12              String sexo = "Masculino";
13              double renda = 14567.56;
14              Statement st = conn.createStatement();
15              String sql = "update cliente set nome= '" + nome + "', telefone= '" +
                telefone + "', sexo= '" + sexo + "', renda=" + renda + " where id= " +
                id + "";
16              int r = st.executeUpdate(sql);
17              if (r == 1) {
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
18                  System.out.println("Os dados do cliente foram alterados");
19              } else {
20                  System.out.println("cliente não encontrado");
21              }
22              conn.close();
23          } catch (ClassNotFoundException | SQLException ex) {
24              System.out.println("Erro ao tentar se conectar ao banco de dados");
25          }
26      }
27  }
```

Observe que as linhas responsáveis por executar a alteração no registro estão no intervalo de 14 à 16. Veja que na linha 15 a instrução *update* é armazenada em uma variável *String*, concatenando-se as partes da instrução com o conteúdo das variáveis que contém os dados a serem alterados. Perceba que é muito trabalhoso e complexo gerar esta linha.

Na linha 16 a instrução *update* contida na variável *String* é executada, retornando o valor 1 caso a operação tenha sido executada com sucesso, e 0, caso não tenha sido executada (no caso de não haver um registro com o *id* informado para alteração). Se a operação teve sucesso (linha 17), exibe a mensagem contida na linha 18. Caso contrário, exibe a mensagem contida na linha 20.

7.3.6 Diferenças entre as interfaces PreparedStatement e Statement

Como você pôde perceber nos exemplos apresentados nos tópicos anteriores, para executar qualquer operação em uma tabela do banco de dados você precisará de um objeto das interfaces *PreparedStatement* ou *Statement*. Por meio deste objeto você conseguirá acessar os métodos para executar os comandos *insert*, *updade*, *select* e *delete* da linguagem SQL. Para executar as instruções *insert*, *update* e *delete*, você deverá utilizar o método *executeUpdate*. Para executar a instrução *select*, precisará do método *executeQuery*.

Se você utilizar um objeto da interface *PreparedStatement*, sua instrução SQL poderá receber, como parâmetro, os valores a serem inseridos. Se utilizar a interface *Statement*, terá que montar a instrução SQL concatenando sua partes com os valores a serem inseridos, o que dará bastante trabalho e a instrução estará propensa a erros.

Procure utilizar a interface *PreparedStatement*, mais fácil de utilizar, mais segura e mais utilizada pela comunidade de desenvolvedores Java.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.4 Exemplo de aplicação CRUD

CRUD é a abreviação de *Create*, *Read*, *Update* e *Delete*.

Uma aplicação CRUD precisa executar estas operações em um banco de dados. *Create* é a funcionalidade de cadastro, executada pela instrução SQL *insert*. *Read* é a funcionalidade de consulta, executada pela instrução SQL *select*. *Update* é a funcionalidade de alteração, executada pela instrução SQL *update*. *Delete* é a funcionalidade de exclusão, executada pela instrução SQL *delete*.

Neste tópico criaremos uma aplicação CRUD utilizando os recursos das APIs *swing* e *awt* para criar as janelas para entrada e apresentação de dados, e da API JDBC para executar operações no banco de dados e na tabela que criamos no início deste Capítulo.

Crie então um projeto chamado *ProjetoBD* e uma classe principal chamada *Menu.java*, conforme mostra a Figura 7.30.



Figura 7.30: Projeto para criar uma aplicação CRUD.

Adicione o *driver* JDBC à pasta "Bibliotecas" do projeto, clicando com o botão direito sobre o nome da pasta, como mostra a Figura 7.31.

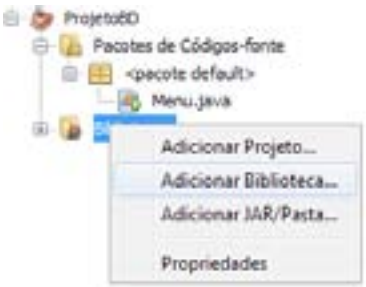


Figura 7.31: Adicionando o driver JDBC à biblioteca.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Selecione a opção “Adicionar Biblioteca”, que aparecerá a Figura 7.32.

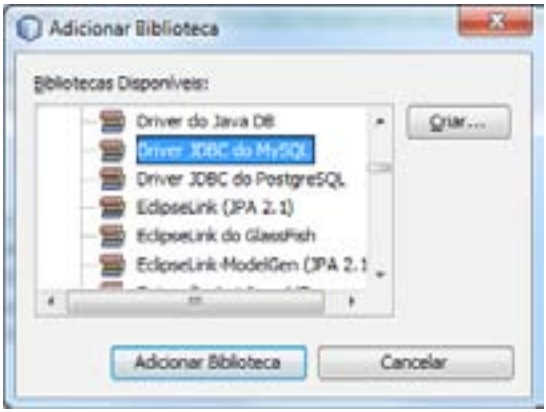


Figura 7.32: Escolha do driver JDBC entre as bibliotecas disponíveis.

Selecione “*Driver JDBC do MySQL*” e clique no botão “Adicionar Biblioteca”.

Pronto!

Veja na Figura 7.33 que o *driver* já está na biblioteca do nosso projeto.

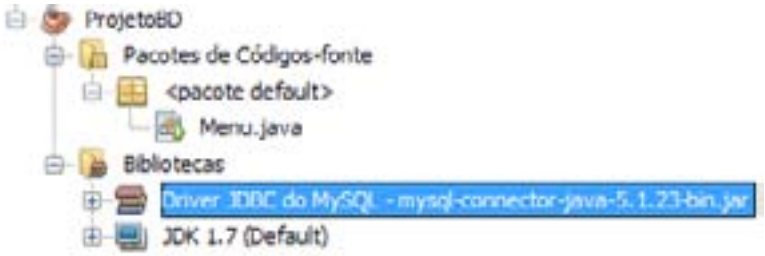


Figura 7.33: Driver JDBC do MySQL na pasta “Bibliotecas” do projeto.

Agora já podemos programar a classe *Menu.java*, mas antes vamos à apresentação de algumas informações sobre a aplicação que facilitará o entendimento do código-fonte.

A classe *Menu.java* criará três formulários (janelas): uma janela do menu principal, uma janela para cadastro e alteração de dados e uma janela para consulta e exclusão de dados.

Ao executar a aplicação aparecerá a janela principal, mostrada na Figura 7.34.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 7.34: janela do menu principal da aplicação.

Ao clicar no botão “Cadastro” no menu principal aparecerá a janela mostrada na Figura 7.35.



Figura 7.35: Janela para cadastro e alteração de dados.

Após cadastrar os dados nos campos, a janela terá a aparência mostrada na Figura 7.36.



Figura 7.36: Entrando com dados para o cadastro.

Ao clicar no botão “Salvar”, aparecerá a mensagem mostrada na Figura 7.37.

UNINOVE – USO EXCLUSIVO PARA ALUNO

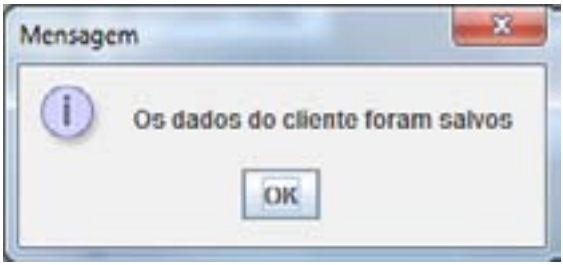


Figura 7.37: Confirmação de cadastro.

Ao clicar no botão "Voltar", a janela atual é escondida e a tela do menu principal será apresentada novamente.

Ao clicar no botão "Outras Operações", no menu principal, aparecerá a janela mostrada na Figura 7.38.

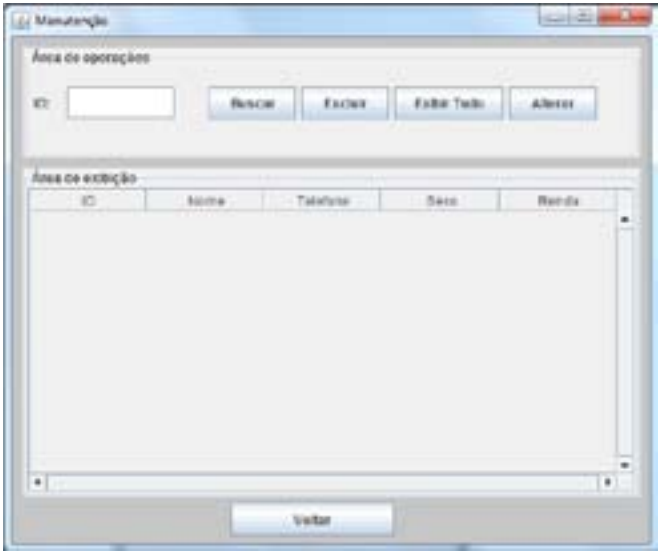


Figura 7.38: Janela para consultas e exclusão de dados.

Veja que você tem um painel na parte superior onde poderá realizar as operações de consulta, exclusão e alteração.

Ao digitar no campo *ID* o valor 1 e clicar no botão "Buscar", caso exista este registro no banco de dados, os dados serão carregados na tabela na parte inferior da janela, como mostra a Figura 7.39.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 7.39: Exibição do registro retornado do banco de dados.

Ao clicar no botão “Exibir Tudo”, todos os registros na tabela do banco de dados serão apresentados, como mostra a Figura 7.40.



Figura 7.40: Exibição de todos os registros da tabela do banco de dados.

Ao digitar 2 no campo *ID* e clicar no botão “Excluir”, será apresentado o que mostra a Figura 7.41, pedindo a confirmação da exclusão.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 7.41: Operação de exclusão de registro.

Ao clicar no botão no botão “Sim”, aparecerá a caixa de diálogo mostrada na Figura 7.42.



Figura 7.42: Confirmação da exclusão de um registro.

Digitando o valor 1 no campo *ID* e clicando no botão “Alterar”, a mesma janela usada no cadastro será exibida, porém, com os dados do registro cujo *ID* é 1 e com o botão “Salvar Alteração” na parte de baixo da janela, como mostra a Figura 7.43.



Figura 7.43: Carregamento dos dados para alteração.

Veja na janela mostrada na Figura 7.44 que foi feita uma modificação nos campos *nome* e *telefone*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

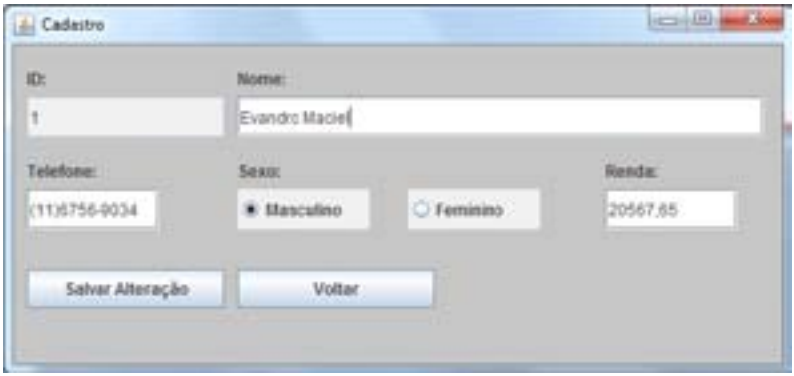


Figura 7.44: Modificando dados.

Ao clicar no botão "Salvar Alteração", aparecerá a caixa de diálogo mostrada na Figura 7.45.

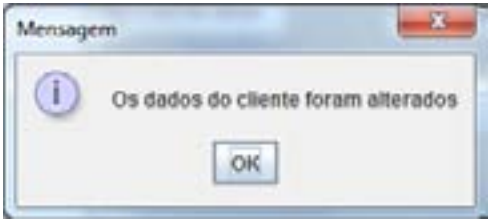


Figura 7.45: Confirmação da operação de alteração.

Ao clicar no botão "OK", a tela de operações será mostrada. Ao clicar no botão "Voltar", o menu principal será exibido novamente.

As três janelas utilizadas, assim como o tratamento de eventos de clique nos botões, foram geradas pela mesma classe, o que deixou o código-fonte bastante longo. Uma alternativa seria usar uma classe para cada janela, mas seria um pouco mais difícil de alternar entre elas, principalmente quando é necessário passar parâmetros de uma tela para outra.

Agora que você já viu as telas geradas pelas operações da aplicação, vamos analisar o código-fonte da classe *Menu.java*, apresentado a seguir:

```
1    import java.awt.*;
2    import java.awt.event.*;
3    import java.sql.Connection;
4    import java.sql.DriverManager;
5    import java.sql.PreparedStatement;
6    import java.sql.ResultSet;
7    import java.sql.SQLException;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
8   import java.text.DecimalFormat;
9   import java.text.ParseException;
10  import javax.swing.*;
11  import javax.swing.border.TitledBorder;
12  import javax.swing.table.DefaultTableModel;
13  import javax.swing.text.DefaultFormatterFactory;
14  import javax.swing.text.MaskFormatter;
15  import javax.swing.text.NumberFormatter;
16
17  public class Menu extends JFrame implements ActionListener {
18
19      JFrame frmCadastrarAlterar, frmManutencao;
20      JButton btnCadastro, btnOp, btnVoltaCadAlt, btnVoltaManutencao,
        btnSair;
21      JButton btnSalvar, btnSalvarAlteracao, btnAlterar, btnBuscar, btnExcluir,
        btnExibirTudo;
22      JLabel lblId, lblNome, lblTelefone, lblSexo, lblRenda;
23      JTextField txtNome;
24      JFormattedTextField txtTelefone, txtId, txtIdConsulta, txtRenda;
25      MaskFormatter mskTelefone, mskId;
26      ButtonGroup gruSexo;
27      JRadioButton rdoMasculino, rdoFeminino;
28      JPanel pnlCampos, pnlTabela;
29      TitledBorder tituloPnlCampos, tituloPnlTabela;
30      DefaultTableModel tblConsultaModel;
31      JTable tblConsulta;
32      JScrollPane scrRolagem;
33      Connection conn;
34      PreparedStatement st;
35      ResultSet rs;
36
37      public Menu() {
38
39          //Programação da janela principal (Menu)
40
41          setTitle("Menu Principal");
42          setDefaultCloseOperation(EXIT_ON_CLOSE);
43          setLayout(null);
44          setBounds(500, 300, 300, 210);
45          btnCadastro = new JButton("Cadastro");
46          btnCadastro.setBounds(50, 20, 150, 30);
47          btnCadastro.addActionListener(this);
48          btnOp = new JButton("Outras Operações");
49          btnOp.setToolTipText("Consultas, exclusão e alteração de
            dados");
50          btnOp.setBounds(50, 70, 150, 30);
51          btnOp.addActionListener(this);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
52          btnSair = new JButton("Sair");
53          btnSair.setBounds(50, 120, 150, 30);
54          btnSair.addActionListener(this);
55          add(btnCadastro);
56          add(btnOp);
57          add(btnSair);
58
59          //Programação da janela para cadastro e alteração de dados
60
61          frmCadastrarAlterar = new JFrame("Cadastro");
62          frmCadastrarAlterar.setDefaultCloseOperation(EXIT_ON_CLOSE);
63          frmCadastrarAlterar.setLayout(null);
64          frmCadastrarAlterar.setBounds(400, 200, 600, 280);
65          frmCadastrarAlterar.getContentPane().setBackground
            (Color.LIGHT_GRAY);
66          lblId = new JLabel("ID:");
67          lblId.setBounds(10, 10, 50, 30);
68          frmCadastrarAlterar.add(lblId);
69
70          txtId = new JFormattedTextField();
71          txtId.setFormatterFactory(new DefaultFormatterFactory(new
            NumberFormatter(new DecimalFormat("#"))));
72          txtId.setBounds(10, 40, 150, 30);
73          txtId.requestFocus();
74          frmCadastrarAlterar.add(txtId);
75
76          lblNome = new JLabel("Nome:");
77          lblNome.setBounds(170, 10, 50, 30);
78          frmCadastrarAlterar.add(lblNome);
79
80          txtNome = new JTextField(40);
81          txtNome.setBounds(170, 40, 400, 30);
82          frmCadastrarAlterar.add(txtNome);
83
84          lblTelefone = new JLabel("Telefone:");
85          lblTelefone.setBounds(10, 80, 150, 30);
86          frmCadastrarAlterar.add(lblTelefone);
87
88          try {
89             mskTelefone = new MaskFormatter("(##)####-####");
90             mskTelefone.setPlaceholderCharacter('_');
91          } catch (ParseException ex) {
92          }
93          txtTelefone = new JFormattedTextField(mskTelefone);
94          txtTelefone.setBounds(10, 110, 100, 30);
95          frmCadastrarAlterar.add(txtTelefone);
96
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
97          lblSexo = new JLabel("Sexo:");
98          lblSexo.setBounds(170, 80, 150, 30);
99          frmCadastrarAlterar.add(lblSexo);
100
101         rdoMasculino = new JRadioButton("Masculino");
102         rdoMasculino.setBounds(170, 110, 100, 30);
103         rdoFeminino = new JRadioButton("Feminino");
104         rdoFeminino.setBounds(300, 110, 100, 30);
105         gruSexo = new ButtonGroup();
106         gruSexo.add(rdoMasculino);
107         gruSexo.add(rdoFeminino);
108         frmCadastrarAlterar.add(rdoMasculino);
109         frmCadastrarAlterar.add(rdoFeminino);
110
111         lblRenda = new JLabel("Renda:");
112         lblRenda.setBounds(450, 80, 150, 30);
113         frmCadastrarAlterar.add(lblRenda);
114
115         txtRenda = new JFormattedTextField();
116         txtRenda.setFormatterFactory(new DefaultFormatterFactory(new
            NumberFormatter(new DecimalFormat("#0.00"))));
117         txtRenda.setBounds(450, 110, 100, 30);
118         txtRenda.setToolTipText("Utilize vírgula para separar as casas
            decimais");
119         frmCadastrarAlterar.add(txtRenda);
120
121         btnSalvar = new JButton("Salvar");
122         btnSalvar.setBounds(10, 170, 150, 30);
123         btnSalvar.addActionListener(this);
124         btnSalvar.setVisible(false);
125         frmCadastrarAlterar.add(btnSalvar);
126
127         btnSalvarAlteracao = new JButton("Salvar Alteração");
128         btnSalvarAlteracao.setBounds(10, 170, 150, 30);
129         btnSalvarAlteracao.addActionListener(this);
130         btnSalvarAlteracao.setVisible(false);
131         frmCadastrarAlterar.add(btnSalvarAlteracao);
132
133         btnVoltaCadAlt = new JButton("Voltar");
134         btnVoltaCadAlt.setBounds(170, 170, 150, 30);
135         btnVoltaCadAlt.addActionListener(this);
136         frmCadastrarAlterar.add(btnVoltaCadAlt);
137
138         //Programação da janela de consulta e exclusão (Outras Operações)
139
140         frmManutencao = new JFrame("Manutenção");
141         frmManutencao.setDefaultCloseOperation(EXIT_ON_CLOSE);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
142         frmManutencao.setLayout(null);
143         frmManutencao.setBounds(400, 200, 600, 500);
144         frmManutencao.getContentPane().setBackground(Color.LIGHT_
            GRAY);
145
146         pnlCampos = new JPanel(null);
147         tituloPnlCampos = BorderFactory.createTitledBorder("Área de
            operações");
148         pnlCampos.setBorder(tituloPnlCampos);
149         pnlCampos.setBounds(10, 10, 565, 100);
150
151         lblId = new JLabel("ID:");
152         lblId.setBounds(10, 35, 50, 30);
153         pnlCampos.add(lblId);
154
155         txtIdConsulta = new JFormattedTextField();
156         txtId.setFormatterFactory(new DefaultFormatterFactory(new
            NumberFormatter(new DecimalFormat("#"))));
157         txtIdConsulta.setBounds(40, 35, 100, 30);
158         pnlCampos.add(txtIdConsulta);
159
160         btnBuscar = new JButton("Buscar");
161         btnBuscar.setBounds(170, 35, 80, 30);
162         btnBuscar.addActionListener(this);
163         pnlCampos.add(btnBuscar);
164
165         btnExcluir = new JButton("Excluir");
166         btnExcluir.setBounds(255, 35, 80, 30);
167         btnExcluir.addActionListener(this);
168         pnlCampos.add(btnExcluir);
169
170         btnExibirTudo = new JButton("Exibir Tudo");
171         btnExibirTudo.setBounds(340, 35, 100, 30);
172         btnExibirTudo.addActionListener(this);
173         pnlCampos.add(btnExibirTudo);
174
175         btnAlterar = new JButton("Alterar");
176         btnAlterar.setBounds(445, 35, 80, 30);
177         btnAlterar.addActionListener(this);
178         pnlCampos.add(btnAlterar);
179
180         frmManutencao.add(pnlCampos);
181
182         pnlTabela = new JPanel(null);
183         pnlTabela.setLayout(new GridLayout(1, 1));
184         tituloPnlTabela = BorderFactory.createTitledBorder("Área de exibição");
185         pnlTabela.setBorder(tituloPnlTabela);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
186         pnlTabela.setBounds(10, 120, 565, 300);
187         frmManutencao.add(pnlTabela);
188
189         String[] cols = {"ID", "Nome", "Telefone", "Sexo", "Renda"};
190         tblConsultaModel = new DefaultTableModel(cols, 5);
191         tblConsulta = new JTable(tblConsultaModel);
192         tblConsultaModel.setNumRows(0);
193         scrRolagem = new JScrollPane(tblConsulta,
            JScrollPane.VERTICAL_SCROLLBAR_ALWAYS,
            JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
194
195         pnlTabela.add(scrRolagem);
196
197         btnVoltaManutencao = new JButton("Voltar");
198         btnVoltaManutencao.setBounds(200, 425, 150, 30);
199         frmManutencao.add(btnVoltaManutencao);
200         btnVoltaManutencao.addActionListener(this);
201
201     }
203
204     //Programação dos botões
205     @Override
206     public void actionPerformed(ActionEvent e) {
207         Object btn = e.getSource();
208
209         if (btn == btnCadastro) {
210             setVisible(false);
211             btnSalvar.setVisible(true);
212             frmCadastrarAlterar.setVisible(true);
213         }
214
215         if (btn == btnOp) {
216             setVisible(false);
217             tblConsultaModel.setRowCount(0);
218             txtIdConsulta.setText("");
219             frmManutencao.setVisible(true);
220         }
221
222         if (btn == btnVoltaCadAlt) {
223             setVisible(true);
224             limparCampos();
225             frmCadastrarAlterar.setVisible(false);
226         }
227
228         if (btn == btnVoltaManutencao) {
229             setVisible(true);
230             frmManutencao.setVisible(false);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
231         }
232
233         if (btn == btnSair) {
234            System.exit(0);
235         }
236
237         if (btn == btnSalvar) {
238            int id;
239            double renda;
240            String nome, telefone, sexo;
241            id = Integer.parseInt(txtId.getText());
242            nome = txtNome.getText();
243            telefone = txtTelefone.getText();
244            if (rdoMasculino.isSelected()) {
245               sexo = rdoMasculino.getText();
246            } else if (rdoFeminino.isSelected()) {
247               sexo = rdoFeminino.getText();
248            } else {
249               sexo = "";
250            }
251            renda = Double.parseDouble(txtRenda.getText().replace(",", "."));
252            if (conectar() == null) {
253               JOptionPane.showMessageDialog(btnSalvar, "Erro ao tentar
                  Se conectar ao banco de dados");
254            } else {
255               try {
256                  st = conn.prepareStatement("insert into cliente (id, nome,
                     telefone, sexo, renda) values (?, ?, ?, ?, ?)");
257                  st.setInt(1, id);
258                  st.setString(2, nome);
259                  st.setString(3, telefone);
260                  st.setString(4, sexo);
261                  st.setDouble(5, renda);
262                  st.executeUpdate();
263                  JOptionPane.showMessageDialog(btnSalvar, "Os dados do
                     cliente foram salvos");
264                  limparCampos();
265               } catch (SQLException ex) {
266                  if (ex.getErrorCode() == 1062) {
267                     JOptionPane.showMessageDialog(btnSalvar, "Já existe um
                        cliente com este ID");
268                     txtId.requestFocus();
269                  } else {
270                     JOptionPane.showMessageDialog(btnSalvar, "Erro ao tentar
                        salvar os dados");
271                  }
272               }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
273                 desconectar();
274             }
275         }
276
277         if (btn == btnBuscar) {
278             try {
279                 if (conectar() == null) {
280                     JOptionPane.showMessageDialog(btnBuscar, "Erro ao tentar
                        conectar ao banco de dados");
281                 } else {
282                     if (txtIdConsulta.getText().isEmpty()) {
283                         JOptionPane.showMessageDialog(btnBuscar, "Por favor,
                            digite um valor no campo ID");
284                         txtIdConsulta.requestFocus();
285                     } else {
286                         int id = Integer.parseInt(txtIdConsulta.getText());
287                         st=conn.prepareStatement("select * from cliente where id=? ");
288                         st.setInt(1, id);
289                         tblConsultaModel.setRowCount(0);
290                         rs = st.executeQuery();
291                         if (rs.next()) {
292                             String[] linha = {rs.getString("id"), rs.getString("nome"),
                                rs.getString("telefone"), rs.getString("sexo"),
                                rs.getString("renda").replace(".", ",")};
293                             tblConsultaModel.addRow(linha);
294                         } else {
295                             JOptionPane.showMessageDialog(btnBuscar, "Este cliente
                                não foi encontrado");
296                         }
297                         desconectar();
298                     }
299                 }
300             } catch (SQLException ex) {
301                 JOptionPane.showMessageDialog(btnBuscar, "Erro na consulta");
302             }
303
304         }
305
306         if (btn == btnExibirTudo) {
307             try {
308                 if (conectar() == null) {
309                     JOptionPane.showMessageDialog(btnExibirTudo, "Erro ao
                        tentar conectar ao banco de dados");
310                 } else {
311                     st = conn.prepareStatement("select * from cliente");
312                     rs = st.executeQuery();
313                     tblConsultaModel.setRowCount(0);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
314                 while (rs.next()) {
315                     String[] linha = {rs.getString("id"), rs.getString("nome"),
                        rs.getString("telefone"), rs.getString("sexo"),
                        rs.getString("renda").replace(".", ",")};
316                     tblConsultaModel.addRow(linha);
317                 }
318                 desconectar();
319             }
320         } catch (SQLException ex) {
321             JOptionPane.showMessageDialog(btnExibirTudo, "Erro na
                    consulta");
322         }
323     }
324
325     if (btn == btnExcluir) {
326         try {
327             if (conectar() == null) {
328                 JOptionPane.showMessageDialog(btnSalvar, "Erro ao tentar se
                        conectar ao banco de dados");
329             } else {
330                 if (txtIdConsulta.getText().isEmpty()) {
331                     JOptionPane.showMessageDialog(btnExcluir, "Por favor,
                            digite um valor no campo ID");
332                     txtIdConsulta.requestFocus();
333                 } else {
334                     int op = JOptionPane.showConfirmDialog(btnExcluir, "Tem
                            certeza que deseja excluir este cliente?",
                            JOptionPane.OPTIONS_PROPERTY,
                            JOptionPane.YES_NO_OPTION);
335                     if (op == 0) {
336                         int id = Integer.parseInt(txtIdConsulta.getText());
337                         st=conn.prepareStatement("delete from cliente where
                                id=?");
338                         st.setInt(1, id);
339                         int r = st.executeUpdate();
340                         if (r == 1) {
341                             JOptionPane.showMessageDialog(btnExcluir, "Cliente
                                    excluído com sucesso");
342                             tblConsultaModel.setRowCount(0);
343                             txtIdConsulta.setText("");
344                             txtIdConsulta.requestFocus();
345                         } else {
346                             JOptionPane.showMessageDialog(btnExcluir, "Este
                                    cliente não está cadastrado");
347                         }
348                         desconectar();
349                     }
350                 }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
351                 }
352             } catch (SQLException ex) {
353                 JOptionPane.showMessageDialog(btnExcluir, "Erro na exclusão");
354             }
355         }
356
357         if (btn == btnAlterar) {
358             try {
359                 if (conectar() == null) {
360                     JOptionPane.showMessageDialog(btnAlterar, "Erro ao tentar se
                        conectar ao banco de dados");
361                 } else {
362                     if (txtIdConsulta.getText().isEmpty()) {
363                         JOptionPane.showMessageDialog(btnAlterar, "Por favor,
                            digite um valor no campo ID");
364                         txtIdConsulta.requestFocus();
365                     } else {
366                         int id = Integer.parseInt(txtIdConsulta.getText());
367                         st=conn.prepareStatement("select * from cliente where id=?");
368                         st.setInt(1, id);
369                         rs = st.executeQuery();
370                         if (rs.next()) {
371                             txtId.setText(rs.getString("id"));
372                             txtId.setEditable(false);
373                             txtNome.setText(rs.getString("nome"));
374                             txtTelefone.setText(rs.getString("telefone"));
375                             if (rs.getString("sexo").equalsIgnoreCase("masculino")) {
376                                 rdoMasculino.setSelected(true);
377                             }
378                             if (rs.getString("sexo").equalsIgnoreCase("feminino")) {
379                                 rdoFeminino.setSelected(true);
380                             }
381                             txtRenda.setText(rs.getString("renda").replace(".", ","));
382                             btnSalvar.setVisible(false);
383                             btnSalvarAlteracao.setVisible(true);
384
385                             frmManutencao.setVisible(false);
386                             frmCadastrarAlterar.setVisible(true);
387                         } else {
388                             JOptionPane.showMessageDialog(btnAlterar, "Este cliente
                                não foi encontrado");
389                         }
390                     }
391                 }
392             } catch (SQLException ex) {
393                 JOptionPane.showMessageDialog(btnAlterar, "Erro na consulta");
394             }
395
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
396             }
397
398             if (btn == btnSalvarAlteracao) {
399                 int id;
400                 double renda;
401                 String nome, telefone, sexo;
402                 id = Integer.parseInt(txtId.getText());
403                 nome = txtNome.getText();
404                 telefone = txtTelefone.getText();
405                 if (rdoMasculino.isSelected()) {
406                     sexo = rdoMasculino.getText();
407                 } else if (rdoFeminino.isSelected()) {
408                     sexo = rdoFeminino.getText();
409                 } else {
410                     sexo = "";
411                 }
412                 renda = Double.parseDouble(txtRenda.getText().replace(",", "."));
413                 try {
414                     st = conn.prepareStatement("update cliente set nome=?,
                        telefone=?, sexo=?, renda=? where id=?");
415                     st.setString(1, nome);
416                     st.setString(2, telefone);
417                     st.setString(3, sexo);
418                     st.setDouble(4, renda);
419                     st.setInt(5, id);
420                     st.executeUpdate();
421                     JOptionPane.showMessageDialog(btnSalvarAlteracao, "Os dados
                        do cliente foram alterados");
422                     limparCampos();
423                 } catch (SQLException ex) {
424                     JOptionPane.showMessageDialog(btnSalvarAlteracao, "Erro ao
                        tentar salvar os dados");
425                 }
426                 desconectar();
427                 limparCampos();
428                 frmCadastrarAlterar.setVisible(false);
429                 tblConsultaModel.setRowCount(0);
430                 txtIdConsulta.setText("");
431                 txtIdConsulta.requestFocus();
432                 frmManutencao.setVisible(true);
433             }
434         }
435
436         //programação de métodos de apoio
437         private Connection conectar() {
438             try {
439                 Class.forName("com.mysql.jdbc.Driver");
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
440             conn = DriverManager.getConnection
                ("jdbc:mysql://localhost:3306/bd002", "root", "teruel");
441             return conn;
442          } catch (ClassNotFoundException | SQLException ex) {
443             return null;
444          }
445       }
446
447       private void desconectar() {
448          try {
449             conn.close();
450          } catch (SQLException ex) {
451             JOptionPane.showMessageDialog(btnSalvar, "Erro ao tentar se
                desconectar do banco de dados");
452          }
453       }
454
455       private void limparCampos() {
456          txtId.setEditable(true);
457          txtId.setText("");
458          txtNome.setText("");
459          txtTelefone.setText("");
460          gruSexo.clearSelection();
461          txtRenda.setText("");
462          txtId.requestFocus();
463       }
464
465       // método principal de star da aplicação
466       public static void main(String[] args) {
467          Menu m = new Menu();
468          m.setVisible(true);
469       }
470    }
```

Nas linhas de 1 à 15 são importadas as classes utilizadas na aplicação. Note que são importadas as classes da API *awt*, *swing* e JDBC.

Observe na linha 17 que a classe menu herda a classe *JFrame* e implementa o método abstrato da interface *ActionListener*. Lembre-se que implementar a classe *ActionListener* obriga sobrescrever o método *actionPerformed*, que trata eventos de ação de clique do mouse.

Nas linhas de 19 à 35 são declarados os objetos que serão incluídos nas janelas e também os que serão utilizados nas operações com o banco de dados.

Nas linhas de 31 à 201 está o construtor da classe *Menu.java*. Este trecho de código gerará as janelas que serão utilizadas na aplicação. Será utilizado

UNINOVE – USO EXCLUSIVO PARA ALUNO

posicionamento estático dos elementos, ou seja, não serão utilizados gerenciadores de *layout*. Os objetos (elementos) terão sua posição e tamanho configurados um a um por meio do método *setBounds*. Lembre-se que este método recebe 4 parâmetros, sendo os dois primeiros referentes à posição do elemento (eixos *x* e *y*) e os dois últimos ao tamanho do elemento (largura e altura). Poderíamos utilizar os métodos *setLocation* e *setSize*, mas optamos por usar apenas o método *setBounds*.

As linhas de 41 à 57 criam a janela do menu principal, que aparecerá na tela quando a aplicação for compilada e executada. Essa janela é mostrada na Figura 7.46.



Figura 7.46: Tela do menu principal.

Note a linha 49, que por meio do método *setToolTipText* aplica uma mensagem ao botão ao botão *btnOp*, que será mostrada quando o ponteiro do mouse for posicionado sobre ele.

Observe também as linhas 47, 51 e 54, que adicionam um *Listener* (ouvinte) aos botões, de tal forma que quando eles receberem o clique do mouse, será feito um desvio para o método *actionPerformed*, que tratará os eventos de clique sobre os botões. As linhas de 55 à 57 adicionam os botões "Cadastro", "Outras Operações" e "Sair" à janela principal.

As linhas de 59 à 136 criam a janela com os campos. Esta janela será utilizada para fazer o cadastro e a alteração nos dados de um cliente. A janela gerada é mostrada na Figura 7.47.

UNINOVE – USO EXCLUSIVO PARA ALUNO

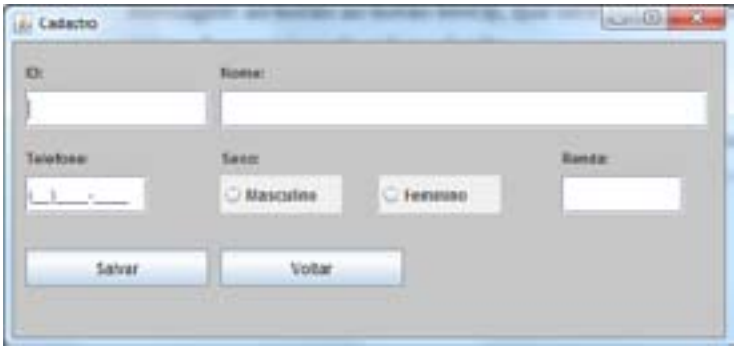


Figura 7.47: tela para cadastro e alteração de dados.

Não há novidades na criação desta janela em relação ao que já vimos no Capítulo anterior, exceto o que consta nas linhas 71 e 116. A linha 71 faz com que seja permitido no campo ID apenas números inteiros. A linha 116 permite apenas a entrada de números e da vírgula para separar as casas decimais. Caso você inclua outro *caracter* que não sejam os permitidos, eles automaticamente desaparecerão.

Note também que três botões são adicionados a esta janela: "Salvar", "Salvar Alteração" e "Voltar", mas apenas um é definido como visível, o botão "Voltar". O botão "Salvar Alteração" ficará oculto (linha 130) e será visualizado somente quando esta janela for aberta para alteração de dados. Já o botão "Salvar" será visualizado somente quando a tela for aberta para cadastro de um novo cliente. Veja que os botões "Salvar" e "Salvar Alteração" ocupam a mesma posição, pois nunca ficarão visíveis ao mesmo tempo e dependerão da operação a ser realizada.

Da linha 138 à linha 200 é criada a janela para consulta e exclusão de dados, referente à opção "Outras Operações" do menu principal. Esta janela é mostrada pela Figura 7.48.

UNINOVE – USO EXCLUSIVO PARA ALUNO



Figura 7.48: Janela para consultas e exclusão de dados.

Não há novidades na criação desta janela em relação ao que vimos no Capítulo anterior. Há dois painéis, um abaixo do outro. O painel superior contém um campo de busca e alguns botões para realizar operações de busca, exclusão e para chamar a operação de alteração. O painel inferior, que usa o gerenciador de *layout GridLayout* (linha 183) para que a tabela se ajuste a única célula do *layout*, é utilizado para exibir dados.

As linhas de 204 à 434 compõem o método *actionPerformed*, que é chamado sempre que um botão das janelas for clicado. Isso porque a todos os botões foi adicionado um *Listener* (ouvinte) que monitora o botão. Dentro deste método está programado o que ocorrerá após cada botão ser clicado. A linha 207 captura o nome do botão que foi clicado e armazena na variável *btn*, testando em seguida esta variável para saber o botão que foi clicado e realizar a operação adequada para cada botão. Já estudamos o tratamento de eventos de ação de clique no capítulo anterior. Se precisar de mais detalhes, releia o tópico sobre tratamento de eventos no capítulo 6.

As linhas de 209 à 213 tratam o evento de clique no botão "Cadastrar" (*btnCadastro*) do menu principal. Veja que este método apenas esconde o formulário de menu (linha 210) e exibe o formulário do cadastro (linha 212). Veja que antes de exibir o formulário de cadastro, o botão "Salvar" deste formulário torna-se visível (linha 211), já que esse botão havia sido criado oculto (linha 124).

As linhas de 215 à 220 tratam o evento de clique no botão "Outras Operações" (*btnOp*) do menu principal. Este método torna o formulário de menu invisível

UNINOVE – USO EXCLUSIVO PARA ALUNO

(linha 216), limpa a tabela do formulário (*frmManutencao* – linha 217), limpa o campo *ID* (linha 218) e exibe o formulário para consulta e exclusão de dados (*frmManutencao*).

As linhas de 222 à 226 tratam o evento de clique no botão "Voltar" (*btnVoltaCadAlt*) da janela onde será feito cadastro e alteração de dados (*frmCadastrarAlterar*). Veja que este método torna o formulário de menu ativo e visível (linha 223), chama o método *limparCampos*, para limpar os campos dos formulários (veja o método *limparCampos* das linhas 445 à 463) e esconde o formulário para cadastro e alteração de dados (linha 230).

As linhas de 228 à 231 referem-se ao botão "Voltar" do formulário de consulta e exclusão de dados (*frmManutencao*). Ao clicar nesse botão, a janela ativa ficará oculta e a janela do menu será visualizada.

As linhas de 233 à 235 referem-se ao botão "Sair" do menu principal. Ao clicar nesse botão, a aplicação será finalizada (linha 234).

As linhas de 237 à 275 referem-se ao botão "Salvar", pressionado no formulário para cadastro. As linhas de 241 à 251 capturam os valores incluidos nos campos do formulário e armazena em variáveis. Note, nas linhas de 244 à 250, que o texto da opção selecionada para sexo é armazenada na variável sexo. Isso é feito verificando qual opção foi selecionada. Na linha 251 a renda digitada é capturada. Como a renda pode ter sido digitada com vírgula para separar as casas decimais, a vírgula é trocada por ponto, já que o tipo *double* usa ponto no lugar da vírgula.

A linha 252 chama o método *conectar*, escrito da linha 437 à linha 445. Este método retorna uma conexão com o banco de dados, ou nulo (*null*), caso a conexão não tenha ocorrido. Se a conexão com o banco de dados não ocorreu (linha 252), a mensagem da linha 253 será exibida. Senão, se a conexão ocorreu (linha 254), as linhas de 255 à 273 são executadas. Na linha 256 uma instrução SQL *insert* é preparada, aguardando como parâmetro os dados que serão inseridos. Estes dados parâmetros são passados das linhas 257 à 261, completando a instrução *insert*, que é executada na linha 262. Em seguida, na linha 263 é exibida uma mensagem avisando que os dados foram inseridos na tabela e os campos do formulário são limpos por meio de uma chamada ao método *limparCampos* (linha 264). As instruções que limpam os campos estão nas linhas de 455 à 463.

Observe o intervalo de linhas de 266 à 271, dentro do bloco *catch* que é executado se houve algum problema na execução do comando *insert*. Na linha 266 é verificado se o código do erro gerado é 1062. Este código refere-se à tentativa de cadastrar um valor no campo chave primária, que já foi cadastrado anteriormente, ou seja, à tentativa de cadastrar um cliente com *id* que já foi cadastrado. Neste caso, será exibida a mensagem da linha 267 e o campo *ID*

UNINOVE – USO EXCLUSIVO PARA ALUNO

receberá o foco do cursor (linha 268). Se o erro não for o de número 1062, a mensagem da linha 270 será exibida.

Após a operação ter sido realizada, a linha 273 chama o método *desconectar*, escrito das linhas 447 à 453. Este método fecha a conexão com o banco de dados.

As linhas de 277 à 304 referem-se ao botão "Buscar", pressionado no formulário para consulta e exclusão. Se a conexão com o banco de dados não foi estabelecida (linha 279), a mensagem de erro da linha 280 é mostrada. Senão, se a conexão foi estabelecida com sucesso (linha 281), as linhas de 282 à 297 são executadas.

Se o campo *ID* estiver vazio (linha 282), a mensagem pedindo o preenchimento do campo será exibida (linha 283) e o cursor será posicionado no campo *ID* (linha 284). Senão, se o campo *ID* tiver um valor de *ID* válido (linha 285), este valor é capturado e armazenado na variável *id* (linha 286).

A linha 287 prepara uma instrução SQL *select* aguardando o *id* como parâmetro. Este *id* é passado para a instrução na linha 288. Na linha 289 a tabela que exibirá os dados é limpa e a instrução *select* é executada na linha 290. O resultado retornado da tabela do banco de dados é armazenado no objeto *rs* da interface *ResultSet* (linha 290). Se este objeto recebeu um registro (linha 291), ele é carregado na tabela da parte inferior da janela (linhas 292 e 293). Senão, uma mensagem será exibida informando que o cliente não foi encontrado.

Caso tenha ocorrido algum erro na execução da instrução *select*, a linha 300 será executada e a mansegem da linha 301 será exibida na tela.

As linhas de 306 à 333 são referentes ao botão "Exibir Tudo" do formulário para consulta e exclusão de dados. Ao clicar nesse botão, todos os registros cadastrados na tabela do banco de dados serão exibidos em uma tabela na parte inferior da janela. Veja que a programação deste botão é muito semelhante à programação do botão "Buscar". Quando a conexão com o banco de dados é estabelecida (linha 310), uma instrução *select* é preparada (linha 311) e executada (linha 312). Em seguida, da linha 314 à 317, um laço de repetição percorre o objeto *rs* da interface *ResultSet*, que contém os dados, carregando cada registro em uma linha da tabela que fica na parte inferior da janela.

As linhas de 325 à 355 são referentes ao botão "Excluir" do formulário de consulta e exclusão de dados. Ao clicar nesse botão, uma caixa de diálogo pergunta se o usuário realmente deseja excluir o registro cujo ID foi informado no campo *ID*. Se o usuário confirmar, o registro será excluído.

A programação deste botão é semelhante à programação dos demais botões. Quando a conexão com o banco de dados é estabelecida, as linhas de 330 à 348 são executadas. A linha 330 verifica se o campo *ID* está vazio e mostra uma mensagem pedindo seu preenchimento, caso esteja. Se o campo *ID* não

UNINOVE – USO EXCLUSIVO PARA ALUNO

estiver vazio, a linha 334 abre uma caixa de diálogo pedindo a confirmação da exclusão. Se o usuário clicar na opção "Sim" desta caixa de diálogo (linha 335), o ID digitado no campo *ID* é capturado (linha 336), uma instrução SQL *delete* é preparada (linha 337) e o *id* é passado como parâmetro para esta instrução (linha 338). A linha 339 executa a instrução *delete* e guarda o retorno desta execução em uma variável. Se o retorno for 1, ou seja, se a exclusão ocorreu com sucesso (linha 340), será mostrada uma mensagem avisando que o registro foi excluído (linha 341). Em seguida, a tabela que está na parte inferior da janela é limpa, assim como o campo *ID* (linha 343), e o cursor é posicionado neste campo (linha 344). Se a exclusão não ocorreu (linha 345), será mostrada a mensagem da linha 346. Em seguida, a conexão com o banco de dados é encerrada (linha 348). Se houve alguma exceção na execução da instrução *delete* (linha 352), será mostrada a mensagem da linha 353.

As linhas de 357 à 396 são referentes ao botão "Alterar" do formulário de consulta e exclusão. Quando este botão é pressionado, os dados do cliente cujo ID está no campo *ID* desta janela são carregados no formulário para cadastro e alteração, para serem alterados. Basicamente este trecho executa uma consulta na tabela do banco de dados, como o botão "Buscar", já explicado anteriormente. A diferença é que os dados retornados nesta consulta são carregados na janela para cadastro e alteração (linhas de 371 à 381). Quando os dados consultados são carregados no formulário, todos os campos poderão ser editados, exceto o campo ID, pois ele é chave primária da tabela do banco de dados e não poderá ser modificado nesta operação. Isso é mostrado na linha 372. Note também na linha 381 que a renda obtida da tabela do banco de dados vem com ponto para separar as casas decimais. Quando esta renda é carregada no campo *renda* do formulário (linha 381), o ponto é trocado por vírgula, exibindo o valor com a vírgula separando as casas decimais. Após os dados consultados serem carregados nos campos do formulário, o botão "Salvar Alteração" é exibido (linha 383), o formulário ativo de consulta e exclusão é escondido (linha 385) e o formulário com os dados a serem alterados é mostrado (linha 386). Veja que na programação deste botão ("Alterar") a conexão com o banco de dados não foi fechada, pois isto será feito apenas depois que os dados forem alterados e salvos novamente no banco de dados, por meio do botão "Salvar Alteração".

As linhas de 398 à 434 são referentes ao botão "Salvar Alteração", do formulário que exibe os dados para alteração. Após os dados terem sido modificados neste formulário, esse botão permite atualizar o registro na tabela do banco de dados, executando a instrução *update* da linguagem SQL.

Note que na programação deste botão não foi feita conexão com o banco de dados, pois a conexão já foi aberta quando os dados foram carregados no formulário por meio do pressionamento do botão "Alterar". Observe que os

UNINOVE – USO EXCLUSIVO PARA ALUNO

dados do formulário são capturados e armazenados em variáveis (linhas de 402 à 412). Em seguida uma instrução SQL *update* é preparada na linha 414 e os dados das variáveis que contém os valores dos campos do formulário são passados como parâmetro para esta instrução (linhas 415 à 419). A linha 420 executa a instrução *update* atualizando o registro na tabela do banco de dados. Agora sim, o banco de dados é fechado na linha 426.

Nas linhas de 436 à 463 estão os métodos *conectar*, *desconectar* e *limparCampos*, chamados durante a programação dos eventos dos botões. As instruções contidas nestes métodos não são novas e já foram vistas em exemplos anteriores. As linhas de 465 à 469 são do método *main*, que inicia a janela principal da aplicação quando o projeto é compilado e executado.

Compile e execute o projeto por meio do pressionamento da tecla *F6* para testar a aplicação.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.5 O padrão Data Access Object (DAO)

O *design pattern* (padrão de projeto) *Data Access Object* (DAO) é um padrão que recomenda que todas as operações de acesso a banco de dados sejam separadas em uma classe que chamamos de classe DAO. Os métodos para conectar/desconectar do banco de dados e para executar inclusão, exclusão, consulta e alterações, devem ser colocados nesta classe, em métodos específicos para cada uma destas operações. Isso permite separar as operações de acesso a dados da interface do usuário das classes que implementam as regras de negócio.

Em aplicações que utilizam duas, três ou mais camadas, a classe DAO é representada na camada de Persistência, também chamada de camada de Dados ou de Modelo.

No que se refere à arquitetura de aplicações, o modelo de camadas mais utilizado é o modelo de três camadas. Neste modelo, as classes que geram as janelas criadas utilizando as APIs *swing* e *awt* são representadas na camada de Apresentação, e a classe DAO, na camada de Persistência. Entre estas camadas, são criadas classes que implementam as regras de negócio, em uma camada chamada de camada de Negócio. Regras de negócio são operações específicas de uma empresa ou negócio em particular. Por exemplo, cálculos de saldo bancário e verificações de saldo, cálculo e verificação de notas de alunos, cálculos e operações de impostos etc.

Nos tópicos a seguir será criada uma nova aplicação apenas com a operação de cadastro, utilizando o padrão DAO.

7.6 CRIANDO UMA APLICAÇÃO DE CADASTRO COM DUAS CAMADAS

Neste tópico vamos criar uma pequena aplicação com duas camadas: camada de Apresentação (*view*) e camada de Persistência (*persistence*). Não criaremos a camada de Negócio porque a aplicação é muito simples e não terá nenhuma regra de negócio para ser implementada.

Para iniciar, crie um projeto chamado *ProjetoCamadas*, uma classe principal chamada *TelaCliente.java* em um pacote chamado *view*, e duas classes em um pacote chamado *persistence*: *Cliente.java* e *ClienteDao.java*.

Não esqueça de adicionar o *driver* JDBC na biblioteca do projeto. Para isso, clique com o botão direito do mouse na pasta "Bibliotecas" e selecione a opção "Adicionar Biblioteca". Na janela que aparece, selecione "Driver JDBC do MySQL" e clique no botão "Adicionar Biblioteca".

Ao terminar, o projeto deverá estar como mostra a Figura 7.49.

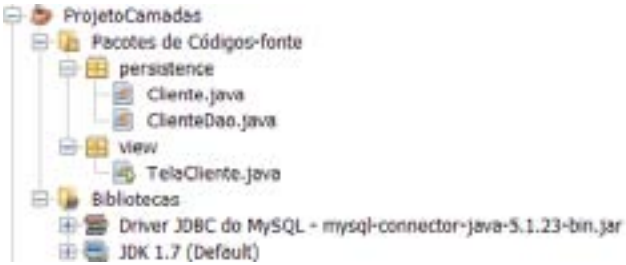


Figura 7.49: Projeto usando o padrão DAO.

Note que os pacotes *view* e *persistence* simbolizam as camadas de Apresentação e de Persistência.

A relação de dependência entre as três classes que serão desenvolvidas é apresentada no diagrama de classes mostrado na Figura 7.50.

UNINOVE – USO EXCLUSIVO PARA ALUNO

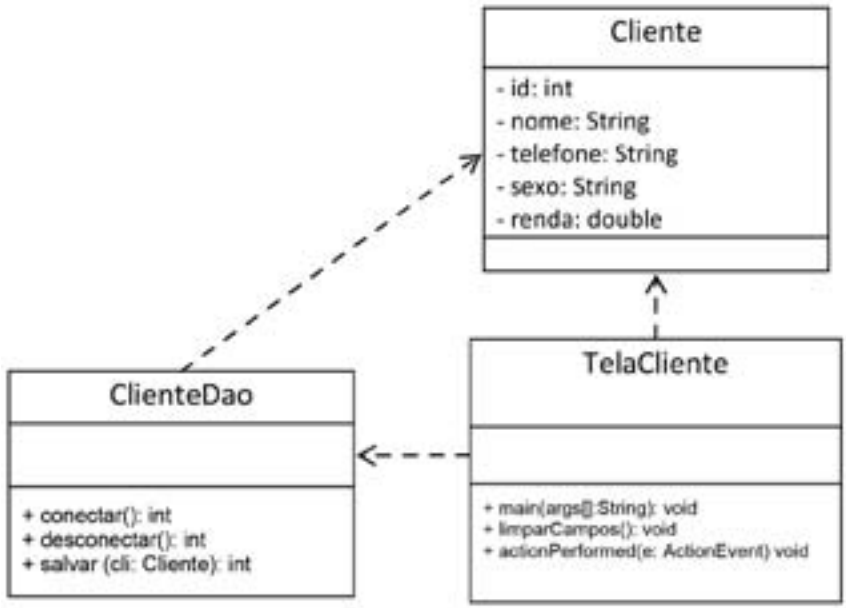


Figura 7.50: Diagrama de classes - relações entre as classes da aplicação.

Note que os métodos de acesso ao banco de dados estão concentrados na classe ClienteDao.java.

A classe *Cliente.java* representa o modelo da tabela *cliente* do banco de dados, por isso, seus atributos têm relação direta com os campos da tabela. Os dados cadastrados nos campos da janela (formulário), criada pela classe *TelaCliente.java*, serão encapsulados em um objeto da classe *Cliente.java* e este objeto será enviado ao método *salvar* da classe *ClienteDao.java* para ser inserido na tabela do banco de dados.

7.6.1 A classe DAO

A classe DAO concentra os métodos de acesso a banco de dados. Estes métodos serão chamados do interior do método *actionPerformed* da classe *TelaCliente.java*, onde são tratados os eventos de clique nos botões.

A seguir é apresentado o código-fonte da classe *ClienteDao.java*:

```
1    package persistence;
2
3    import java.sql.*;
4
5    public class ClienteDao {
6
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
7       String url = "jdbc:mysql://localhost:3306/bd002";
8       String usuario = "root";
9       String senha = "";
10      Connection con;
11      PreparedStatement st;
12
13
14      public int conectar() {
15          try {
16              Class.forName("com.mysql.jdbc.Driver");
17              con = DriverManager.getConnection(url, usuario, senha);
18              return 1;
19          } catch (ClassNotFoundException ex) {
20              return 2;
21          } catch (SQLException ex) {
22              return 3;
23          }
24      }
25
26      public int salvar(Cliente cli) {
27          try {
28              st = con.prepareStatement("insert into cliente (id, nome, telefone,
                sexo, renda) values (?, ?, ?, ?, ?)");
29              st.setInt(1, cli.getId());
30              st.setString(2, cli.getNome());
31              st.setString(3, cli.getTelefone());
32              st.setString(4, cli.getSexo());
33              st.setDouble(5, cli.getRenda());
34              int retorno = st.executeUpdate();
35              return retorno;
36          } catch (SQLException ex) {
37              int c = ex.getErrorCode();
38              if (c == 1062) {
39                  return 2;
40              } else {
41                  return 3;
42              }
43          }
44
45      }
46
47      public int desconectar() {
48          try {
49              con.close();
50              return 1;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
51              } catch (SQLException ex) {
52                  return 0;
53              }
54          }
55      }
```

Observe na linha 2 a indicação de que esta classe está no pacote *persistence*. Na linha 3, todas as classes e interfaces do pacote *java.sql* são importadas. Isso inclui as classes *DriverManager* e *SQLException* e as interfaces *PreparedStatement* e *Connection*.

As linhas 7, 8 e 9 criam variáveis que armazenam os parâmetros que serão utilizados na conexão com o banco de dados. Estes parâmetros são passados ao método *getConnection* da classe *DriverManager* na linha 17 para estabelecer a conexão com o banco de dados.

As linhas de 14 à 24 compõem o método *conectar*, que faz a conexão com o banco de dados e retornará o valor inteiro 1 (se a conexão foi estabelecida – linha 18), 2 (se a classe do *driver* não estiver na biblioteca do projeto – linha 20) ou 3 (se os parâmetros de conexão estiverem incorretos – linha 22).

As linhas de 26 à 45 compõem o método *salvar*. Veja que este método recebe um objeto da classe *Cliente.java* como parâmetro, contendo encapsulados os dados digitados nos campos do formulário. Se este método realizar a inclusão dos dados na tabela do banco de dados corretamente, ele devolverá o valor inteiro 1 (linha 35). Se a inclusão não for realizada porque o *id* que está sendo inserido (que é chave primária na tabela) já estiver cadastrado, este método retornará o valor 2 (linha 39). Se der qualquer outro erro na execução do comando *insert*, este método retornará o valor 3 (linha 41). Note na linha 28 que uma instrução *insert* é preparada aguardando como parâmetro os dados a serem inseridos. As linhas de 29 à 33 passam os parâmetros à instrução *insert*. Observe que como os valores a serem inseridos estão no objeto *cli* da classe *Cliente.java*, foi necessário retirar estes valores do objeto por meio dos metodos *getter*. Assim, por exemplo, na linha 29, obteve-se o *id* contido no objeto *cli* por meio da instrução *cli.getId()* e esse valor foi passado como parâmetro para o método *setInt* da interface *PreparedStatement*, como parâmetro de número (índice) 1. Note que o mesmo tipo de operação ocorre nas linhas de 31 à 33.

A linha 34 executa o comando *insert* preparado e recebe da tabela do banco de dados o retorno da operação. Este retorno será 1 se a operação for realizada com sucesso, ou 0, se a operação não teve sucesso. O retorno de uma operação realizada pelo método *executeUpdate* é sempre o número de linhas da tabela que foram afetadas pela operação. Seja qual for o valor retornado na operação, ele é retornado a quem chamar o método *salvar*, na linha 35.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Se ocorrer algum problema na execução da instrução SQL *insert*, um objeto *ex* da classe *SQLException* é recebido como parâmetro na instrução *catch* da linha 36. Por meio do objeto se obtém o número do erro na linha 37. Se o número do erro for 1062, que indica tentativa de duplicação de valor no campo chave primária da tabela, é retornado o valor 2, senão, se houver qualquer outro erro na execução da instrução *insert*, é retornado o valor 3 (linha 41).

A comparação da linha 38 tem o objetivo de identificar se você está tentando cadastrar um número de *id* que já está cadastrado. Isso é necessário quando o valor do campo chave primária é inserido pelo usuário na aplicação. Uma alternativa para evitar esse tipo de problema é usar o *id* como auto numeração, de forma que ele seja incrementado automaticamente sem a intervenção do usuário.

As linhas de 47 à 54 compõem o método *desconectar*. Veja que a linha que desconecta do banco de dados é a 49. Está linha deve estar dentro de uma estrutura de tratamento de exceção *try...catch*, pois você pode, por engano, tentar fechar uma conexão que nunca foi estabelecida. Nesse caso, geraria uma exceção do tipo *SQLException*. Se a conexão for fechada com sucesso, este método retornará 1, senão, retornará 0.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.6.2 A CLASSE MODELO DA TABELA DO BANCO DE DADOS

A classe *Cliente.java* é um modelo da tabela do banco de dados. Ela terá como atributo o que é campo na tabela, respeitando, inclusive, os mesmos tipos de dados. Este tipo de classe também é chamada de classe *bean* ou POJO (*Plain Old Java Objects*). Basicamente são classes que possuem atributos, um construtor e métodos *setter* e *getter* para os atributos.

Nesta aplicação de cadastro você verá como este tipo de classe, estudada no Capítulo 3, é utilizada com janelas e acesso a banco de dados. Basicamente os dados cadastrados em um formulário que está em uma janela são encapsulados em um objeto desta classe para serem transportados para métodos de outras classes ou para o banco de dados. Isto é essencial na orientação a objetos, transportar dados encapsulados em objetos.

Veja a seguir o código-fonte da classe *Cliente.java*:

```
1    package persistence;
2
3    public class Cliente {
4    private int id;
5    private String nome;
6    private String telefone;
7    private String sexo;
```

```
8       private double renda;
9
10          public Cliente() {
11          }
12
13          public Cliente(int id, String nome, String telefone, String sexo, double
            renda) {
14              this.id = id;
15              this.nome = nome;
16              this.telefone = telefone;
17              this.sexo = sexo;
18              this.renda = renda;
19          }
20
21          public int getId() {
22              return id;
23          }
24
25          public void setId(int id) {
26              this.id = id;
27          }
28
29          public String getNome() {
30              return nome;
31          }
32
33          public void setNome(String nome) {
34              this.nome = nome;
35          }
36
37          public String getTelefone() {
38              return telefone;
39          }
40
41          public void setTelefone(String telefone) {
42              this.telefone = telefone;
43          }
44
45          public String getSexo() {
46              return sexo;
47          }
48
49          public void setSexo(String sexo) {
50              this.sexo = sexo;
51          }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
52
53          public double getRenda() {
54              return renda;
55          }
56
57          public void setRenda(double renda) {
58              this.renda = renda;
59          }
60
61      }
```

Note que esta classe possui dois construtores, um vazio (linhas 10 e 11) e um que recebe os parâmetros que serão guardados em atributos do objeto (linhas de 13 à 19). Observe que para cada atributo é definido um método *setter* e um método *getter* equivalente.

7.6.3 A CLASSE PARA GERAÇÃO DO FORMULÁRIO DE CADASTRO

Para o usuário cadastrar os dados será gerado um formulário com os campos necessários. Utilizaremos a mesma tela já gerada em aplicações anteriores, para podermos focar a explicação apenas no que representa alguma novidade. A tela que será gerada é mostrada na Figura 7.51.



Figura 7.51: Tela de cadastro da aplicação utilizando o padrão DAO.

A classe que gera a janela mostrada na Figura 7.51 é a classe *TelaCliente.java*, cujo código-fonte é apresentado a seguir:

```
1       package view;
2
3       import java.awt.event.ActionEvent;
4       import java.awt.event.ActionListener;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
5   import java.text.DecimalFormat;
6   import java.text.ParseException;
7   import javax.swing.*;
8   import javax.swing.text.DefaultFormatterFactory;
9   import javax.swing.text.MaskFormatter;
10  import javax.swing.text.NumberFormatter;
11  import persistence.Cliente;
12  import persistence.ClienteDao;
13
14  public class TelaCliente extends JFrame implements ActionListener {
15
16      JButton btnSalvar;
17      JLabel lblId, lblNome, lblTelefone, lblSexo, lblRenda;
18      JTextField txtNome;
19      JFormattedTextField txtTelefone, txtId, txtRenda;
20      MaskFormatter mskTelefone, mskId;
21      ButtonGroup gruSexo;
22      JRadioButton rdoMasculino, rdoFeminino;
23
24      public TelaCliente() {
25
26          setTitle("Cadastro");
27          setDefaultCloseOperation(EXIT_ON_CLOSE);
28          setLayout(null);
29          setBounds(400, 200, 600, 280);
30          lblId = new JLabel("ID:");
31          lblId.setBounds(10, 10, 50, 30);
32          add(lblId);
33
34          txtId = new JFormattedTextField();
35          txtId.setFormatterFactory(new DefaultFormatterFactory(new
            NumberFormatter(new DecimalFormat("#"))));
36          txtId.setBounds(10, 40, 150, 30);
37          txtId.requestFocus();
38          add(txtId);
39
40          lblNome = new JLabel("Nome:");
41          lblNome.setBounds(170, 10, 50, 30);
42          add(lblNome);
43
44          txtNome = new JTextField(40);
45          txtNome.setBounds(170, 40, 400, 30);
46          add(txtNome);
47
48          lblTelefone = new JLabel("Telefone:");
```

```
49              lblTelefone.setBounds(10, 80, 150, 30);
50              add(lblTelefone);
51
52              try {
53                 mskTelefone = new MaskFormatter("(##)####-####");
54                 mskTelefone.setPlaceholderCharacter('_');
55              } catch (ParseException ex) {
56              }
57              txtTelefone = new JFormattedTextField(mskTelefone);
58              txtTelefone.setBounds(10, 110, 100, 30);
59              add(txtTelefone);
60
61              lblSexo = new JLabel("Sexo:");
62              lblSexo.setBounds(170, 80, 150, 30);
63              add(lblSexo);
64
65              rdoMasculino = new JRadioButton("Masculino");
66              rdoMasculino.setBounds(170, 110, 100, 30);
67              rdoFeminino = new JRadioButton("Feminino");
68              rdoFeminino.setBounds(300, 110, 100, 30);
69              gruSexo = new ButtonGroup();
70              gruSexo.add(rdoMasculino);
71              gruSexo.add(rdoFeminino);
72              add(rdoMasculino);
73              add(rdoFeminino);
74
75              lblRenda = new JLabel("Renda:");
76              lblRenda.setBounds(450, 80, 150, 30);
77              add(lblRenda);
78
79              txtRenda = new JFormattedTextField();
80              txtRenda.setFormatterFactory(new DefaultFormatterFactory(new
                NumberFormatter(new DecimalFormat("#0.00"))));
81              txtRenda.setBounds(450, 110, 100, 30);
82              txtRenda.setToolTipText("Utilize vírgula para separar as casas
                decimais");
83              add(txtRenda);
84
85              btnSalvar = new JButton("Salvar");
86              btnSalvar.setBounds(10, 170, 150, 30);
87              btnSalvar.addActionListener(this);
88              add(btnSalvar);
89          }
90
91          public static void main(String[] args) {
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
92              TelaCliente tela = new TelaCliente();
93              tela.setVisible(true);
94
95          }
96
97          @Override
98          public void actionPerformed(ActionEvent e) {
99              String mensagem;
100             if (e.getSource() == btnSalvar) {
101                 int id;
102                 double renda;
103                 String nome, telefone, sexo;
104                 id = Integer.parseInt(txtId.getText());
105                 nome = txtNome.getText();
106                 telefone = txtTelefone.getText();
107                 if (rdoMasculino.isSelected()) {
108                     sexo = rdoMasculino.getText();
109                 } else if (rdoFeminino.isSelected()) {
110                     sexo = rdoFeminino.getText();
111                 } else {
112                     sexo = "";
113                 }
114                 renda = Double.parseDouble(txtRenda.getText().replace(",", "."));
115                 Cliente cli = new Cliente(id, nome, telefone, sexo, renda);
116                 ClienteDao dao = new ClienteDao();
117                 int r = dao.conectar();
118                 if (r == 2) {
119                     mensagem = "Driver de conexão não foi encontrado";
120                 } else if (r == 3) {
121                     mensagem = "Os dados de conexão com o banco de dados
                        estão incorretos";
122                 } else {
123                     int x = dao.salvar(cli);
124                     if (x == 1) {
125                         mensagem = "Dados gravados com sucesso";
126                         limparCampos();
127                     } else if (x == 2) {
128                         mensagem = "Você está tentando cadastrar um ID que já
                            existe";
129                     } else {
130                         mensagem = "Dados gravados com sucesso";
131                     }
132                     dao.desconectar();
133                 }
134                 JOptionPane.showMessageDialog(null, mensagem);
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
135             }
136         }
137
138         private void limparCampos() {
139             txtId.setText("");
140             txtNome.setText("");
141             txtTelefone.setText("");
142             gruSexo.clearSelection();
143             txtRenda.setText("");
144             txtId.requestFocus();
145         }
146     }
```

Observe que a janela e seus componentes são gerados no construtor da classe, que vai da linha 24 à linha 89.

Além dos campos, haverá no formulário apenas o botão "Salvar", criado das linhas 85 à 88. Note que a este botão foi adicionado um *Listener* (ouvinte) para que quando ele for clicado seja executado o método *actionPerformed* da linha 98. Neste método, o objeto *e* do tipo *ActionEvent* permitirá identificar a origem do clique por meio do método *getSource*. Se houve um clique no botão "Salvar" (*btnSalvar* – linha 100), a operação de gravação será executada (das linhas 101 à 135).

As linhas de 104 à 114 capturam os valores inseridos nos campos do formulário e guardam em variáveis. Em seguida, na linha 115, estes dados são armazenados (encapsulados) no objeto *cli* da classe *Cliente.java*.

A linha 116 instancia um objeto da classe *ClienteDao.java* para que seja possível chamar o método *conectar* desta classe por meio deste objeto (linha 117). Na chamada ao método *conectar* (linha 117) é retornado o valor inteiro 1 (se a conexão foi estabelecida), 2 (se a classe do *driver* não estiver na biblioteca do projeto) ou 3 (se os parâmetros de conexão estiverem incorretos). Se o retorno foi 2 (linha 118), a mensagem da linha 119 é armazenada na variável *mensagem*. Se o retorno foi 3 (linha 120), a mensagem da linha 121 é armazenada na variável *mensagem*. Se o retorno foi 1 (linha 122), executa as linhas de 123 à 132, que realizam a operação de cadastro.

A linha 123 chama o método *salvar* da classe *ClienteDao.java* e passa como parâmetro o objeto da classe *Cliente.java* que contém os dados cadastrados no formulário. O método *salvar,* após ser executado, retornará 1 (se a operação de cadastro for realizada com sucesso), 2 (se o *id* já existe cadastrado na tabela do banco de dados) ou 3 (se ocorreu qualquer outro erro na inclusão dos dados na tabela do banco de dados). Se o retorno foi 1 (linha 124), armazena a mensagem da linha 125 na variável *mensagem* e limpa os campos por meio de

UNINOVE – USO EXCLUSIVO PARA ALUNO

uma chamada ao método *limparCampos* (linha 126). Se o retorno foi 2 (linha 127), armazena a mensagem da linha 128 na variável *mensagem*. Se o retorno for 3 (linha 129), armazena a mensagem da linha 130 na variável *mensagem*.

A linha 132 chama o método *desconectar* da classe *ClienteDao.java*, que encerra a conexão com o banco de dados.

A linha 134 exibe a mensagem que foi armazenada anteriormente na variável *mensagem*.

As linhas de 138 à 145 compõem o método *limparCampos* que limpa os campos e posiciona o cursor no campo *ID* para um novo cadastro.

DICA

Quando você precisa exibir apenas uma dentre muitas mensagens que são geradas em uma estrutura de seleção (*if..else if..else*), crie uma variável para receber a mensagem e exiba o conteúdo desta variável somente após sair da estrutura de seleção. Isto diminui a execução de comandos de exibição e pode melhorar a performance da aplicação.

Agora que você terminou de digitar o código-fonte das classes da aplicação, compile e execute o projeto por meio do pressionamento da tecla *F6* e cadastre alguns clientes.

7.6.4 Outras operações na classe DAO

A classe *ClienteDao.java* que desenvolvemos na aplicação apresentada anteriormente tinha apenas os métodos *conectar*, *desconectar* e *salvar*, pois a aplicação realizava apenas a operação de cadastro.

Se a aplicação realizar outras operações como alteração, consulta e exclusão, será necessário adicionar métodos para realizar estas operações na classe DAO e chamar estes métodos quando os botões referentes a estas funcionalidades forem clicados na interface do usuário.

Não vamos desenvolver neste livro estas funcionalidades, mas será apresentada a seguir a classe *ClienteDao.java* com os métodos que executam as principais operações na tabela do banco de dados. Você poderá, a partir desta classe, implementar as demais funcionalidades na classe *TelaCliente.java*.

O código-fonte a seguir apresenta a classe *ClienteDao.java* com métodos para conectar ao banco de dados, salvar um cliente, consultar um cliente, consultar todos os clientes, excluir um cliente e alterar os dados de um cliente.

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
1   package persistence;
2
3   import java.sql.*;
4   import java.util.ArrayList;
5
6   public class ClienteDao {
7
8       private String url = "jdbc:mysql://localhost:3306/bd002";
9       private String usuario = "root";
10      private String senha = "";
11      private Connection con;
12      private PreparedStatement st;
13      private ResultSet rs;
14
15      //Método para conectar ao banco de dados
16      public int conectar() {
17          try {
18              Class.forName("com.mysql.jdbc.Driver");
19              con = DriverManager.getConnection(url, usuario, senha);
20              return 1;
21          } catch (ClassNotFoundException ex) {
22              return 2;
23          } catch (SQLException ex) {
24              return 3;
25          }
26      }
27
28      //Método para salvar os dados de um cliente
29      public int salvar(Cliente cli) {
30          try {
31              st = con.prepareStatement("insert into cliente (id, nome, telefone,
                sexo, renda) values (?, ?, ?, ?, ?)");
32              st.setInt(1, cli.getId());
33              st.setString(2, cli.getNome());
34              st.setString(3, cli.getTelefone());
35              st.setString(4, cli.getSexo());
36              st.setDouble(5, cli.getRenda());
37              int retorno = st.executeUpdate();
38              return retorno;
39          } catch (SQLException ex) {
40              int c = ex.getErrorCode();
41              if (c == 1062) {
42                  return 2;
43              } else {
44                  return 3;
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
45              }
46          }
47      }
48
49      //Método para consultar apenas um cliente cujo id foi informado
50      public Cliente buscar(int id) {
51          try {
52              st = con.prepareStatement("select * from cliente where id = ? ");
53              st.setInt(1, id);
54              rs = st.executeQuery();
55              if (rs.next()) {
56                  Cliente cli = new Cliente();
57                  cli.setId(rs.getInt("id"));
58                  cli.setNome(rs.getString("nome"));
59                  cli.setTelefone(rs.getString("telefone"));
60                  cli.setSexo(rs.getString("sexo"));
61                  cli.setRenda(rs.getDouble("renda"));
62                  return cli;
63              } else {
64                  return null;
65              }
66          } catch (SQLException ex) {
67              return null;
68          }
69      }
70
71      //Método para buscar todos os registros cadastrados
72      public ArrayList buscarTudo() {
73          ArrayList<Cliente> listaClientes = new ArrayList<>();
74          try {
75              st = con.prepareStatement("select * from cliente");
76              rs = st.executeQuery();
77              while (rs.next()) {
78                  Cliente cli = new Cliente();
79                  cli.setId(rs.getInt("id"));
80                  cli.setNome(rs.getString("nome"));
81                  cli.setTelefone(rs.getString("telefone"));
82                  cli.setSexo(rs.getString("sexo"));
83                  cli.setRenda(rs.getDouble("renda"));
84                  listaClientes.add(cli);
85              }
86              return listaClientes;
87          } catch (SQLException ex) {
88              return null;
89          }
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

```
90          }
91
92          //Método para excluir um registro cujo id foi informado
93          public int excluir(int id) {
94             try {
95                st = con.prepareStatement("delete from cliente where id = ? ");
96                st.setInt(1, id);
97                int r = st.executeUpdate();
98                return r;
99             } catch (SQLException ex) {
100               return 0;
101            }
102         }
103
104         // Método para alterar o registro cujo id foi informado
104         public int salvarAlteracao(Cliente cli) {
106            try {
107               st = con.prepareStatement("update cliente  set nome=?, telefone=?,
                  sexo=?, renda=? where id=?");
108               st.setString(1, cli.getNome());
109               st.setString(2, cli.getTelefone());
110               st.setString(3, cli.getSexo());
111               st.setDouble(4, cli.getRenda());
112               st.setInt(5, cli.getId());
113               int r = st.executeUpdate();
114               return r;
115            } catch (SQLException ex) {
116               return 0;
117            }
118         }
119
120         //Método para se desconectar do banco de dados
121         public int desconectar() {
122            try {
123               con.close();
124               return 1;
125            } catch (SQLException ex) {
126               return 0;
127            }
128         }
129      }
```

Como já foram explicados os métodos *conectar*, *desconectar* e *salvar* no exemplo anterior, as próximas linhas comentarão apenas os métodos para consultar, alterar e excluir dados.

UNINOVE – USO EXCLUSIVO PARA ALUNO

O método para consultar apenas um cliente é apresentado das linhas 50 à 69. Este método recebe o *id* do cliente a ser consultado, realiza a busca na tabela do banco de dados e armazena os dados retornados em um objeto da classe *Cliente.java*. Em seguida, retorna este objeto com os dados a quem chamar este método de busca. Caso ocorra algum problema na consulta ou a consulta não retorne nenhum cliente do banco de dados, será retornado *null* a quem chamar este método de busca.

Observe da linha 72 à 90 o método para buscar todos os clientes cadastrados no banco de dados. Os dados retornados por meio da execução da instrução *select* são armazenados em um objeto da interface *ResultSet* (linha 76). Em seguida, no laço de repetição das linhas de 77 à 85, estes dados do objeto da interface *ResultSet* são encapsulados em objetos da classe *Cliente.java* e armazenados em uma lista do tipo *ArrayList*. Ao final da execução do laço você terá uma lista de objetos da classe *Cliente.java* com os dados retornados do banco de dados. Esta lista será retornada a quem chamar este método.

O método para realizar a exclusão é composto pelas linhas de 93 à 102. Este método executa uma instrução *delete* e retorna 1 se a operação ocorreu com sucesso ou 0 se a operação falhou.

Das linhas 104 à 118 está o método para alterar os dados de um cliente. Este método é semelhante ao método *salvar*, porém, executa uma instrução *update* ao invés de uma instrução *insert*.

7.6.5 Criação de interface DAO

Se você precisa criar uma aplicação que controle, por exemplo, produtos, funcionários e clientes, terá que construir uma classe *ProdutoDao.java*, uma classe *FuncionarioDao.java* e uma classe *ClienteDao.java*. Normalmente terá que construir uma classe DAO para cada tabela do banco de dados em que você executará operações. Todas estas classes DAO terão que implementar alguns métodos comuns, porém, de maneiras diferentes. Estes métodos são salvar, consultar, alterar, excluir etc. Neste caso, é indicada a criação de uma interface DAO com a assinatura destes métodos que serão implementados nas respectivas classes DAO da aplicação. Veja essa representação no diagrama de classes mostrado na Figura 7.52.

UNINOVE – USO EXCLUSIVO PARA ALUNO

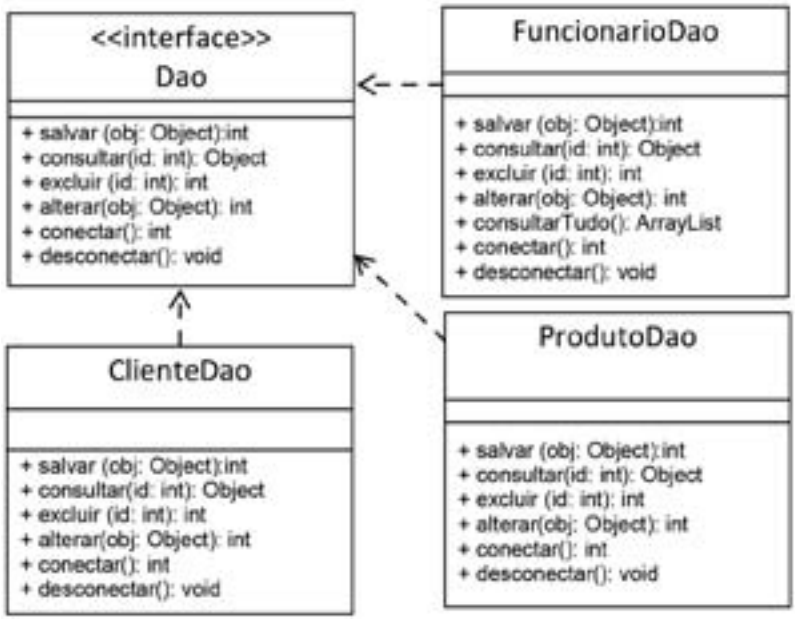


Figura 7.52: Diagrama de classes utilizando uma interface DAO na aplicação.

Observe que os métodos assinados na interface são sobrescritos em todas as classes que implementam as interfaces. Observe também que o método *salvar* recebe como parâmetro um objeto genérico da classe *Object* que pode ser um objeto da classe *Cliente.java* na classe *ClienteDao.java*; *Produto.java* na classe *ProdutoDao.java* e *Funcionario.java* na classe *FuncionarioDao.java*.

Observe também que a classe *FuncionarioDao.java* possui um método que não está assinado na interface. Isso é totalmente permitido. Além dos métodos assinados na interface que terão implementação obrigatória na classe, ela poderá implementar quantos outros métodos específicos forem necessários.

Observando o diagrama apresentado é possível concluir que a interface *Dao.java* terá o seguinte código-fonte:

```
public interface Dao {
   public int salvar (Object obj);
   public Object consultar(int id);
   public int excluir(int id);
   public int  alterar(Object obj);
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Já as classes que implementam a interface *Dao.java*, por exemplo, a classe *ClienteDao.java*, terá o seguinte código-fonte:

```
public class ClienteDao implements Dao{

   @Override
   public int salvar(Object obj) {
      //implementação do método aqui
   }

   @Override
   public Object consultar(int id) {
      //implementação do método aqui
   }

   @Override
   public int excluir(int id) {
      //implementação do método aqui
   }

   @Override
   public int alterar(Object obj) {
      //implementação do método aqui
   }
   @Override
   public int conectar() {
      //implementação do método aqui
   }

   @Override
   public void desconectar() {
      //implementação do método aqui
   }
}
```

UNINOVE – USO EXCLUSIVO PARA ALUNO

Trabalhar com interfaces garante uma padronização maior na aplicação e facilita o entendimento, mas pode impor maior complexidade na programação. Você deve analisar quais recursos são relevantes e implementá-los de forma a reaproveitar o máximo de código possível, evitando repetição e de forma que não comprometa a performance da aplicação.

7.7 Resumo

Para acessar bancos de dados em aplicações Java existem dois padrões que determinam as regras para acesso de maneira eficiente: *Open Database Connectivity* (ODBC) e *Java Database Connectivity* (JDBC). Para utilizar

estes padrões, os fabricantes de SGBDs criam os *drivers* (que são bibliotecas de classes) para seguir as regras impostas por esses padrões.

ODBC é uma interface genérica padrão *Microsoft Windows* que permite a ligação de aplicações feitas em qualquer linguagem de programação com qualquer banco de dados. Já JDBC é uma interface padrão, independente de plataforma, que permite a comunicação entre aplicações baseadas em Java e SGBDs.

Para que sua aplicação possa se conectar a um SGBD específico é necessário entrar no site do fabricante e baixar o *driver* JDBC. Este *driver* deve ser adicionado à pasta "Bibliotecas" do projeto criado com o *NetBeans*.

Quando você deseja criar uma aplicação que acessa banco de dados, primeiro deve entrar no SGBD e criar o banco de dados e as tabelas que pretente utilizar. O *NetBeans* permite criar o banco de dados e as tabelas na sua própria área de trabalho, na guia "Serviços", mas para isso o servidor de banco de dados deverá ter sido inicializado (startado).

Em uma aplicação que acessa banco de dados, você terá que utilizar classe e interfaces do pacote *java.sql*. Este pacote traz as interfaces *Connection*, *Statement*, *PreparedStatement* e *ResultSet*, e as classes *DriverManager* e *SQLException*.

A interface *Connection* estabelece e mantém uma conexão ativa com o banco de dados.

As interfaces *Statement* ou *PreparedStatement* permitem executar comandos SQL no banco de dados, como *insert*, *select*, *delete* e *update*. A diferença entre estas duas interfaces é que a interface *PreparedStatement* prepara uma instrução SQL que pode receber parâmetros posteriormente. Já com a interface *Statement,* a instrução SQL não poderá receber parâmetros. Por meio de métodos assinados nestas interfaces é possível utilizar os comandos SQL. O método *executeQuery* permite executar o comando *select* e o método *executeUpdate* permite executar os comandos *insert*, *update* e *delete*.

Objetos da interface *ResultSet* recebem o retorno das consultas realizadas na tabela por meio do comando *select*.

Um objeto da classe *SQLException* é instanciado quando uma exceção relacionada a uma operação no banco de dados ocorre. Este objeto é recebido como parâmetro na cláusula *catch* de um tratamento de exceção iniciado pela instrução *try*. Por meio deste objeto é possível obter informações do erro, como o número (método *getErrorCode*), a mensagem de erro (método *getErrorMessage*) etc.

UNINOVE – USO EXCLUSIVO PARA ALUNO

A classe *DriverManager* passa, por meio do método *getConnection*, os parâmetros para realizar a conexão com o banco de dados, como o caminho, a porta, o nome do banco de dados, o usuário e a senha, e gerencia o uso do *driver* de conexão fornecido pelo fabricante do SGBD.

Para se conectar a qualquer SGBD bastará modificar apenas duas linhas na aplicação:

```
Class.forName("com.mysql.jdbc.Driver");
Connection conn = DriverManager.getConnection ("jdbc:mysql://localhost:3306/bd002",
"root", "teruel");
```

Bastará mudar os parâmetros passados para o método *forName* da classe *Class* e *getConnection* da classe *Drivermanager* para se conectar a qualquer SGBD.

As aplicações com acesso a banco de dados normalmente executam as funcionalidades chamadas de CRUD (*Create*, *Read*, *Update* e *Delete*). *Create* é a funcionalidade de cadastro, executada pela instrução SQL *insert*. *Read* é a funcionalidade de consulta, executada pela instrução SQL *select*. *Update* é a funcionalidade de alteração, executada pela instrução SQL *update*. *Delete* é a funcionalidade de exclusão, executada pela instrução SQL *delete*.

Apesar de você poder realizar as operações de acesso a banco de dados em qualquer local da aplicação, é comum separar essas operações em uma classe que chamamos de classe DAO.

O *design pattern* (padrão de projeto) *Data Access Object* (DAO) é um padrão que recomenda que todas as operações de acesso a banco de dados sejam separadas em classes DAO.

Em aplicações que utilizam duas, três ou mais camadas, a classe DAO é representada na camada de Persistência, também chamada de camada de Dados ou de Modelo.

Se você precisar criar uma aplicação que necessita de muitas classes DAO e elas precisem implementar um conjunto de métodos com assinatura comum, mas com implementação diferente, é aconselhável o uso de uma interface DAO para centralizar a assinatura dos métodos comuns.

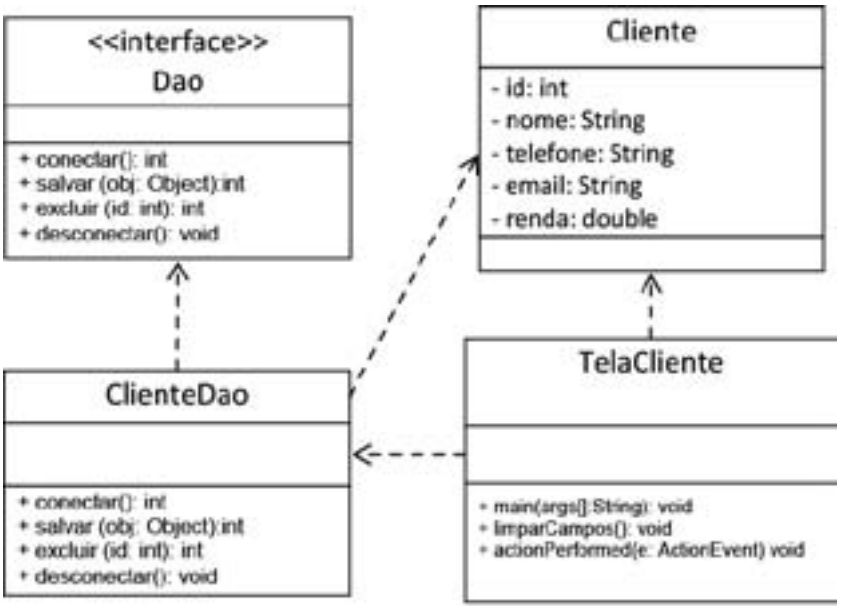
7.8 Exercícios

Os exercícios presentes neste tópico são divididos em duas categorias: práticos e conceituais. Os exercícios práticos precisam ser realizados em um computador com ambiente de programação Java instalado e configurado. Os exercícios conceituais cobram conceitos referentes aos temas abordados neste capítulo.

UNINOVE – USO EXCLUSIVO PARA ALUNO

7.8.1 EXERCÍCIOS PRÁTICOS

1) Crie um projeto no *NetBeans* e implemente a aplicação mostrada no diagrama de classes a seguir:



2) Implemente uma aplicação Java utilizando as APIs *swing* e *awt* para criar a interface do usuário. Esta aplicação deverá executar as operações CRUD em um banco de dados chamado *BdProd* com uma tabela *produto* contendo os seguintes campos:

id – *int, not null, primary key*
nomeProduto – *varchar* (50)
marca – *varchar* (30)
categoria – *varchar* (30)
preco – *double precision*

7.8.2 EXERCÍCIOS CONCEITUAIS

1) Qual a diferença entre os padrões ODBC e JDBC?

2) O que muda em uma aplicação se você quiser fazer a conexão com um SGBD diferente do utilizado atualmente?

UNINOVE – USO EXCLUSIVO PARA ALUNO

3) Qual a diferença entre as interfaces *Statement* e *PreparedStatement*? Dê um exemplo.

4) O que é *driver* e onde ele deve ser adicionado no projeto Java que acessa banco de dados?

5) Quais métodos das interfaces *Statement* e *preparedStatement* são utilizados para executar as instruções SQL *insert*, *update*, *delete* e *select*?

6) Qual a finalidade de uma classe DAO?

7) Quando, em uma aplicação que possui diversos módulos e diversas tabelas no banco de dados, é aconselhável utilizar interfaces?

8) Que exceção (ou exceções) pode gerar um método que realiza conexão com banco de dados?

9) Como pode ser verificado em uma operação de cadastro se o valor que está sendo inserido na chave primária da tabela já foi cadastrado?

10) O que o método *executeUpdate* das interfaces *Statement* e *PreparedStatement* retorna? Para que pode ser utilizado este valor de retorno?

11) O que o método *executeQuery* das interfaces *Statement* e *PreparedStatement* retorna?

12) Como é possível percorrer o conteúdo de um objeto da interface *ResultSet* até o seu final?

13) Quais são as principais classes e/ou interfaces contidas no pacote *java.sql*?

UNINOVE – USO EXCLUSIVO PARA ALUNO

8. Recomendações para complementar a formação em Java

Após ler e praticar os conteúdos vistos até aqui neste livro, você estará pronto para aprofundar seus conhecimentos em Java estudando assuntos mais complexos como *Collections*, Mapeamento Objeto-Relacional com *JPA* e *Hibernate*, desenvolvimento de sites ou aplicações distribuídas com Java EE (*JSP*, *servlet*, *EJB*, *JSF* etc.), desenvolvimento Java para celulares e *tablets* que usam o sistema Android, *design patterns* (padrões de projeto) e *frameworks*. Poderá ainda, se desejar, fazer provas para certificações Java reconhecidas internacionalmente. Este capítulo apresenta uma introdução sobre estes assuntos.

8.1 Collections

Agora que você já conheceu e utilizou as diversas formas de manipulação de dados com variáveis simples, constantes, objetos, tipos primitivos e de referência, um bom caminho é estudar um tipo de estrutura mais complexa chamada de coleção (*collection*).

As coleções (*collections*) em Java são estruturas de dados semelhantes as *arrays*, porém, que manipulam dados como objetos, são dimensionadas dinamicamente (ou seja, não tem tamanho pré-definido) e possuem um conjunto de métodos úteis para manipular dados como listas, filas etc.

Da interface *Collection* descendem as interfaces *Set*(conjunto), *Queue* (filas) e *List* (listas) que formam a base das coleções genéricas da linguagem Java, como pode ser visto na Figura 8.1.

A interface *Set* define uma coleção que não contém valores duplicados; *Queue* define uma coleção que representa uma fila no modelo FIFO (*First-In, First-Out*); e *List* define uma coleção ordenada que pode conter elementos duplicados.

UNINOVE – USO EXCLUSIVO PARA ALUNO

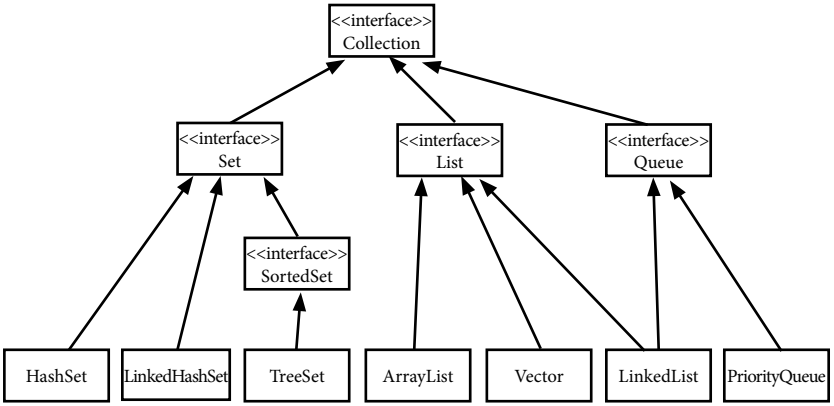


Figura 8.1: Hierarquia das coleções em java.
Fonte: http://www.w3resource.com/java-tutorial/java-collections.php

Note que as classes concretas *HashSet*, *LinkedHashSet* e *TreeSet* implementam a interface *Set*; *ArrayList*, *Vector* e *LinkedList* implementam a interface *List*; e *LinkedList* e *PriorityQueue* implementam a interface *Queue*.

8.2 Mapeamento Objeto-Relacional

Executar operações em banco de dados relacionais utilizando Java com JDBC e controlando as operações no código-fonte é uma tarefa difícil, pois inclui o trabalho de gerenciar as conexões com o banco de dados e executar operações SQL de forma a manter a consistência dos dados. Da forma mostrada no Capítulo 7 deste livro, um método que recebe, por exemplo, um objeto contendo dados em uma classe DAO, não pode enviá-lo diretamente para a tabela correspondente do banco de dados, tendo que retirar os valores contidos no objeto, colocá-los em uma instrução INSERT da linguagem SQL e depois enviar os dados através da execução deste comando.

Para que o objeto possa ser enviado diretamente para a tabela correspondente do banco de dados relacional, de forma que os atributos sejam mapeados corretamente para os campos da tabela correspondente, Java possui uma API chamada Java Persistence API (JPA). Para implementar JPA é necessário o uso de um *framework* de persistência como *EclipseLink*, *Oracle TopLink* ou *Hibernate*.

O uso mais comum é de *JPA* com *Hibernate*. Utilizando estes recursos, você poderá delegar o gerenciamento de conexões com o banco de dados, assim como a execução de operações *SQL* ao *JPA*, focando seus esforços na implementação das regras de negócio da aplicação ou em outros aspectos que necessitam da atenção do desenvolvedor. *JPA* mapeia o envio de objetos da aplicação para as tabelas e de registros das tabelas para objetos da aplicação, reduzindo significativamente o esforço de programação.

UNINOVE – USO EXCLUSIVO PARA ALUNO

8.3 JAVA EE

Hoje em dia, a maioria das aplicações funcionam na *web*, o que torna atrativo aprender Java EE. Com os recursos disponíveis no Java EE você pode desenvolver sites e aplicações corporativas distribuídas, como aquelas que disponibilizam um conjunto de recursos acessados de aplicações *web*, *desktop* e *mobile*, oferecendo uma interface adequada para cada uma. Dentre os recursos mais importantes do Java EE que vale a pena estudar estão *JSP*, *servlet*, *JSTL*, *EJB* e *JSF*. Para criar aplicações Java EE você vai precisar de um servidor de aplicação como *TomEE*, *Glasfish*, *JBoss*, *Weblogic*, *Tomcat* (apenas contêiner *web*) etc.

8.4 PADRÕES DE PROJETO

Outro assunto que você não pode deixar de estudar são os padrões de projeto (*design patterns*).

Os padrões de projeto são soluções reutilizáveis, pré-definidas e modeladas para problemas comuns dentro de um determinado contexto (ou domínio) de aplicação. O padrão de projeto dita ou apresenta uma maneira de fazer algo de forma já testada em que os resultados positivos já foram comprovados. É uma maneira de não ter que "reinventar a roda".

Alguns dos principais padrões de projeto foram apresentado em um livro escrito por quatro autores conhecidos como a Gangue dos Quatro (*Gang of Four – GoF*). Eles apresentaram os padrões de projetos divididos em três grupos: padrões de criação, relacionados à criação de objetos; padrões estruturais, que tratam das associações entre classes e objetos; e padrões comportamentais, que tratam das interações e divisões de responsabilidades entre as classes ou objetos. Os padrões de cada grupo são apresentados a seguir:

Padrões de criação:
– *Abstract Factory.*
– *Builder.*
– *Factory Method.*
– *Prototype.*
– *Singleton.*

Padrões estruturais:
– *Adapter.*
– *Bridge.*
– *Composite.*
– *Decorator.*
– *Façade.*
– *Flyweight.*
– *Proxy.*

Padrões comportamentais:
– *Chain of Responsibility.*
– *Command.*
– *Interpreter.*
– *Iterator.*
– *Mediator.*
– *Memento.*
– *Observer.*
– *State.*
– *Strategy.*
– *Template Method.*
– *Visitor.*

UNINOVE – USO EXCLUSIVO PARA ALUNO

Além destes padrões apresentados pelo *Gang of Four – GoF*, há diversos outros padrões de projeto disponíveis, como *Model-View-Controller* (MVC), o *Data Access Object* (DAO) etc.

O MVC é um dos mais famosos padrões de projeto para o desenvolvimento de aplicações *web* em três camadas. Ele sugere o agrupamento dos componentes da aplicação em Apresentação (*View*), Controle (*Controller*) e Modelo (*Model*). No grupo *View* são representados os componentes que apresentam informações aos usuários, como páginas *HTML*, arquivos *CSS*, *JavaScript* e *JSP*. No grupo *Controller*, são representadas as *servlets* Java que recebem as requisições dos usuários, passam dados a componentes do grupo *Model* e devolvem dados do *Model* aos usuários. No grupo *Model* são representadas as clesses *bean* (ou *JavaBean*) que representam o modelo das tabelas do banco de dados, as classes que implementam as regras de negócio e as classes de acesso a dados, que executam operações *SQL* no banco de dados.

DAO é um padrão para persistência de dados que permite separar as regras de negócio das regras de acesso ao banco de dados. Normalmente em aplicações que utiliza o padrão *MVC*, todas as funcionalidades de bancos de dados, tais como conexões, mapeamento de objetos para as tabelas do banco de dados e execução de comandos *SQL* são feitas utilizando classes no padrão *DAO*.

8.5 Frameworks

Depois de entender o conceito de padrão de projeto, ou *design pattern*, você deve estudar um pouco sobre os *frameworks*. Enquanto os *design patterns* representam modelos de procedimentos que comprovadamente funcionam, os *frameworks* trazem normalmente algo pronto, ou seja, pelo menos uma parte dos códigos necessários para executar os procedimentos definidos em algum padrão de projeto. Alguns *frameworks*, principalmente voltados para o desenvolvimento de aplicações *web*, são listados a seguir:

Spring – http://www.springsource.org/

EJB – http://www.oracle.com/technetwork/java/javaee/ejb/index.html

Hibernate – http://www.hibernate.org/

EJB – http://www.oracle.com/technetwork/java/javaee/ejb/index.html

Struts – http://struts.apache.org/

JSF – http://www.oracle.com/technetwork/java/javaee/javaserverfaces-139869.html

Estes *frameworks* normalmente já vem integrados a *IDEs* como o *NetBeans* ou Eclipse, porém, em alguns casos, precisarão ser baixados e integrados à *IDE*.

UNINOVE – USO EXCLUSIVO PARA ALUNO

8.6 Android e iOS

Java é a linguagem de programação base para o desenvolvimento de aplicativos para dispositivos móveis (celulares e *tablets*) que utilizam o Sistema Operacional *Android*, baseado no núcleo do Sistema Operacional *Linux*. Se você quiser aproveitar este mercado, vale a pena dedicar um tempo para aprender a usar o Java para programação voltada a dispositivos com *Android*.

Caso queira desenvolver para dispositivos da *Apple* que usam o Sistema Operacional *iOS*, como *iPhones* e *iPads*, terá que aprender outra linguagem com sintaxe bem diferente do Java, a *Objective-C*, e utilizar um ambiente de desenvolvimento (IDE) chamada *xCode*.

8.7 Principais certificações Java

Uma boa opção para conseguir espaço no mercado de trabalho tão concorrido, é fazer uma certificação Java reconhecida internacionalmente, principalmente se você não tem experiência comprovada em desenvolvimento Java.

As duas principais certificações Java SE são *OCAJP* e *OCPJP*.

OCAJP – Oracle Certified Associate Java SE Programmer (Programmer I), cobra as bases do desenvolvimento Java. Trata-se de uma prova com 70 questões de múltipla escolha e o candidato terá 120 minutos para respondê-la. É preciso acertar pouco mais de 60% das questões para passar nesta prova. A prova inicia com temas básicos sobre laços de repetição e declaração de variáveis, e segue até tópicos mais complexos, como *arrays* e exceções. Não cobre tópicos específicos como criação de *GUIs* e programação para Internet.

OCPJP – Oracle Certified Professional Java SE Programmer (Programmer II), é a próxima certificação, que cobra conhecimentos aprofundados do Java SE. Esta prova cobra conhecimentos de construção de laços de repetição complexos, *Threads*, *Collections* e *Generics*. Não abrange domínios tecnológicos específicos, como a criação de programação *GUI*, *web* e distribuída. A prova é composta por 60 questões e o candidato tem 150 minutos para responder. Para ser aprovado o candidato deve acertar pouco mais de 60% das questões.

Você pode conhecer estas e outras certificações disponíveis no site da *Oracle:*

<https://education.oracle.com/pls/web_prod-plq-dad/db_pages.getpage?page_id=632>

As provas estão disponíveis em vários idiomas, inclusive em português.

UNINOVE – USO EXCLUSIVO PARA ALUNO

8.8 Resumo

As coleções (*collections*) são estruturas de dados semelhantes às *arrays*, que manipulam dados como objetos, são dimensionadas dinamicamente e possuem um conjunto de métodos úteis para manipular dados como listas, filas e conjunto. Da interface *Collection* descendem as interfaces *Set* (conjuntos), *Queue* (filas) e *List* (listas), que formam a base das coleções em Java.

Quando precisar fazer a comunicação de aplicação com o banco de dados, você pode utilizar a *API JPA* com o *framework Hibernate* para mapear objetos para o banco de dados diretamente, sem ter necessariamente que utilizar comandos SQL, e receber registros do banco de dados como objetos, sem ter necessariamente que utilizar comandos de consulta *SQL*. A criação e execução dos comandos SQL pode ser deixada a cargo da *JPA*.

Para criar sites e aplicações distribuídas e multiplataforma, você pode utilizar recurso do Java EE como *JSP*, *servlets*, *EJB*, *JSF*, *Struts* etc. Estas aplicações necessitam de servidores, alguns deles integrados à IDE, como *Glassfish*, *Tomcat* entre outros.

Muitas das situações e problemas encontrados no projeto e desenvolvimento de *software* já foram resolvidos e as soluções já foram testadas e tiveram a eficiência comprovada. Estas soluções foram desenhadas e documentadas em padrões de projeto, para evitar que novos desenvolvedores tenham os mesmos problemas e as mesmas dificuldades para solucioná-los. Há um conjunto de padrões de projeto utilizados com frequência no mercado que é interessante o desenvolvedor conhecer, como o MVC e DAO. Além dos padrões de projeto, que apresentam modelos de soluções, existem os *frameworks*, que apresentam parte da solução já desenvolvida, normalmente partes de código já prontos para facilitar a implementação dos *design patterns*. Alguns dos principais *frameworks* são *JSF*, *Hibernate*, *Struts* e *EJB*.

Se desejar programar para celulares e *tablets*, ou você utiliza o Java ME, ou utiliza programação Java para dispositivos que utilizam o Sistema Operacional *Android*. O desenvolvimento desse tipo de aplicação tem crescido muito devido à popularização dos dispositivos móveis e do acesso sem fio à Internet.

Uma boa opção para conseguir espaço no mercado de trabalho tão concorrido, é fazer uma certificação Java internacional, principalmente se você não tem experiência comprovada em desenvolvimento com Java. As duas certificações Java SE mais importantes hoje são OCAJP e OCPJP. A primeira aborda aspectos mais básicos e a segunda, conceitos mais avançados.

UNINOVE – USO EXCLUSIVO PARA ALUNO

Referências

CAELUM. *Exceções e controle de erros*. Disponível em: <http://www.caelum.com.br/apostila-java-orientacao-objetos/excecoes-e-controle-de-erros>. Acesso em: 15 ago. 2014.

CODEJAVA. *JComboBox basic tutorial and examples*. Disponível em: < http://www.codejava.net/java-se/swing/jcombobox-basic-tutorial-and-examples>. Acesso em: 08 ago. 2014.

DEITEL, H.; DEITEL, P. *Java*: como programar. 8. ed. São Paulo: Pearson Prentice-Hall, 2010.

HORSTMANN, Key. *Conceitos de computação com Java*. 5. ed. Porto Alegre: Bookman, 2009.

JAVA COLLECTION FRAMEWORK. Disponível em: <http://www.w3resource.com/java-tutorial/java-collections.php>. Acesso em: 28 set. 2015.

TERUEL, Evandro Carlos. *Arquitetura de sistemas para web com Java utilizando design patterns e frameworks*. Rio de Janeiro: Ciência Moderna, 2012.

O GESTOR DE POSICIONAMENTO GRIDBAGLAYOUT. Disponível em: <http://www.dei.isep.ipp.pt/~mouta/LPG2-Programas/progJavaGestPos/GridBagLayout.htm>. Acesso em: 05 ago. 2014.

ORACLE JAVA DOCUMENTATION. *Trail*: creating a GUI With JFC/Swing. Disponível em: <http://docs.oracle.com/javase/tutorial/uiswing/index.html>. Acesso em: 05 ago. 2014.

UNINOVE – USO EXCLUSIVO PARA ALUNO

UNINOVE – USO EXCLUSIVO PARA ALUNO

UNINOVE – USO EXCLUSIVO PARA ALUNO

UNINOVE – USO EXCLUSIVO PARA ALUNO

Este livro apresenta de maneira prática e simplificada a programação orientada a objetos utilizando Java. Você aprenderá a diferença entre a programação estruturada e orientada a objetos, as estruturas e comandos fundamentais do Java, os conceitos essenciais da programação orientada a objetos (como encapsulamento, herança, polimorfismo, interfaces etc.), a criação de interfaces gráficas para desktop, tratamento de exceções e acesso a banco de dados utilizando a API Java Database Connectivity (JDBC).

Cada capítulo é estruturado com uma introdução, apresentação dos conteúdos teóricos com exemplos, resumo e lista de exercícios.

Um livro para se aprender, definitivamente, a programação com Java.



ISBN 978-85-89852-27-2

